The Parse Tree of Knowledge

# Formal Languages and Their Applications

An Introduction via Lark

— 2026 —

Prof. Dr. Karl Stroetmann

October 13, 2026

These lecture notes, along with their LaTeX source files and any associated programs, are available on the following GitHub repository:

https://github.com/karlstroetmann/Formal-Languages.

Given the dynamic nature of the field of computer science, these notes are subject to periodic updates. If you have `git` installed on your computer, you can clone the repository using the following command:

```
git clone https://github.com/karlstroetmann/Formal-Languages.git
```

To update to the most recent version of these notes, execute the command

```
git pull
```

from the repository's root directory.

Since these notes are frequently updated, they may contain typos or errors. If you find any, please consider submitting a pull request or contacting me via Discord (username: `karl.stroetmann`) or email at karl.stroetmann@dhbw.de.

# Contents


**List of Notation** **4**

**1 Introduction and Motivation** **6**
- 1.1 Basic Definitions . . . 7
- 1.2 Overview . . . 10
- 1.3 Literature . . . 11
- 1.4 Check your Understanding . . . 12

**2 Regular Expressions** **13**
- 2.1 Preliminary Definitions . . . 14
- 2.2 The Formal Definition of Regular Expressions . . . 15
- 2.3 Algebraic Simplification of Regular Expressions . . . 18
- 2.4 Check your Understanding . . . 20

**3 Building Scanners** **21**
- 3.1 Overview . . . 21
- 3.2 Introduction . . . 22
- 3.3 Building a Scanner with Regular Expressions . . . 24
- 3.4 The Syntax of Regular Expressions in *Python* . . . 27
  - 3.4.1 The Function `findall` . . . 27
  - 3.4.2 Characters and Meta-Characters . . . 28
  - 3.4.3 Concatenation . . . 29
  - 3.4.4 Choice . . . 29
  - 3.4.5 Quantifiers . . . 30
  - 3.4.6 Non-Greedy Quantifiers . . . 32
  - 3.4.7 Character Classes . . . 32
  - 3.4.8 Grouping and Back-References . . . 34
  - 3.4.9 The Dot . . . 35
  - 3.4.10 Named Groups . . . 35
  - 3.4.11 The Start and the End of a Line . . . 36
  - 3.4.12 Lookahead Assertions . . . 36
  - 3.4.13 Examples . . . 37
  - 3.4.14 Further Reading . . . 38
- 3.5 A Complex Example: Evaluating an Exam . . . 38
  - 3.5.1 Computing the Mark . . . 39
  - 3.5.2 Extracting the Maximum Number of Points . . . 40
  - 3.5.3 Processing the Student Records . . . 41
  - 3.5.4 Running the Evaluation . . . 43
- 3.6 Converting Html to Text . . . 43

3.7 Check your Understanding . . . 56

**4 Finite State Machines** **57**
4.1 Deterministic Finite Automata . . . 58
4.2 Non-Deterministic Finite Automata . . . 62
4.3 Transforming NFAs to DFAs . . . 65
4.3.1 Implementation . . . 69
4.4 From Regular Expression to NFAs . . . 70
4.4.1 Implementation . . . 74
4.5 Translating a DFA into a Regular Expression . . . 74
4.5.1 Implementation . . . 77
4.6 Minimization of Finite State Machines . . . 77
4.6.1 Implementation . . . 81
4.7 Conclusion . . . 81
4.8 Check your Understanding . . . 82

**5 The Theory of Regular Languages** **83**
5.1 Closure Properties of Regular Languages . . . 84
5.2 Recognizing Empty Languages . . . 88
5.3 Equivalence of Regular Expressions . . . 89
5.4 Limits of Regular Languages . . . 89
5.5 Check your Understanding . . . 95

**6 Context-Free Languages** **96**
6.1 Context-Free Grammars . . . 97
6.1.1 Derivations . . . 100
6.1.2 Parse Trees . . . 102
6.1.3 Ambiguous Grammars . . . 103
6.2 Top-Down Parser . . . 104
6.2.1 Implementing a Recursive Descent Parser that Uses an EBNF Grammar . . . 105
6.3 Check your Understanding . . . 107

**7 Using Lark as a Parser Generator** **108**
7.1 A Symbolic Calculator . . . 109
7.2 Symbolic Differentiation . . . 115
7.3 Check your Understanding . . . 123

**8 Implementing a Simple Interpreter** **124**
8.1 Check your Understanding . . . 136

**9 Earley Parser** **137**
9.1 The Earley Algorithm . . . 138
9.2 Implementing Earley's Algorithm in *Python* . . . 143
9.3 Check Your Understanding . . . 144

**10 Bottom-up Parsers** **145**
10.1 Bottom-Up-Parser . . . 146
10.2 Shift-Reduce-Parser . . . 147
10.3 SLR-Parser . . . 155
10.3.1 The Functions *First* and *Follow* . . . 158
10.3.2 Computing the Function *action* . . . 162

10.3.3 Shift/Reduce and Reduce/Reduce Conflicts . . . 166
10.4 Canonical LR Parsers . . . 168
10.5 LALR-Parser* . . . 172
10.6 Comparison of SLR, LR, and LALR Parsers . . . 175
10.6.1 SLR Language ⊊ LALR Language . . . 175
10.6.2 LALR Language ⊊ Canonical LR Language . . . 176
10.6.3 Evaluation of the Different Methods . . . 177
10.7 Check your Understanding . . . 178

**11 Advanced Features of Lark** **179**
11.1 Shift-Reduce and Reduce-Reduce Conflicts . . . 180
11.1.1 Making Lark Talk About Its Conflicts . . . 181
11.1.2 Inspecting the LALR States . . . 182
11.2 Operator Precedence . . . 183
11.3 Resolving Shift-Reduce and Reduce-Reduce Conflicts . . . 186
11.3.1 Look-Ahead Conflicts . . . 186
11.3.2 Mysterious Reduce-Reduce Conflicts . . . 188
11.4 Check your Understanding . . . 191

**12 Assembler** **192**
12.1 Introduction into Jasmin Assembler . . . 193
12.2 Assembler Instructions . . . 196
12.2.1 Instructions to Manipulate the Stack . . . 199
12.3 An Example Program . . . 203
12.4 Disassembler* . . . 206

**13 Development of a Simple Compiler** **209**
13.1 The Programming Language *Integer*-C . . . 210
13.2 Developing the Scanner and the Parser . . . 211
13.3 Code Generation . . . 220
13.3.1 Translation of Arithmetic Expressions . . . 220
13.3.2 Translation of Boolean Expressions . . . 224
13.3.3 Translation of Statements . . . 228
13.3.4 Translation of a Function Definition . . . 234
13.3.5 Compiling a Program . . . 236

**Bibliography** **241**

**Index** **242**

# List of Notation

| Notation | Meaning | Page |
|---|---|---|
| $\Sigma$ | alphabet | 7 |
| $\lambda$ | empty string | 7 |
| $\Sigma^*$ | set of all strings over the alphabet $\Sigma$ | 7 |
| $\lvert w \rvert$ | length of the string $w$ | 8 |
| $w[i]$ | $i$-th character of the string $w$ (counting from $0$) | 8 |
| $L_1 \cdot L_2$ | product (concatenation) of the languages $L_1$ and $L_2$ | 14 |
| $L^n$ | $n$-th power of the language $L$ | 14 |
| $L^*$ | Kleene closure of the language $L$ | 15 |
| $s^n$ | $n$-th power of the string $s$ | 15 |
| $L(r)$ | language described by the regular expression $r$ | 15 |
| $\mathsf{RegExp}_\Sigma$ | set of regular expressions over $\Sigma$ | 15 |
| $\emptyset$ | regular expression denoting the empty language | 15 |
| $\varepsilon$ | regular expression denoting $\{\lambda\}$ | 15 |
| $2^M$ | power set of the set $M$ | 15 |
| $r_1 + r_2$ | choice between the regular expressions $r_1$ and $r_2$ | 16 |
| $r_1 \cdot r_2$ | concatenation of the regular expressions $r_1$ and $r_2$ | 16 |
| $r^*$ | Kleene star of the regular expression $r$ | 16 |
| $r_1 \doteq r_2$ | $r_1$ and $r_2$ are equivalent, i.e. $L(r_1) = L(r_2)$ | 18 |
| $\langle Q, \Sigma, \delta, q_0, A \rangle$ | finite state machine: states, alphabet, transition function, start state, accepting states | 59 |
| $\Omega$ | undefined value, e.g. a missing transition | 59 |
| $\delta^*(q, w)$ | state reached from $q$ after reading the string $w$ | 59 |
| $L(F)$ | language accepted by the automaton $F$ | 60 |
| ☠ | dead state | 60 |
| $\rightsquigarrow$ | transition between configurations of an NFA | 65 |
| $\rightsquigarrow^*$ | reflexive-transitive closure of $\rightsquigarrow$ | 65 |
| $\mathsf{ec}(q)$ | $\varepsilon$-closure of the state $q$ | 66 |
| $\mathit{det}(F)$ | deterministic automaton constructed from the NFA $F$ | 67 |
| $p_1 \sim p_2$ | the states $p_1$ and $p_2$ are equivalent | 78 |
| $[p]_\sim$ | equivalence class of the state $p$ | 78 |
| $\Sigma^* \backslash L$ | complement of the language $L$ | 83 |
| $s^R$, $L^R$ | reversal of the string $s$, of the language $L$ | 88 |
| $\langle V, T, R, S \rangle$ | context-free grammar: variables, terminals, rules, start symbol | 99 |
| $\alpha \Rightarrow_G \beta$ | $\beta$ results from $\alpha$ by one derivation step of $G$ | 100 |
| $\alpha \Rightarrow_G^* \beta$ | $\beta$ can be derived from $\alpha$ in any number of steps | 100 |
| $L(G)$ | language generated by the grammar $G$ | 101 |
| $\langle a \to \beta \bullet \gamma, k \rangle$ | Earley object | 138 |

| Notation | Meaning | Page |
|---|---|---|
| $\widehat{s}$ | new start symbol of the augmented grammar | 138 |
| $\langle Q, q_0, \mathit{action}, \mathit{goto}\rangle$ | shift-reduce parser | 147 |
| $\mathit{action}(q, t)$ | action of a shift-reduce parser in state $q$ on token $t$ | 148 |
| $\mathit{goto}(q, v)$ | successor state of $q$ after reducing to the variable $v$ | 148 |
| $a \to \beta \bullet \gamma$ | marked rule | 155 |
| $\mathit{closure}(\mathcal{M})$ | closure of the set of marked rules $\mathcal{M}$ | 156 |
| \$ | end-of-input token | 157 |
| $\mathit{goto}(\mathcal{M}, c)$ | successor state of $\mathcal{M}$ after reading the symbol $c$ | 157 |
| $\mathit{First}(\alpha)$ | tokens that can start a string derived from $\alpha$ | 158 |
| $\mathit{Follow}(a)$ | tokens that can follow the variable $a$ | 160 |
| $a \to \beta \bullet \gamma : L$ | extended marked rule with the follow tokens $L$ | 168 |
| $\mathcal{M} \uplus \mathcal{N}$ | extended union of two sets of extended marked rules | 174 |

# Chapter 1

# Introduction and Motivation

Figure 1.1: An ancient clay tablet with cuneiform script.

This lecture explores the theory of formal languages and delves into various applications, with a focus on the development of scanners,[1] parsers,[2] interpreters,[3] and compilers.[4] We also introduce the

[1] A *scanner* (also called a *lexer* or *tokenizer*) splits a text into *tokens* such as numbers, names, keywords, and operators, e.g. "`x1 + 23`" is split into the tokens "`x1`", "`+`", and "`23`".

[2] A *parser* checks whether a sequence of tokens conforms to the *grammar* of a language and builds a tree that shows the structure of the input.

[3] An *interpreter* executes a program directly, instruction by instruction. For example *Python* programs are executed via an interpreter.

[4] A *compiler* translates a program into a lower-level language like machine code or *Java* byte code.

tool Lark, which can be used to generate a parser from a given formal description of a programming language.

## 1.1 Basic Definitions

The cornerstone of this lecture is the concept of a *formal language*, essentially defined as a set of strings characterized by precise mathematical criteria. To establish this concept, we first introduce some foundational definitions.

**Definition 1 (Alphabet)** An alphabet $\Sigma$ is a finite, non-empty set of characters:

$$\Sigma = \{c_1, \cdots, c_n\}.$$

The term symbol is occasionally used interchangeably with the term character. ◇

**Examples**:

(a) $\Sigma = \{0, 1\}$ is the alphabet for representing binary numbers.

(b) $\Sigma = \{$ "a", $\cdots$, "z", "A", $\cdots$, "Z" $\}$ is the standard alphabet for the English language.

(c) $\Sigma_{\text{ASCII}} = \{0, 1, \cdots, 127\}$ is known as the ASCII-alphabet. Here, numbers correspond to letters, digits, punctuation marks, and control characters. For instance, the set of numbers $\{65, \cdots, 90\}$ represents the letters $\{\text{A}, \cdots, \text{Z}\}$: The number $65$ represents the letter "A", the number $66$ represents the letter "B", and the number $90$ represents the letter "Z". Table 1.1 on the next page shows all printable characters of the ASCII-alphabet. The remaining codes, i.e. the first 32 codes and the code $127$, are non-printable control codes, for example the number $10$ encodes a line feed control symbol, while the number $127$ encodes the delete character.

(d) $\Sigma_{\text{UNICODE}} = \{0, 1, \cdots, 1\,114\,111\}$ represents the UNICODE-alphabet. In hexadecimal notation, the largest code is 0x10FFFF. Currently, i.e. in version 18.0, 172 808 codes from the Unicode-alphabet are assigned to character. This extensive set accommodates a wide array of characters, including 175 scripts, various symbols, and even emojis. Table 1.2 shows the Unicode encoding of the 100 most frequent Chinese characters according to the frequency list of Jun Da. The characters are ordered by frequency, row by row. A *Jupyter* notebook that displays arbitrary ranges of Unicode characters in a table can be found at

Python/Chapter-01/Show-Unicode.ipynb. ◇

**Definition 2 (Strings)** For a given alphabet $\Sigma$, a string is a sequence of characters selected from $\Sigma$. In formal language theory, these sequences are written without brackets or separating commas. Thus, for $c_1, \cdots, c_n \in \Sigma$, we write

$$w = c_1 \cdots c_n \quad \text{instead of} \quad w = [c_1, \cdots, c_n].$$

The empty string is represented by $\lambda$, so $\lambda =$ "". The set of all possible strings formed from the alphabet $\Sigma$ is denoted as $\Sigma^*$. Strings are often enclosed in quotation marks for emphasis. ◇

**Examples**:

(a) Let $\Sigma = \{0, 1\}$. If we define

$$w_1 := \text{"01110"} \text{ and } w_2 := \text{"11001"},$$

then both $w_1$ and $w_2$ are strings over $\Sigma$. Hence, we can state:

$$w_1 \in \Sigma^* \text{ and } w_2 \in \Sigma^*.$$

| Character | Code | Character | Code | Character | Code |
|---|---|---|---|---|---|
| | | | | Space | 32 |
| ! | 33 | " | 34 | # | 35 |
| $ | 36 | % | 37 | & | 38 |
| ' | 39 | ( | 40 | ) | 41 |
| * | 42 | + | 43 | , | 44 |
| - | 45 | . | 46 | / | 47 |
| 0 | 48 | 1 | 49 | 2 | 50 |
| 3 | 51 | 4 | 52 | 5 | 53 |
| 6 | 54 | 7 | 55 | 8 | 56 |
| 9 | 57 | : | 58 | ; | 59 |
| < | 60 | = | 61 | > | 62 |
| ? | 63 | @ | 64 | A | 65 |
| B | 66 | C | 67 | D | 68 |
| E | 69 | F | 70 | G | 71 |
| H | 72 | I | 73 | J | 74 |
| K | 75 | L | 76 | M | 77 |
| N | 78 | O | 79 | P | 80 |
| Q | 81 | R | 82 | S | 83 |
| T | 84 | U | 85 | V | 86 |
| W | 87 | X | 88 | Y | 89 |
| Z | 90 | [ | 91 | \ | 92 |
| ] | 93 | ^ | 94 | _ | 95 |
| ‘ | 96 | a | 97 | b | 98 |
| c | 99 | d | 100 | e | 101 |
| f | 102 | g | 103 | h | 104 |
| i | 105 | j | 106 | k | 107 |
| l | 108 | m | 109 | n | 110 |
| o | 111 | p | 112 | q | 113 |
| r | 114 | s | 115 | t | 116 |
| u | 117 | v | 118 | w | 119 |
| x | 120 | y | 121 | z | 122 |
| { | 123 | \| | 124 | } | 125 |
| ~ | 126 | DEL | 127 | | |

Table 1.1: ASCII Encoding Table

(b) Consider $\Sigma = \{\mathtt{a}, \cdots, \mathtt{z}\}$. With

$$w := \text{"}\mathtt{example}\text{"},$$

it follows that $w \in \Sigma^*$. ◇

The *length* of a string $w$ is denoted by $|w|$ and represents the number of characters in $w$. We employ array notation for character extraction: given a string $w$ and a natural number $i < |w|$, $w[i]$ specifies the character at position $i$ in $w$. Positions are counted from 0, so $w[0]$ is the first character of $w$. This adheres to conventions in modern programming languages like C, *Java*, and *Python*. Next, we introduce the concept of concatenation of two strings $w_1$ and $w_2$. Concatenation results in a new string $w$ formed by appending $w_2$ to $w_1$. It is represented as $w_1 \cdot w_2$. In most books the concatenation of $w_1$ and $w_2$ is written without the dot as $w_1 w_2$.

**Example**: For $\Sigma = \{0, 1\}$ and $w_1 = \text{"}01\text{"}$ and $w_2 = \text{"}10\text{"}$, the concatenated strings are:

| Glyph | Unicode | Glyph | Unicode | Glyph | Unicode | Glyph | Unicode |
|---|---|---|---|---|---|---|---|
| 的 | U+7684 | 一 | U+4E00 | 是 | U+662F | 不 | U+4E0D |
| 了 | U+4E86 | 在 | U+5728 | 人 | U+4EBA | 有 | U+6709 |
| 我 | U+6211 | 他 | U+4ED6 | 这 | U+8FD9 | 个 | U+4E2A |
| 们 | U+4EEC | 中 | U+4E2D | 来 | U+6765 | 上 | U+4E0A |
| 大 | U+5927 | 为 | U+4E3A | 和 | U+548C | 国 | U+56FD |
| 地 | U+5730 | 到 | U+5230 | 以 | U+4EE5 | 说 | U+8BF4 |
| 时 | U+65F6 | 要 | U+8981 | 就 | U+5C31 | 出 | U+51FA |
| 会 | U+4F1A | 可 | U+53EF | 也 | U+4E5F | 你 | U+4F60 |
| 对 | U+5BF9 | 生 | U+751F | 能 | U+80FD | 而 | U+800C |
| 子 | U+5B50 | 那 | U+90A3 | 得 | U+5F97 | 于 | U+4E8E |
| 着 | U+7740 | 下 | U+4E0B | 自 | U+81EA | 之 | U+4E4B |
| 年 | U+5E74 | 过 | U+8FC7 | 发 | U+53D1 | 后 | U+540E |
| 作 | U+4F5C | 里 | U+91CC | 用 | U+7528 | 道 | U+9053 |
| 行 | U+884C | 所 | U+6240 | 然 | U+7136 | 家 | U+5BB6 |
| 种 | U+79CD | 事 | U+4E8B | 成 | U+6210 | 方 | U+65B9 |
| 多 | U+591A | 经 | U+7ECF | 么 | U+4E48 | 去 | U+53BB |
| 法 | U+6CD5 | 学 | U+5B66 | 如 | U+5982 | 都 | U+90FD |
| 同 | U+540C | 现 | U+73B0 | 当 | U+5F53 | 没 | U+6CA1 |
| 动 | U+52A8 | 面 | U+9762 | 起 | U+8D77 | 看 | U+770B |
| 定 | U+5B9A | 天 | U+5929 | 分 | U+5206 | 还 | U+8FD8 |
| 进 | U+8FDB | 好 | U+597D | 小 | U+5C0F | 部 | U+90E8 |
| 其 | U+5176 | 些 | U+4E9B | 主 | U+4E3B | 样 | U+6837 |
| 理 | U+7406 | 心 | U+5FC3 | 她 | U+5979 | 本 | U+672C |
| 前 | U+524D | 开 | U+5F00 | 但 | U+4F46 | 因 | U+56E0 |
| 只 | U+53EA | 从 | U+4ECE | 想 | U+60F3 | 实 | U+5B9E |

Table 1.2: UNICODE Encoding of the 100 Most Frequent Chinese Characters

$w_1 \cdot w_2 =$ "0110" and $w_2 \cdot w_1 =$ "1001". ◇

**Definition 3 (Formal Language)**
Given an alphabet $\Sigma$, a subset $L \subseteq \Sigma^*$ is called a formal language if it is *precisely defined*. ◇

You may wonder what it means for a subset to be *precisely defined*. A subset $L$ is precisely defined if for every $w \in \Sigma^*$ the notion $w \in L$ is unambiguously defined. For instance, English doesn't qualify as a formal language because it lacks a precise rule set for valid sentences.

The definition of the concept of a formal language is intentionally broad. As we progress through the lecture, we will explore specialized versions, most notably regular languages and context-free languages. These are particularly relevant in computer science.

**Examples**:

1. Assume that $\Sigma = \{0, 1\}$. Define

   $$L_{\mathbb{N}} = \{1 \cdot w \mid w \in \Sigma^*\} \cup \{0\}$$

   Then $L_{\mathbb{N}}$ is the language consisting of all strings that can be interpreted as natural numbers given in binary notation. The language contains all strings from $\Sigma^*$ that start with the character 1 as well as the string 0, which only contains the character 0. For example, we have

   "100" $\in L_{\mathbb{N}}$, but "010" $\notin L_{\mathbb{N}}$.

Let us define a function

$$\mathit{value} : L_{\mathbb{N}} \to \mathbb{N}$$

on the set $L_{\mathbb{N}}$. We define $\mathit{value}(w)$ by induction on the length of $w$. We call $\mathit{value}(w)$ the interpretation of $w$. The idea is that $\mathit{value}(w)$ computes the number that is represented by the string $w$:

(a) $\mathit{value}(0) = 0$, $\mathit{value}(1) = 1$,

(b) $|w| > 0 \to \mathit{value}(w0) = 2 \cdot \mathit{value}(w)$,

(c) $|w| > 0 \to \mathit{value}(w1) = 2 \cdot \mathit{value}(w) + 1$.

2. Again we have $\Sigma = \{0, 1\}$. Define the language $L_{\mathbb{P}}$ to be the set of all strings from $L_{\mathbb{N}}$ whose value is a prime number:

$$L_{\mathbb{P}} := \{w \in L_{\mathbb{N}} \mid \mathit{value}(w) \in \mathbb{P}\}$$

Here, $\mathbb{P}$ denotes the set of prime numbers, which is the set of all natural numbers $p$ bigger than $1$ that have no divisor other than $1$ or $p$:

$$\mathbb{P} = \left\{p \in \mathbb{N} \;\middle|\; p > 1 \wedge \{t \in \mathbb{N} \mid \exists k \in \mathbb{N} : k \cdot t = p\} = \{1, p\}\right\}.$$

3. Define $\Sigma_{\text{ASCII}} = \{0, \cdots, 127\}$. Furthermore, define $L_C$ as the set of all strings of the form

```
char* f(char* x) { ··· }
```

that are, furthermore, valid C function definitions. Therefore, $L_C$ contains all those strings that can be interpreted as a C function $f$ such that $f$ takes a single argument which is a string and returns a value which is also a string.

4. Define $\Sigma := \Sigma_{\text{ASCII}} \cup \{\dagger\}$, where $\dagger$ is some new symbol that is different from all symbols in $\Sigma_{\text{ASCII}}$. The universal language $L_u$ is the set of all strings of the form

$$p\dagger x\dagger y$$

such that

(a) $p \in L_C$,

(b) $x, y \in \Sigma^*_{\text{ASCII}}$,

(c) if $f$ is the function that is defined by $p$, then $f(x)$ yields the result $y$. ◇

The above examples illustrate the expansive scope of what can be considered a formal language. While it's straightforward to identify strings in $L_{\mathbb{N}}$, this becomes substantially more challenging for languages like $L_{\mathbb{P}}$ or $L_C$. Moreover, since the halting problem is undecidable, no algorithm can definitively determine membership in the universal language $L_u$. Nevertheless, $L_u$ is semi-decidable: if a string $w$ belongs to $L_u$, we can eventually establish this fact.

## 1.2 Overview

The aim of this lecture is to cover a series of interconnected topics. While this overview will introduce some terms that may not be immediately clear, rest assured that these terms will be elaborated upon as the lecture progresses. The lecture is structured into three main parts:

1. The first part delves into the theory of regular languages. These are those languages that can be described by a regular expression.

   (a) We begin with an exploration of regular expressions. After formally defining this term, we show how regular expressions can be simplified algebraically.
   (b) We then demonstrate how to create scanners using the regular expressions of *Python*.
   (c) Subsequently, we introduce finite state machines, both deterministic and non-deterministic ones. We show how a regular expression can be translated into a finite state machine and, conversely, how a finite state machine can be translated into a regular expression. Hence both concepts describe the same class of languages. Furthermore, we show how a finite state machine can be minimized.
   (d) Next, we prove that regular languages are closed under union, intersection, and complement. Using these closure properties and an algorithm that decides whether a regular language is empty, we can decide whether two regular expressions are equivalent.
   (e) We conclude this part by discussing the limitations of regular languages, focusing particularly on the Pumping Lemma.

2. The second part is centered on context-free languages, which are instrumental in describing the syntax of programming languages.
   (a) We first introduce context-free grammars, which are employed to define context-free languages, and show how to implement a top-down parser for a given grammar.
   (b) Next, we talk about using Lark as a parser generator. As examples, we implement a symbolic calculator and a program for symbolic differentiation.
   (c) Then we use Lark to implement an interpreter for a small fragment of the programming language C.
   (d) Essential theory for understanding parsing algorithms is then presented, including the Earley parser and bottom-up parsers (such as SLR, LR, and LALR parsers).

3. The final part of this lecture covers compilers.
   (a) We introduce the Java virtual machine and its assembly language Jasmin.
   (b) Then we discuss how to implement a compiler that translates a subset of the C language (Integer-C) into Jasmin assembler, which is then translated into Java byte code.

## 1.3 Literature

Besides these lecture notes, I highly recommend the following three books and one additional resource:

(a) *Introduction to Automata Theory, Languages, and Computation* [HMU06] — Often considered the definitive text on formal languages, this book encompasses all the theoretical concepts discussed in this lecture, although we will only touch on a subset of its content.

(b) *Introduction to the Theory of Computation* [Sip12] — This book provides another accessible introduction to formal languages and extends to the theory of computability, which is beyond the scope of this lecture.

(c) *Compilers — Principles, Techniques and Tools* [ASUL06] — A standard reference in compiler theory, this book also covers a substantial portion of formal languages.

(d) The large language models Gemini, Grok, Claude, DeepSeek, and ChatGPT have reached a level of maturity that allows them to serve as a tutor for many concepts discussed in this course.

## 1.4 Check your Understanding

(a) Define the notion of an alphabet.

(b) Based on a given alphabet, define the concept of a string.

(c) Explain the concept of a formal language.

# Chapter 2

# Regular Expressions

Figure 2.1: Stephen Cole Kleene, the father of regular languages.

*Regular expressions* are linguistic constructs designed to specify formal languages simple enough to be recognized by a finite state machine. These machines will be introduced in Chapter 4. At the core, regular expressions enable the specification of:

(a) Alternatives among different options,

(b) Concatenation of elements, and

(c) Repetition of elements.

Regular expressions form the backbone of many modern scripting languages. For instance, the initial surge in the popularity of the programming language *Perl* can be largely attributed to its efficient handling of regular expressions. Today, most popular high-level programming languages (e.g. *Python*, C++, *Java*, C#, `JavaScript`, `TypeScript`, PHP, `Ruby`, `Tcl`) support regular expressions via corresponding libraries. In the programming language C regular expressions are available via the C POSIX library.

Moreover, various UNIX utilities, including `grep`, `sed`, and `awk`, are based on regular expressions. Consequently, proficiency in regular expressions is indispensable for a budding computer scientist.

In this chapter, we will introduce a definition of regular expressions that is more concise than those typically used in programming languages. This streamlined definition will facilitate our theoretical examination of regular languages in Chapters 4 and 5. The syntax of regular expressions as used in *Python* will be covered in the subsequent chapter.

## 2.1 Preliminary Definitions

Before delving into the syntax and semantics of regular expressions, we need to introduce some preliminary definitions.

**Definition 4 (Product of Languages)** Given an alphabet $\Sigma$ and formal languages $L_1, L_2 \subseteq \Sigma^*$, the product of $L_1$ and $L_2$ is denoted as $L_1 \cdot L_2$ and is defined as the set of all concatenations $w_1 \cdot w_2$ where $w_1 \in L_1$ and $w_2 \in L_2$. Formally,

$$L_1 \cdot L_2 := \{w_1 \cdot w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2\}. \qquad \diamond$$

**Example**: Suppose $\Sigma = \{\mathsf{a}, \mathsf{b}, \mathsf{c}\}$ and $L_1$ and $L_2$ are given by

$$L_1 = \{\mathsf{ab}, \mathsf{bc}\} \quad \text{and} \quad L_2 = \{\mathsf{ac}, \mathsf{cb}\},$$

then $L_1 \cdot L_2 = \{\mathsf{abac}, \mathsf{abcb}, \mathsf{bcac}, \mathsf{bccb}\}$. $\diamond$

**Definition 5 (Power of a Language)** Let $\Sigma$ be an alphabet, $L \subseteq \Sigma^*$ a formal language, and $n \in \mathbb{N}$. The $n$-th power of $L$, denoted as $L^n$, is defined inductively as follows:

B.C.: $n = 0$:

$L^0 := \{\lambda\}$, where $\lambda$ is the empty string.

I.S.: $n \mapsto n + 1$:

$L^{n+1} := L^n \cdot L$. $\diamond$

**Example**: For $\Sigma = \{\mathsf{a}, \mathsf{b}\}$ and $L = \{\mathsf{ab}, \mathsf{ba}\}$, we find:

(a) $L^0 = \{\lambda\}$,

(b) $L^1 = L$,

(c) $L^2 = \{\mathsf{abab}, \mathsf{abba}, \mathsf{baab}, \mathsf{baba}\}$. $\diamond$

**Definition 6 (Kleene Closure)** Given an alphabet $\Sigma$ and a formal language $L \subseteq \Sigma^*$, the Kleene closure of $L$, denoted as $L^*$, is the union of all languages $L^n$ for $n \in \mathbb{N}$:

$$L^* := \bigcup_{n \in \mathbb{N}} L^n = L^0 \cup L^1 \cup L^2 \cup \cdots$$

Note that $\lambda \in L^*$, so $L^*$ is never empty, even when $L = \{\}$.

The Kleene closure is named after Stephen Cole Kleene (1909–1994), who invented regular expressions. ◇

**Definition 7 (Power of a string, $s^n$)** For a string $s$ and a non-negative integer $n \in \mathbb{N}$, the expression $s^n$ is defined inductively:

B.C.: $n = 0$:

$s^0 := \lambda$.

I.S.: $n \mapsto n + 1$:

$s^{n+1} := s^n \cdot s$, where $s^n \cdot s$ represents the concatenation of $s^n$ and $s$. ◇

**Example**: Let $\Sigma = \{\mathsf{a}, \mathsf{b}\}$ and $L = \{\mathsf{a}\}$. Then

$$L^* = \{\mathsf{a}^n \mid n \in \mathbb{N}\},$$

where $\mathsf{a}^n$ represents a string of $n$ consecutive a's. ◇

The previous example highlights that the Kleene closure of a finite language can be infinite. Specifically, $L^*$ will be infinite whenever $L$ contains a string different from the empty string $\lambda$.

## 2.2 The Formal Definition of Regular Expressions

We proceed to define the set of regular expressions for a given alphabet $\Sigma$. This set is denoted as $\mathsf{RegExp}_\Sigma$, and is defined inductively. Simultaneously, we define the function

$$L : \mathsf{RegExp}_\Sigma \to 2^{\Sigma^*},$$

which interprets each regular expression $r$ as a formal language $L(r) \in 2^{\Sigma^*}$.[1]

**Definition 8 (Regular Expressions, Stephen Cole Kleene)** The set $\mathsf{RegExp}_\Sigma$ of regular expressions on the alphabet $\Sigma$ is defined inductively as follows:

1. $\emptyset \in \mathsf{RegExp}_\Sigma$

   The regular expression $\emptyset$ denotes the empty language, we have

   $$L(\emptyset) := \{\}.$$

   In order to avoid confusion we assume that the symbol $\emptyset$ is not a member of the alphabet $\Sigma$, i.e. we have $\emptyset \notin \Sigma$.

2. $\varepsilon \in \mathsf{RegExp}_\Sigma$

   The regular expression $\varepsilon$ denotes the language that only contains the empty string $\lambda$:

   $$L(\varepsilon) := \{\lambda\}.$$

   Furthermore, we assume that $\varepsilon \notin \Sigma$.

[1] Given a set $M$, the power set of $M$, i.e., the set of all subsets of $M$, is denoted as $2^M$.

3. $c \in \Sigma \to c \in \mathsf{RegExp}_\Sigma$.

   Every character from the alphabet $\Sigma$ is a regular expression. This expression denotes the language that contains only the string $c$:

   $$L(c) := \{c\}.$$

   Observe that we identify characters with strings of length one.

4. $r_1 \in \mathsf{RegExp}_\Sigma \wedge r_2 \in \mathsf{RegExp}_\Sigma \to r_1 + r_2 \in \mathsf{RegExp}_\Sigma$

   Starting from two regular expressions $r_1$ and $r_2$ we can use the infix operator "$+$" to build a new regular expression. This regular expression denotes the union of the languages described by $r_1$ and $r_2$:

   $$L(r_1 + r_2) := L(r_1) \cup L(r_2).$$

   We have to assume that the symbol "+" does not occur in the alphabet $\Sigma$, i.e. we have "+" $\notin \Sigma$.

5. $r_1 \in \mathsf{RegExp}_\Sigma \wedge r_2 \in \mathsf{RegExp}_\Sigma \to r_1 \cdot r_2 \in \mathsf{RegExp}_\Sigma$

   Starting from the regular expressions $r_1$ and $r_2$ we can use the infix operator "$\cdot$" to build a new regular expression. This regular expression denotes the product of the languages of $r_1$ and $r_2$:

   $$L(r_1 \cdot r_2) := L(r_1) \cdot L(r_2).$$

   We have to assume that the symbol "$\cdot$" does not occur in the alphabet $\Sigma$, i.e. we have "$\cdot$" $\notin \Sigma$.

6. $r \in \mathsf{RegExp}_\Sigma \to r^* \in \mathsf{RegExp}_\Sigma$

   Given a regular expression $r$, the postfix operator "$^*$" can be used to create a new regular expression. This new regular expression denotes the Kleene closure of the language described by $r$:

   $$L(r^*) := \big(L(r)\big)^*.$$

   We have to assume that "$^*$" $\notin \Sigma$.

7. $r \in \mathsf{RegExp}_\Sigma \to (r) \in \mathsf{RegExp}_\Sigma$

   Regular expressions can be surrounded by parentheses. This does not change the language denoted by the regular expression:

   $$L\big((r)\big) := L(r).$$

   We have to assume that the parentheses "(" and ")" do not occur in the alphabet $\Sigma$, i.e. we have "(" $\notin \Sigma$ and ")" $\notin \Sigma$. ◇

Given the preceding definition, it is unclear whether we interpret the regular expression

$$\mathtt{a} + \mathtt{b} \cdot \mathtt{c}$$

as either

$$(\mathtt{a} + \mathtt{b}) \cdot \mathtt{c} \quad \text{or instead as} \quad \mathtt{a} + (\mathtt{b} \cdot \mathtt{c}).$$

To remove this ambiguity, we assign operator precedences as follows:

1. The postfix operator "$^*$" has the highest precedence.
2. The infix operator "$\cdot$" has a lower precedence than the postfix operator "$^*$" but a higher precedence than the infix operator "$+$".
3. The infix operator "$+$" has the lowest precedence.

With these conventions, the expression

$$\mathtt{a} + \mathtt{b} \cdot \mathtt{c}^*$$

is interpreted as

$$\mathtt{a} + \big(\mathtt{b} \cdot (\mathtt{c}^*)\big).$$

Precedences do not tell us how to read an expression like $\mathtt{a} + \mathtt{b} + \mathtt{c}$, where the same operator occurs twice. We assume that the operators "$+$" and "$\cdot$" both associate to the left. Therefore,

$a + b + c$ is interpreted as $(a + b) + c$

and

$a \cdot b \cdot c$ is interpreted as $(a \cdot b) \cdot c$

However, we will see in the next section that the operators "$+$" and "$\cdot$" are associative, e.g. the regular expression $(a + b) + c$ and $a + (b + c)$ are equivalent.

**Examples**: In the following examples, the alphabet $\Sigma$ is defined as

$$\Sigma := \{\mathtt{a}, \mathtt{b}, \mathtt{c}\}.$$

1. $r_1 := (\mathtt{a} + \mathtt{b} + \mathtt{c}) \cdot (\mathtt{a} + \mathtt{b} + \mathtt{c})$

   This expression $r_1$ denotes the set of all strings from $\Sigma^*$ that have length 2:

   $$L(r_1) = \big\{w \in \Sigma^* \;\big|\; |w| = 2\big\}.$$

2. $r_2 := (\mathtt{a} + \mathtt{b} + \mathtt{c}) \cdot (\mathtt{a} + \mathtt{b} + \mathtt{c})^*$

   This expression $r_2$ denotes the set of all strings from $\Sigma^*$ that have at least length 1:

   $$L(r_2) = \big\{w \in \Sigma^* \;\big|\; |w| \geq 1\big\}.$$

3. $r_3 := (\mathtt{b} + \mathtt{c})^* \cdot \mathtt{a} \cdot (\mathtt{b} + \mathtt{c})^*$

   This expression $r_3$ denotes the set of all strings from $\Sigma^*$ that have exactly one occurrence of the letter a:

   $$L(r_3) = \Big\{w \in \Sigma^* \;\Big|\; \#\big\{i \in \mathbb{N} \;\big|\; w[i] = \mathtt{a}\big\} = 1\Big\}.$$

4. $r_4 := (\mathtt{b} + \mathtt{c})^* \cdot \mathtt{a} \cdot (\mathtt{b} + \mathtt{c})^* + (\mathtt{a} + \mathtt{c})^* \cdot \mathtt{b} \cdot (\mathtt{a} + \mathtt{c})^*$

   This expression $r_4$ denotes the set of all strings from $\Sigma^*$ that either contain exactly one occurrence of the letter a or exactly one occurrence of the letter b:

   $$L(r_4) = \Big\{w \in \Sigma^* \;\Big|\; \#\big\{i \in \mathbb{N} \;\big|\; w[i] = \mathtt{a}\big\} = 1\Big\} \cup \Big\{w \in \Sigma^* \;\Big|\; \#\big\{i \in \mathbb{N} \;\big|\; w[i] = \mathtt{b}\big\} = 1\Big\}. \quad \diamond$$

**Remark**: The syntax of regular expressions given here is the same as the syntax used in [HMU06]. However, the syntax used for regular expressions in programming languages like *Python* differs. These differences will be discussed later. $\diamond$

**Exercise 1**:

(a) Assume $\Sigma = \{\mathtt{a}, \mathtt{b}, \mathtt{c}\}$. Define a regular expression for the language $L \subseteq \Sigma^*$ that consists of those strings that contain at least one occurrence of the letter "a" and at least one occurrence of the letter "b".

(b) Assume $\Sigma = \{\mathtt{a}, \mathtt{b}\}$. Give a concise mathematical description for the language generated by the regular expression

$$\mathtt{b}^* \cdot (\mathtt{a} \cdot \mathtt{b}^* \cdot \mathtt{a} \cdot \mathtt{b}^*)^*.$$

**Hint**: Use the function $\texttt{count}(s, c)$ that counts the number of occurrences of the character $c$ in the string $s$.

(c) Assume $\Sigma = \{0, 1\}$. Specify a regular expression for the language $L \subseteq \Sigma^*$ that consists of those strings $s$ such that the antepenultimate character is the symbol "1".

(d) Again, assume $\Sigma = \{0, 1\}$. Define a regular expression for the language $L \subseteq \Sigma^*$ containing all those strings that do not contain the substring $110$.

(e) Again, assume $\Sigma = \{0, 1\}$. What is the language $L$ generated by the regular expression

$$(1 + \varepsilon) \cdot (0 \cdot 0^* \cdot 1)^* \cdot 0^*?$$ ◇

## 2.3 Algebraic Simplification of Regular Expressions

Given two regular expressions $r_1$ and $r_2$, we use the notation

$$r_1 \doteq r_2 \quad \text{iff} \quad L(r_1) = L(r_2),$$

to indicate that $r_1$ and $r_2$ describe the same language. If the equation $r_1 \doteq r_2$ holds, then we refer to $r_1$ and $r_2$ as equivalent. The following algebraic laws hold:

(a) $r_1 + r_2 \doteq r_2 + r_1$

This equation is true because the union operator is commutative for sets:

$$L(r_1 + r_2) = L(r_1) \cup L(r_2) = L(r_2) \cup L(r_1) = L(r_2 + r_1).$$

(b) $(r_1 + r_2) + r_3 \doteq r_1 + (r_2 + r_3)$

This equation is true because the union operator is associative for sets.

(c) $(r_1 \cdot r_2) \cdot r_3 \doteq r_1 \cdot (r_2 \cdot r_3)$

This equation is true because the concatenation of strings is associative, for any strings $u$, $v$, and $w$ we have

$$(uv)w = u(vw).$$

This implies

$$\begin{aligned} L\big((r_1 \cdot r_2) \cdot r_3\big) &= \big\{xw \;\big|\; x \in L(r_1 \cdot r_2) \wedge w \in L(r_3)\big\} \\ &= \big\{(uv)w \;\big|\; u \in L(r_1) \wedge v \in L(r_2) \wedge w \in L(r_3)\big\} \\ &= \big\{u(vw) \;\big|\; u \in L(r_1) \wedge v \in L(r_2) \wedge w \in L(r_3)\big\} \\ &= \big\{uy \;\big|\; u \in L(r_1) \wedge y \in L(r_2 \cdot r_3)\big\} \\ &= L\big(r_1 \cdot (r_2 \cdot r_3)\big). \end{aligned}$$

The following equations are more or less obvious.

(d) $\emptyset \cdot r \doteq r \cdot \emptyset \doteq \emptyset$

(e) $\varepsilon \cdot r \doteq r \cdot \varepsilon \doteq r$

(f) $\emptyset + r \doteq r + \emptyset \doteq r$

(g) $(r_1 + r_2) \cdot r_3 \doteq r_1 \cdot r_3 + r_2 \cdot r_3$

(h) $r_1 \cdot (r_2 + r_3) \doteq r_1 \cdot r_2 + r_1 \cdot r_3$

(i) $r + r \doteq r$, because

$$L(r + r) = L(r) \cup L(r) = L(r).$$

(j) $(r^*)^* \doteq r^*$

We have

$$L(r^*) = \bigcup_{n \in \mathbb{N}} L(r)^n$$

and that implies $L(r) \subseteq L(r^*)$. This holds true if we replace $r$ by $r^*$. Therefore

$$L(r^*) \subseteq L\big((r^*)^*\big)$$

holds. In order to prove the inclusion

$$L\big((r^*)^*\big) \subseteq L(r^*),$$

we consider the structure of the strings $w \in L\big((r^*)^*\big)$. Because of

$$L\big((r^*)^*\big) = \bigcup_{n \in \mathbb{N}} L(r^*)^n$$

we have $w \in L\big((r^*)^*\big)$ if and only if there is an $n \in \mathbb{N}$ such that there are strings $u_1, \cdots, u_n \in L(r^*)$ satisfying

$$w = u_1 \cdots u_n.$$

Because of $u_i \in L(r^*)$ we find a number $m(i) \in \mathbb{N}$ for every $i \in \{1, \cdots, n\}$ such that for $j = 1, \cdots, m(i)$ there are strings $v_{j,i} \in L(r)$ satisfying

$$u_i = v_{1,i} \cdots v_{m(i),i}.$$

Combining these equations yields

$$w = v_{1,1} \cdots v_{m(1),1} v_{1,2} \cdots v_{m(2),2} \cdots v_{1,n} \cdots v_{m(n),n}.$$

Hence $w$ is a concatenation of strings from the language $L(r)$ and hence we have

$$w \in L\big(r^*\big).$$

This shows the inclusion $L\big((r^*)^*\big) \subseteq L(r^*)$.

(k) $\emptyset^* \doteq \varepsilon$

(l) $\varepsilon^* \doteq \varepsilon$

(m) $\varepsilon + r^* \cdot r \doteq r^*$

(n) $(\varepsilon + r)^* \doteq r^*$

Used as rewrite rules, these laws can simplify regular expressions. Most of them are read from left to right for this purpose. However, the laws of distributivity (g) and (h) have to be read from right to left: they are used to factor out common parts, e.g. $\mathtt{a} \cdot \mathtt{b} + \mathtt{a} \cdot \mathtt{c}$ is replaced by $\mathtt{a} \cdot (\mathtt{b} + \mathtt{c})$. The *Jupyter* notebook

Python/Chapter-02/Regular-Expressions.ipynb

contains a parser for the regular expressions defined in this chapter, a function `simplify_regexp` that simplifies regular expressions using the laws given above, and an interpreter for regular expressions: the function `matches` checks whether a string is an element of the language $L(r)$ of a regular expression $r$.

## 2.4 Check your Understanding

(a) What is the definition of $L^*$ for a given formal language $L$?

(b) How is the set $\mathsf{RegExp}_\Sigma$ formally defined?

(c) How is the function $L : \mathsf{RegExp}_\Sigma \to 2^{\Sigma^*}$ formally defined?

(d) For $r_1, r_2 \in \mathsf{RegExp}_\Sigma$, how is the equivalence $r_1 \doteq r_2$ defined?

(e) Simplify the following regular expressions:

i. $\emptyset^*$

ii. $\varepsilon^*$

iii. $\varepsilon + r^* \cdot r$

iv. $(\varepsilon + r)^*$

# Chapter 3

# Building Scanners with Regular Expressions

Figure 3.1: The scanner component of a medieval compiler.

## 3.1 Overview

In the previous chapter we have investigated regular expressions from a theoretical point of view. This chapter is devoted to their most important practical application: the construction of scanners. A scanner is the program that splits a structured text into the smallest units that carry a meaning of their own. These units are called tokens. Every compiler starts by running a scanner, but scanners are also useful in many situations that have nothing to do with compilers, and we will see several examples of this kind.

It is worth pointing out that we do *not* use any scanner generator in this chapter. Traditionally, scanners have been produced by tools like `Lex` or `Ply` , which read a scanner specification and

generate the corresponding program. As *Python* already provides regular expressions in its module `re`, which is part of the standard library, we can do without such a tool: everything that is needed in order to build a scanner is contained in the module `re`. This keeps the number of tools that have to be learned small and it makes the inner workings of a scanner much more visible.

As this chapter is rather long, we start with a road map.

1. Section 3.2, *Introduction*, discusses what a scanner is supposed to do. It presents the kinds of tokens that occur in a typical programming language and shows how a small C program is split into its tokens.

2. Section 3.3, *Building a Scanner with Regular Expressions*, develops the recipe that is used throughout this chapter: every token is described by a regular expression, all of these regular expressions are combined into a single master pattern built from named groups, and this pattern is then used in order to scan the input from left to right. The running example of this section is a scanner for arithmetic expressions.

3. Section 3.4, *The Syntax of Regular Expressions in Python*, discusses the syntax that is supported by the module `re` in a systematic way. Among other things, it covers meta-characters, quantifiers and their non-greedy variants, character classes, groups and back-references, the dot, named groups, the anchors that match the start and the end of a line, and lookahead assertions. The section concludes with some examples that analyze the book *Alice's Adventures in Wonderland* and with Exercise 2, which converts dollar amounts into euros.

4. Section 3.5, *A Complex Example: Evaluating an Exam*, presents a program that I use in order to evaluate the exams that you have to write. This program reads a file that contains the number of points that the students have achieved, computes the corresponding marks, and prints the mark of every student. Along the way, this section introduces the functions `re.search` and `re.compile` and discusses match objects in more detail. It shows that regular expressions are useful for much more than just building compilers.

5. Section 3.6, *Converting* HTML *to Text*, solves the problem of extracting the text that is embedded in an HTML document. Interestingly, this problem is *not* solved by building a scanner. Instead, the input is pushed through a pipeline of substitutions, which is built with the function `re.sub`. This section also shows the limits of our approach: we will see in Chapter 5 why no regular expression is able to describe the syntax of HTML.

6. Exercise 3, which concludes that section, asks you to translate a small subset of LaTeX into MATHML. In order to solve this exercise, two techniques are needed that go beyond a plain token specification: the scanner has to switch between different scanner states, and it has to maintain a stack in order to keep track of the delimiters that are currently open.

All the programs that are discussed in this chapter are available as *Jupyter* notebooks. As programming is not a spectator sport, you are strongly encouraged to run these notebooks on your own and to experiment with them while you read.

## 3.2 Introduction

A scanner is a program designed to segment a given string into a sequence of *tokens*, where a token is a contiguous string of characters that logically group together. To illustrate, consider the input for a C-compiler, which is an ASCII-string representing a valid C program. The compiler initially organizes these characters into various *tokens*, including:

1. Keywords or reserved words, such as, for example, “`if`”, “`while`”, “`for`”, and “`switch`”.
2. Operator symbols, for example “+”, “+=”, “<”, and “<=”.
3. Parentheses, which include “`(`”, “`[`”, “`{`”, and the corresponding closing symbols.
4. Constants, with C recognizing three types:
   (a) Numerical constants, such as the integer “`42`” or the floating-point number “`1.23e2`”.
   (b) String constants enclosed in double quotes, like “`"hello"`”.
   Note that the double quote character “`"`” is part of the string constant.
   (c) Single-character constants wrapped in single quotes, as in “`'a'`”.
   Note that the single quote character “`'`” is part of the character constant.
5. Identifiers, which may serve as variable names, function names, or type names.
6. Comments, available in two forms: Single-line comments start with “`//`” and extend to the end of the line, while multi-line comments begin with “`/*`” and end with “`*/`”.
7. Whitespace characters, such as the blank character `' '`, the tab character `'\t'`, the newline character `'\n'`, and the carriage return character `'\r'`.
   Both whitespace characters and comments are generally discarded by the scanner.

```c
/* Hello World program */
#include <stdio.h>

int main() {
    printf("Hello World!\n");
    return 0;
}
```

Figure 3.2: A straightforward C program.

For a more tangible example, consider Figure 3.2 on page 23, which presents a C program that outputs the string “`Hello World!`” along with a newline character. The scanner processes this program into the following list of tokens:

[“#”, “include”, “<”, “stdio.h”, “>”, “int”, “main”, “(”, “)”, “{”,
“printf”, “(”, “"Hello World!\n"”, “)”, “;”, “return”, “0”, “;”, “}”]

Note that the scanner discards all white space characters, as these are only used to demarcate tokens. The exception to this are those white space characters that occur inside string constants. Furthermore, the comment has been removed by the scanner.

While it is certainly possible to implement a scanner by inspecting the input character by character, doing so by hand is tedious and error-prone. *Python*’s `re` module makes this task considerably easier: We specify every kind of token as a regular expression and let `re` do the work of matching. We develop this approach in the following section.

## 3.3 Building a Scanner with Regular Expressions

In order to build a scanner using regular expressions, we proceed in three steps that mirror the three parts of any scanner specification:

1. We decide on the names of all token types that our scanner is supposed to recognize.
2. For every token name, we specify a regular expression that describes the set of strings that are to be recognized as tokens of this type.
3. We combine all of these regular expressions into a single master pattern and use this pattern to scan the input string from left to right.

To illustrate this approach, we develop a scanner that tokenizes simple arithmetic expressions. Figure 3.3 on page 24 and Figure 3.4 on page 26 show the complete implementation, which we discuss line by line below.

```
1  import re
2  import collections
3  from collections.abc import Iterator
4
5  token_specification = [
6      ('NUMBER',   r'0|[1-9][0-9]*'),
7      ('PLUS',     r'\+'),
8      ('MINUS',    r'-'),
9      ('TIMES',    r'\*'),
10     ('DIVIDE',   r'/'),
11     ('LPAREN',   r'\('),
12     ('RPAREN',   r'\)'),
13     ('NEWLINE',  r'\n'),
14     ('SKIP',     r'[ \t]+'),
15     ('MISMATCH', r'.'),
16 ]
17 master_pattern = \
18   '|'.join(f'(?P<{name}>{regexp})'
19            for name, regexp in token_specification)
20
21 Token = collections.namedtuple(
22            'Token', ['type', 'value', 'line', 'column'])
```

Figure 3.3: A scanner for simple arithmetic expressions, implemented with regular expressions.

1. Line 1 imports the module `re`, while line 2 imports the module `collections`, which we need in order to define the class of our tokens. Line 3 imports the type `Iterator`, which is used in the type annotation of the function `tokenize`.
2. Lines 5–16 contain the token specification. Syntactically, this is a list of pairs. The first component of every pair is the name of a token and, by convention, this name is written using capital letters. The second component is a regular expression using the syntax that is specified in the *Python*

`re` module. This regular expression describes the set of strings that are recognized as tokens of the given name.

Three of the names occurring in the token specification, namely `NEWLINE`, `SKIP`, and `MISMATCH`, do not name genuine tokens. Instead, they serve a bookkeeping purpose:

(a) `NEWLINE` matches a single newline character. Whenever this token is recognized, we have to increment our line counter.

(b) `SKIP` matches blanks and tabs. These characters are only used to separate tokens and are therefore discarded.

(c) `MISMATCH` matches any single character that has not already been matched by one of the preceding regular expressions. If this token is recognized, an unexpected character has been found in the input and the scanner has to report an error.

3. Line 17 combines the individual regular expressions into the master pattern. Every regular expression is wrapped inside a named group, which is written as

   `(?P<`*NAME*`>`*regexp*`)`.

   These named groups are then joined using the alternative operator "`|`". Later, once a match has been found, we can ask which of the named groups is responsible for the match.

   **Remark**: Unlike some scanner generators, which choose the *longest* matching alternative irrespective of the order in which the alternatives are written down, *Python*'s `re` module chooses the *first* alternative—reading from left to right—that matches at the current position. Therefore, the order of the pairs in the token specification is significant: More specific patterns have to be listed before more general ones. For instance, if our language had keywords, the regular expressions for these keywords would have to precede the regular expression used for identifiers, since otherwise every keyword would incorrectly be recognized as an identifier. ◇

4. Lines 21–22 define the class `Token` using the function `collections.namedtuple`. Every object of this class has four attributes, namely `type`, `value`, `line`, and `column`, which we discuss in more detail below.

5. Lines 23–40 define the function `tokenize`, which is a generator function: Instead of returning a single value, it `yields` a sequence of `Token` objects, one for every token found in its argument `code`. Accordingly, the type annotation of `tokenize` states that it takes a string and returns an `Iterator` of `Token` objects.

   (a) Lines 24–25 initialize the variables `ln` (short for *line number*) and `line_start`. The variable
   `ln` counts the lines of the input, starting with the value 1. The variable `line_start` stores the position at which the current line begins; it is needed to compute the column of a token.

   (b) Line 26 uses the function `finditer` from the module `re` to scan the input string `code` for non-overlapping matches of the `master_pattern`. This function returns an iterator of match objects, one for every token that is found, in the order in which these tokens occur in the input.

   (c) Line 27 uses the attribute `lastgroup` of the match object `mo` to determine the name of the named group that is responsible for the match, i.e. the name of the token that has been found.

   (d) Line 28 uses the method `mo.group()` to extract the string that has been matched.

```
23 def tokenize(code: str) -> Iterator[Token]:
24     ln         = 1    # number of the current line
25     line_start = 0    # position where the current line starts
26     for mo in re.finditer(master_pattern, code):
27         kind  = mo.lastgroup
28         value = mo.group()
29         col   = mo.start() - line_start  # column of the token
30         if kind == 'NUMBER':
31             value = int(value)
32         elif kind == 'NEWLINE':
33             line_start   = mo.end()
34             ln += 1
35             continue
36         elif kind == 'SKIP':
37             continue
38         elif kind == 'MISMATCH':
39             raise RuntimeError(f"{value!r}: line {ln}, col {col}.")
40         yield Token(kind, value, ln, col)
41
42 data = '3 + 4 * 10 + 007 + (-20) * 2'
43 for tok in tokenize(data):
44     print(tok)
```

Figure 3.4: A scanner for simple arithmetic expressions, part 2.

(e) Line 29 computes the column at which the token starts by subtracting `line_start` from `mo.start()`, the position at which the match begins.

(f) Lines 30–31 deal with the token NUMBER: the matched string is converted into an `int`.

(g) Lines 32–35 deal with the token NEWLINE: the variable `line_start` is updated to the position right after the newline character, the line counter is incremented, and the loop continues with the `continue` statement—no token is generated.

(h) Lines 36–37 deal with the token SKIP: nothing needs to be done except to continue with the next match.

(i) Lines 38–39 deal with the token MISMATCH: an exception is raised that reports the illegal character together with the line and column at which it was found.

(j) Line 40 is only reached for genuine tokens. It constructs a `Token` object and `yields` it to the caller.

6. Line 42 defines the string `data` that we want to tokenize.

7. Lines 43–44 show how the scanner is used: We simply iterate over the values produced by `tokenize(data)` and print every token that is found.

Executing the program depicted in Figures 3.3 and 3.4 yields the following output:

```
Token(type='NUMBER', value=3, line=1, column=0)
```

```
Token(type='PLUS', value='+', line=1, column=2)
Token(type='NUMBER', value=4, line=1, column=4)
...
Token(type='NUMBER', value=2, line=1, column=27)
```

The scanner returns tokens as instances of the class `Token`, which possess four distinct attributes:

1. The `type` attribute indicates the type of the token. It holds a string value that corresponds to one of the names occurring in the token specification.
2. The `value` attribute usually contains the recognized string. However, as shown in the case of the token NUMBER, this value can be transformed before the token is constructed.
3. The `line` attribute specifies the line number at which the token was found.
4. The `column` attribute specifies the column at which the token starts. Note that, unlike a counter that simply counts characters from the very beginning of the input, this column is reset to $0$ at the start of every line, which matches the way most editors report cursor positions.

The scanner discussed in this section can be found at:

Python/Chapter-03/01-RegExp-Scanner.ipynb

## 3.4 The Syntax of Regular Expressions in *Python*

In the preceding chapter, we introduced regular expressions using a minimalistic syntax. While this simplicity is advantageous for the theoretical discussions in the upcoming chapter—where we explore the implementation of regular expressions via finite state machines—it is limiting in practical applications. To address this, the *Python* module `re` offers a number of syntactic abbreviations that allow us to express complex patterns concisely. In this section, we discuss those features of the module `re` that we will need in the remainder of these lecture notes.

The module `re` is not part of the core language, but it is part of the *Python* standard library. Therefore, no installation is necessary, but in order to use it we have to import it:

```
import re
```

The complete documentation of this module is available at https://docs.python.org/3/library/re.html.

As in the previous chapter, we interpret a regular expression as a description of a language, where a language is a set of strings. In the following, $\Sigma$ denotes the set of all UNICODE characters, while $\Sigma^*$ denotes the set of all strings that can be built from these characters. We define the set $\mathrm{RegExp}$ of all *Python* regular expressions inductively, and in parallel we define a function

$$\mathcal{L} : \mathrm{RegExp} \to 2^{\Sigma^*}$$

such that $\mathcal{L}(r)$ is the language described by the regular expression $r$.

### 3.4.1 The Function `findall`

In order to experiment with regular expressions, we use the function `findall` that is provided by the module `re`. This function is called as follows:

$$\texttt{re.findall}(r, s, \mathrm{flags} = 0)$$

The arguments are interpreted as follows:

1. $r$ is a string that is interpreted as a regular expression.
2. $s$ is the string that is searched for substrings matching $r$.
3. $\mathrm{flags}$ is an optional argument of type `int` that defaults to $0$.

   This argument can be used to set flags that modify the interpretation of the regular expression $r$. For example, setting the flag `re.IGNORECASE` makes the search case-insensitive.

The function `findall` returns the list of all *non-overlapping* substrings of $s$ that match the regular expression $r$. These substrings are returned in the order in which they occur in $s$. For example, the call

```
re.findall(r'a', 'abcabcABC')
```

returns the list `['a', 'a']`, since the character "a" occurs twice in the given string. If we set the flag `re.IGNORECASE`, the capital letter "A" is found as well and hence the call

```
re.findall(r'a', 'abcabcABC', re.IGNORECASE)
```

returns the list `['a', 'a', 'A']` that has three elements.

### 3.4.2 Characters and Meta-Characters

Before we can define the set $\mathrm{RegExp}$, we have to introduce the set `MetaChars` of all meta-characters:

$$\texttt{MetaChars} := \{\ \texttt{.}, \texttt{\^{}}, \texttt{\$}, \texttt{*}, \texttt{+}, \texttt{?}, \texttt{\{}, \texttt{\}}, \texttt{[}, \texttt{]}, \texttt{\textbackslash}, \texttt{|}, \texttt{(}, \texttt{)}\ \}.$$

These characters serve as operator symbols, or as parts of operator symbols, inside a regular expression. Now the inductive definition of the set $\mathrm{RegExp}$ can be started:

1. Every Unicode character $c$ that is *not* a meta-character is a regular expression. This regular expression matches the character $c$, i.e. we have

   $$\mathcal{L}(c) = \{c\}.$$

2. If $c$ is a meta-character, then the string $\backslash c$ is a regular expression that matches the character $c$, i.e. we have

   $$\mathcal{L}(\backslash c) = \{c\}.$$

   In this case, we say that the meta-character $c$ has been escaped.

For example, the character "+" is a meta-character and therefore has to be escaped if we want to search for a literal plus sign:

```
re.findall(r'\+', '+-+')
```

This call returns the list `['+', '+']`.

**Remark**: In the example above we have used a raw string. A string is a raw string if the opening quote character is preceded by the character "`r`". Inside a raw string, the backslash character is not interpreted as the beginning of an escape sequence of *Python* but is passed on to the module `re` unchanged. Since regular expressions use the backslash character extensively, it is good practice to always write regular expressions as raw strings.

If we do not want to use a raw string, we have to double every backslash character, because otherwise the *Python* interpreter would try to interpret the backslash itself. Hence, the call shown above is equivalent to the following call:

```
re.findall('\\+', '+-+')
```

Obviously, the version using the raw string is easier to read. ◇

### 3.4.3 Concatenation

The next rule shows how regular expressions can be concatenated:

3. If $r_1$ and $r_2$ are regular expressions, then $r_1 r_2$ is a regular expression. This regular expression matches any string $s$ that can be split into two substrings $s_1$ and $s_2$ such that $r_1$ matches $s_1$ and $r_2$ matches $s_2$. Formally, we have

$$\mathcal{L}(r_1 r_2) := \big\{ s_1 \cdot s_2 \mid s_1 \in \mathcal{L}(r_1) \wedge s_2 \in \mathcal{L}(r_2) \big\}.$$

Note that in the preceding chapter we have written $r_1 \cdot r_2$ for the concatenation of $r_1$ and $r_2$, while in *Python* the operator symbol "$\cdot$" is simply omitted. Using concatenation, we can search for words:

```
re.findall(r'the', 'The horse, the dog, and the cat.',
           flags=re.IGNORECASE)
```

This call returns the list `['The', 'the', 'the']`.

### 3.4.4 Choice

The operator "|" is used to choose between alternatives:

4. If $r_1$ and $r_2$ are regular expressions, then $r_1 \,|\, r_2$ is a regular expression. This regular expression matches every string $s$ that is matched by either $r_1$ or $r_2$. Formally, we have

$$\mathcal{L}(r_1 \,|\, r_2) := \mathcal{L}(r_1) \cup \mathcal{L}(r_2).$$

In the preceding chapter, we have written $r_1 + r_2$ instead of the *Python* notation $r_1 \,|\, r_2$. Here is an example:

```
re.findall(r'The|a', 'The horse, the dog, and a cat.',
           flags=re.IGNORECASE)
```

This call returns the list `['The', 'the', 'a', 'a', 'a']`. Note that the last three occurrences of the character "a" stem from the words "and", "a", and "cat".

### 3.4.5 Quantifiers

The most interesting operators of regular expressions are the quantifiers. The official documentation calls them repetition qualifiers, but in the following we use the shorter name. Syntactically, quantifiers are postfix operators.

5. If $r$ is a regular expression, then $r$+ is a regular expression. This regular expression matches every string $s$ that can be split into a list of $n$ substrings $s_1, \cdots, s_n$ such that $r$ matches $s_i$ for all $i \in \{1, \cdots, n\}$. Formally, we have

$$\mathcal{L}(r\texttt{+}) := \Big\{ s \;\Big|\; \exists n \in \mathbb{N} : \big(n \geq 1 \wedge \exists s_1, \cdots, s_n : (s_1 \cdots s_n = s \wedge \forall i \in \{1, \cdots, n\} : s_i \in \mathcal{L}(r))\big)\Big\}.$$

   Informally, $r$+ matches $r$ any positive number of times.

```
re.findall(r'a+', 'abaabaAaba', flags=re.IGNORECASE)
```

This call returns the list `['a', 'aa', 'aAa', 'a']`.

6. If $r$ is a regular expression, then $r$* is a regular expression. This regular expression matches either the empty string or any string that is matched by $r$+. Formally, we have

$$\mathcal{L}(r\texttt{*}) := \{\text{''}\} \cup \mathcal{L}(r\texttt{+}).$$

   Informally, $r$* matches $r$ any number of times, including zero times.

Since $r$* also matches the empty string, the result of the following call contains a number of empty strings. For example, in the string `'abaabbaaaba'` the regular expression `a*` finds an empty string in front of every occurrence of the character `'b'`: at these positions, no `'a'` can be matched and therefore `a*` matches the empty string. The final occurrence of the empty string is found at the end of the string.

```
re.findall(r'a*', 'abaabbaaaba')
```

This call returns the list

```
['a', '', 'aa', '', '', 'aaa', '', 'a', '']
```

7. If $r$ is a regular expression, then $r$? is a regular expression. This regular expression matches either the empty string or any string that is matched by $r$. Formally, we have

$$\mathcal{L}(r\texttt{?}) := \{\text{''}\} \cup \mathcal{L}(r).$$

   Informally, $r$? matches $r$ at most one time, but possibly also zero times.

```
re.findall(r'a?', 'abaa')
```

This call returns the list `['a', '', 'a', 'a', '']`. Here, the first empty string is found in front of the character `'b'`, while the second is found at the end of the string.

8. If $r$ is a regular expression and $m, n \in \mathbb{N}$ such that $m \leq n$, then $r\{m,n\}$ is a regular expression. This regular expression matches any number $k$ of repetitions of $r$ such that $m \leq k \leq n$. Formally, we have

$$\mathcal{L}(r\{m,n\}) = \Big\{ s \;\Big|\; \exists k \in \mathbb{N} : \big(m \leq k \leq n \wedge \exists s_1, \cdots, s_k : (s_1 \cdots s_k = s \wedge \forall i \in \{1, \cdots, k\} : s_i \in \mathcal{L}(r))\big)\Big\}.$$

   Informally, $r\{m,n\}$ matches $r$ at least $m$ times and at most $n$ times.

```
re.findall(r'a{2,3}', 'aaaa')
```

This call returns the list `['aaa']`. The regular expression `a{2,3}` matches the string `'aaaa'` only once, since the first match already consumes three occurrences of the character `'a'` and after that only a single `'a'` is left, which is not enough for a second match.

The following three abbreviations are frequently useful:

9. If $r$ is a regular expression and $n \in \mathbb{N}$, then $r\{n\}$ is a regular expression that matches exactly $n$ repetitions of $r$:

$$\mathcal{L}(r\{n\}) = \mathcal{L}(r\{n,n\}).$$

10. If $r$ is a regular expression and $n \in \mathbb{N}$, then $r\{,n\}$ is a regular expression that matches up to $n$ repetitions of $r$:

$$\mathcal{L}(r\{,n\}) = \mathcal{L}(r\{0,n\}).$$

11. If $r$ is a regular expression and $n \in \mathbb{N}$, then $r\{n,\}$ is a regular expression that matches $n$ or more repetitions of $r$:

$$\mathcal{L}(r\{n,\}) = \mathcal{L}(r\{n\}r^*).$$

The following three calls illustrate these abbreviations:

```
re.findall(r'a{2}',  'aabaaaba')
re.findall(r'a{,2}', 'aabaaabba')
re.findall(r'a{2,}', 'aabaaaba')
```

These calls return the lists

1. `['aa', 'aa']`,
2. `['aa', '', 'aa', 'a', '', '', 'a', '']`, and
3. `['aa', 'aaa']`.

**Remark**: The quantifiers bind stronger than concatenation, and concatenation binds stronger than the choice operator "|". Hence, the regular expression `ab+` matches the character `a` followed by any positive number of occurrences of the character `b`, and the regular expression `ab|cd` matches either the string `'ab'` or the string `'cd'`. If a different grouping is required, parentheses have to be used, as discussed in Subsection 3.4.8 below. ◇

### 3.4.6 Non-Greedy Quantifiers

The quantifiers ?, +, *, {$m$,$n$}, {$n$}, {,$n$}, and {$n$,} are greedy, i.e. they match the longest possible substring. Suffixing these operators with the character "?" makes them non-greedy, i.e. they then match the shortest possible substring. For example, the regular expression a{2,3}? matches either two or three occurrences of the character a, but it prefers to match only two characters. Therefore, the regular expression a{2,3}? finds two matches in the string 'aaaa', while the greedy version a{2,3} finds only a single match:

```
re.findall(r'a{2,3}?', 'aaaa'), re.findall(r'a{2,3}', 'aaaa')
```

This call returns the pair

```
(['aa', 'aa'], ['aaa']).
```

### 3.4.7 Character Classes

In order to match one out of a set of characters, we use a character class. If $c_1, \cdots, c_n$ are *Unicode* characters, then [$c_1 \cdots c_n$] is a regular expression that matches any of the characters from the set $\{c_1, \cdots, c_n\}$:

$$\mathcal{L}\big([c_1 \cdots c_n]\big) := \{c_1, \cdots, c_n\}.$$

Note that inside a character class, most meta-characters lose their special meaning and hence do not need to be escaped.

```
re.findall(r'[abc]+', 'abcdcba')
```

This call returns the list ['abc', 'cba'].

A character class can also contain ranges. Syntactically, a range has the form $c_1$-$c_2$, where $c_1$ and $c_2$ are Unicode characters. Such a range matches every character $c$ satisfying $c_1 \leq c \leq c_2$, where the characters are compared according to their Unicode code points. For example, the regular expression [0-9] contains the range 0-9 and therefore matches any decimal digit. In order to find all natural numbers that are embedded in a string, we can use the regular expression [1-9][0-9]*|0. This regular expression matches either a string that starts with a non-zero digit and is followed by any number of digits, or a single zero.

```
re.findall(r'[1-9][0-9]*|0', '11 abc 12 2345 007 42 0')
```

This call returns the list

```
['11', '12', '2345', '0', '0', '7', '42', '0'].
```

Note that the string '007' is decomposed into the three matches '0', '0', and '7', since a number is not allowed to start with the digit 0.

At first sight, the following call looks quite similar, but it produces a very different result:

```
re.findall(r'[0-9]|[1-9][0-9]*', '11 abc 12 2345 007 42 0')
```

Here, the result is the list

```
['1', '1', '1', '2', '2', '3', '4', '5', '0', '0', '7', '4', '2', '0'],
```

which consists of single digits only. The reason is the same as the reason discussed in the remark on page 24: the module `re` does not choose the *longest* alternative but rather the *first* alternative—reading from left to right—that matches at the current position. Since the alternative `[0-9]` comes first and already matches a single digit, the search restarts immediately after this digit and the second alternative never gets a chance. If we exchange the two alternatives, we recover the previous result:

```
re.findall(r'[1-9][0-9]*|[0-9]', '11 abc 12 2345 007 42 0')
```

This is exactly the phenomenon that forced us to list the more specific token definitions before the more general ones when we built the master pattern of our scanner in the previous section.

There are a number of predefined character classes:

1. `\d` matches any digit.
2. `\D` matches any character that is not a digit.
3. `\s` matches any whitespace character.
4. `\S` matches any character that is not a whitespace character.
5. `\w` matches any alphanumeric character and the underscore "_".

   If we were to restrict ourselves to Ascii characters, this would be equivalent to the character class `[0-9a-zA-Z_]`.
6. `\W` matches any character that is not alphanumeric.
7. `\b` matches at a word boundary. The string that is matched is the empty string. Strictly speaking, `\b` is therefore not a character class but an assertion: it does not match a character but only checks a condition at the current position.
8. `\B` matches at any position that is **not** a word boundary. Again, the string that is matched is the empty string.

With the exception of `\b` and `\B`, these escape sequences can also be used inside square brackets:

```
re.findall(r'[\dabcde]+', '11 abc12 1a2 2b3c4d5')
```

This call returns the list `['11', 'abc12', '1a2', '2b3c4d5']`.

A character class is negated if the first character following the opening bracket "`[`" is the character "`^`". For example, the regular expression `[^abc]` matches any character that is different from a, b, and c:

```
re.findall(r'[^abc]+', 'axyzbuvwchij')
```

This call returns the list `['xyz', 'uvw', 'hij']`.

The assertion `\b` is useful in order to extract complete words. For example, the regular expression `\b\w+\b` matches a maximal sequence of alphanumeric characters that is surrounded by word boundaries:

```
re.findall(r'\b\w+\b',
           'This is some text where we want to extract the words.')
```

This call returns the list

```
['This', 'is', 'some', 'text', 'where', 'we', 'want', 'to',
 'extract', 'the', 'words'].
```

### 3.4.8 Grouping and Back-References

If $r$ is a regular expression, then `(`$r$`)` is a regular expression that describes the same language as $r$, i.e. we have

$$\mathcal{L}\big(\texttt{(}r\texttt{)}\big) = \mathcal{L}(r).$$

There are two reasons for using parentheses:

1. Parentheses can be used to override the precedence of the operators. This is the same concept as in programming languages. For example, the regular expression `ab+` matches the character a followed by any positive number of occurrences of the character b, since the precedence of a quantifier is higher than the precedence of concatenation. In contrast, the regular expression `(ab)+` matches the strings `'ab'`, `'abab'`, `'ababab'`, and so on.

2. Parentheses can be used for back-references: Inside a regular expression, we can refer to the substring that has been matched by a subexpression enclosed in parentheses using the syntax $\backslash n$, where $n \in \{1, \cdots, 99\}$. Here, $\backslash n$ refers to the $n^{\text{th}}$ group of the regular expression, where a group is any part of the regular expression that is enclosed in parentheses. Counting of the groups starts with the opening parentheses, i.e. the groups are numbered in the order in which their opening parentheses occur. For example, the regular expression

   ```
   (a(b|c)*d)?ef(gh)+
   ```

   has three groups:

   (a) `(a(b|c)*d)` is the first group,
   (b) `(b|c)` is the second group, and
   (c) `(gh)` is the third group.

For example, if we want to recognize a number that is followed by some whitespace and then followed by the **same** number, we can use the regular expression `(\d+)\s+\1`:

```
re.findall(r'(\d+)\s+\1', '12 12 12, 23 23 17 18')
```

This call returns the list `['12', '23']`.

**Remark**: If a regular expression contains groups, then `findall` does not return the complete matches but rather the substrings that have been matched by the groups. If there is more than one group, a list of tuples is returned. This explains why the call shown above returns the list `['12', '23']` and not the list `['12 12', '23 23']`. ◇

Since groups change the value returned by `findall`, the following call is a convenient way to extract complete numbers: the parentheses are needed in order to override the precedence of the choice operator, while the word boundaries ensure that we do not accept a number that is part of a longer word:

```
re.findall(r'\b([1-9][0-9]*|0)\b', '11 abc 12 2345 007 42 0')
```

This call returns the list `['11', '12', '2345', '42', '0']`. Note that the string `'007'` is no longer matched at all, since the digit `'0'` at the beginning of this string is not followed by a word boundary.

### 3.4.9 The Dot

The regular expression `.` matches any character except the newline character. For example, the regular expression `c.*?t` matches any string that starts with the character `c`, ends with the character `t`, and does not contain any newline character. Since we have used the non-greedy version of the quantifier `*`, the matches are as short as possible:

```
re.findall(r'c.*?t', 'ct cat caat could we look at that!')
```

This call returns the list

`['ct', 'cat', 'caat', 'could we look at']`.

Note that the dot "`.`" has no special meaning if it is used inside a character class. Hence, the regular expression `[.]` matches only the character "`.`" and is therefore equivalent to the regular expression "`\.`".

### 3.4.10 Named Groups

Referring to a group via the syntax $\backslash n$, where $n$ is a natural number, is both cumbersome and error-prone: as soon as a new group is inserted at the beginning of a regular expression, all the numbers change. Instead, we can use named groups. The syntax to define a named group is

`(?P<`*name*`>`*r*`)`,

where *name* is the name of the group and *r* is a regular expression. In order to refer to the string that has been matched by this group, we use the syntax

`(?P=`*name*`)`.

For example, the following call tries to find a string of alphanumeric characters that is enclosed either in single quotes or in double quotes. The character class `[\'"]` matches either a single quote or a double quote. By referring to the group that has been named `quote`, we ensure that an opening single quote is matched by a closing single quote and that an opening double quote is matched by a closing double quote.

```
re.findall(r'((?P<quote>[\'"])\w*(?P=quote))', 'abc "uvw" and \'xyz\'')
```

This call returns the list

`[('"uvw"', '"'), ("'xyz'", "'")]`.

Since the regular expression contains two groups, `findall` returns a list of pairs: the first component

of every pair is the string matched by the outer group, while the second component is the quote character that has been matched by the group named `quote`.

**Remark**: Named groups are the mechanism that made the master pattern of our scanner work: in Section 3.3 we have wrapped the regular expression of every token into a named group, and after a match had been found we used the attribute `lastgroup` of the match object in order to determine the name of the group—and hence the type of the token—that was responsible for the match. ◇

### 3.4.11 The Start and the End of a Line

The regular expression ^ matches at the start of a string. If the flag `re.MULTILINE` is set—which we will usually do when working with this regular expression—then ^ also matches at the beginning of every line, i.e. it matches after every newline character.

Similarly, the regular expression $ matches at the end of a string. If the flag `re.MULTILINE` is set, then $ also matches at the end of every line, i.e. it matches in front of every newline character. Note that both of these regular expressions match the empty string; they only constrain the *position* at which a match may occur.

The program shown in Figure 3.5 on page 36 extracts all non-empty lines from a given string.

```
data = \
'''
This is a text containing five lines, two of which are empty.
This is the second non-empty line,

and this is the third non-empty line.
'''
re.findall(r'^.+$', data, flags=re.MULTILINE)
```

Figure 3.5: Extracting all non-empty lines of a string.

This program returns a list containing the three non-empty lines of the string `data`. The empty lines are not part of the result, because the regular expression ^.+$ requires at least one character between the start and the end of a line. Furthermore, observe that the newline characters themselves are not part of the matches, since the dot does not match a newline character.

### 3.4.12 Lookahead Assertions

Sometimes we have to look ahead in order to decide whether we have found what we are looking for. Consider the task of adding up all those numbers in a text that are followed by a dollar symbol, while all other numbers are to be ignored. In this case, a lookahead assertion comes in handy. The syntax of a lookahead assertion is

$$r_1(?{=}r_2).$$

Here, $r_1$ and $r_2$ are regular expressions and ?= is the lookahead operator. The regular expression $r_1$ describes what we are searching for, while $r_2$ describes the lookahead. Note that the lookahead is **not** part of the match: it is only checked whether an occurrence of $r_1$ is followed by $r_2$, but only the substring matching $r_1$ is returned. Syntactically, the lookahead $r_2$ has to be preceded by the lookahead operator, and both have to be enclosed in parentheses.

The program shown in Figure 3.6 on page 37 sums up all numbers that are followed by a dollar symbol, where the dollar symbol may be preceded by whitespace.

```
text = 'Here is 1$, here are 21 EUR, and there are 42 $.'
L    = re.findall(r'([0-9]+)(?=\s*\$)', text)
print(f'L = {L}')
sum(int(x) for x in L)
```

Figure 3.6: Summing up all numbers that are followed by a dollar symbol.

This program prints the list `L = ['1', '42']` and returns the number $43$. The number $21$ is ignored, since it is not followed by a dollar symbol.

There are also negative lookahead assertions. The syntax is

$$r_1(?!r_2),$$

where `?!` is the negative lookahead operator. This regular expression matches all those occurrences of $r_1$ that are **not** followed by $r_2$. The program shown in Figure 3.7 on page 37 sums up all numbers that are *not* followed by a dollar symbol.

```
text = 'Here is 1$, here are 21 EUR, and there are 42 $.'
L    = re.findall(r'[0-9]+(?![0-9]*\s*\$)', text)
print(f'L = {L}')
sum(int(x) for x in L)
```

Figure 3.7: Summing up all numbers that are not followed by a dollar symbol.

This program prints the list `L = ['21']` and returns the number $21$.

**Remark**: Note that the lookahead expression in Figure 3.7 has to contain the subexpression `[0-9]*`. Without it, the regular expression `[0-9]+` could match the substring `'4'` of the string `'42 $'`, and this substring is indeed not followed by a dollar symbol but rather by the digit `'2'`. In general, negative lookahead is quite tricky and I recommend to use it sparingly. ◇

### 3.4.13 Examples

In order to have some material to play with, let us read the file `alice.txt`, which contains the book *Alice's Adventures in Wonderland* written by Lewis Carroll:

```
with open('alice.txt', 'r') as f:
    text = f.read()
```

The first question that we ask is how many non-empty lines this story has. A line is non-empty if it contains at least one character that is not a whitespace character:

```
len(re.findall(r'^.*\S.*$', text, flags=re.MULTILINE))
```

This call returns the number 2725.

Next, let us check whether this text is suitable for minors. In order to do so, we search for all four-letter words that start with either d, f, or s and that end with k or t:

```
set(re.findall(r'\b[dfs]\w{2}[kt]\b', text, flags=re.IGNORECASE))
```

This call returns a set containing, among others, the words 'duck', 'fact', 'foot', 'salt', and 'suit', but nothing that would be objectionable. Note the use of the quantifier {2} and of the word boundaries \b, which together ensure that the words found have exactly four characters.

Finally, let us count how many words this text contains and how many *different* words are used:

```
L = re.findall(r'\b\w+\b', text.lower())
S = set(L)
print(f'There are {len(L)} words in this book')
print(f'and {len(S)} different words.')
```

This program prints the following line:

```
There are 27344 words in this book and 2579 different words.
```

### 3.4.14 Further Reading

The material presented in this section is also available as a *Jupyter* notebook that can be found at

Python/Chapter-03/02-Regexp-Tutorial.ipynb.

Since regular expressions are best understood by experimentation, you are strongly encouraged to work through this notebook interactively and to modify the examples given there.

Furthermore, the web page regex101.com can be used to test regular expressions for various programming languages. It shows, step by step, how a given regular expression is matched against a given string and is therefore an invaluable tool for debugging complex regular expressions. Finally, the web page https://regexlearn.com/learn/regex101 provides a free interactive tutorial on regular expressions.

**Exercise 2**: Write a program that reads a given text, finds all numbers inside this text that are followed by a "$" symbol, and that converts these numbers into their equivalent value in €. Assume that $1\,\$ = 0.85\,€$ and note that the numbers may contain a decimal point. Your program should replace every occurrence of a dollar amount by the corresponding euro amount, while leaving the remainder of the text unchanged.

**Hint**: Have a look at the function `re.sub`, which is documented at https://docs.python.org/3/library/re.html. This function can take a *function* as its second argument. This function is then called with the match object and has to return the string that is used as the replacement. ◇

## 3.5 A Complex Example: Evaluating an Exam

This section presents a more complex example that shows some of the power of the module `re`. The task at hand is the evaluation of an exam. When I mark an exam, I create a file that has a format similar

to the example shown in Figure 3.8 on page 39.

```
Class: Advanced Witchcraft
Group: TINF22AI1
MaxPoints = 60

Exercise:      1. 2. 3. 4. 5. 6.
Jim Smith:     9 12 10  6  6  0
John Slow:     4  4  2  0  -  -
Susi Sorglos:  9 12 12  9  9  6
1609922:       7  4 12  5  5  3
```

Figure 3.8: The results of an exam.

The structure of this file is as follows:

1. The first line contains the keyword "`Class`", a colon "`:`", and then the name of the lecture.
2. The second line specifies the group that has taken the exam.
3. The third line specifies the number of points that are necessary in order to obtain the best mark.
4. The fourth line is empty.
5. The fifth line numbers the exercises.
6. After that, there is a table. Every row of this table lists the scores achieved by a single student. The row starts with the name of the student, which is followed by a colon and then by the list of the scores achieved for each exercise. If an exercise has not been attempted at all, the corresponding column contains a hyphen "`-`".

   Note that a student need not be identified by a name: students who prefer to stay anonymous are identified by their seven-digit matriculation number instead. The last row of Figure 3.8 shows such a student.

Our goal is to write a program that reads data of this kind and computes the mark of every student. In the following, we assume that the contents of this file are stored in the string `data`. Note that in contrast to the scanner developed in Section 3.3, we do *not* tokenize the whole input into a single stream of tokens. Instead, we process the input line by line and use a different regular expression for every kind of information that we want to extract. For many text-processing tasks, this lightweight approach is both simpler and more robust than writing a full-blown scanner.

### 3.5.1 Computing the Mark

Before we turn to the regular expressions, we implement the function `mark`, which is shown in Figure 3.9 on page 40. This function takes two arguments:

1. `max_points` is the number of points that have to be achieved in order to get the best mark $1.0$.
2. `points` is the number of points that have been achieved by the student whose mark is to be computed.

We assume that the relation between the mark of an exam and the number of points achieved in this exam is essentially linear. Furthermore, a student who has achieved $50\%$ of `max_points` points should get the mark $4.0$, while a student who has achieved $100\%$ of `max_points` points should get the mark $1.0$. Since the straight line through the two points $\left(\frac{1}{2} \cdot \texttt{max_points}, 4.0\right)$ and $\left(\texttt{max_points}, 1.0\right)$ is given by the equation

$$\text{grade} = 7 - 6 \cdot \frac{\texttt{points}}{\texttt{max_points}},$$

we can use this formula to compute the mark. However, the worst mark is $5.0$. Therefore, the function `min` is used to ensure that the computed mark never exceeds $5.0$. Finally, the resulting number is rounded to a single digit after the decimal point.

```
1  import re
2
3  def mark(max_points: int, points: int) -> float:
4      grade = 7 - 6 * points / max_points
5      return round(min(5.0, grade), 1)
```

Figure 3.9: Computing the mark from the number of points achieved.

For example, if `max_points` has the value $60$, then a student who has achieved $40$ points gets the mark $3.0$, while a student who has achieved only $20$ points gets the mark $5.0$. A student achieving only $10$ points would still get the mark $5.0$, since every value above $5.0$ is capped by the function `min`.

### 3.5.2 Extracting the Maximum Number of Points

In order to evaluate the exam, we first have to isolate the definition of the maximum number of points. To this end, we search the complete input string for the pattern "`MaxPoints = number`". The corresponding function `get_max_points` is shown in Figure 3.10 on page 40.

```
1  def get_max_points(text: str) -> int:
2      match = re.search(r'MaxPoints\s*=\s*([1-9][0-9]*)', text)
3      if match:
4          return int(match.group(1))
5      raise ValueError('MaxPoints definition could not be found.')
```

Figure 3.10: Extracting the maximum number of points.

1. Line 2 uses the function `re.search`, which is called as

   `re.search`$(r, s)$.

   This function scans the string $s$ from left to right and returns a match object describing the *first* position at which the regular expression $r$ matches. If there is no match at all, the function returns the value None.

   Note the difference to the function `findall` that we have used in the previous section: `findall` returns *all* matches as a list of strings, while `search` returns a single match object that also

carries information about the position of the match and about the groups occurring in the regular expression.

In the regular expression `MaxPoints\s*=\s*([1-9][0-9]*)`, the subexpression `\s*` allows for arbitrary whitespace surrounding the equality sign, while the parentheses turn the regular expression `[1-9][0-9]*` that describes the number into a group.

2. Line 3 tests whether a match has been found. This test works because a match object is always *truthy*, while the value `None` is *falsy*.

3. Line 4 uses the method `group` of the match object in order to extract the string that has been matched by the first group, i.e. the digits of the number. This string is then converted into an `int`.

   In general, `match.group(`$n$`)` returns the substring that has been matched by the $n^{\text{th}}$ group, while `match.group(0)` returns the substring matched by the complete regular expression. Hence, in our example `match.group(0)` would return the string `'MaxPoints = 60'`, while `match.group(1)` returns the string `'60'`.

4. Line 5 is only reached if no match has been found. In this case, the input file is malformed and an exception is raised.

Therefore, the call `get_max_points(data)` returns the number $60$.

### 3.5.3 Processing the Student Records

Next, we need a function that evaluates the individual students. This function is shown in Figure 3.11 on page 42.

A row describing a student consists of an identifier that is followed by a colon. This identifier is either a name—which may contain upper-case letters, lower-case letters, blanks, and hyphens—or a seven-digit matriculation number. The colon is followed by the list of the points achieved.

1. Lines 2 and 3 define the two regular expressions that we need. Both of them are turned into pattern objects using the function `re.compile`. This function translates a regular expression into an internal representation once and for all. Since both regular expressions are used repeatedly inside the loop that follows, precompiling them is more efficient than passing the regular expression as a string to `re.search` or `re.findall` on every iteration. A pattern object provides the methods `match`, `search`, and `findall`, which are used in the same way as the corresponding functions of the module `re`, except that the regular expression is no longer given as an argument.

   (a) The regular expression `student` matches a row that describes a student. It consists of the following parts:

      i. `^\s*` matches the whitespace at the beginning of the line.

      ii. `([A-Za-z \-]+|[0-9]{7})` is the first group. It matches the identifier of the student, which is either a non-empty sequence of letters, blanks, and hyphens, or a sequence of exactly seven digits.
      Note that the hyphen has been escaped as "`\-`", since inside a character class an unescaped hyphen would be interpreted as the range operator that we have discussed in the previous section.

      iii. `:\s*` matches the colon and the whitespace that separates the identifier from the scores.

      iv. `(.*)$` is the second group. It matches the remainder of the line, which contains the scores.

```
1  def evaluate_students(text: str, max_points: int) -> None:
2      student = re.compile(r'^\s*([A-Za-z \-]+|[0-9]{7}):\s*(.*)$')
3      pattern = re.compile(r'0|[1-9][0-9]*')
4      for line in text.strip().split('\n'):
5          line = line.strip()
6          if not line or line.startswith(
7                  ('Class:', 'Group:', 'MaxPoints', 'Exercise:')):
8              continue
9          match = student.match(line)
10         if match:
11             name   = match.group(1).strip()
12             scores = match.group(2)
13             points = sum(int(p) for p in pattern.findall(scores))
14             grade  = mark(max_points, points)
15             print(f'{name} has {points:2d} points and '
16                   f'achieved the mark {grade}.')
17         else:
18             print(f"Warning: could not parse the line '{line}'.")
```

Figure 3.11: Evaluating the records of the individual students.

(b) The regular expression `pattern` matches a natural number, i.e. either the digit `0` or a sequence of digits that does not start with the digit `0`. It is used to extract the scores and it automatically ignores the hyphens that mark those exercises that have not been attempted.

2. Line 4 removes the leading and trailing whitespace of the input using the method `strip` and then splits the resulting string into a list of lines using the method `split`. The `for` loop then iterates over these lines.

3. Line 5 removes the leading and trailing whitespace of the current line.

4. Lines 6–8 skip those lines that do not describe a student, i.e. empty lines as well as the four lines that make up the header of the file.

5. Line 9 uses the method `match` of the pattern object `student`. In contrast to `search`, the method `match` only checks whether the regular expression matches *at the beginning* of the given string. Again, the result is either a match object or the value `None`.

6. Line 11 extracts the identifier of the student, which has been matched by the first group. The method `strip` removes the trailing blanks that separate the name from the colon.

7. Line 12 extracts the string containing the scores, which has been matched by the second group.

8. Line 13 uses the method `findall` of the pattern object `pattern` in order to extract the list of all scores from this string. These strings are converted into integers and are then added up using the function `sum`.

9. Line 14 computes the mark using the function `mark` that has been discussed above.

10. Lines 15–16 print the result. The format specifier ":2d" right-aligns the number of points in a field of width $2$ so that the output is properly aligned.
11. Lines 17–18 deal with the case that a line could not be parsed. In this case, a warning is printed but the evaluation continues with the next line.

**Remark**: At first sight, the test in lines 6–8 looks superfluous: after all, the regular expression `student` has been designed to match the rows describing students. Unfortunately, this regular expression is not selective enough. For example, the line

```
Group: TINF22AI1
```

does match `student`: the first group matches the string `'Group'`, while the second group matches the string `'TINF22AI1'`. Since the regular expression `pattern` then finds the numbers $22$ and $1$ inside this string, we would end up crediting a non-existing student named "Group" with $23$ points. Similarly, the line that numbers the exercises would yield a student named "Exercise" with $1+2+3+4+5+6=21$ points.

### 3.5.4 Running the Evaluation

Finally, we execute the program by first extracting the maximum number of points and then evaluating the individual students:

```
max_points = get_max_points(data)
evaluate_students(data, max_points)
```

This program produces the following output:

```
Jim Smith has 43 points and achieved the mark 2.7.
John Slow has 10 points and achieved the mark 5.0.
Susi Sorglos has 57 points and achieved the mark 1.3.
1609922 has 36 points and achieved the mark 3.4.
```

Note that the two exercises that have not been attempted by John Slow do not contribute anything to his score, since the hyphens are silently ignored by the regular expression `pattern`.

**Remark**: The regular expression `pattern` is written as `0|[1-9][0-9]*`, i.e. the alternative matching a single `0` comes first. In the previous section we have seen that the order of the alternatives usually matters, since the module `re` always chooses the *first* alternative that matches at the current position. Here, however, the order is irrelevant: the two alternatives start with disjoint sets of characters, so at any given position at most one of them can match at all. Therefore, the regular expression `[1-9][0-9]*|0` would work equally well. ◇

The program discussed in this section can be found at

Python/Chapter-03/03-Exam-Evaluation.ipynb.

In addition to the code discussed above, this notebook contains a plot of the function `mark` that visualizes the relation between the number of points achieved and the resulting mark.

## 3.6 Converting Html to Text

In this section, we develop a program that converts an HTML file into a plain text file. This program is actually quite useful: Some years ago I had a student who was blind. If he read a web page, he would

use his Braille display. For him, the HTML markup was of no use, so if the markup was removed, he could read web pages faster.

Figure 3.12 on page 44 shows an excerpt of the web page that we are going to use as our running example; the complete page is contained in the notebook that accompanies this section. Inspecting this excerpt, we can identify four different kinds of markup that have to be dealt with:

1. The header of the document, i.e. everything between the tags `<head>` and `</head>`, contains no text that is of any interest to the reader. The same is true for any *JavaScript* code, which is enclosed in the tags `<script>` and `</script>`. Therefore, these parts have to be discarded *completely*, including their contents.

2. All remaining tags, for example `<div id="row1">` or `<br/>`, have to be removed, but the text that is enclosed by these tags has to be preserved.

3. Named entities like "`ü`" or "` `" have to be replaced by the characters that they denote, in this case by the German umlaut "ü" and by a non-breaking space.

4. Numerical entities like "`Ⅻ`" specify a UNICODE character by its numerical code point and have to be replaced accordingly.

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Homepage of Prof. Dr. Karl Stroetmann</title>
    <link type="text/css" rel="stylesheet" href="style.css" />
  </head>
  <body>
    <div id="row1">
      <div class="right">
        <a id="dhbw" href="http://www.ba-stuttgart.de">Duale
           Hochschule Baden-W&uuml;rttemberg</a>
        <br/>Coblitzallee 1-9
        <br/>Office:     Raum 344B
      </div>
    </div>
    <div class="citation">
      <div id="sign">His holiness Pope Hugo &#8555;.</div>
    </div>
  </body>
</html>
```

Figure 3.12: An excerpt of the HTML page that is to be converted into plain text.

Note that we do *not* build a scanner for this task. Instead, we push the input through a pipeline of substitutions: every step of this pipeline uses one regular expression in order to remove or to rewrite one kind of markup, and the output of one step is the input of the next step. The tool that makes this possible is the function `re.sub`, which we discuss below. In the following, we assume that the string `data` contains the complete HTML document.

The program is shown in Figure 3.13 on page 45 and in Figure 3.14 on page 46. We discuss it line by line below.

```
1  import re
2  from html.entities import html5
3
4  def replace_named(match: re.Match[str]) -> str:
5      s = match.group(0)
6      return html5.get(s[1:], s)  # chop off '&'
7
8  def replace_unicode(match: re.Match[str]) -> str:
9      s = match.group(0)
10     if s[-1] == ';':
11         number_str = s[2:-1]    # chop off '&#' and ';'
12     else:
13         number_str = s[2:]      # only chop off '&#'
14     try:
15         return chr(int(number_str))
16     except ValueError:
17         return s
```

Figure 3.13: Converting Html into text, part 1: the replacement functions.

Before we discuss the program, we have to introduce the function `re.sub`, which is called as follows:

$$\texttt{re.sub}(r, \mathit{repl}, s, \mathrm{flags} = 0)$$

This function searches the string $s$ for all non-overlapping substrings matching the regular expression $r$ and replaces each of them. The result is a *new* string; the string $s$ itself is left unchanged, since strings are immutable in *Python*. The argument *repl* can take two different forms:

1. If *repl* is a string, then every match is replaced by this string. In particular, if *repl* is the empty string '', then every match is simply deleted.

2. If *repl* is a *function*, then this function is called once for every match. It receives the corresponding match object as its only argument and has to return the string that is used as the replacement. This variant is considerably more powerful, since the replacement can then depend on the text that has been matched.

Furthermore, we need two of the flags that are provided by the module `re`:

1. `re.DOTALL` changes the meaning of the regular expression "`.`" so that it matches *every* character, including the newline character. Recall that, by default, the dot does not match a newline.

2. `re.IGNORECASE` makes the matching case-insensitive, which we need because Html tags may be written using capital letters as well.

Several flags are combined using the operator "`|`", which denotes the bitwise *or*.

Now we are ready to discuss the program.

1. Line 1 imports the module `re`.

2. Line 2 imports the dictionary `html5` from the module `html.entities`. This dictionary maps the names of the HTML5 entities to the characters that these entities denote. Note that these names include the terminating semicolon. For example, `html5['uuml;']` returns the string `'ü'`. A few entities, e.g. `uuml` and `nbsp`, are also contained without the semicolon, since HTML tolerates these entities even if the semicolon is missing.

3. Lines 4–6 define the function `replace_named`, which computes the replacement of a named entity.

   (a) Line 5 uses the method `group(0)` of the match object in order to retrieve the complete string that has been matched, for example the string `'ü'`.

   (b) Line 6 chops off the ampersand at the beginning. What remains, e.g. the string `'uuml;'`, is a key of the dictionary `html5`. Note the use of the method `get`: if the string is not a key of this dictionary, then `get` returns its second argument, which is the string `s` that has been matched. Hence, an entity that we do not know is left untouched instead of being deleted.

4. Lines 8–17 define the function `replace_unicode`, which computes the replacement of a numerical entity. Here, the string `'&#'` is chopped off at the beginning and, if it is present, the semicolon is chopped off at the end. The remaining digits are converted into an `int`. The built-in function `chr` then computes the UNICODE character that has this number as its code point. For example, the call `chr(128034)` returns the character with the code point 128034, which is the emoji showing a turtle.

   The `try-except` statement in lines 14–17 catches the case that the given number does not denote a valid UNICODE character. In this case, the entity is left untouched.

```
18 flgs = re.IGNORECASE|re.DOTALL
19 text = data
20 text = re.sub(r'<head>.*?</head>'              , '', text, flags=flgs)
21 text = re.sub(r'<script\b[^>]*>.*?</script>', '', text, flags=flgs)
22 text = re.sub(r'<[^>]+>'      , ''              , text)
23 text = re.sub(r'&[a-zA-Z]+;?', replace_named   , text)
24 text = re.sub(r'&#[0-9]+;?'  , replace_unicode, text)
25 text = re.sub(r'(\s*\n\s*)+' , '\n'            , text)
26 print(text.strip())
```

Figure 3.14: Converting HTML into text, part 2: the pipeline of substitutions.

5. Line 19 initializes the variable `text` with the HTML document. From here on, every line overwrites this variable with the result of the previous transformation. This is the pipeline that we have announced above.

6. Lines 20–21 remove the header and any *JavaScript* code, *including* the text enclosed by the corresponding tags. The part `\b[^>]*` of the regular expression in line 21 allows the opening tag to contain attributes, as in `<script src="code.js">`. The word boundary `\b` ensures that the name of the tag is exactly `script`.

   Note that the quantifier is used in its non-greedy form "`.*?`". This is essential: if a document contained more than one `<script>` element, the greedy quantifier "`.*`" would match everything

from the *first* opening tag up to the *last* closing tag and would therefore delete all the text in between.

Furthermore, the flag `re.DOTALL` is needed here, since the header spans many lines and, without this flag, the regular expression "`.`" would not match the newline characters.

7. Line 22 removes all remaining tags. The regular expression `<[^>]+>` matches a string that starts with the character "<" and ends with the character ">". Between these two characters, there has to be a non-empty sequence of characters that are all different from ">". Since these tags are replaced by the empty string, they are deleted, but the text that they enclose is preserved.

8. Line 23 replaces the named entities. The regular expression `&[a-zA-Z]+;?` matches an ampersand that is followed by a non-empty sequence of letters and, optionally, by a semicolon. Here, the second argument of `re.sub` is the *function* `replace_named` that has been discussed above.

9. Line 24 replaces the numerical entities in the same manner. The regular expression `&#[0-9]+;?` matches an ampersand that is followed by the character "#", a non-empty sequence of digits, and, optionally, a semicolon.

   Note that the order of the lines 23 and 24 does not matter: since the character "#" is not a letter, the regular expression of line 23 does not match any numerical entity, while the regular expression of line 24 only matches numerical entities.

10. Line 25 tidies up the result. After all the markup has been removed, the text is littered with lines that contain nothing but whitespace. The regular expression `(\s*\n\s*)+` matches a non-empty sequence of whitespace that contains at least one newline character, and every such sequence is condensed into a single newline character.

11. Line 26 prints the result. The method `strip` removes the leading and trailing whitespace.

Running this program on the complete web page produces the output that is shown in Figure 3.15 on page 48. The original web page can be found at

https://wwwlehre.dhbw-stuttgart.de/~stroetma/

Today, the styling of this website is a little outdated, but I can assure you that it was very trendy when it was first conceived!
Two details of this output deserve a comment. First, the blanks that separate the labels "`Office:`", "`Phone:`", "`Fax:`", and "`Skype:`" from their values are in fact *non-breaking* spaces, since they originate from the entity "` `". Second, the last line ends with the single Unicode character U+216B, which is the Roman numeral twelve; since this character is not available in the font that is used for these lecture notes, it has been rendered as "`XII`" above.

**Remark**: It is worth noting how we have dealt with the header of the document. The traditional way to solve this problem is to equip the scanner with different scanner states: the scanner starts in a state in which it echoes the text that it reads, and as soon as it encounters the tag `<head>` it switches into a state in which everything is discarded until the tag `</head>` is found. Scanner generators like `Lex` or `Ply` offer explicit support for this technique.

The pipeline that we have developed achieves the same effect without any notion of state: because we remove the complete header in a separate step *before* we deal with the remaining tags, the later steps never see the header at all. Splitting a problem into a sequence of simple substitutions is often much easier than getting a single stateful scanner right. ◇

**Remark**: The program developed in this section works for the web page shown above, but it is easy to construct Html documents for which it fails. For example, an attribute value may itself contain the

```
Prof. Dr. Karl Stroetmann
Duale Hochschule Baden-Württemberg
Coblitzallee 1-9
68163 Mannheim
Germany
Office:     Raum 344B
Phone:    +49 621 4105-1376
Fax:        +49 621 4105-1194
Skype:     karlstroetmann
Some links:
My lecture notes,
as well as the programs presented in class, can be found
at
https://github.com/karlstroetmann.
Most of my papers can be found at researchgate.net.
The programming language SetlX can be downloaded at
http://randoom.org/Software/SetlX.
As I am getting old and wise, I have to accept the limits of
my own capabilities.  I have condensed these deep philosophical
insights into a most beautiful pearl of poetry.  I would like
to share these humble words of wisdom:
I am a teacher by profession,
mostly really by obsession;
But even though I boldly try,
I just cannot teach pigs to fly.
Instead, I slaughter them and fry.
Any sufficiently advanced poetry is indistinguishable from divine
wisdom.
His holiness Pope Hugo XII.
```

Figure 3.15: The text that has been extracted from the HTML page.

character “>”, as in

```
<a title="a > b">,
```

and in this case the regular expression `<[^>]+>` matches only a proper prefix of the tag.

This is not an accident. In Chapter 5, we will prove that the language $\{a^n b^n \mid n \in \mathbb{N}\}$ is *not* a regular language. It follows that the language of all strings with balanced parentheses is not regular either, and the nesting of HTML elements is an instance of exactly this problem. Therefore, no regular expression can ever describe the syntax of HTML. Our program nevertheless works, because we never have to *understand* the nesting of the elements: apart from the header, all we have to do is to delete the markup, and for this task a regular expression is sufficient. ◇

The program discussed in this section can be found at

Python/Chapter-03/04-Html2Text.ipynb.

The notebook contains the complete HTML document and, in addition, it prints the intermediate result after every step of the pipeline. Inspecting these intermediate results is quite instructive, as it shows

how the document is stripped of its markup step by step.

**Exercise 3**: The purpose of the following exercise is to transform LaTeX into MathML. LaTeX is a document markup language that is especially well suited to present text that contains mathematical formulas. In fact, these lecture notes have all been typeset using LaTeX. MathML is the part of Html that deals with the representation of mathematical formulas. As LaTeX provides a very rich document markup language and we can only afford to spend a few hours on this exercise, we confine ourselves to a small subset of LaTeX. Figure 3.16 on page 49 shows the example input file that we want to transform into Html. If this example file is typeset using LaTeX, it is displayed as shown in Figure 3.17 on page 50. The program that you are going to develop should transform the LaTeX input file into an Html file. For your convenience, all these files are available in the github directory

Python/Chapter-03/05-LaTeX2HTML.

This directory contains also the *Jupyter* notebook LaTeX2HTML-Exercise.ipynb. This notebook provides the complete framework of the scanner: it contains all the functions that write the MathML, the driver of the scanner, and the processing of those parts of the input that are not mathematical formulas. Your task is to fill in the parts that are marked in this notebook, i.e. the regular expressions of the tokens that occur inside a formula and the actions that are associated with these tokens.

```
\documentclass{article}

\begin{document}
The sum of the squares of the first $n$ natural numbers is given as:
$$ \sum\limits_{i=1}^{n} i^{2} = \frac{1}{6} \cdot n \cdot (n+1)
 ↪  \cdot (2\cdot n + 1). $$
According to Pythagoras, the length of the hypotenuse of a
 ↪  right-angled triangle is
the square root of the sum of the squares of the lengths of the two
 ↪  catheti:
$$ c = \sqrt{a^{2} + b^{2}}.  $$
The area of a circle is given as
$$  A = \pi \cdot r^{2},   $$
while its circumference satisfies
$$ C = 2 \cdot \pi \cdot r.  $$
\end{document}
```

Figure 3.16: An example LaTeX input file.

In order to do this exercise, you have to understand a little bit about LaTeX and about MathML. In the following, we discuss those features of these two languages that are needed in order to solve the given problem.

1. A LaTeX input file has the following structure:

   (a) The file starts with its preamble. The preamble specifies the type of the document and loads the packages that are used. In our example, the preamble consists of the single line

   \documentclass{article}.

   As our translator does not support any of the options that LaTeX offers here, the preamble is simply discarded. Instead, the following Html header is written:

The sum of the squares of the first $n$ natural numbers is given as:

$$\sum_{i=1}^{n} i^2 = \frac{1}{6} \cdot n \cdot (n+1) \cdot (2 \cdot n + 1).$$

According to Pythagoras, the length of the hypotenuse of a right-angled triangle is the square root of the sum of the squares of the lengths of the two catheti:

$$c = \sqrt{a^2 + b^2}.$$

The area of a circle is given as

$$A = \pi \cdot r^2,$$

while its circumference satisfies

$$C = 2 \cdot \pi \cdot r.$$

Figure 3.17: Output produced by the LaTeX file shown in Figure 3.16

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Converted LaTeX Document</title>
<script src="https://cdn.jsdelivr.net/npm/mathjax@3/es5/te
↪ x-mml-chtml.js"
↪ async>
</script>
</head>
```

Here, the `<script>` tag loads MathJax, which is the *JavaScript* library that renders the MathML in the browser. Modern browsers can display MathML on their own, but *MathJax* ensures that the formulas look the same in every browser.

(b) The content of the document is started by the line

`\begin{document}`

and this line is translated into the tag

`<body>`.

(c) After that, the LaTeX file consists of text that contains mathematical formulas.

(d) The LaTeX input file finishes with a line of the form

`\end{document}`.

This line is translated into the tags

`</body></html>`

and everything that follows it is discarded.

2. In LaTeX, an inline formula is started and ended with a single dollar symbol “$”. In MathML, an inline formula is written as

```
<math xmlns="http://www.w3.org/1998/Math/MathML"
    display="inline">···</math>.
```

Here, I have used "···" to represent the mathematical content of the formula.

3. In LaTeX, a formula that is displayed in its own line is started and ended with the string "$$". In MathML, these formulas are called block formulas and are written as

```
<math xmlns="http://www.w3.org/1998/Math/MathML"
    display="block">···</math>.
```

Again, I have used "···" to represent the mathematical content of the formula.

4. While in LaTeX a mathematical variable does not need any special markup, in MathML a mathematical variable is written using the tags <mi> and </mi>. For example, the mathematical variable $n$ is written as

```
<mi>n</mi>.
```

5. While in LaTeX a number does not need any special markup, in MathML a number is written using the tags <mn> and </mn>. For example, the number $3.14159$ is written as

```
<mn>3.14159</mn>.
```

6. In LaTeX the mathematical constant $\pi$ is written using the command "\pi". In MathML, we have to make use of the Html entity "π". Since $\pi$ is the name of a constant, it is enclosed in the same tags as a variable, and hence we would write $\pi$ as

```
<mi>&pi;</mi>.
```

7. While in LaTeX most operator symbols stand for themselves, in MathML an operator is surrounded by the tags <mo> and </mo>. For example, the operator $+$ is written as

```
<mo>+</mo>.
```

8. In LaTeX the multiplication operator "·" is written using the command "\cdot". In MathML, we have to make use of the Html entity "⋅" and hence we would write "·" as

```
<mo>&sdot;</mo>.
```

Similarly, the comparison operators "$\leq$" and "$\geq$", which are written as "\leq" and "\geq" in LaTeX, make use of the entities "≤" and "≥".

9. In LaTeX, raising an expression $e$ to the $n$th power is done using the operator "^". Furthermore, the exponent should be enclosed in the curly braces "{" and "}". For example, the code to produce the term $x^2$ is

```
x^{2}.
```

In MathML, raising an expression to a power is achieved using the tags <msup> and </msup>. This tag takes two arguments: the base and the exponent. For example, in order to display the term $x^2$, we have to write

```
<msup><mi>x</mi><mrow><mn>2</mn></mrow></msup>.
```

10. In LaTeX, taking the square root of an expression is done using the command "\sqrt". The argument has to be enclosed in curly braces. For example, in order to produce the output $\sqrt{a+b}$, we have to write

```
\sqrt{a+b}.
```

In MathML, taking the square root makes use of the tags `<msqrt>` and `</msqrt>`. The example shown above can be written as

`<msqrt><mrow><mi>a</mi><mo>+</mo><mi>b</mi></mrow></msqrt>`.

11. In LaTeX, writing a fraction is done using the command "`\frac`". This command takes two arguments, the numerator and the denominator. Both of these have to be enclosed in curly braces. For example, in order to produce the output $\frac{a+b}{2}$, we have to write

    `\frac{a+b}{2}`.

    In MathML, a fraction is created via the tags `<mfrac>` and `</mfrac>`. The example shown above can be written as

    `<mfrac><mrow><mi>a</mi><mo>+</mo><mi>b</mi></mrow>`
    `<mrow><mn>2</mn></mrow></mfrac>`.

12. In LaTeX, writing a sum is done using the command "`\sum\limits`". This command takes two arguments: The first argument gives the indexing variable together with its lower bound, while the second argument gives the upper bound. The first argument is started using the string "`_{`" and ended using the string "`}`", while the second argument is started using the string "`^{`" and ended using the string "`}`". For example, in order to produce the output

    $$\sum_{i=1}^{n} i,$$

    we have to write

    `\sum\limits_{i=1}^{n} i`.

    In MathML, a sum with lower and upper limits is created via the tags `<munderover>` and `</munderover>` and the Html entity "`∑`". The tag `munderover` takes three arguments:

    (a) The first argument is the operator, so in this case it is the entity "`∑`".
    (b) The second argument initializes the indexing variable of the sum.
    (c) The third argument provides the upper bound.

    Hence, the example shown above would be written as follows:

```
<munderover>
    <mo>&sum;</mo>
    <mrow>
        <mi>i</mi> <mo>=</mo> <mn>1</mn>
    </mrow>
    <mrow>
        <mi>n</mi>
    </mrow>
</munderover>
```

Strictly speaking, the tags `<mrow>` and `</mrow>` that occur in the examples above are only required if the corresponding argument consists of more than a single element. However, enclosing an argument in these tags is never wrong, and it is much simpler to write them unconditionally than to figure out, at the time the argument is started, whether more than one element will follow. Therefore, the scanner always encloses the arguments of `<msup>`, `<msqrt>`, `<mfrac>`, and `<munderover>` in a row.

Now that we know what has to be produced, we can discuss how the scanner has to be organized. The crucial observation is that the rules that apply inside a mathematical formula are completely different from the rules that apply in ordinary text: inside a formula, the string "`\frac`" has to be translated, while in ordinary text it is nothing but a sequence of five characters. Therefore, the scanner needs scanner states. We have already met this notion in Section 3.6, where we have deliberately *avoided* it by using a pipeline of substitutions. Here, we cannot avoid it, since the output depends on the order in which the tokens occur. Our scanner uses the four states that are shown in Table 3.1 on page 53.

| State | Meaning |
|---|---|
| PREAMBLE | This is the state the scanner starts in. Everything that occurs before the string `\begin{document}` is discarded. |
| TEXT | The scanner is reading ordinary text, i.e. text that is not part of a mathematical formula. This text is echoed unchanged. |
| FORMULA | The scanner is inside a formula, i.e. it has read either a "$" or a "$$". The input has to be translated into MathML. |
| DONE | The string `\end{document}` has been read and the scanner stops. |

Table 3.1: The states of the scanner translating LaTeX into MathML.

Since every state uses its own set of tokens, we do not have a single token specification anymore. Instead, we have a *dictionary* `token_specifications` that maps every state to the token specification that is used in this state. In the state DONE the scanner does not read anything anymore and therefore this state does not need a token specification. The states PREAMBLE and TEXT are simple: apart from the strings `\begin{document}`, `\end{document}`, "$", and "$$", these states only need a token that matches an arbitrary character, which is then either discarded or echoed. The interesting state is the state FORMULA, whose tokens are listed in Table 3.2 on page 54.
Figure 3.18 on page 55 shows how the scanner is driven. It differs from the scanner discussed at the beginning of this chapter in two respects.

1. Lines 1–4 build one master pattern for every state. Note that the names of the groups only have to be unique *within* a single master pattern. Therefore, the token DOLLAR_DOLLAR can occur both in the state TEXT and in the state FORMULA, even though the actions associated with this token are different in these two states.

2. The scanner switches between different master patterns and therefore the function `re.finditer` can not be used here, since this function scans a string from its beginning to its end using one fixed regular expression. Instead, the master patterns are compiled via `re.compile` and line 11 uses the method `match` of the resulting objects. This method is called as

   $$\textit{pattern}.\texttt{match}(s, \textit{pos})$$

   and tries to find a match that starts *exactly* at the position *pos* of the string $s$. Hence, the scanner has to keep track of the position where the next token starts itself, which is done in line 15.

   Note that this is precisely the behaviour that a scanner needs: it must not skip any characters, since every character of the input belongs to some token.

For every state there is a function that processes the tokens of this state. This function writes the MathML that corresponds to the given token and returns the state that the scanner has to switch to. For example, the function `process_text` returns the state FORMULA if it has been called with the token DOLLAR or with the token DOLLAR_DOLLAR, while in all other cases it returns the state TEXT.

| Token | Matches |
|---|---|
| DOLLAR_DOLLAR | the string "$$", which ends a block formula |
| DOLLAR | the string "$", which ends an inline formula |
| SUM_LIMITS | the string \sum\limits_{ |
| SQRT | the string \sqrt{ |
| FRAC | the string \frac{ |
| VARIABLE_HAT | a variable that is followed by "^{", e.g. i^{ |
| NUMBER_HAT | a number that is followed by "^{", e.g. 2^{ |
| CB_HAT_OB | the string }^{, which separates the limits of a sum |
| CB_OB | the string }{, which separates numerator and denominator |
| CB | the string }, which closes the innermost construct |
| CDOT | the string \cdot |
| LEQ | the string \leq |
| GEQ | the string \geq |
| PI | the string \pi |
| NUMBER | a natural number |
| VARIABLE | a sequence of letters |
| OPERATOR | one of the characters +, -, *, /, =, <, >, ., ,, (, ) |
| WS | whitespace, which is discarded |
| MISMATCH | any other character, which is an error |

Table 3.2: The tokens of the state FORMULA.

Note that the state is nothing but a string that is passed around: there is no magic involved, and this is the reason why we do not need a scanner generator in order to implement scanner states.

**Remark**: The most important problem that you have to solve is the following: Once you encounter a closing brace "}" you have to know whether this brace closes the argument of a square root, a fraction, a sum, or an exponent. You should be aware that, for example, square roots and fractions can be nested. Hence, it is not enough to have a single variable that remembers whether you are translating, say, a square root or a fraction. Instead, you need a stack.

Every time you encounter a string that opens a curly brace, you push a marker onto this stack that records which construct has been opened. Every time you encounter a string that closes a curly brace, you pop this marker again and it tells you which MathML tags have to be written. This is the reason why the function `scan` shown in Figure 3.18 maintains the variable `stack` and hands it to the function `process_formula`. Table 3.3 on page 54 lists the markers that are needed.

| Marker | pushed when reading | popped when reading |
|---|---|---|
| 'SQRT' | \sqrt{ | } |
| 'FRAC_NUM' | \frac{ | }{ |
| 'FRAC_DEN' | }{ | } |
| 'SUM_LOWER' | \sum\limits_{ | }^{ |
| 'SUM_UPPER' | }^{ | } |
| 'SUPER' | i^{ | } |

Table 3.3: The markers that are stored on the stack.

Note that the markers 'FRAC_NUM' and 'SUM_LOWER' must not be closed by a simple closing brace: the numerator of a fraction is always followed by its denominator, while the lower limit of a sum is always followed by its upper limit. If a formula ends while the stack is not empty, the input contains

```
master_patterns = \
    { state: re.compile('|'.join(f'(?P<{name}>{regexp})'
                                 for name, regexp in spec))
      for state, spec in token_specifications.items() }

def scan(text: str) -> None:
    state = 'PREAMBLE'
    stack: list[str] = []
    pos   = 0
    while state != 'DONE' and pos < len(text):
        mo    = master_patterns[state].match(text, pos)
        assert mo is not None and mo.lastgroup is not None
        token = mo.lastgroup
        value = mo.group()
        pos   = mo.end()
        if state == 'PREAMBLE':
            state = process_preamble(token, value)
        elif state == 'TEXT':
            state = process_text(token, value)
        else:
            state = process_formula(token, value, stack)
    if state != 'DONE':
        raise SyntaxError(r'missing "\end{document}"')
```

Figure 3.18: The driver of the scanner that translates LaTeX into MathML.

an opening brace that is never closed and your program should report an error. ◇

**Remark**: There is a subtlety hidden in the tokens VARIABLE_HAT and NUMBER_HAT. At first sight, the string `i^{2}` seems to consist of the variable "`i`", the operator "^", and the number "2" enclosed in braces. However, the corresponding MathML is

```
<msup><mi>i</mi><mrow><mn>2</mn></mrow></msup>
```

and here the tag `<msup>` has to be written *before* the variable "`i`". At the time the scanner reads this variable it does not yet know whether an exponent follows, and once it knows, the variable has already been written.

The solution is to recognize the string `i^{` as a *single* token. This token then carries the information that an exponent follows and the corresponding action can write the opening tag, the base, and the tag `<mrow>` in the correct order. This trick is a poor man's version of a lookahead: instead of inspecting the characters that follow the variable, we simply make them part of the token. Note that the token VARIABLE_HAT therefore has to precede the token VARIABLE in the token specification, since otherwise the string `i^{` would be recognized as an ordinary variable. ◇

**Remark**: As we have discussed in the remark on page 24, *Python* chooses the *first* alternative of the master pattern that matches at the current position. Hence, the order of the tokens listed in Table 3.2 is significant and you should keep it:

1. VARIABLE_HAT has to precede VARIABLE and NUMBER_HAT has to precede NUMBER.

2. CB_HAT_OB has to precede CB_OB, which in turn has to precede CB, since otherwise the string `}^{` would be recognized as a single closing brace that is followed by garbage.

3. DOLLAR_DOLLAR has to precede DOLLAR, since otherwise a block formula would be ended by the first of the two dollar symbols.

Finally, note that the token `MISMATCH` matches an arbitrary character and therefore has to be the *last* token of the specification. Since this token matches everything that the preceding tokens do not match, the method `match` always succeeds. However, a type checker cannot know this, since in general the method `match` returns `None` if there is no match, and the attribute `lastgroup` is `None` if no named group has matched. Therefore, line 12 of the function `scan` asserts that neither of these cases occurs. Any LaTeX command that is not supported, for example the command "`\alpha`", is caught by this token and produces a readable error message instead of silently disappearing from the output. ◇

## 3.7 Check your Understanding

(a) How are regular expressions defined in *Python*?

(b) Do you understand the structure of the scanner discussed at the beginning of this chapter?

(c) Why does the order of the pairs in a token specification matter, and which rule does *Python* use in order to choose between several alternatives that all match at the current position?

(d) What is a scanner state and how can scanner states be implemented using the module `re`? Why can the function `re.finditer` not be used in this case?

(e) Why is a single variable not sufficient in order to remember which construct is closed by a closing brace, and what has to be used instead?

(f) Are you able to use regular expressions to write a program that reads a given text, finds all numbers inside this text that are followed by a $ symbol and that then converts these numbers to their equivalent value in €? Assume that $1\$ = 0.85€$.

# Chapter 4

# Finite State Machines

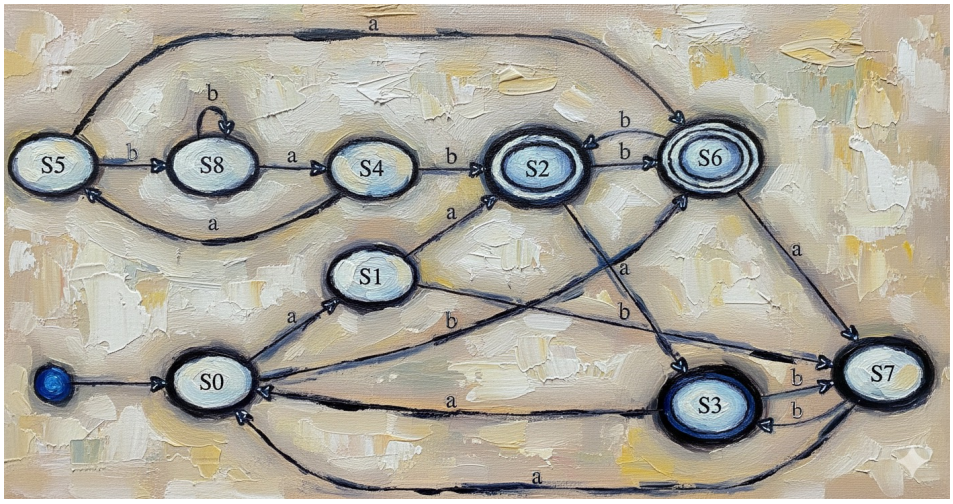



Figure 4.1: A finite state machine.

In the previous chapter we have seen how we can generate scanners from regular expressions. In this chapter we learn how regular expressions can be implemented using finite state machines. In the literature, finite state machines are also called finite automata. There are two kinds of finite state machines: the deterministic finite automata, abbreviated as DFAs, and the non-deterministic finite automata, abbreviated as NFAs. Although NFAs seem to be more powerful than DFAs, we will see that every NFA can be transformed into an equivalent DFA. After proving this result, we show how a regular expression can be translated into an equivalent NFA. Then we show that the language recognized by any DFA can be described by an equivalent regular expression. Therefore, the central result of this chapter is the equivalence of finite state machines and regular expressions. Finally, we show how the number of states of a DFA can be minimized. In detail, the results proved in this chapter are as follows:

(a) The language described by a regular expression can be defined by an NFA.

(b) Every NFA can be transformed into an equivalent DFA.

(c) For every DFA there is a regular expression specifying the language recognized by this DFA.

(d) For every DFA there is an equivalent DFA that has the minimal number of states.

## 4.1 Deterministic Finite Automata

The finite state machines that we are going to discuss in this chapter are used to read a string and to decide whether this string is an element of a given language. Hence, the input of these DFAs is a string, while the output is either the value `True` or the value `False`. The name giving feature of a DFA is the fact that a DFA only has a finite number of possible states. Reading a character causes the DFA to change its state. A DFA accepts its input if it has reached a so called accepting state after reading all characters of the input string. Let me explain this idea more precisely:

(a) Initially, the DFA is in a state that is known as the start state.

(b) In every step of its computation, the DFA reads one character of the input string $s$. Every time a character is processed, the state of the DFA might change.

(c) Some states of the DFA are designated as accepting states. If the DFA has consumed all characters of the given input string $s$ and the DFA has reached an accepting state, then the input string $s$ is accepted and the DFA returns `True`. Otherwise the DFA returns `False` and the string $s$ is rejected.

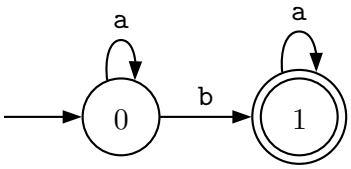



Figure 4.2: A DFA to recognize the language $L(\mathsf{a}^* \cdot \mathsf{b} \cdot \mathsf{a}^*)$.

Finite state machines are best presented graphically. Figure 4.2 depicts a simple DFA that recognizes those strings that are specified by the regular expression

$$\mathsf{a}^* \cdot \mathsf{b} \cdot \mathsf{a}^*.$$

This DFA has but two states. These states are called $0$ and $1$.

1. State $0$ is the start state. In Figure 4.2, the start state is indicated by an arrow coming from the left that points to it.

   If the DFA is in the state $0$ and reads the character "a", then the DFA stays in the state $0$. This is specified in the figure by an arrow labeled with the character "a" that both starts and ends in the state $0$. On the other hand, if the character "b" is read while the DFA is in state 0, then the DFA switches into the state $1$. This is depicted by an arrow labeled with the character "b" that originates from the state $0$ and points to the state $1$.

2. State $1$ is an accepting state. In Figure 4.2 this is specified by the fact that the state $1$ is decorated by a double circle.

   If the character "a" is read while the DFA is in state $1$, then the DFA does not change its state. On the other hand, if the DFA reads the character "b" while in state $1$, then the next state is undefined since there is no arrow originating from state $1$ that is labeled with the character "b".

   In general, a DFA dies if it reads a character $c$ in a state $s$ such that there is no transition from $s$ when $c$ is read. In this case, the DFA does not accept the given input string.

Formally, a *deterministic finite automaton* is defined as a 5-tuple.

**Definition 9 (DFA)** A deterministic finite automaton (abbreviated as DFA) is a 5-tuple

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle$$

where the components $Q$, $\Sigma$, $\delta$, $q_0$, and $A$ have the following properties:

1. $Q$ is the finite set of states.
2. $\Sigma$ is the input alphabet. Therefore, $\Sigma$ is a set of characters and the strings read by the DFA $F$ are strings from the set $\Sigma^*$.
3. $\delta : Q \times \Sigma \to Q \cup \{\Omega\}$

   is the transition function. For every state $q \in Q$ and for all characters $c \in \Sigma$ the expression $\delta(q, c)$ computes the new state of the DFA $F$ that is reached if $F$ reads the character $c$ while in state $q$. If $\delta(q, c) = \Omega$, then $F$ dies when it is in state $q$ and the next character is $c$.

   In the figures depicting DFAs transitions of the form $\delta(q, c) = \Omega$ are not shown.
4. $q_0 \in Q$ is the start state.
5. $A \subseteq Q$ is the set of accepting states. ◇

**Example**: The DFA that is shown in Figure 4.2 is formally defined as follows:

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle,$$

where we have:

1. $Q = \{0, 1\}$,
2. $\Sigma = \{\mathsf{a}, \mathsf{b}\}$,
3. $\delta = \big\{\langle 0, a\rangle \mapsto 0, \langle 0, b\rangle \mapsto 1, \langle 1, a\rangle \mapsto 1, \langle 1, b\rangle \mapsto \Omega\big\}$,
4. $q_0 = 0$,
5. $A = \{1\}$.

In order to formally define the language $L(F)$ that is accepted by a DFA $F$ we generalize the transition function $\delta$ to a new function

$$\delta^* : Q \times \Sigma^* \to Q \cup \{\Omega\}$$

that, instead of a single character, accepts a string as its second argument. The definition of $\delta^*(q, w)$ is given by induction on the string $w$.

I.A. $w = \lambda$: We define

$$\delta^*(q, \lambda) := q,$$

because if a DFA does not read any character, it cannot change its state.

I.S. $w = cv$ where $c \in \Sigma$ and $v \in \Sigma^*$: We define

$$\delta^*(q, cv) := \begin{cases} \delta^*\big(\delta(q, c), v\big) & \text{provided } \delta(q, c) \neq \Omega; \\ \Omega & \text{otherwise.} \end{cases}$$

If $F$ reads the string $cv$, it first reads the character $c$. Now if this causes $F$ to change into the state $\delta(q, c)$, then $F$ has to read the string $v$ in the state $\delta(q, c)$. However, if $\delta(q, c)$ is undefined, then $\delta^*(q, cv)$ is undefined too.

**Definition 10 (Accepted Language, $L(F)$)** For a DFA $F = \langle Q, \Sigma, \delta, q_0, A \rangle$ the language accepted by $F$ is called $L(F)$ and is defined as

$$L(F) := \big\{ s \in \Sigma^* \mid \delta^*(q_0, s) \in A \big\}.$$

Hence, the accepted language of $F$ is the set of all those strings that take $F$ from its start state into an accepting state. ◇

**Exercise 4**: Specify a DFA $F$ such that $L(F)$ is the set of all those strings $s \in \{a, b\}^*$, such that $s$ contains the substring "aba". ◇

**Complete Finite State Machines** Occasionally it is beneficial for a DFA $F$ to be complete: A DFA

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle,$$

is complete if the transition function $\delta$ never returns the undefined value $\Omega$, i.e. we have

$$\delta(q, c) \neq \Omega \quad \text{for all } q \in Q \text{ and } c \in \Sigma.$$

**Proposition 11** For every DFA $F$ there exists a complete DFA $\widehat{F}$ that accepts the same language as the DFA $F$, i.e. we have:

$$L(\widehat{F}) = L(F).$$

**Proof**: Assume $F$ is given as

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle.$$

The idea is to define $\widehat{F}$ by adding a new state to the set of states $Q$. This new state is called the dead state. If there is no next state for a given state $q \in Q$ when a character $c$ is processed, i.e. if we have

$$\delta(q, c) = \Omega,$$

then $F$ changes into the dead state. Once $F$ has reached a dead state, it will never leave this state.

The formal definition of the DFA $\widehat{F}$ is done as follows: We introduce a new state ☠ which serves as the dead state. The only requirement is that $☠ \notin Q$.

1. $\widehat{Q} := Q \cup \{☠\}$,
   the dead state is added to the set $Q$.
2. $\widehat{\delta} : \widehat{Q} \times \Sigma \to \widehat{Q}$,
   where the function $\widehat{\delta}$ is defined as follows:
   (a) $\delta(q, c) \neq \Omega \to \widehat{\delta}(q, c) = \delta(q, c)$ for all $q \in Q$ and $c \in \Sigma$.
   If the state transition function is defined for the state $q$ and the character $c$, then $\widehat{\delta}(q, c)$ is the same as $\delta(q, c)$.
   (b) $\delta(q, c) = \Omega \to \widehat{\delta}(q, c) = ☠$ for all $q \in Q$ and $c \in \Sigma$.
   If the state transition function $\delta$ is undefined for the state $q$ and the character $c$, then $\widehat{\delta}(q, c)$ returns the dead state ☠.
   (c) $\widehat{\delta}(☠, c) = ☠$ for all $c \in \Sigma$,
   because there is no escape from death.

Hence the DFA $\widehat{F}$ is given as follows:

$$\widehat{F} = \langle \widehat{Q}, \Sigma, \widehat{\delta}, q_0, A \rangle.$$

If $F$ reads a string $s$ without reaching an undefined state, then the behavior of $F$ and $\widehat{F}$ is the same. However, if $F$ reaches an undefined state, then $\widehat{F}$ instead switches into the dead state ☠ and remains in this state regardless of the rest of the input string. As the dead state ☠ is not an accepting state, the languages accepted by $F$ and $\widehat{F}$ are identical. □

**Exercise 5**: Define a DFA that accepts the language specified by the regular expression

$$r := (\mathtt{a} + \mathtt{b})^* \cdot \mathtt{b} \cdot (\mathtt{a} + \mathtt{b}) \cdot (\mathtt{a} + \mathtt{b}).$$ ◇

**Solution**: The regular expression $r$ specifies those strings $s$ from the alphabet $\Sigma = \{\mathtt{a}, \mathtt{b}\}$ such that the antepenultimate character of $s$ is the character "b". In order to recognize this fact, the DFA has to remember the last three characters. As there are eight different possible combinations for the last three characters, the DFA needs to have eight states. Let us number these states $0$, $1$, $2$, $\cdots$, $7$. We describe the purpose of these states in the following:

**State 0:** In this state, the character "b" has not yet been seen. Depending on how many characters have been read, there are four cases:

(a) At least three characters have been read. In this case, the last three characters are "aaa".

(b) Two characters have been read. In this case, the string that has been read so far is the string "aa".

(c) Only one character has been read so far. In this case, the string that has been read is "a".

(d) Nothing has yet been read and therefore the string that has been read is $\lambda$.

For the remaining states we list the last three characters that have been read without further comment.

**State 1:** "aab".

This case also covers the cases where the strings "ab" and "b" have been read.

**State 2:** "aba".

This case also covers the case where the string "ba" has been read.

**State 3:** "abb".

This case also covers the case where the string "bb" has been read.

**State 4:** "bab".

**State 5:** "bba".

**State 6:** "bbb".

**State 7:** "baa".

Obviously, the states 4, 5, 6 and 7 are the accepting states because here the antepenultimate character is the character "b". Next, we construct the transition function $\delta$.

0. First, let us consider the state 0. If the last three characters that have been read are "aaa" and if we read the character "a" next, then the last three characters read will again be "aaa". Hence, we must have

$$\delta(0, \mathtt{a}) = 0.$$

However, if instead we read the character "b" in state 0, then the last three characters that have been read are "aab", which is exactly the last three characters that have been read in state 1. Hence we have

$$\delta(0, \mathtt{b}) = 1.$$

1. Next we consider state 1. If the last three characters are “aab” and we read the character “a” next, then the last three characters are “aba”. This corresponds to the state 2. Therefore, we must have

   $$\delta(1, \mathtt{a}) = 2.$$

   If instead we read the character “b” while in state 1, then the last three characters will be “abb”, which corresponds to the state number 3. Hence we have

   $$\delta(1, \mathtt{b}) = 3.$$

The remaining transitions are found in a similar way. Figure 4.3 on page 62 shows the resulting DFA. We still have to explain how we have chosen the start state. When the computation starts, the finite state machine has not read any character. In particular, this implies that neither of the last three characters is the character “b”. Hence we can use the state 0 as the start state of our DFA.

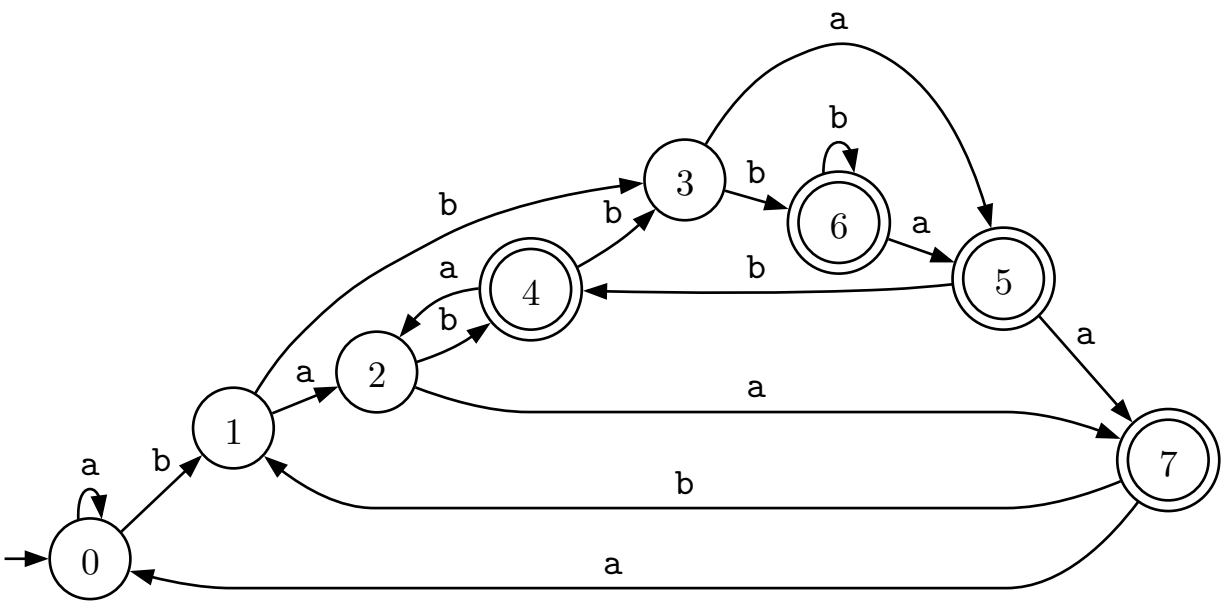



Figure 4.3: A DFA accepting $L\big((\mathtt{a} + \mathtt{b})^* \cdot \mathtt{b} \cdot (\mathtt{a} + \mathtt{b}) \cdot (\mathtt{a} + \mathtt{b})\big)$.

**Remark**: There is a nice tool available that can be used to better understand finite state machines. This tool is available at

https://ivanzuzak.info/noam/webapps/fsm_simulator/.

## 4.2 Non-Deterministic Finite Automata

For many applications, the DFAs introduced in the previous section are unwieldy because they have a large numbers of states. For example, a DFA that recognizes the language

$$L\big((\mathtt{a} + \mathtt{b})^* \cdot \mathtt{b} \cdot (\mathtt{a} + \mathtt{b}) \cdot (\mathtt{a} + \mathtt{b})\big)$$

needs 8 different states since the DFA needs to remember the last three characters that have been read and there are $2^3 = 8$ combinations of these characters. It would be possible to simplify this finite state machine if the machine would be permitted to *choose* its next state from a given set of states. Figure 4.4 presents a non-deterministic finite automaton (abbreviated as NFA) that accepts the language specified by the regular expression

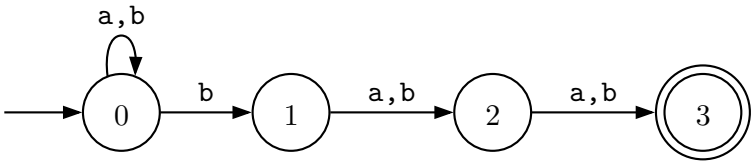



Figure 4.4: A non-deterministic finite automaton to recognize $L\big((\mathsf{a}+\mathsf{b})^* \cdot \mathsf{b} \cdot (\mathsf{a}+\mathsf{b}) \cdot (\mathsf{a}+\mathsf{b})\big)$.

$$(\mathsf{a}+\mathsf{b})^* \cdot \mathsf{b} \cdot (\mathsf{a}+\mathsf{b}) \cdot (\mathsf{a}+\mathsf{b}).$$

This NFA has only 4 different states that are named $0$, $1$, $2$ and $3$.

1. $0$ is the start state. If the NFA reads the letter "a" while it is in this state, the NFA will stay in state $0$. However, if the NFA reads the character "b", then the NFA has a *choice*: It can either stay in state $0$, or it might switch to the state $1$.
2. In state $1$ the NFA switches to state $2$ if it reads either the character "a" or the character "b".
3. In state $2$ the NFA switches to state $3$ if it reads either the character "a" or the character "b".
4. State $3$ is the accepting state. There is no transition from this state. Hence, if the NFA is in state 3 and there are still characters to read, then the NFA dies.

The finite state machine in Figure 4.4 is non-deterministic because it has to guess the next state if it is in state $0$ and reads the character "b". Let us consider a possible *computation* of the NFA when it reads the input "abab":

$$0 \overset{a}{\mapsto} 0 \overset{b}{\mapsto} 1 \overset{a}{\mapsto} 2 \overset{b}{\mapsto} 3$$

In this computation, the NFA has chosen the correct transition when reading the first occurrence of the character "b". If the NFA had stayed in the state $0$ instead of switching into the state 1, it would have been impossible to reach the accepting state 3 later because then the computation would have worked out as follows:

$$0 \overset{a}{\mapsto} 0 \overset{b}{\mapsto} 0 \overset{a}{\mapsto} 0 \overset{b}{\mapsto} 1$$

Here, the NFA is in state 1 after consuming the input string "abab" and as state 1 is not an accepting state, the NFA would have falsely rejected the string "abab". Let us consider a different example where the input is the string "bbbbb":

$$0 \overset{b}{\mapsto} 0 \overset{b}{\mapsto} 1 \overset{b}{\mapsto} 2 \overset{b}{\mapsto} 3 \overset{b}{\mapsto} \Omega$$

Here, the NFA has switched too early into the state 1. In this case, the NFA dies when reading the last character "b". If the NFA had stayed in state $0$ when reading the second occurrence of the character "b", then it would have correctly accepted the string "bbbbb" since then the computation could have been as follows:

$$0 \overset{b}{\mapsto} 0 \overset{b}{\mapsto} 0 \overset{b}{\mapsto} 1 \overset{b}{\mapsto} 2 \overset{b}{\mapsto} 3.$$

The previous examples show that in order to avoid premature death, the given non-deterministic NFA has to choose its successor state wisely. If $F$ is a non-deterministic NFA and $s$ is a string such that $F$ can, when reading $s$, choose its successor so that it reaches an accepting state after having read $s$, then the string $s$ is an element of the language $L(F)$.

It seems that the concept of an NFA is far more powerful than the concept of a DFA. After all, an NFA appears to have some form of clairvoyance for else it could not guess which states to choose. However, we will prove in the next section that both DFAs and NFAs have the same power to recognize

languages: Every language recognized by an NFA is also recognized by a DFA. In order to prove this claim, we have to formalize the notion of an NFA. The definition that follows is more general than the informal description of an NFAs given so far, as we will allow the NFA to also have $\varepsilon$-transitions. An $\varepsilon$ transition allows the NFA to switch its state without reading any character. For example, if there is an $\varepsilon$-transition from the state 1 into the state 2, we write

$$1 \overset{\varepsilon}{\mapsto} 2.$$

**Definition 12 (NFA)** A non-deterministic finite automaton (abbreviated as NFA) is a 5-tuple

$$\langle Q, \Sigma, \delta, q_0, A \rangle,$$

such that the following holds:

1. $Q$ is the finite set of states.
2. $\Sigma$ is the input alphabet.
3. $\delta$ is a function from $Q \times (\Sigma \cup \{\varepsilon\})$ that assigns a set of states $\delta(q, a) \subseteq Q$ to every pair $\langle q, a \rangle$ from $Q \times (\Sigma \cup \{\varepsilon\})$:

   $$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q.$$

   If $a \in \Sigma$, then $\delta(q, a)$ is the set of states the NFA can switch to after reading the character $a$ in state $q$. The set $\delta\big(q, \varepsilon\big)$ is the set of states that can be reached from the state $q$ without reading a character.

   As in the deterministic case, $\delta$ is called the transition function.
4. $q_0 \in Q$ is the start state.
5. $A \subseteq Q$ is the set of accepting states. ◇

If $F = \langle Q, \Sigma, \delta, q_0, A \rangle$ is an NFA and we have $q_2 \in \delta(q_1, \varepsilon)$, then the NFA has an $\varepsilon$-transition from the state $q_1$ into the state $q_2$. This is written as

$$q_1 \overset{\varepsilon}{\mapsto} q_2.$$

If $c \in \Sigma$ and $q_2 \in \delta(q_1, c)$, we write

$$q_1 \overset{c}{\mapsto} q_2.$$

**Example**: For the NFA $F$ shown in Figure 4.4 on page 63 we have

$$F = \langle Q, \Sigma, \delta, 0, A \rangle \quad \text{where}$$

1. $Q = \{0, 1, 2, 3\}$.
2. $\Sigma = \{\mathsf{a}, \mathsf{b}\}$.
3. $\delta = \big\{\langle 0, \mathsf{a}\rangle \mapsto \{0\}, \langle 0, \mathsf{b}\rangle \mapsto \{0, 1\}, \langle 0, \varepsilon\rangle \mapsto \{\}, \langle 1, \mathsf{a}\rangle \mapsto \{2\}, \langle 1, \mathsf{b}\rangle \mapsto \{2\}, \langle 1, \varepsilon\rangle \mapsto \{\},$
   $\langle 2, \mathsf{a}\rangle \mapsto \{3\}, \langle 2, \mathsf{b}\rangle \mapsto \{3\}, \langle 2, \varepsilon\rangle \mapsto \{\},$
   $\langle 3, \mathsf{a}\rangle \mapsto \{\}, \langle 3, \mathsf{b}\rangle \mapsto \{\}, \langle 3, \varepsilon\rangle \mapsto \{\}\big\}.$

   It is more convenient to specify the transition function $\delta$ as follows:

   $$\delta := \big\{0 \overset{\mathsf{a}}{\mapsto} 0,\ 0 \overset{\mathsf{b}}{\mapsto} 0,\ 0 \overset{\mathsf{b}}{\mapsto} 1,\ 1 \overset{\mathsf{a}}{\mapsto} 2,\ 1 \overset{\mathsf{b}}{\mapsto} 2,\ 2 \overset{\mathsf{a}}{\mapsto} 3,\ 2 \overset{\mathsf{b}}{\mapsto} 3\ \big\}.$$
4. The start state is $0$.

5. $A = \{3\}$, hence the only accepting state is $3$. ◇

In order to formally define how an NFA processes its input we introduce the notion of a configuration of an NFA. A configuration is defined as a pair

$$\langle q, s \rangle$$

where $q$ is a state and $s$ is a string. Here, $q$ is the current state of the NFA and $s$ is the part of the input that has not yet been consumed. We define a binary relation $\rightsquigarrow$ on configurations as follows:

$$\langle q_1, cs \rangle \rightsquigarrow \langle q_2, s \rangle \quad \text{iff} \quad q_1 \overset{c}{\mapsto} q_2, \quad \text{i.e. if } q_2 \in \delta(q_1, c).$$

Therefore, we have $\langle q_1, cs \rangle \rightsquigarrow \langle q_2, s \rangle$ if and only if the NFA transitions from the state $q_1$ into the state $q_2$ when the character $c$ is consumed. Furthermore, we have

$$\langle q_1, s \rangle \rightsquigarrow \langle q_2, s \rangle \quad \text{iff} \quad q_1 \overset{\varepsilon}{\mapsto} q_2, \quad \text{i.e. if } q_2 \in \delta(q_1, \varepsilon).$$

This accounts for the $\varepsilon$ transitions. The reflexive-transitive closure[1] of the relation $\rightsquigarrow$ is written as $\rightsquigarrow^*$. The language accepted by an NFA $F$ is denoted as $L(F)$ and is defined as

$$L(F) := \big\{ s \in \Sigma^* \mid \exists p \in A : \langle q_0, s \rangle \rightsquigarrow^* \langle p, \lambda \rangle \big\}.$$

Here, $q_0$ is the start state and $A$ is the set of accepting states. Hence, a string $s$ is an element of the language $L(F)$, iff there is an accepting state $p$ such that the configuration $\langle p, \lambda \rangle$ is reachable from the configuration $\langle q_0, s \rangle$.

**Example**: The NFA $F$ shown in Figure 4.4 accepts those strings $w \in \{\mathtt{a}, \mathtt{b}\}^*$ such that the antepenultimate character of $w$ is the character "b":

$$L(F) = \big\{ w \in \{\mathtt{a}, \mathtt{b}\}^* \;\big|\; |w| \geq 3 \wedge w[-3] = \mathtt{b} \big\}$$ ◇

**Exercise 6**: Specify an NFA $F$ such that $L(F)$ is the set of those strings from the language $\{a, b\}^*$ that contain the substring "aba". ◇

## 4.3 Transforming NFAs to DFAs

In this section we show how an NFA

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle$$

can be transformed into a DFA $\mathit{det}(F)$ such that both finite state machines accept the same language, i.e. we have

$$L(F) = L\big(\mathit{det}(F)\big)$$

The idea behind this transformation is that $\mathit{det}(F)$ has to compute the **set** of all states that the $F$ could be in after reading a string in the start state. Hence the states of $\mathit{det}(F)$ are **sets** of states of $F$. A set of these states contains all those states that $F$ could have reached when reading a character. Furthermore, a set $M$ of states of $F$ is an accepting state of $\mathit{det}(F)$ if the set $M$ contains an accepting state of $F$.

[1] The reflexive-transitive closure of a binary relation $R$ on a set $S$ is denoted $R^*$, and is defined as the smallest binary relation $T$ on $S$ such that

(a) $T$ contains $R$, i.e. $R \subseteq T$ holds,

(b) $T$ is reflexive on $S$, i.e. $\forall x \in S : \langle x, x \rangle \in T$ holds, and

(c) $T$ is transitive, i.e. $\forall x, y, z \in S : \big(\langle x, y \rangle \in T \wedge \langle y, z \rangle \in T \rightarrow \langle x, z \rangle \in T\big)$ holds.

The *Jupyter notebook* `Python/Chapter-04-05/ReflexiveTransitiveClosure.ipynb` shows how $R^*$ can be computed. As an application, it solves the puzzle of the missionaries and the infidels who have to cross a river.

In order to present the construction of $\mathit{det}(F)$ we first have to define two auxiliary functions. We start with the $\varepsilon$-closure of a given state. For every state $q$ of the NFA $F$ the function

$$\mathit{ec} : Q \to 2^Q$$

computes the set $\mathit{ec}(q)$ of all those states that $F$ can reach by $\varepsilon$ transitions from the state $q$. Formally, the set $\mathit{ec}(q)$ is computed inductively:

B.C.: $q \in \mathit{ec}(q)$.

I.S.: $p \in \mathit{ec}(q) \wedge r \in \delta(p, \varepsilon) \;\rightarrow\; r \in \mathit{ec}(q)$.

If the state $p$ is an element of the $\varepsilon$-closure of the state $q$ and there is an $\varepsilon$-transition from $p$ to some state $r$, then $r$ is also an element of the $\varepsilon$-closure of $q$.

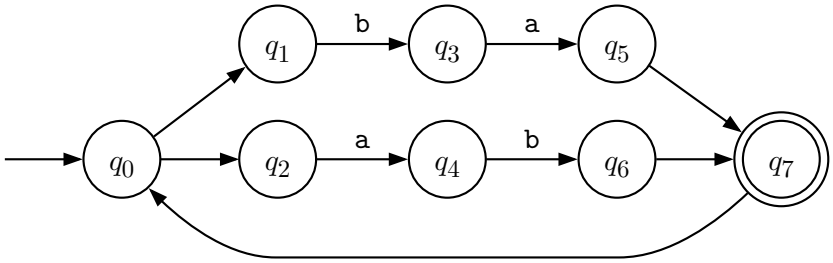



Figure 4.5: An NFA with $\varepsilon$-transitions.

**Example**: Figure 4.5 shows an NFA with $\varepsilon$-transitions. In this figure, the $\varepsilon$-transitions are shown as unlabelled arrows. We compute the $\varepsilon$-closure for all states:

1. $\mathit{ec}(q_0) = \{q_0, q_1, q_2\}$,
2. $\mathit{ec}(q_1) = \{q_1\}$,
3. $\mathit{ec}(q_2) = \{q_2\}$,
4. $\mathit{ec}(q_3) = \{q_3\}$,
5. $\mathit{ec}(q_4) = \{q_4\}$,
6. $\mathit{ec}(q_5) = \{q_5, q_7, q_0, q_1, q_2\}$,
7. $\mathit{ec}(q_6) = \{q_6, q_7, q_0, q_1, q_2\}$,
8. $\mathit{ec}(q_7) = \{q_7, q_0, q_1, q_2\}$. □

In order to transform an NFA into a DFA $\mathit{det}(F)$ we have to extend the function $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ into the function

$$\widehat{\delta} : Q \times \Sigma \to 2^Q.$$

The idea is that given a state $q$ and a character $c$, $\widehat{\delta}(q, c)$ is the set of all states that $F$ could reach when it reads the character $c$ in state $q$ and then performs an arbitrary number of $\varepsilon$-transitions. Formally, the definition of $\widehat{\delta}$ is as follows:

$$\widehat{\delta}(q_1, c) := \bigcup \{\mathit{ec}(q_2) \mid q_2 \in \delta(q_1, c)\}.$$

This formula is to be read as follows:

(a) For every state $q_2 \in Q$ that can be reached from the state $q_1$ by reading the character $c$ we compute the $\varepsilon$-closure $ec(q_2)$.

(b) Then we take the union of all the sets $ec(q_2)$ where $q_2 \in \delta(q_1, c)$.

**Example**: In continuation of the previous example (shown in Figure 4.5) we have:

1. $\widehat{\delta}(q_0, \mathsf{a}) = \{\}$,

   because in state $q_0$ there is no transition on reading the character a. Note that in our definition of the function $\widehat{\delta}$ the $\varepsilon$-transitions are done only after the character has been read.

2. $\widehat{\delta}(q_1, \mathsf{b}) = \{q_3\}$,

   because when the letter 'b' is read in the state $q_1$ the NFA switches into the state $q_3$ and the state $q_3$ has no $\varepsilon$-transitions.

3. $\widehat{\delta}(q_3, \mathsf{a}) = \{q_5, q_7, q_0, q_1, q_2\}$,

   because when the letter 'a' is read in the state $q_3$ the NFA switches into the state $q_5$. From $q_5$ the states $q_7$, $q_0$, $q_1$ and $q_2$ are reachable by $\varepsilon$-transitions. ◇

The function $\widehat{\delta}$ takes a state and a character and maps it into a set of states. Since $det(F)$ uses sets of states of $F$ as its states we need a function that takes a set of states and a character and maps it into a set of states. Hence we generalize the function $\widehat{\delta}$ to the function

$$\Delta : 2^Q \times \Sigma \to 2^Q$$

such that for a set $M$ of states and a character $c$ the expression $\Delta(M, c)$ computes the set of all those states that $F$ could be in if it is in a state from $M$, then reads the character $c$, and finally makes some $\varepsilon$-transitions. The formal definition is as follows:

$$\Delta(M, c) := \bigcup \Big\{ \widehat{\delta}(q, c) \mid q \in M \Big\}.$$

This formula is easy to understand: For every state $q \in M$ we compute the set of states that $F$ could be in after reading the character $c$ and doing some $\varepsilon$-transitions afterwards. Then we take the union of these sets.

**Example**: Continuing our previous example (shown in Figure 4.5) we have:

1. $\Delta(\{q_0, q_1, q_2\}, \mathsf{a}) = \{q_4\}$,
2. $\Delta(\{q_0, q_1, q_2\}, \mathsf{b}) = \{q_3\}$,
3. $\Delta(\{q_3\}, \mathsf{a}) = \{q_5, q_7, q_0, q_1, q_2\}$,
4. $\Delta(\{q_3\}, \mathsf{b}) = \{\}$,
5. $\Delta(\{q_4\}, \mathsf{a}) = \{\}$,
6. $\Delta(\{q_4\}, \mathsf{b}) = \{q_6, q_7, q_0, q_1, q_2\}$. ◇

Now we are ready to formally define how $det(F)$ is constructed from $F := \langle Q, \Sigma, \delta, q_0, A \rangle$.

$$det(F) := \langle 2^Q, \Sigma, \Delta, ec(q_0), \widehat{A} \rangle$$

where the components of this tuple are defined as follows:

1. The set of states of *det*$(F)$ is the set of all subsets of $Q$ and therefore it is equal to the power set $2^Q$.

   Later we will see that we do not need all of these subsets. The reason is that the states are those subsets that can be reached from the start state $q_0$ when some string has been read. In most cases there are some combinations of states that can not be reached and the corresponding sets are not really needed as states.

2. The input alphabet $\Sigma$ does not change when going from $F$ to *det*$(F)$. After all, the finite state machine *det*$(F)$ has to recognize the same language as $F$.

3. The previously defined function $\Delta$ specifies how the set of states changes when a character is read.

4. The start state *ec*$(q_0)$ of *det*$(F)$ is the set of all states that can be reached from the start state $q_0$ of $F$ via $\varepsilon$-transitions.

5. The set of accepting states $\widehat{A}$ is the set of those subsets of $Q$ that do contain an accepting state of $F$:

$$\widehat{A} := \big\{M \in 2^Q \mid M \cap A \neq \{\}\big\}.$$

**Exercise 7**: Transform the NFA $F$ that is shown in Figure 4.4 on page 63 into a DFA *det*$(F)$. ◇

**Solution**: We start by computing the set of states.

1. As we have *ec*$(0) = \{0\}$, the start state of *det*$(F)$ is the set containing $0$.

$$S_0 := \textit{ec}(0) = \{0\}.$$

2. As we have $\delta(0, \mathsf{a}) = \{0\}$ and there are no $\varepsilon$-transitions we have

$$\Delta(S_0, \mathsf{a}) = \Delta(\{0\}, \mathsf{a}) = \{0\} = S_0.$$

3. As we have $\delta(0, \mathsf{b}) = \{0, 1\}$ we conclude

$$S_1 := \Delta(S_0, \mathsf{b}) = \Delta(\{0\}, \mathsf{b}) = \{0, 1\}.$$

4. We have that $\delta(0, \mathsf{a}) = \{0\}$ and $\delta(1, \mathsf{a}) = \{2\}$. Hence

$$S_2 := \Delta(S_1, \mathsf{a}) = \Delta(\{0, 1\}, \mathsf{a}) = \{0, 2\}.$$

5. We have $\delta(0, \mathsf{b}) = \{0, 1\}$ and $\delta(1, \mathsf{b}) = \{2\}$. Therefore

$$S_3 := \Delta(S_1, \mathsf{b}) = \Delta(\{0, 1\}, \mathsf{b}) = \{0, 1, 2\}$$

   Similarly we derive the following:

6. $S_4 := \Delta(S_2, \mathsf{a}) = \Delta(\{0, 2\}, \mathsf{a}) = \{0, 3\}$.
7. $S_5 := \Delta(S_2, \mathsf{b}) = \Delta(\{0, 2\}, \mathsf{b}) = \{0, 1, 3\}$.
8. $S_6 := \Delta(S_3, \mathsf{a}) = \Delta(\{0, 1, 2\}, \mathsf{a}) = \{0, 2, 3\}$.
9. $S_7 := \Delta(S_3, \mathsf{b}) = \Delta(\{0, 1, 2\}, \mathsf{b}) = \{0, 1, 2, 3\}$.
10. $\Delta(S_4, \mathsf{a}) = \Delta(\{0, 3\}, \mathsf{a}) = \{0\} = S_0$.
11. $\Delta(S_4, \mathsf{b}) = \Delta(\{0, 3\}, \mathsf{b}) = \{0, 1\} = S_1$.
12. $\Delta(S_5, \mathsf{a}) = \Delta(\{0, 1, 3\}, \mathsf{a}) = \{0, 2\} = S_2$.

13. $\Delta(S_5, \mathsf{b}) = \Delta(\{0,1,3\}, \mathsf{b}) = \{0,1,2\} = S_3$.
14. $\Delta(S_6, \mathsf{a}) = \Delta(\{0,2,3\}, \mathsf{a}) = \{0,3\} = S_4$.
15. $\Delta(S_6, \mathsf{b}) = \Delta(\{0,2,3\}, \mathsf{b}) = \{0,1,3\} = S_5$.
16. $\Delta(S_7, \mathsf{a}) = \Delta(\{0,1,2,3\}, \mathsf{a}) = \{0,2,3\} = S_6$.
17. $\Delta(S_7, \mathsf{b}) = \Delta(\{0,1,2,3\}, \mathsf{b}) = \{0,1,2,3\} = S_7$.

These are all possible sets of states that the DFA $\mathit{det}(F)$ can reach. For a better overview let us summarize the definitions of the individual states of the DFA $\mathit{det}(F)$:

$$S_0 = \{0\},\ S_1 = \{0,1\},\ S_2 = \{0,2\},\ S_3 = \{0,1,2\},\ S_4 = \{0,3\},$$
$$S_5 = \{0,1,3\},\ S_6 = \{0,2,3\},\ S_7 = \{0,1,2,3\}$$

Therefore the set $\widehat{Q}$ of the states of $\mathit{det}(F)$ is given as follows:

$$\widehat{Q} := \{S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_7\}.$$

The transition function $\Delta$ is shown as a table:

| $\Delta$ | $S_0$ | $S_1$ | $S_2$ | $S_3$ | $S_4$ | $S_5$ | $S_6$ | $S_7$ |
|---|---|---|---|---|---|---|---|---|
| a | $S_0$ | $S_2$ | $S_4$ | $S_6$ | $S_0$ | $S_2$ | $S_4$ | $S_6$ |
| b | $S_1$ | $S_3$ | $S_5$ | $S_7$ | $S_1$ | $S_3$ | $S_5$ | $S_7$ |

Finally we recognize that only the sets $S_4$, $S_5$, $S_6$ and $S_7$ contain the accepting state $3$. Therefore we have

$$\widehat{A} := \{S_4, S_5, S_6, S_7\}.$$

Therefore we have now found $\mathit{det}(F)$. We have

$$\mathit{det}(F) := \langle \widehat{Q}, \Sigma, \Delta, S_0, \widehat{A} \rangle.$$

This DFA is shown in Figure 4.6 on page 70.

We realize that $\mathit{det}(F)$ has 8 different states. The NFA $F$ has 4 different states $Q = \{0,1,2,3\}$. Hence the power set $2^Q$ has 16 elements. Why then has $\mathit{det}(F)$ only 8 and not $2^4 = 16$ states? The reason is that we can only reach those sets of states from the start $0$ that contain the state $0$ because no matter whether we read an a or a b, $F$ can always choose to switch to the state $0$. Therefore, every set of states that is reachable from the state $0$ has to contain the state $0$. Therefore, sets that do not contain $0$ are not needed as states of $\mathit{det}(F)$.

**Exercise 8**: Transform the NFA $F$ that is shown in Figure 4.5 on page 66 into an equivalent DFA $\mathit{det}(F)$. ◇

### 4.3.1 Implementation

It is straightforward to implement the theory developed so far. The *Jupyter notebook*

`Python/Chapter-04-05/01-NFA-2-DFA.ipynb`

contains a program that takes a NFA $F$ and computes the DFA $\mathit{det}(F)$.

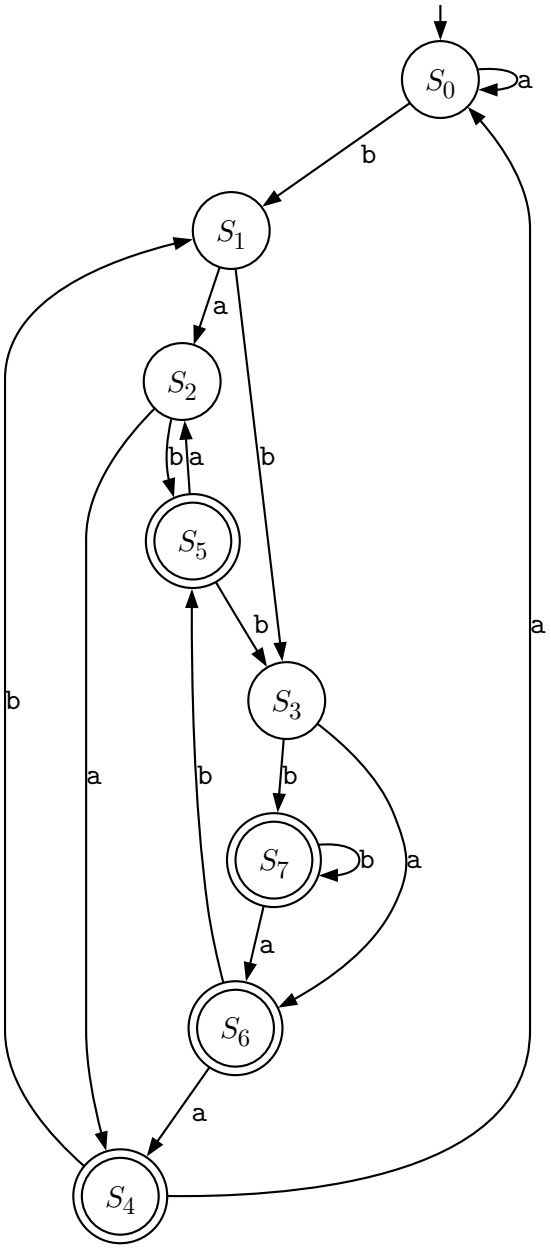



Figure 4.6: The DFA $det(F)$.

## 4.4 From Regular Expression to NFAs

In this section we show how a regular expression can be implemented as an NFA. Given a regular expression $r$ we will construct an NFA $A(r)$ that accepts the language that is described by the regular expression $r$, i.e. we will have

$$L\big(A(r)\big) = L(r).$$

The NFA $A(r)$ is defined by induction on the regular expression $r$. The NFA $A(r)$ will have the following properties:

1. $A(r)$ does not have a transition into its start state.

2. $A(r)$ has exactly one accepting state. Furthermore, there are no transitions out of this state.

In the following we assume that $\Sigma$ is the alphabet that has been used when constructing the regular expression $r$. Then we can define $A(r)$ as follows:

1. The NFA $A(\emptyset)$ is defined as

   $$A(\emptyset) = \big\langle \{q_0, q_1\}, \Sigma, \{\}, q_0, \{q_1\} \big\rangle.$$

   Note that this NFA has no transitions at all.

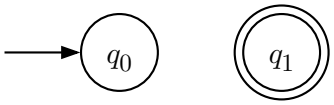



Figure 4.7: The NFA $A(\emptyset)$.

   Figure 4.7 shows the NFA $A(\emptyset)$. It is obvious that we have $L\big(A(\emptyset)\big) = \{\}$.

2. The NFA $A(\varepsilon)$ is defined as

   $$A(\varepsilon) = \big\langle \{q_0, q_1\}, \Sigma, \big\{\langle q_0, \varepsilon\rangle \mapsto \{q_1\}\big\}, q_0, \{q_1\} \big\rangle.$$

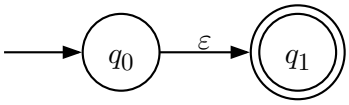



Figure 4.8: The NFA $A(\varepsilon)$.

   Figure 4.8 shows the NFA $A(\varepsilon)$. We have that $L\big(A(\varepsilon)\big) = \{\lambda\}$, i.e. the NFA accepts only the empty string.

3. For a letter $c \in \Sigma$ the NFA $A(c)$ is defined as

   $$A(c) = \big\langle \{q_0, q_1\}, \Sigma, \big\{\langle q_0, c\rangle \mapsto \{q_1\}\big\}, q_0, \{q_1\} \big\rangle.$$

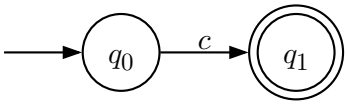



Figure 4.9: The NFA $A(c)$.

   Figure 4.9 shows $A(c)$. We have that $L\big(A(c)\big) = \{c\}$, i.e. the NFA accepts only the character $c$.

4. In order to define the NFA $A(r_1 \cdot r_2)$ for the concatenation $r_1 \cdot r_2$ we assume that we have already constructed NFAs $A(r_1)$ and $A(r_2)$ such that $L\big(A(r_1)\big) = L(r_1)$ and $L\big(A(r_2)\big) = L(r_2)$. Furthermore, without loss of generality we assume that the states in the NFAs $A(r_1)$ and $A(r_2)$ are different. This can always be achieved by renaming the states of $A(r_2)$. Next, we assume that $A(r_1)$ and $A(r_2)$ have the following form:

   (a) $A(r_1) = \big\langle Q_1, \Sigma, \delta_1, q_1, \{q_2\} \big\rangle$,

   (b) $A(r_2) = \big\langle Q_2, \Sigma, \delta_2, q_3, \{q_4\} \big\rangle$,

   (c) $Q_1 \cap Q_2 = \{\}$.

Then we can build the NFA $A(r_1 \cdot r_2)$ from $A(r_1)$ and $A(r_2)$ as follows:

$$A(r_1 \cdot r_2) := \big\langle Q_1 \cup Q_2, \Sigma, \big\{\langle q_2, \varepsilon\rangle \mapsto \{q_3\}\big\} \cup \delta_1 \cup \delta_2, q_1, \{q_4\}\big\rangle$$

Here, the notation $\{\langle q_2, \varepsilon\rangle \mapsto \{q_3\}\} \cup \delta_1 \cup \delta_2$ specifies that $A(r_1 \cdot r_2)$ contains all transitions from both $A(r_1)$ and $A(r_2)$ and, furthermore, contains an $\varepsilon$-transition from $q_2$ to $q_3$. Formally, this transition function $\delta$ can be specified as follows:

$$\delta(q, c) := \begin{cases} \{q_3\} & \text{if } q = q_2 \text{ and } c = \varepsilon, \\ \delta_1(q, c) & \text{if } q \in Q_1 \text{ and } \langle q, c\rangle \neq \langle q_2, \varepsilon\rangle, \\ \delta_2(q, c) & \text{if } q \in Q_2. \end{cases}$$

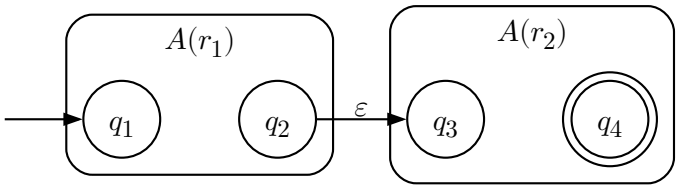



Figure 4.10: The NFA $A(r_1 \cdot r_2)$.

Figure 4.10 shows the NFA $A(r_1 \cdot r_2)$.

Instead of having an $\varepsilon$-transition from $q_2$ to $q_3$ we can identify the states $q_2$ and $q_3$. The advantage is that the resulting NFA is smaller. We will do this when creating NFAs by hand.

I haven't done this identification in the definition above because both the graphical representation and the implementation get more complicated if we identify these states.

5. In order to define the NFA $A(r_1 + r_2)$ we assume that we have already constructed NFAs $A(r_1)$ and $A(r_2)$ such that $L\big(A(r_1)\big) = L(r_1)$ and $L\big(A(r_2)\big) = L(r_2)$. Furthermore, without loss of generality we assume that the states in the NFAs $A(r_1)$ and $A(r_2)$ are different and that $A(r_1)$ and $A(r_2)$ have the following form:

   (a) $A(r_1) = \big\langle Q_1, \Sigma, \delta_1, q_1, \{q_3\}\big\rangle$,
   (b) $A(r_2) = \big\langle Q_2, \Sigma, \delta_2, q_2, \{q_4\}\big\rangle$,
   (c) $Q_1 \cap Q_2 = \{\}$.

   Then the NFA $A(r_1 + r_2)$ is defined as follows:

   $$\big\langle \{q_0, q_5\} \cup Q_1 \cup Q_2, \Sigma, \big\{\langle q_0, \varepsilon\rangle \mapsto \{q_1, q_2\}, \langle q_3, \varepsilon\rangle \mapsto \{q_5\}, \langle q_4, \varepsilon\rangle \mapsto \{q_5\}\big\} \cup \delta_1 \cup \delta_2, q_0, \{q_5\}\big\rangle$$

   Figure 4.11 shows the NFA $A(r_1 + r_2)$. In addition to the states of $A(r_1)$ and $A(r_2)$ there are two more states:

   (a) $q_0$ is the start state of the NFA $A(r_1 + r_2)$,
   (b) $q_5$ is the only accepting state of the NFA $A(r_1 + r_2)$.

   In addition to the transitions of $A(r_1)$ and $A(r_2)$ the NFA $A(r_1 + r_2)$ has four more $\varepsilon$-transitions.

   (a) The new start state $q_0$ has two $\varepsilon$-transitions leading to the start states $q_1$ and $q_2$ of the NFAs $A(r_1)$ and $A(r_2)$.
   (b) Each of the accepting states $q_3$ and $q_4$ of the NFAs $A(r_1)$ and $A(r_2)$ has an $\varepsilon$-transition to the new accepting state $q_5$.

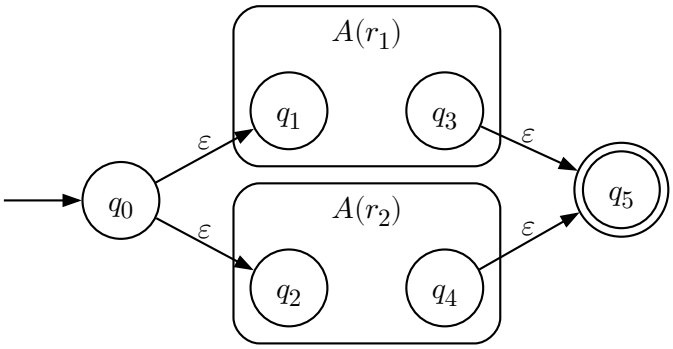



Figure 4.11: The NFA $A(r_1 + r_2)$.

In order to simplify this NFA we could identify the three states $q_0$, $q_1$ and $q_2$ and the three states $q_3$, $q_4$ and $q_5$. However, the resulting NFA would be more difficult to understand and hence we are not doing this when creating NFAs by hand.

6. In order to define the NFA $A(r^*)$ we assume that we have already constructed an NFA $A(r)$ such that $L\big(A(r)\big) = L(r)$ and that $A(r)$ has the form

$$A(r) = \big\langle Q, \Sigma, \delta, q_1, \{q_2\}\big\rangle.$$

Then $A(r^*)$ is defined as follows:

$$\big\langle \{q_0, q_3\} \cup Q, \Sigma, \big\{\langle q_0, \varepsilon\rangle \mapsto \{q_1, q_3\}, \langle q_2, \varepsilon\rangle \mapsto \{q_1, q_3\}\big\} \cup \delta, q_0, \{q_3\}\big\rangle.$$

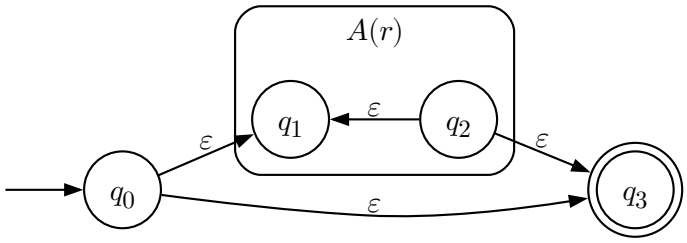



Figure 4.12: The NFA $A(r^*)$.

Figure 4.12 shows the NFA $A(r^*)$. In comparison with $A(r)$ this NFA has two additional states.

(a) $q_0$ is the start state of $A(r^*)$,
(b) $q_3$ is the only accepting state of $A(r^*)$.

The NFA $A(r^*)$ has four more $\varepsilon$-transitions than $A(r)$:

(a) The new start state $q_0$ has $\varepsilon$-transitions to the states $q_1$ and $q_3$.
(b) $q_2$ has an $\varepsilon$-transition back to the state $q_1$.
(c) $q_2$ also has an $\varepsilon$-transition to the state $q_3$.

**Attention**: If we would identify the two states $q_0$ and $q_1$ and the two states $q_2$ and $q_3$, then the resulting NFA would no longer be correct!

**Exercise 9**: Construct an NFA for the regular expression

$$(\mathsf{a} + \mathsf{b}) \cdot \mathsf{a}^* \cdot \mathsf{b}. \qquad \diamond$$

### 4.4.1 Implementation

The *Jupyter notebook*

Python/Chapter-04-05/03-Regexp-2-NFA.ipynb

implements the theory discussed in this section.

## 4.5 Translating a DFA into a Regular Expression

In this last section we start with a DFA $F$ and construct a regular expression $r$ such that we have

$$L(r) = L(F).$$

We assume that the DFA $F$ is given as follows:

$$F = \big\langle \{q_0, q_1, \cdots, q_n\}, \Sigma, \delta, q_0, A \big\rangle.$$

For every pair of states $\langle p_1, p_2 \rangle \in Q \times Q$ we define a regular expression $r(p_1, p_2)$ such that $r(p_1, p_2)$ describes those strings $w$ that take the DFA $F$ from the state $p_1$ to the state $p_2$, i.e. we have

$$L\big(r(p_1, p_2)\big) = \big\{w \in \Sigma^* \mid \langle p_1, w\rangle \rightsquigarrow^* \langle p_2, \lambda\rangle\big\}.$$

The definition of $r(p_1, p_2)$ is done by first defining auxiliary regular expressions $r^{(k)}(p_1, p_2)$ for all $k = 0, \cdots, n+1$. The regular expression $r^{(k)}(p_1, p_2)$ specifies those strings that take the DFA $F$ from the state $p_1$ to the state $p_2$ while only visiting states from the set

$$Q_k := \big\{q_i \mid i \in \{0, \cdots, k-1\}\big\}$$

in between. Observe that

$$Q_0 = \{\} \quad \text{and} \quad Q_{n+1} = \{q_0, \cdots, q_n\} = Q.$$

To this end we define the ternary relation

$$\mapsto_k \subseteq (Q \times \Sigma^* \times Q).$$

Given two states $p, q \in Q$ and a string $w$ we have that

$$p \overset{w}{\mapsto}_k q$$

holds iff the DFA $F$ switches from the state $p$ to the state $q$ when it reads the string $w$ but on reading $w$ only visits states from the set $Q_k$ in-between. Here, "in-between" specifies that the states $p$ and $q$ may well be outside of the set $Q_k$, only the states between $p$ and $q$ are required to be elements of $Q_k$. The formal definition of $p \overset{w}{\mapsto}_k q$ is done by induction on $w$:

B.C.: $|w| \leq 1$. Then there are two cases:

(a) $p \overset{\lambda}{\mapsto}_k p$,
because when the empty string is read we can only reach the state $p$ if we start in the state $p$.

(b) $\delta(p, c) = q \;\Rightarrow\; p \overset{c}{\mapsto}_k q$,
because when the DFA reads the character $c$ and switches from state $p$ directly to the state $q$, there are no states "inbetween".

I.S.: $w = cv$ where $|v| \geq 1$.

Here we have: $p \overset{c}{\mapsto} q \wedge q \in Q_k \wedge q \overset{v}{\mapsto}_k r \;\Rightarrow\; p \overset{cv}{\mapsto}_k r$.

Now we are ready to define the regular expressions $r^{(k)}(p_1, p_2)$ for all $k = 0, \cdots, n+1$. This definition will be done such that we have

$$L\big(r^{(k)}(p_1, p_2)\big) = \big\{w \in \Sigma^* \mid p_1 \overset{w}{\mapsto}_k p_2\big\}.$$

The definition of the regular expressions $r^{(k)}(p_1, p_2)$ is done by induction on $k$.

B.C.: $k = 0$.

Then we have $Q_0 = \{\}$. Therefore, when the DFA switches from the state $p_1$ to the state $p_2$ it must not visit any states in-between. There are two cases.

(a) $p_1 \neq p_2$: Then we can have $p_1 \overset{w}{\mapsto}_0 p_2$ only if $w$ contains but a single letter. Define

$$\{c_1, \cdots, c_l\} := \{c \in \Sigma \mid \delta(p_1, c) = p_2\}$$

as the set of all letters that take the DFA from the state $p_1$ to the state $p_2$. If this set is not empty we define

$$r^{(0)}(p_1, p_2) := c_1 + \cdots + c_l.$$

If this set is empty, then there is no direct transition from $p_1$ to $p_2$ and we define

$$r^{(0)}(p_1, p_2) := \emptyset.$$

(b) $p_1 = p_2$: Again we define

$$\{c_1, \cdots, c_l\} := \{c \in \Sigma \mid \delta(p_1, c) = p_1\}.$$

If this set is not empty we have

$$r^{(0)}(p_1, p_1) := c_1 + \cdots + c_l + \varepsilon.$$

Otherwise we have

$$r^{(0)}(p_1, p_1) := \varepsilon.$$

I.S.: $k \mapsto k + 1$.

We have that

$$Q_{k+1} = Q_k \cup \{q_k\}$$

Therefore, when compared to the regular expression $r^{(k)}(p_1, p_2)$, the regular expression $r^{(k+1)}(p_1, p_2)$ describes strings that cause the DFA to potentially visit the state $q_k$ in addition to the states from the set $Q_k$. If the DFA reads a string $w$ that switches the state $p_1$ to the state $p_2$ while only using states from the set $Q_{k+1}$, then there are two cases.

(a) We already have $p_1 \overset{w}{\mapsto}_k p_2$, i.e. the DFA uses only states from $Q_k$ and the state $q_k$ isn't used.

(b) The string $w$ can be written as $w = w_1 s_1 \cdots s_l w_2$ where we have:

- $p_1 \overset{w_1}{\mapsto}_k q_k$,
- $q_k \overset{s_i}{\mapsto}_k q_k$ for all $i \in \{1, \cdots, l\}$,
- $q_k \overset{w_2}{\mapsto}_k p_2$.

Therefore we define

$$r^{(k+1)}(p_1, p_2) := r^{(k)}(p_1, p_2) + r^{(k)}(p_1, q_k) \cdot \big(r^{(k)}(q_k, q_k)\big)^* \cdot r^{(k)}(q_k, p_2).$$

Now we are ready to define the regular expressions $r(p_1, p_2)$ for all states $p_1$ and $p_2$:

$$r(p_1, p_2) := r^{(n+1)}(p_1, p_2).$$

This regular expression specifies all strings that take the DFA from the state $p_1$ to the state $p_2$ while using any state from the set $Q_{n+1}$ in-between. Since we have

$$Q_{n+1} = \big\{q_0, \cdots, q_n\big\} = Q$$

we can use any state in-between.

In order to construct a regular expression that specifies the language accepted by a DFA $F$ we write the set $A$ of accepting states of $F$ as

$$A = \{t_1, \cdots, t_m\}.$$

Then the regular expression $r(F)$ is defined as

$$r(F) := r(q_0, t_1) + \cdots + r(q_0, t_m).$$

This regular expression specifies those strings that take the DFA $F$ from its start state $q_0$ into any of its accepting states. If $F$ has no accepting states at all, then $F$ does not accept any string and we define $r(F) := \emptyset$. ◇

**Exercise 10**: Take the DFA shown in Figure 4.2 and construct an equivalent regular expression.

**Solution**: The DFA has two states: $0$ and $1$. We start by computing the regular expressions $r^{(k)}(i, j)$ for all $i, j \in \{0, 1\}$ for $k = 0$, 1, and 2:

1. For $k = 0$ we have:
   (a) $r^{(0)}(0, 0) = \mathsf{a} + \varepsilon$,
   (b) $r^{(0)}(0, 1) = \mathsf{b}$,
   (c) $r^{(0)}(1, 0) = \emptyset$,
   (d) $r^{(0)}(1, 1) = \mathsf{a} + \varepsilon$.
2. For $k = 1$ we have:
   (a) For $r^{(1)}(0, 0)$ we have:
   $$\begin{array}{rcl} r^{(1)}(0,0) & = & r^{(0)}(0,0) + r^{(0)}(0,0) \cdot \big(r^{(0)}(0,0)\big)^* \cdot r^{(0)}(0,0) \\ & \doteq & r^{(0)}(0,0) \cdot \big(r^{(0)}(0,0)\big)^* \end{array}$$
   In the last step we used the fact that
   $$\begin{array}{rcl} r + r \cdot r^* \cdot r & \doteq & r \cdot (\varepsilon + r^* \cdot r) \\ & \doteq & r \cdot r^* \end{array}$$
   to simplify the result. If we substitute for $r^{(0)}(0, 0)$ the expression $\mathsf{a} + \varepsilon$ we get
   $$r^{(1)}(0, 0) \doteq (\mathsf{a} + \varepsilon) \cdot (\mathsf{a} + \varepsilon)^*.$$
   As we have $(\mathsf{a} + \varepsilon) \cdot (\mathsf{a} + \varepsilon)^* \doteq \mathsf{a}^*$ we have
   $$r^{(1)}(0, 0) \doteq \mathsf{a}^*.$$
   (b) For $r^{(1)}(0, 1)$ we have:
   $$\begin{array}{rcl} r^{(1)}(0,1) & \doteq & r^{(0)}(0,1) + r^{(0)}(0,0) \cdot \big(r^{(0)}(0,0)\big)^* \cdot r^{(0)}(0,1) \\ & \doteq & \mathsf{b} + (\mathsf{a} + \varepsilon) \cdot (\mathsf{a} + \varepsilon)^* \cdot \mathsf{b} \\ & \doteq & \mathsf{b} + \mathsf{a}^* \cdot \mathsf{b} \\ & \doteq & (\varepsilon + \mathsf{a}^*) \cdot \mathsf{b} \\ & \doteq & \mathsf{a}^* \cdot \mathsf{b} \end{array}$$
   (c) For $r^{(1)}(1, 0)$ we have:
   $$\begin{array}{rcl} r^{(1)}(1,0) & \doteq & r^{(0)}(1,0) + r^{(0)}(1,0) \cdot \big(r^{(0)}(0,0)\big)^* \cdot r^{(0)}(0,0) \\ & \doteq & \emptyset + \emptyset \cdot (\mathsf{a} + \varepsilon)^* \cdot (\mathsf{a} + \varepsilon) \\ & \doteq & \emptyset \end{array}$$

(d) For $r^{(1)}(1,1)$ we have

$$\begin{aligned} r^{(1)}(1,1) &\doteq r^{(0)}(1,1) + r^{(0)}(1,0) \cdot \big(r^{(0)}(0,0)\big)^* \cdot r^{(0)}(0,1) \\ &\doteq (\mathsf{a} + \varepsilon) + \emptyset \cdot (\mathsf{a} + \varepsilon)^* \cdot \mathsf{b} \\ &\doteq (\mathsf{a} + \varepsilon) + \emptyset \\ &\doteq \mathsf{a} + \varepsilon \end{aligned}$$

3. For $k = 2$ we only have to compute the regular expression $r^{(2)}(0,1)$:

$$\begin{aligned} r^{(2)}(0,1) &\doteq r^{(1)}(0,1) + r^{(1)}(0,1) \cdot \big(r^{(1)}(1,1)\big)^* \cdot r^{(1)}(1,1) \\ &\doteq \mathsf{a}^* \cdot \mathsf{b} + \mathsf{a}^* \cdot \mathsf{b} \cdot (\mathsf{a} + \varepsilon)^* \cdot (\mathsf{a} + \varepsilon) \\ &\doteq \mathsf{a}^* \cdot \mathsf{b} + \mathsf{a}^* \cdot \mathsf{b} \cdot \mathsf{a}^* \\ &\doteq \mathsf{a}^* \cdot \mathsf{b} \cdot (\varepsilon + \mathsf{a}^*) \\ &\doteq \mathsf{a}^* \cdot \mathsf{b} \cdot \mathsf{a}^*. \end{aligned}$$

As the state $\mathtt{0}$ is the start state and the state $1$ is the only accepting state we have

$$r(F) = r^{(2)}(0,1) \doteq \mathsf{a}^* \cdot \mathsf{b} \cdot \mathsf{a}^*. \qquad \square$$

**Exercise 11**: Take the DFA shown in Figure 4.13 on page 77 and construct a regular expression specifying the same language as this DFA.

**Hint**: In order to avoid unnecessary computations, you should only compute those regular expression $r^{(k)}(p,q)$ that are needed in order to compute $r^{(3)}(q_0, q_2)$. $\diamond$

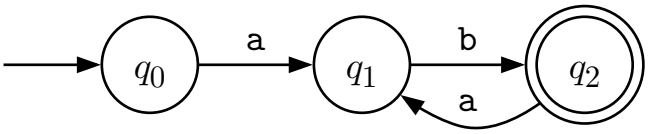



Figure 4.13: A DFA.

### 4.5.1 Implementation

The *Jupyter notebook*

Python/Chapter-04-05/05-DFA-2-RegExp.ipynb

contains a program that takes a DFA $F$ and computes a regular expression $r$ such that $r$ specifies the language accepted by $F$, i.e. we have $L(r) = L(F)$.

## 4.6 Minimization of Finite State Machines

In this section we show how to minimize the number of states of a DFA

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle.$$

Without loss of generality we want to assume that the DFA $F$ is complete: Therefore, we assume that for each state $q \in Q$ and each letter $c \in \Sigma$ the expression $\delta(q, c)$ returns a state of $Q$. Our goal is to find a complete DFA

$$\mathtt{Min}(F) := \big\langle \mathcal{Q}_\sim, \Sigma, \Delta, [q_0]_\sim, \widehat{A} \big\rangle$$

which accepts the same language as $F$, so we have

$$L\big(\texttt{Min}(F)\big) = L(F)$$

and for which the number of states of the set $\mathcal{Q}_\sim$ is minimal. You might be wondering about the meaning associated with $\mathcal{Q}_\sim$ and $[q_0]_\sim$: We will introduce an equivalence relation[2] $\sim$ on the set of states $Q$. Basically, $p_1, p_2 \in Q$ are equivalent if they serve the same purpose in the DFA $F$, whatever that means. If $p_1$ and $p_2$ are equivalent, this will be written as

$$p_1 \sim p_2.$$

Given a state $p \in Q$, the equivalence class of $p$ w.r.t. the relation $\sim$, written $[p]_\sim$ is the set of all states that are equivalent to $p$, i.e. we have

$$[p]_\sim := \{q \in Q \mid p \sim q\}.$$

The new states of the DFA $\texttt{Min}(F)$ will be equivalence classes of the equivalence relation $\sim$. Therefore we have

$$\mathcal{Q}_\sim := \big\{[q]_\sim \;\big|\; q \in Q\big\}.$$

The new start state $[q_0]_\sim$ is the set of all states that are equivalent to the state $q_0$. The formal definitions of the equivalence relation $\sim$, the function $\Delta : \mathcal{Q}_\sim \times \Sigma \to \mathcal{Q}_\sim$ and the set $\widehat{A}$ will be given later.

We start with the function

$$\delta^* : Q \times \Sigma^* \to Q$$

which extends the function $\delta$ by accepting a string instead of a single character as its second argument. The function call $\delta^*(q, s)$ calculates the state $p$ which the DFA $F$ enters if it reads the string $s$ in the state $q$. As we have assumed the DFA $F$ to be complete, we have that

$$\delta(q, c) \neq \Omega \quad \text{for all } q \in Q \text{ and all } c \in \Sigma$$

and then $\delta^*(q, w)$ is defined by induction on $w$ as follows:

(a) $\delta^*(q, \lambda) := q$ for all $q \in Q$.

(b) $\delta^*(q, cv) := \delta^*(\delta(q, c), v)$ for all $q \in Q$, $c \in \Sigma$, and $v \in \Sigma^*$.

Obviously, in a DFA $F = \langle Q, \Sigma, \delta, q_0, A\rangle$ we can remove all the states $p \in Q$ which are not reachable from the start state. A state $p$ is reachable iff a string $w \in \Sigma^*$ is given, such that

$$\delta^*(q_0, w) = p$$

holds. It is clear that an unreachable state is useless. Therefore, w.l.o.g. we assume in the following that all states of the DFA $F = \langle Q, \Sigma, \delta, q_0, A\rangle$ are reachable.

In general, we can minimize a DFA by identifying certain states. If we look for example at the DFA shown in figure 4.14, we can identify the states $q_1$ and $q_2$ as well as $q_3$ and $q_4$ without changing the language of the DFA. The resulting DFA is shown in Figure 4.15 on page 81. The central idea in minimizing a DFA is that we compute those states which **must not** be identified and consider all other states as equivalent. States that must not be identified are called separable states. This notion is defined next.

[2] A binary relation $R \subseteq M \times M$ is an equivalence relation on $M$ iff it has the following properties:

(a) $R$ is reflexive on $M$, i.e. $\forall x \in M : \langle x, x\rangle \in R$.

(b) $R$ is symmetric, i.e. $\forall x, y \in M : \big(\langle x, y\rangle \in R \to \langle y, x\rangle \in R\big)$.

(c) $R$ is transitive, i.e. $\forall x, y, z \in M : \big(\langle x, y\rangle \in R \wedge \langle y, z\rangle \in R \to \langle x, z\rangle \in R\big)$.

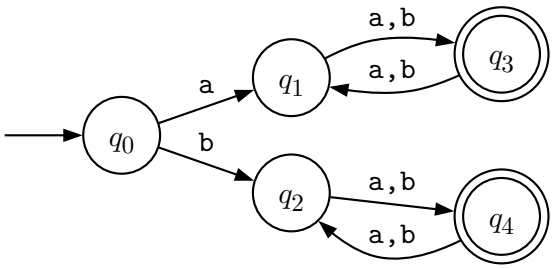



Figure 4.14: A DFA with equivalent states.

**Definition 13 (Separable States)** Assume $F = \langle Q, \Sigma, \delta, q_0, A\rangle$ is a DFA. Two states $p_1, p_2 \in Q$ are called separable if and only if there exists a string $s \in \Sigma^*$ such that either

1. $\delta^*(p_1, s) \in A$ and $\delta^*(p_2, s) \notin A$ or
2. $\delta^*(p_1, s) \notin A$ and $\delta^*(p_2, s) \in A$

holds. In this case, the string $s$ separates the states $p_1$ and $p_2$. ◇

If two states $p_1$ and $p_2$ are separable, then these states must not be identified since doing so would change the language that is accepted. We define an equivalence relation $\sim$ on the set $Q$ of all states by setting

$$p_1 \sim p_2 \quad \text{iff} \quad \forall s \in \Sigma^* : \big(\delta^*(p_1, s) \in A \leftrightarrow \delta^*(p_2, s) \in A\big).$$

Hence, two states $p_1$ and $p_2$ are considered to be equivalent iff they are not separable. The claim is that we can identify all pairs $\langle p_1, p_2\rangle$ of equivalent states.

The question remains how we can determine which states are separable. One possibility is to create a set $V$ with pairs of states. We add the pair $\langle p, q\rangle$ to the set $V$ if we have discovered that $p$ and $q$ are separable. We recognize $p$ and $q$ as separable if there is a letter $c \in \Sigma$ and two states $s$ and $t$ that are already known to be separable such that

$$\delta(p, c) = s, \quad \delta(q, c) = t, \quad \text{and} \quad \langle s, t\rangle \in V$$

holds. Formally, we define a sequence $(V_k)_{k\in\mathbb{N}}$ of sets of pairs of separable states by induction.

I.A.: $k = 0$

Since an accepting state can never be equivalent to a non-accepting state we define

$$V_0 := \big(A \times (Q \backslash A)\big) \cup \big((Q \backslash A) \times A\big).$$

Therefore, initially only the accepting states are separable from the states that are not accepting.

I.S.: $k \mapsto k + 1$

If we have a pair $\langle p, q\rangle \in Q \times Q$ and a letter $c$ such that the pair of states $\langle \delta(p, c), \delta(q, c)\rangle$ is already separable, then $\langle p, q\rangle$ is separable, too. Therefore we define:

$$V_{k+1} := V_k \cup \big\{\langle p, q\rangle \;\big|\; \exists c \in \Sigma : \langle \delta(p, c), \delta(q, c)\rangle \in V_k\big\}$$

The sequence $(V_k)_{k\in\mathbb{N}}$ is monotonically increasing, i.e. we have

$$V_k \subseteq V_{k+1}.$$

Since $V_k \subseteq Q \times Q$ and the set $Q \times Q$ is finite, there must be a $K \in \mathbb{N}$ such that

$$V_{K+1} = V_K.$$

We can therefore define two states $p$ and $q$ to be equivalent iff $\langle p, q\rangle \notin V_K$:

$$p \sim q \overset{\text{def}}{\iff} \langle p, q\rangle \notin V_K.$$

Next, for a state $p \in Q$ we define the equivalence class of $p$ as the set

$$[p]_\sim := \{q \in Q \mid p \sim q\}.$$

We define the set $\mathcal{Q}_\sim$ of all equivalence classes as

$$\mathcal{Q}_\sim := \big\{[q]_\sim \;\big|\; q \in Q\big\}.$$

Next, we define the function $\Delta : \mathcal{Q}_\sim \times \Sigma \to \mathcal{Q}_\sim$ as follows:

$$\Delta\big([q]_\sim, c\big) := \big[\delta(q, c)\big]_\sim.$$

Finally, we define the set of accepting equivalence classes $\widehat{A}$ as

$$\widehat{A} := \big\{[q]_\sim \;\big|\; q \in A\big\}.$$

Then, the DFA

$$\texttt{Min}(F) := \big\langle \mathcal{Q}_\sim, \Sigma, \Delta, [q_0]_\sim, \widehat{A}\big\rangle$$

is the minimal complete DFA that accepts the same language as the DFA $F$.

**Example**: We consider the DFA shown in figure 4.14 and apply the algorithm described above to this DFA. We use a table for this. The columns and rows of this table are numbered with the different states. If we have recognized in the first step that the states $i$ and $j$ are separable, we insert a $1$ in this table in the $i$-th row and the $j$-th column. Furthermore, since the states $i$ and $j$ can also be distinguished from the states $j$ and $i$, we also insert a $1$ in the $j$-th row and the $i$-th column.

1. In the first step we recognize that the two accepting states $q_3$ and $q_4$ are separable from all non-accepting states. Therefore the pairs $\langle q_0, q_3\rangle$, $\langle q_0, q_4\rangle$, $\langle q_1, q_3\rangle$, $\langle q_1, q_4\rangle$, $\langle q_2, q_3\rangle$ and $\langle q_2, q_4\rangle$ are separable. Hence the table now has the following configuration:

| | $q_0$ | $q_1$ | $q_2$ | $q_3$ | $q_4$ |
|---|---|---|---|---|---|
| $q_0$ | | | | 1 | 1 |
| $q_1$ | | | | 1 | 1 |
| $q_2$ | | | | 1 | 1 |
| $q_3$ | 1 | 1 | 1 | | |
| $q_4$ | 1 | 1 | 1 | | |

2. Next we see that the states $q_0$ and $q_1$ are separable because

   $$\delta(q_0, a) = q_1, \quad \delta(q_1, a) = q_3 \quad \text{and} \quad q_1 \not\sim q_3.$$

   Also we see that the states $q_0$ and $q_2$ are separable, because

   $$\delta(q_0, b) = q_2, \quad \delta(q_2, b) = q_4 \quad \text{and} \quad q_2 \not\sim q_4.$$

   Since we have found in the second step that $q_0 \not\sim q_1$ and $q_0 \not\sim q_2$ holds true, we will insert a $2$ at the corresponding positions in the table. Therefore the table now has the following form:

| | $q_0$ | $q_1$ | $q_2$ | $q_3$ | $q_4$ |
|---|---|---|---|---|---|
| $q_0$ | | 2 | 2 | 1 | 1 |
| $q_1$ | 2 | | | 1 | 1 |
| $q_2$ | 2 | | | 1 | 1 |
| $q_3$ | 1 | 1 | 1 | | |
| $q_4$ | 1 | 1 | 1 | | |

3. Now we do not find any more pairs of separable states, because if we take a look at the pair $\langle q_1, q_2 \rangle$ we can see that

   $$\delta(q_1, \mathsf{a}) = q_3 \quad \text{and} \quad \delta(q_2, \mathsf{a}) = q_4,$$

   but because the states $q_3$ and $q_4$ are not separable so far, this does not yield a new separable pair. Neither does

   $$\delta(q_1, \mathsf{b}) = q_3 \quad \text{and} \quad \delta(q_2, \mathsf{b}) = q_4$$

   yield a new separable pair. At this point, the two states $q_3$ and $q_4$ remain. We find

   $$\delta(q_3, c) = q_1 \quad \text{and} \quad \delta(q_4, c) = q_2 \quad \text{for all } c \in \{\mathsf{a}, \mathsf{b}\}$$

   and because the states $q_1$ and $q_2$ are not yet known to be separable, we have not found any new separable states. Therefore, we can complete the table as shown below:

| | $q_0$ | $q_1$ | $q_2$ | $q_3$ | $q_4$ |
|---|---|---|---|---|---|
| $q_0$ | $\sim$ | 2 | 2 | 1 | 1 |
| $q_1$ | 2 | $\sim$ | $\sim$ | 1 | 1 |
| $q_2$ | 2 | $\sim$ | $\sim$ | 1 | 1 |
| $q_3$ | 1 | 1 | 1 | $\sim$ | $\sim$ |
| $q_4$ | 1 | 1 | 1 | $\sim$ | $\sim$ |

   Next, we read the equivalent states from the table and see that:

   (a) $q_1 \sim q_2$,

   (b) $q_3 \sim q_4$.

   Figure 4.15 shows the corresponding reduced finite DFA.

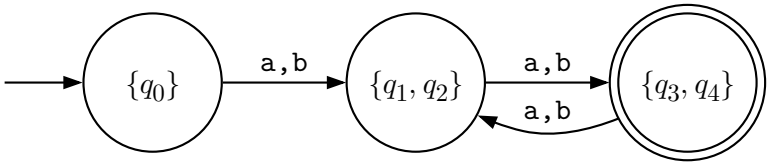



Figure 4.15: The reduced DFA.

**Exercise 12**: Construct the minimal DFA that recognizes the language $L\big(a \cdot (b \cdot a)^*\big)$. Perform the following steps:

(a) Construct an NFA which recognizes this language.

(b) Transform this NFA into a DFA.

(c) Minimize the number of states of this DFA with the algorithm given above.

### 4.6.1 Implementation

The *Jupyter notebook*

`Python/Chapter-04-05/07-Minimize.ipynb`

contains a program that takes a DFA $F$ and returns an equivalent DFA that has the minimum number of states.

## 4.7 Conclusion

In this chapter we have shown that the concept of a finite state machine and the concept of a regular expression are equivalent in their ability to describe formal languages.

(a) Every DFA can be translated into an equivalent regular expression.

(b) Every regular expression can be translated into an equivalent NFA.

(c) Every NFA can be transformed into an equivalent DFA.

Furthermore, we have shown that every DFA can be minimized.

**Historical Remark** Stephen Cole Kleene (1909 – 1994) has shown in 1956 that the concepts of finite state machines and regular expressions have the same strength [Kle56].

## 4.8 Check your Understanding

(a) How are DFAs and NFAs defined?

(b) Given an NFA $F$, can you convert it into an equivalent DFA $det(F)$ that accepts the same language as the NFA $F$?

(c) Given a regular expression $r$, can you describe the steps necessary to construct an NFA $F$ that accepts the language specified by $r$?

(d) Given a DFA $F$ can you describe how to construct a regular expression $r$ that specifies the language accepted by $F$?

(e) How do we minimize a DFA?

# Chapter 5

# The Theory of Regular Languages

Figure 5.1: A pumping lama.

A formal language $L \subseteq \Sigma^*$ is called a regular language if there is a regular expression $r$ such that the language $L$ is specified by $r$, i.e. if

$$L = L(r)$$

holds. In Chapter 4 we have shown that the regular languages are those languages that are recognized by a finite automaton. In this chapter, we show that the class of regular languages has certain closure properties:

(a) The union $L_1 \cup L_2$ of two regular languages $L_1$ and $L_2$ is a regular language, too.

(b) The intersection $L_1 \cap L_2$ of two regular languages $L_1$ and $L_2$ is again a regular language.

(c) The complement $\Sigma^* \backslash L$ of a regular language $L$ is also a regular language.

As an application of these closure properties we will then show how it is possible to decide whether two regular expressions are equivalent, i.e. we present an algorithm that takes two regular expressions $r_1$ and $r_2$ as input and checks whether

$$r_1 \doteq r_2$$

holds. After that, we discuss the limits of regular languages. To this end, we prove the *pumping lemma*. Using the pumping lemma we will be able to show that, for example, the language

$$\{\mathsf{a}^n\mathsf{b}^n \mid n \in \mathbb{N}\}$$

is not regular. To summarize, this chapter

- discusses closure properties of regular languages,
- presents an algorithm for checking the equivalence of regular expressions, and
- shows that certain languages are not regular.

## 5.1 Closure Properties of Regular Languages

In this section we show that regular languages are closed under the Boolean operations of union, intersection and complement. We start with the union.

**Proposition 14** If $L_1$ and $L_2$ are regular languages, then the union of the sets $L_1$ and $L_2$, which is the set $L_1 \cup L_2$, is a regular language, too.

**Proof**: As $L_1$ and $L_2$ are regular languages, there exist regular expressions $r_1$ and $r_2$ such that

$$L_1 = L(r_1) \quad \text{and} \quad L_2 = L(r_2)$$

holds. We define $r := r_1 + r_2$. Then we have

$$L(r) = L(r_1 + r_2) = L(r_1) \cup L(r_2) = L_1 \cup L_2.$$

Since $L_1 \cup L_2$ is the language that is generated by the regular expression $r_1 + r_2$, it is a regular language. □

**Proposition 15** If $L_1$ and $L_2$ are regular languages, then the intersection of the sets $L_1$ and $L_2$, which is the set $L_1 \cap L_2$, is a regular language, too.

**Proof**: While the proof of Proposition 14 follows directly from the definition of regular expressions, we have to do a little more work here. In the previous chapter we saw that for every regular expression $r$ there is an equivalent DFA $F$, which accepts the language specified by $r$, and we can also assume that this DFA is complete. Let $r_1$ and $r_2$ be the regular expressions that define the languages $L_1$ and $L_2$, i.e. we have

$$L_1 = L(r_1) \quad \text{and} \quad L_2 = L(r_2).$$

First, we construct two complete DFAs $F_1$ and $F_2$ that accept these languages, i.e. such that we have

$$L(F_1) = L_1 \quad \text{and} \quad L(F_2) = L_2.$$

Our goal is to construct a DFA $F$ such that $F$ accepts the language $L_1 \cap L_2$ and nothing else. As every DFA can be converted into an equivalent regular expression we will then have shown that the language $L_1 \cap L_2$ is regular. We will use the DFAs $F_1$ and $F_2$ to construct $F$. Assume that

$$F_1 = \langle Q_1, \Sigma, \delta_1, q_1, A_1 \rangle \quad \text{and} \quad F_2 = \langle Q_2, \Sigma, \delta_2, q_2, A_2 \rangle$$

holds. We define $F$ as follows:

$$F := \langle Q_1 \times Q_2, \Sigma, \delta, \langle q_1, q_2\rangle, A_1 \times A_2\rangle,$$

where the state transition function

$$\delta : (Q_1 \times Q_2) \times \Sigma \to Q_1 \times Q_2$$

is defined as

$$\delta\big(\langle p_1, p_2\rangle, c\big) := \big\langle \delta_1(p_1, c), \delta_2(p_2, c)\big\rangle.$$

Effectively, the DFA $F$ runs the DFAs $F_1$ and $F_2$ in parallel. In order to do so, the states of $F$ are pairs of the form $\langle p_1, p_2\rangle$ where $p_1$ is a state from $F_1$ and $p_2$ is a state from $F_2$ and the transition function $\delta$ computes the state $\big\langle \delta_1(p_1, c), \delta_2(p_2, c)\big\rangle$ by simultaneously computing the next states of $p_1$ and $p_2$ in the DFAs $F_1$ and $F_2$ when these DFAs read the character $c$. A string $s$ is accepted by $F$ if and only if both $F_1$ and $F_2$ accept $s$. Therefore, the set $A$ of accepting states of the DFA $F$ is defined as:

$$A := \big\{\langle p_1, p_2\rangle \in Q_1 \times Q_2 \mid p_1 \in A_1 \wedge p_2 \in A_2\big\} = A_1 \times A_2.$$

The DFA $F$ is called the product automaton of $F_1$ and $F_2$ and is written as $F_1 \times F_2$. As $F_1$ and $F_2$ are complete, a simple induction on the string $s$ shows that

$$\delta^*\big(\langle p_1, p_2\rangle, s\big) = \big\langle \delta_1^*(p_1, s), \delta_2^*(p_2, s)\big\rangle \quad \text{for all } p_1 \in Q_1, p_2 \in Q_2, \text{ and } s \in \Sigma^*.$$

Then for all $s \in \Sigma^*$ we have:

$$\begin{array}{ll} & s \in L(F) \\ \Leftrightarrow & \delta^*(\langle q_1, q_2\rangle, s) \in A \\ \Leftrightarrow & \langle \delta_1^*(q_1, s), \delta_2^*(q_2, s)\rangle \in A_1 \times A_2 \\ \Leftrightarrow & \delta_1^*(q_1, s) \in A_1 \ \wedge \ \delta_2^*(q_2, s) \in A_2 \\ \Leftrightarrow & s \in L(F_1) \ \wedge \ s \in L(F_2) \\ \Leftrightarrow & s \in L(F_1) \cap L(F_2) \\ \Leftrightarrow & s \in L_1 \cap L_2 \end{array}$$

Therefore we have shown that

$$L(F) = L_1 \cap L_2$$

and this completes the proof. □

**Remark**: It is possible to define a function

$$\wedge : \mathit{RegExp} \times \mathit{RegExp} \to \mathit{RegExp}$$

that takes two regular expressions $r_1$ and $r_2$ and returns a regular expression $r_1 \wedge r_2$ such that we have

$$L(r_1 \wedge r_2) = L(r_1) \cap L(r_2).$$

To compute $r_1 \wedge r_2$ we first compute NFAs $F_1$ and $F_2$ such that

$$L(F_1) = L(r_1) \quad \text{and} \quad L(F_2) = L(r_2)$$

holds. Then we transform the NFAs $F_1$ and $F_2$ into equivalent DFAs $\mathit{det}(F_1)$ and $\mathit{det}(F_2)$. After that, we build the product automaton of $\mathit{det}(F_1)$ and $\mathit{det}(F_2)$ as shown above. Finally, we convert the DFA $\mathit{det}(F_1) \times \mathit{det}(F_2)$ into an equivalent regular expression. The *Jupyter notebook*

Python/Chapter-04-05/11-Intersection.ipynb

implements the function $\wedge$ in this way. In order to keep the product automaton small, it minimizes the DFAs $\mathit{det}(F_1)$ and $\mathit{det}(F_2)$ before building their product. Nevertheless, the resulting regular expression is absurdly large. The reason is that the algorithm converting a DFA into a regular expression calls itself

four times for every state of the DFA. Therefore, the size of the regular expression computed for a DFA with $n$ states grows like $4^n$. As the product automaton has $|Q_1| \cdot |Q_2|$ states, the size of $r_1 \wedge r_2$ grows exponentially in the *product* of the numbers of states of $det(F_1)$ and $det(F_2)$.

The following table shows this for the regular expressions $r_1$ describing the strings over the alphabet $\{a, b\}$ in which the number of occurrences of a is divisible by $k$ and $r_2$ describing the strings in which the number of occurrences of b is divisible by $m$. For example, for $k = m = 2$ we have $r_1 = (b^*ab^*a)^*b^*$ and $r_2 = (a^*ba^*b)^*a^*$. The size of a regular expression is the number of nodes of its syntax tree, i.e. every character, every occurrence of $\varepsilon$ and $\emptyset$, and every operator is counted. The last column shows the size of $r_1 \wedge r_2$ after it has been simplified with the algebraic laws given in Chapter 2.

| $k$ | $m$ | **states** | **size of** $r_1$ | **size of** $r_2$ | **size of** $r_1 \wedge r_2$ | **simplified** |
|---|---|---|---|---|---|---|
| 2 | 2 | 4 | 13 | 13 | 784 | 195 |
| 2 | 3 | 6 | 13 | 18 | 12 304 | 502 |
| 3 | 3 | 9 | 18 | 18 | 787 456 | 2 836 |

Even though the regular expressions $r_1$ and $r_2$ are tiny, the regular expression $r_1 \wedge r_2$ has almost 800 000 nodes when the product automaton has only nine states. Simplification removes most of these nodes, but the result is still far from readable. For comparison, for $k = m = 2$ the language $L(r_1) \cap L(r_2)$ is also described by the regular expression

$$(aa + bb + (ab + ba)(aa + bb)^*(ab + ba))^*,$$

which has only 33 nodes. Therefore, the function $\wedge$ is of no practical use: If we need the intersection of two regular languages, we work with the product automaton instead. ◇

**Proposition 16** If $L$ is a regular language with the alphabet $\Sigma$, then the complement of $L$, which is defined as the language $\Sigma^* \backslash L$, is regular.

**Proof**: We assume that $r$ is a regular expression describing the language $L$, i.e. $L = L(r)$. We construct an NFA $F$ such that $L(F) = L(r) = L$. We transform this NFA into the DFA $det(F)$ as discussed in the previous chapter and note that this DFA is complete. Assume that we then have

$$det(F) = \langle Q, \Sigma, \delta, q_0, A \rangle.$$

We define the DFA $\overline{det(F)}$ as follows:

$$\overline{det(F)} = \langle Q, \Sigma, \delta, q_0, Q \backslash A \rangle,$$

i.e. the set of accepting states of $\overline{det(F)}$ is the complement of the set of accepting states of $det(F)$. Then we have

$$\begin{aligned}
& w \in L\Big(\overline{det(F)}\Big) \\
\Leftrightarrow \quad & \delta^*(q_0, w) \in Q \backslash A \\
\Leftrightarrow \quad & \delta^*(q_0, w) \notin A \\
\Leftrightarrow \quad & w \notin L\big(det(F)\big) \\
\Leftrightarrow \quad & w \notin L(F) \\
\Leftrightarrow \quad & w \notin L(r) \\
\Leftrightarrow \quad & w \notin L \\
\Leftrightarrow \quad & w \in \Sigma^* \backslash L
\end{aligned}$$

This shows that the language $\Sigma^* \backslash L$ is accepted by the DFA $\overline{det(F)}$ and hence it is a regular language. □

**Corollary 17** If $L_1$ and $L_2$ are regular languages over the common alphabet $\Sigma$, then the set difference $L_1 \backslash L_2$ is a regular language.

**Proof**: A string $w$ is a member of $L_1 \backslash L_2$ iff $w$ is a member of $L_1$ and $w$ is also a member of the complement of $L_2$. Therefore we have

$$L_1 \backslash L_2 = L_1 \cap (\Sigma^* \backslash L_2).$$

The previous proposition shows that for a regular language $L_2$ the complement $\Sigma^* \backslash L_2$ is also a regular language. Since the intersection of regular languages is regular, too, we see that $L_1 \backslash L_2$ is also regular. □

In summary, we have demonstrated that regular languages are closed under the operations of Boolean algebra on sets.

**Exercise 13**: Let $\Sigma$ be a given alphabet. For a string $s = c_1 c_2 \ldots c_{n-1} c_n \in \Sigma^*$, the reversal of $s$ is denoted by $s^R$ and is defined as follows:

$$s^R := c_n c_{n-1} \ldots c_2 c_1.$$

For instance, if $s = \texttt{abc}$, then $s^R = \texttt{cba}$. The reversal $L^R$ of a language $L \subseteq \Sigma^*$ is formally defined as:

$$L^R := \{s^R \mid s \in L\}.$$

Assume that the language $L \subseteq \Sigma^*$ is regular. Prove that $L^R$ is also a regular language.

**Hint**: You may use the following equations in your proof:

(a) If $L_1, L_2 \subseteq \Sigma^*$, then

$$(L_1 \cdot L_2)^R = L_2^R \cdot L_1^R.$$

(b) If $L_1 \subseteq \Sigma^*$, then

$$\left(L_1^*\right)^R = \left(L_1^R\right)^*.$$

◇

## 5.2 Recognizing Empty Languages

In this section we develop an algorithm that takes a DFA

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle$$

as its input and then checks whether the language accepted by $F$ is empty, i.e. it checks whether $L(F) = \{\}$. To this end we interpret $F$ as a directed graph. The nodes of this graph are the states of the set $Q$ and for two states $q_1$ and $q_2$ there is an edge from $q_1$ to $q_2$ iff there exists a character $c \in \Sigma$, such that $\delta(q_1, c) = q_2$. The language $L(F)$ is empty if and only if this graph has no path that starts in the state $q_0$ and leads to an accepting state.

Therefore, in order to answer the question whether $L(F) = \{\}$ holds, we have to compute the set $R$ of all those states that are reachable from the start state $q_0$. The computation of the set $R$ of all reachable states can be done iteratively as follows:

1. $q_0 \in R$, because the start state is obviously reachable from the start state.
2. $p_1 \in R \wedge \delta(p_1, c) = p_2 \;\rightarrow\; p_2 \in R$,

   because if $p_1$ is reachable from $q_0$ and there is a transition from $p_1$ to $p_2$ while reading a character $c$, then $p_2$ is reachable from $q_0$.

   This last step is repeated until there are no more states that can be added to the set $R$.

Then we have $L(F) = \{\}$ if and only if none of the accepting states is reachable, i.e. we have

$$L(F) = \{\} \;\Leftrightarrow\; R \cap A = \{\}.$$

Hence we now have an algorithm for checking whether $L(F) = \{\}$ holds: We compute the states that are reachable from the start state $q_0$ and then we check whether this set contains any accepting states.

**Remark**: If the regular language $L$ is not specified by a DFA $F$, but rather is defined by a regular expression $r$, then there is a simple recursive algorithm for checking whether $L(r)$ is empty:

(a) $L(\emptyset) = \{\}$.

(b) $L(\varepsilon) \neq \{\}$.

(c) $L(c) \neq \{\}$ for all $c \in \Sigma$.

(d) $L(r_1 \cdot r_2) = \{\} \Leftrightarrow L(r_1) = \{\} \vee L(r_2) = \{\}$.

(e) $L(r_1 + r_2) = \{\} \Leftrightarrow L(r_1) = \{\} \wedge L(r_2) = \{\}$.

(f) $L(r^*) \neq \{\}$. ◇

## 5.3 Equivalence of Regular Expressions

In Chapter 2 we have defined two regular expressions $r_1$ and $r_2$ to be equivalent (written $r_1 \doteq r_2$), if the languages specified by $r_1$ and $r_2$ are identical:

$$r_1 \doteq r_2 \stackrel{\text{def}}{\Longleftrightarrow} L(r_1) = L(r_2).$$

In this section, we present an algorithm that takes two regular expressions $r_1$ and $r_2$ as input and then checks whether $r_1 \doteq r_2$ holds.

**Theorem 18** If $r_1$ and $r_2$ are regular expressions, then the question whether $r_1 \doteq r_2$ holds is decidable.

**Proof**: We present an algorithm that decides whether $L(r_1) = L(r_2)$ holds. First, we observe that the sets $L(r_1)$ and $L(r_2)$ are identical iff the set differences $L(r_2)\backslash L(r_1)$ and $L(r_1)\backslash L(r_2)$ are both empty:

$$\begin{array}{lcl} L(r_1) = L(r_2) & \Leftrightarrow & L(r_1) \subseteq L(r_2) \;\wedge\; L(r_2) \subseteq L(r_1) \\ & \Leftrightarrow & L(r_1)\backslash L(r_2) = \{\} \;\wedge\; L(r_2)\backslash L(r_1) = \{\} \end{array}$$

Next, assume that $F_1$ and $F_2$ are DFAs such that

$$L(F_1) = L(r_1) \quad \text{and} \quad L(F_2) = L(r_2)$$

holds. We have seen in Chapter 4 how $F_1$ and $F_2$ can be constructed from $r_1$ and $r_2$. According to Corollary 17 the languages $L(r_1)\backslash L(r_2)$ and $L(r_2)\backslash L(r_1)$ are regular and we have seen how to construct DFAs $F_{1,2}$ and $F_{2,1}$ such that

$$L(r_1)\backslash L(r_2) = L(F_{1,2}) \quad \text{and} \quad L(r_2)\backslash L(r_1) = L(F_{2,1})$$

holds. If $F_1$ and $F_2$ are complete and have the accepting states $A_1$ and $A_2$, then $F_{1,2}$ is the product automaton $F_1 \times F_2$ with the set of accepting states $A_1 \times (Q_2 \backslash A_2)$, where $Q_2$ is the set of states of $F_2$, and $F_{2,1}$ is defined analogously. Hence we have

$$r_1 \doteq r_2 \;\Leftrightarrow\; L(F_{1,2}) = \{\} \wedge L(F_{2,1}) = \{\}$$

and according to Section 5.2 this question is decidable by checking whether any of the accepting states of $F_{1,2}$ or $F_{2,1}$ are reachable from the start state. □

**Remark**: The *Jupyter notebook*

`Python/Chapter-04-05/09-Equivalence.ipynb`

implements this algorithm.

## 5.4 Limits of Regular Languages

In this section we present a theorem that can be used to show that certain languages are not regular. This theorem is known as the pumping lemma for regular languages.

**Theorem 19 (Pumping Lemma for Regular Languages, Rabin and Scott 1959, [RS59])**
Assume $L$ is a regular language. Then there exists a natural number $n \in \mathbb{N}$ such that every string $s \in L$ that has a length of at least $n$ can be split into three substrings $u$, $v$, and $w$ such that the following holds:

1. $s = uvw$,
2. $v \neq \lambda$,
3. $|uv| \leq n$,
4. $\forall h \in \mathbb{N} : uv^h w \in L$.

This theorem can be written as a single formula: If $L$ is a regular language, then

$$\exists n \in \mathbb{N} : \forall s \in L : \big(|s| \geq n \rightarrow \exists u, v, w \in \Sigma^* : s = uvw \wedge v \neq \lambda \wedge |uv| \leq n \wedge \forall h \in \mathbb{N} : uv^h w \in L\big).$$

**Proof**: As $L$ is a regular language, there exists a DFA

$$F = \langle Q, \Sigma, \delta, q_0, A \rangle,$$

such that $L = L(F)$. The number $n$ whose existence is claimed in the Pumping Lemma is defined as the number of states of $F$:

$$n := \mathit{card}(Q).$$

Next, assume a string $s \in L$ is given such that $|s| \geq n$. Define $m := |s|$. Then there are $m$ characters $c_i$ such that

$$s = c_1 c_2 \cdots c_m.$$

Since $|s| \geq n$, we have $m \geq n$. On reading the characters $c_i$ the DFA changes its states as follows:

$$q_0 \overset{c_1}{\longmapsto} q_1 \overset{c_2}{\longmapsto} q_2 \overset{c_3}{\longmapsto} \cdots \overset{c_m}{\longmapsto} q_m,$$

i.e. on reading the character $c_{i+1}$ in the state $q_i$ the DFA switches into the state $q_{i+1}$ for all $i = 0, \ldots, m-1$. In general, for states $p, q \in Q$ and a string $t \in \Sigma^*$ we write $p \overset{t}{\longmapsto} q$ if $\delta^*(p, t) = q$ holds. Since we have $s \in L$ we conclude that $q_m$ must be an accepting state, i.e. $q_m \in A$. As $m \geq n$ and $n$ is the total number of states of $F$, not all of the states

$$q_0, q_1, q_2, \cdots, q_m$$

can be different. Because of

$$\mathit{card}\big(\{0, 1, \cdots, n\}\big) = n + 1$$

we know, that even in the list

$$[q_0, q_1, q_2, \cdots, q_n]$$

at least one state has to occur at least twice. Hence there are natural numbers $k, l \in \{0, \cdots, n\}$ such that

$$q_k = q_l \wedge k < l.$$

Next, we define the strings $u$, $v$, and $w$ as follows:

$$u := c_1 \cdots c_k, \quad v := c_{k+1} \cdots c_l, \quad \text{and} \quad w := c_{l+1} \cdots c_m.$$

As $k < l$ we have that $v \neq \lambda$ and $l \leq n$ implies $|uv| \leq n$. Furthermore, we have the following:

1. Reading the string $u$ changes the state of the DFA $F$ from the start state $q_0$ to the state $q_k$, we have

$$q_0 \overset{u}{\longmapsto} q_k. \tag{5.1}$$

2. Reading the string $v$ changes the state of the DFA $F$ from the state $q_k$ to the state $q_l$. As we have $q_l = q_k$, this implies

$$q_k \overset{v}{\longmapsto} q_k. \tag{5.2}$$

3. Reading the string $w$ changes the state of the DFA $F$ from the state $q_l = q_k$ to the accepting state $q_m$:

$$q_k \overset{w}{\longmapsto} q_m. \tag{5.3}$$

From $q_k \overset{v}{\longmapsto} q_k$ we conclude

$$q_k \overset{v}{\longmapsto} q_k \overset{v}{\longmapsto} q_k, \quad \text{hence} \quad q_k \overset{v^2}{\longmapsto} q_k.$$

As we can repeat reading $v$ in state $q_k$ any number of times, we have

$$q_k \overset{v^h}{\longmapsto} q_k \quad \text{for all } h \in \mathbb{N}. \tag{5.4}$$

Combining the equations (5.1), (5.3), and (5.4) we have

$$q_0 \overset{u}{\longmapsto} q_k \overset{v^h}{\longmapsto} q_k \overset{w}{\longmapsto} q_m.$$

This can be condensed to

$$q_0 \overset{uv^hw}{\longmapsto} q_m$$

and since the state $q_m$ is an accepting state we conclude that $uv^hw \in L$ holds for any $h \in \mathbb{N}$.

Figure 5.2 shows the gist of the proof. □

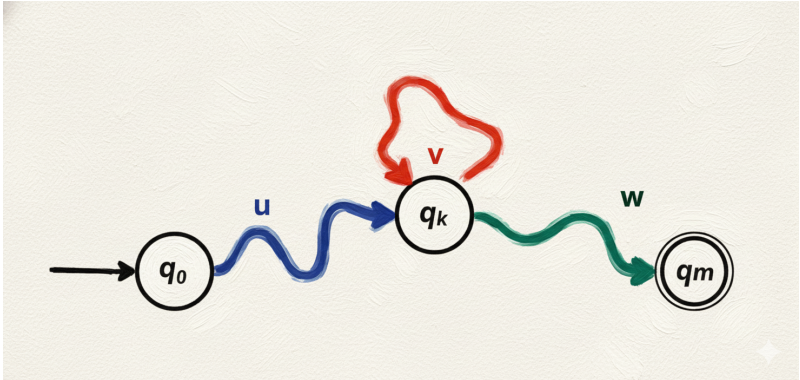



Figure 5.2: The construction of the Pumping Lemma.

**Proposition 20** The alphabet $\Sigma$ is defined as $\Sigma = \{$ "a" , "b" $\}$. Define the language $L$ as the set of all strings of the form $\mathtt{a}^k\mathtt{b}^k$ where $k$ is some natural number:

$$L = \left\{\mathtt{a}^k\mathtt{b}^k \mid k \in \mathbb{N}\right\}.$$

Then the language $L$ is not regular.

**Proof**: This is a proof by contradiction. We assume that $L$ is a regular language. According to the Pumping Lemma there exists a fixed natural number $n > 0$ such that every $s \in L$ that satisfies $|s| \geq n$ can be written as

$$s = uvw$$

where, furthermore, we have that

$$|uv| \leq n, \quad v \neq \lambda, \quad \text{and} \quad \forall h \in \mathbb{N} : uv^h w \in L$$

holds. Let us define the string $s$ as

$$s := \mathtt{a}^n\mathtt{b}^n.$$

Obviously we have $|s| = 2 \cdot n \geq n$. Hence there are strings $u$, $v$, and $w$ such that

$$\mathtt{a}^n\mathtt{b}^n = uvw, \quad |uv| \leq n, \quad v \neq \lambda, \quad \text{and} \quad \forall h \in \mathbb{N} : uv^h w \in L.$$

As $|uv| \leq n$, the string $uv$ is a prefix not only of $s$ but even of $\mathtt{a}^n$. Therefore, and since $v \neq \lambda$ we know that the string $v$ must have the form

$$v = \mathtt{a}^k \quad \text{for some } k \in \mathbb{N} \text{ such that } k > 0.$$

If we take the formula $\forall h \in \mathbb{N} : uv^h w \in L$ and set $h := 0$, we conclude that

$$uw \in L. \tag{5.5}$$

In order to facilitate our argument, we define the function

$$\mathit{count} : \Sigma^* \times \Sigma \to \mathbb{N}.$$

Given a string $t$ and a character $c$ the function $\mathit{count}(t, c)$ counts how often the character $c$ occurs in the string $t$. For the language $L$ we have

$$t \in L \Rightarrow \mathit{count}(t, \text{'a'}) = \mathit{count}(t, \text{'b'}).$$

On one hand we have:

$$\begin{array}{lcl}
\mathit{count}(uw, \text{'a'}) & = & \mathit{count}(uvw, \text{'a'}) - \mathit{count}(v, \text{'a'}) \\
& = & \mathit{count}(s, \text{'a'}) - \mathit{count}(v, \text{'a'}) \\
& = & \mathit{count}(\mathtt{a}^n\mathtt{b}^n, \text{'a'}) - \mathit{count}(\mathtt{a}^k, \text{'a'}) \\
& = & n - k \\
& < & n
\end{array}$$

But on the other hand we have

$$\begin{array}{lcl}
\mathit{count}(uw, \text{'b'}) & = & \mathit{count}(uvw, \text{'b'}) - \mathit{count}(v, \text{'b'}) \\
& = & \mathit{count}(s, \text{'b'}) - \mathit{count}(v, \text{'b'}) \\
& = & \mathit{count}(\mathtt{a}^n\mathtt{b}^n, \text{'b'}) - \mathit{count}(\mathtt{a}^k, \text{'b'}) \\
& = & n - 0 \\
& = & n
\end{array}$$

Therefore, we have

$$\mathit{count}(uw, \text{'a'}) < \mathit{count}(uw, \text{'b'})$$

and this shows that the string $uw$ is not a member of the language $L$ because for all strings in $L$ the number of occurrences of the character "a" is the same as the number of occurrences of the character "b". This contradiction shows that the language $L$ cannot be regular. □

**Exercise 14**: Let us define the language $L$ as the set of all those strings that contain an equal number of occurrences of the characters "a" and "b":

$$L := \left\{w \in \{\text{'a'}, \text{'b'}\}^* \;\middle|\; \mathit{count}(w, \text{'a'}) = \mathit{count}(w, \text{'b'})\right\}$$

Prove that the language $L$ is not regular. ◇

**Remark**: Regular expressions are unable to check even such a simple property as whether the parentheses in an expression are balanced. To see this, let $D$ be the language of all strings over the

alphabet $\{\text{“(”}, \text{“)”}\}$ in which the parentheses are balanced. If $D$ were regular, then the intersection

$$D \cap L\big(\text{“(”}^* \cdot \text{“)”}^*\big) = \big\{\text{“(”}^k \text{“)”}^k \mid k \in \mathbb{N}\big\}$$

would be regular, too. But if we write “a” instead of “(” and “b” instead of “)”, this is the language $\{\mathtt{a}^k\mathtt{b}^k \mid k \in \mathbb{N}\}$, which we have shown to be not regular. Therefore, the concept of regular expressions is not strong enough to describe the syntax of a programming language. The next chapter introduces the notion of context-free languages. These languages are powerful enough to describe modern programming languages.

**Exercise 15**: The language $L_{\text{square}}$ is the set of all strings of the form $\mathtt{a}^m$ where $m$ is a square, we have

$$L_{\text{square}} = \big\{\mathtt{a}^m \mid \exists k \in \mathbb{N} : m = k^2\big\}$$

Prove that the language $L_{\text{square}}$ is not a regular language. ◇

**Solution**: The proof is by contradiction. We begin by assuming that $L_{\text{square}}$ is a regular language. According to the Pumping Lemma, there exists a natural number $n > 0$ such that for any string $s \in L_{\text{square}}$ with $|s| \geq n$, the string can be decomposed into three parts $u$, $v$, and $w$ satisfying the following conditions:

1. $s = uvw$,
2. $|uv| \leq n$,
3. $v \neq \lambda$,
4. $\forall h \in \mathbb{N} : uv^hw \in L_{\text{square}}$.

Let us define $s := \mathtt{a}^{n^2}$. We have $s \in L_{\text{square}}$ and we see that

$$|s| = \left|\mathtt{a}^{n^2}\right| = n^2 \geq n.$$

Hence there have to be strings $u$, $v$ and $w$ such that $s = uvw$ and $u$, $v$, and $w$ have the properties specified above. As ’a’ is the only character that occurs in $s$, the strings $u$, $v$, and $w$ also contain only this character. Hence there must be natural numbers $x$, $y$, and $z$ such that

$$u = \mathtt{a}^x,\ v = \mathtt{a}^y \text{ and } w = \mathtt{a}^z$$

holds. Then we have the following.

(a) The partition $s = uvw$ has the form $\mathtt{a}^{n^2} = \mathtt{a}^x\mathtt{a}^y\mathtt{a}^z$ and hence we have

$$n^2 = x + y + z. \tag{5.6}$$

(b) The inequality $|uv| \leq n$ implies $x + y \leq n$, which implies

$$y \leq n. \tag{5.7}$$

(c) From the condition $v \neq \lambda$ we get

$$y > 0. \tag{5.8}$$

(d) The formula $\forall h \in \mathbb{N} : uv^hw \in L_{\text{square}}$ implies

$$\forall h \in \mathbb{N} : \mathtt{a}^x\mathtt{a}^{y\cdot h}\mathtt{a}^z \in L_{\text{square}}. \tag{5.9}$$

In particular, this must hold for $h = 2$:

$$\mathtt{a}^x \mathtt{a}^{y \cdot 2} \mathtt{a}^z \in L_{\text{square}}.$$

According to the definition of $L_{\text{square}}$ there is a natural number $k$ such that

$$x + 2 \cdot y + z = k^2. \tag{5.10}$$

If we add $y$ on both sides of equation (5.6) we get

$$n^2 + y = x + 2 \cdot y + z = k^2.$$

Because of $y > 0$ this implies

$$n < k. \tag{5.11}$$

On the other hand we have

$$\begin{array}{rcll} k^2 & = & x + 2 \cdot y + z & \text{according to (5.10)} \\ & = & x + y + z + y & \\ & \leq & x + y + z + n & \text{according to (5.7)} \\ & = & n^2 + n & \text{according to (5.6)} \\ & < & n^2 + 2 \cdot n + 1 & \text{since } n + 1 > 0 \\ & = & (n+1)^2 & \end{array}$$

This shows that $k^2 < (n+1)^2$ holds and therefore we have

$$k < n + 1. \tag{5.12}$$

The inequalities (5.11) and (5.12) imply

$$n < k < n + 1.$$

Since $k$ is a natural number and $n$ is also a natural number this is a contradiction because there is no natural number between $n$ and $n + 1$. □

**Exercise 16**: Define $\Sigma := \{\mathtt{a}\}$. Prove that the language

$$L := \{\mathtt{a}^p \mid p \text{ is a prime number}\}$$

is not regular. ◇

**Solution**: The proof is done by contradiction. Assume that $L$ is regular. According to the Pumping Lemma there is a positive natural number $n$ such that all strings $s \in L$ that have a length of at least $n$ can be written as

$$s = uvw$$

where, furthermore, the substrings $u$, $v$, and $w$ have the following properties:

1. $v \neq \lambda$,
2. $|uv| \leq n$, and
3. $\forall h \in \mathbb{N} : uv^h w \in L$.

Let us choose a prime number $p$ such that $p \geq n + 2$. Since there are infinitely many prime numbers, this is always possible. Next, define $s := \mathsf{a}^p$. Then we have $|s| = p \geq n$ and we can use the Pumping Lemma to conclude that there must be substrings $u$, $v$, and $w$ such that

$$\mathsf{a}^p = uvw$$

holds and $u$, $v$, and $w$ have the properties listed above. Hence the substrings $u$, $v$, and $w$ can only contain the letter "a". Therefore there exist natural numbers $x$, $y$, and $z$ such that

$$u = \mathsf{a}^x, \quad v = \mathsf{a}^y, \quad \text{and} \quad w = \mathsf{a}^z.$$

We can conclude that $x$, $y$, and $z$ have the following properties:

1. $x + y + z = p$,
2. $y \neq 0$,
3. $x + y \leq n$,
4. $\forall h \in \mathbb{N} : x + h \cdot y + z \in \mathbb{P}$.

   Here, $\mathbb{P}$ denotes the set of prime numbers.

If we choose to define $h := x + z$ in the last formula, we get

$$x + (x + z) \cdot y + z \in \mathbb{P}.$$

Because of $x + (x + z) \cdot y + z = (x + z) \cdot (1 + y)$ this leads to

$$(x + z) \cdot (1 + y) \in \mathbb{P}.$$

This is impossible. As $y > 0$ we have $1 + y \geq 2$ and because of $x + y \leq n$, $x + y + z = p$, and $p \geq n + 2$ it follows that $z \geq 2$. Therefore, $x + z \geq 2$. But then the product $(x + z) \cdot (1 + y)$ is not a prime number and this contradiction shows that $L$ can not be regular. □

**Exercise 17**: The language $L_{\text{power}}$ is the set of all strings of the form $\mathsf{a}^n$ where $n$ is a power of $2$, i.e. we have

$$L_{\text{power}} = \left\{ \mathsf{a}^{2^k} \mid k \in \mathbb{N} \right\}$$

Prove that the language $L_{\text{power}}$ is not regular. ◇

## 5.5 Check your Understanding

(a) What are the closure properties of regular languages?

(b) How can we check whether two regular expressions are equivalent?

(c) Give the exact wording of the Pumping Lemma.

(d) Can you prove that the language

$$L := \left\{ w \in \{\,\text{'a'}, \text{'b'}\,\}^* \;\middle|\; \mathit{count}(w, \text{'a'}) = \mathit{count}(w, \text{'b'}) \right\}$$

is not regular?

# Chapter 6

# Context-Free Languages

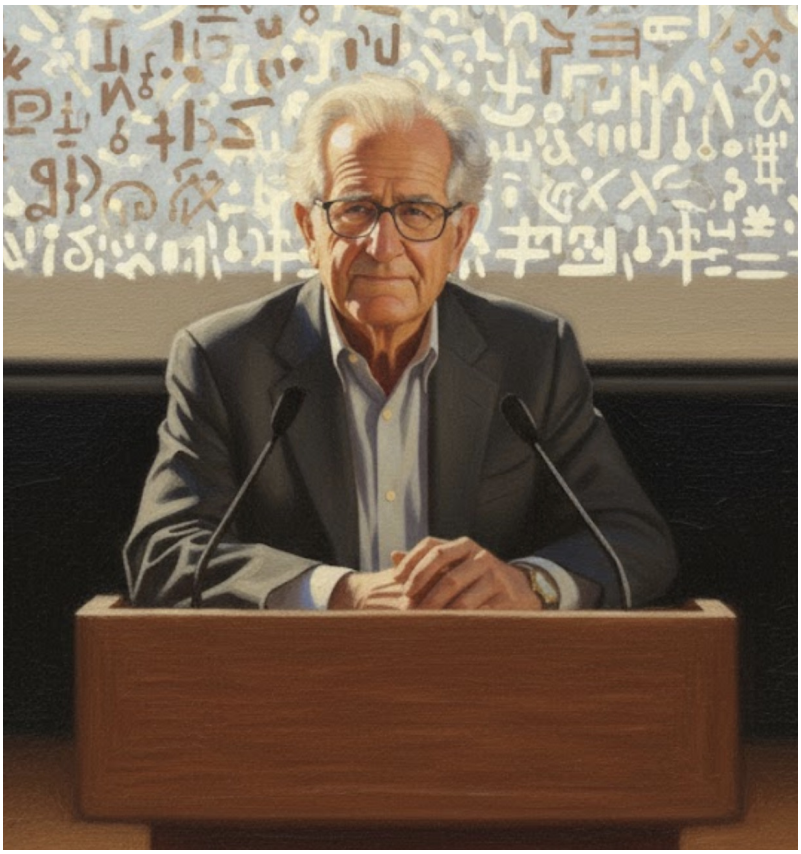


Figure 6.1: The father of context-free languages, Noam Chomsky.

In this chapter we present the notion of a *context-free language*. This concept is much more powerful than the notion of a regular language. The syntax of most modern programming languages can be described by a context-free language. Furthermore, checking whether a string is a member of a context-free language structures the string into a recursive structure known as a parse tree. These parse trees are the basis for understanding the meaning of a string that is to be interpreted as a program fragment. A program that checks whether a given string is an element of a context-free language is called a parser. The task of a parser is to build a parse tree from a given string. Parsing is therefore the first step in an interpreter or a compiler. In this chapter, we first define the notion of context-free languages. Next, we discuss parse trees. We conclude this chapter by introducing top-down parsing, which is one of the less complex algorithms that are available for parsing a string.

## 6.1 Context-Free Grammars

Context-free languages are used to describe programming languages. Context-free languages are formal languages like regular languages, but they are much more expressive than regular languages. When we process a program, we not only want to decide whether the program is syntactically correct, but we also want to understand the *structure* of the program. The process of *structuring* is also referred to as parsing and the program that does this structuring is called a parser. As input a parser usually does not receive the text of a program, but instead a sequence of so-called terminals, which are also called tokens. These tokens are generated by a scanner, which uses regular expressions to split the program text into single words, which we call tokens in this context.

The parser receives a sequence of tokens from the scanner and has the task of constructing a so-called syntax tree. For this purpose the parser uses a grammar which specifies how the input is to be structured. As an example, consider parsing arithmetic expressions. We define the set *expr* of arithmetic expressions inductively. In order to correctly represent the structure of arithmetic expressions, we define simultaneously the sets *product* and *factor*. The set *product* encompasses arithmetic expressions representing products and quotients, while the set *factor* includes numbers, variables, and parenthesized expressions. Defining these additional sets is crucial for ensuring the correct precedence of operators. The basic building blocks of arithmetic expressions are variables, numbers, the operator symbols "+", "-", "*", "/", and the bracket symbols "(" and ")". Based on these symbols the inductive definition of the sets *factor*, *product*, and *expr* proceeds as follows:

1. Each number is a factor:

   $$C \in \text{NUMBER} \Rightarrow C \in \textit{factor}.$$

2. Each variable is a factor:

   $$V \in \text{VARIABLE} \Rightarrow V \in \textit{factor}.$$

3. If $A$ is an arithmetic expression and we enclose this expression in parentheses we get an expression that we can use as a factor:

   $$A \in \textit{expr} \Rightarrow \text{“(”}\ A\ \text{“)”} \in \textit{factor}.$$

   A note on notation: In the preceding formula, $A$ serves as a meta-variable representing an arbitrary arithmetic expression. The strings "(" and ")" should be taken literally and are thus enclosed in quotation marks. These quotation marks are not part of the arithmetic expression; they are used solely for notational clarity.

4. If $F$ is a factor, then $F$ is also a product:

   $$F \in \textit{factor} \Rightarrow F \in \textit{product}.$$

5. If $P$ is a product and if $F$ is a factor, then the strings $P$ "*" $F$ and $P$ "/" $F$ are also products:

   $$P \in \textit{product} \wedge F \in \textit{factor} \Rightarrow P\text{“*”}F \in \textit{product} \ \wedge\ P\text{“/”}F \in \textit{product}.$$

6. Each product is also an arithmetic expression

   $$P \in \textit{product} \Rightarrow P \in \textit{expr}.$$

7. If $A$ is an arithmetic expression and $P$ is a product, then the strings $A$ "+" $P$ and $A$ "-" $P$ are arithmetic expressions:

   $$A \in \textit{expr} \wedge P \in \textit{product} \Rightarrow A\text{“+”}P \in \textit{expr} \ \wedge\ A\text{“-”}P \in \textit{expr}.$$

The sets *factor*, *product*, and *expr* are defined above through mutual recursion. This definition can be succinctly represented using what are known as grammar rules.

$$
\begin{aligned}
\textit{expr} &\to \textit{expr}\ \text{“+”}\ \textit{product} \\
\textit{expr} &\to \textit{expr}\ \text{“-”}\ \textit{product} \\
\textit{expr} &\to \textit{product} \\
\textit{product} &\to \textit{product}\ \text{“*”}\ \textit{factor} \\
\textit{product} &\to \textit{product}\ \text{“/”}\ \textit{factor} \\
\textit{product} &\to \textit{factor} \\
\textit{factor} &\to \text{“(”}\ \textit{expr}\ \text{“)”} \\
\textit{factor} &\to \textsc{Variable} \\
\textit{factor} &\to \textsc{Number}
\end{aligned}
$$

Expressions on the left hand side of a grammar rule are known as syntactic variables or non-terminals, while all other expressions are termed terminals. We adopt the convention of writing syntactic variables in lowercase to align with the syntax used by the parser generator Lark, which we will explore in Chapter 7. However, it is worth noting that in much of the existing literature, the convention is reversed: syntactic variables are typically capitalized, and terminals are presented in lowercase. Additionally, syntactic variables may occasionally be referred to as syntactic categories.

In the example, *expr*, *product*, and *factor* function as the syntactic variables. The remaining elements, namely Number, Variable, and the symbols “+”, “-”, “*”, “/”, “(”, and “)”, are the terminals or tokens. These terminals are precisely the symbols that are not found on the left side of any grammar rule. Terminals can be classified into two types:

1. Operator symbols and separators, like “/” and “(”, are used in their literal sense.
2. Tokens such as Number or Variable carry associated values. For Number, the value is numeric; for Variable, it is a string representing the variable’s name. To distinguish them from syntactic variables, these token types are always written in uppercase letters.

Grammar rules are often expressed in a notation more compact than the notation used previously. For the given example, the compact notation is as follows:

$$
\begin{aligned}
\textit{expr} &\to \textit{expr}\ \text{“+”}\ \textit{product} \\
&\mid \textit{expr}\ \text{“-”}\ \textit{product} \\
&\mid \textit{product} \\
\textit{product} &\to \textit{product}\ \text{“*”}\ \textit{factor} \\
&\mid \textit{product}\ \text{“/”}\ \textit{factor} \\
&\mid \textit{factor} \\
\textit{factor} &\to \text{“(”}\ \textit{expr}\ \text{“)”} \\
&\mid \textsc{Number} \\
&\mid \textsc{Variable}
\end{aligned}
$$

In this format, individual alternatives within a rule are demarcated by the metacharacter “|”. Building on the preceding example, we now introduce the formal definition of a context-free grammar.

**Definition 21 (Context-Free Grammar)** A context-free grammar $G$ is a quadruple

$$G = \langle V, T, R, S \rangle,$$

where

1. $V$ is a set called syntactic variables or non-terminals. In the preceding example, we have

   $$V = \{\textit{expr}, \textit{product}, \textit{factor}\}.$$

2. $T$ is a set referred to as terminals or tokens. The sets $T$ and $V$ are disjoint, hence

   $$T \cap V = \emptyset.$$

   For the aforementioned example, we have

   $$T = \{\textsc{Number}, \textsc{Variable}, \text{“+”}, \text{“-”}, \text{“*”}, \text{“/”}, \text{“(”}, \text{“)”}\}.$$

3. $R$ is a set of grammar rules. Formally, a grammar rule is a pair $\langle A, \alpha \rangle$ where:

   (a) The first component, $A$, is a syntactic variable:

   $$A \in V.$$

   (b) The second component, $\alpha$, is a string composed of syntactic variables and terminals:

   $$\alpha \in (V \cup T)^*.$$

   Consequently, we have

   $$R \subseteq V \times (V \cup T)^*.$$

   A rule $\langle A, \alpha \rangle$ is typically written as

   $$A \to \alpha.$$

   In the example, the first rule is

   $$\textit{expr} \to \textit{expr}\ \text{“+”}\ \textit{product},$$

   which formally corresponds to the pair

   $$\big\langle \textit{expr}, [\textit{expr}, \text{“+”}, \textit{product}] \big\rangle.$$

4. $S$ is a member of $V$ designated as the start symbol. In the given example, *expr* is the start symbol.

◇

When presenting a grammar, we typically enumerate the grammar rules and apply the following conventions to identify the start symbol and to differentiate between syntactic variables and terminals:

1. The start symbol is the syntactic variable appearing on the left side of the first rule.
2. Symbols enclosed within quotes are terminals.
3. Names beginning with an uppercase letter are terminals.

4. Names beginning with a lowercase letter are syntactic variables.

**Example**: The set of grammar rules below defines arithmetic expressions constructed from numbers, variables, and the binary operators "+", "-", "*", and "/".

$$
\begin{aligned}
\textit{expr} &\rightarrow \textit{expr}\ \text{“+”}\ \textit{product} \\
&\mid \textit{expr}\ \text{“-”}\ \textit{product} \\
&\mid \textit{product} \\
\textit{product} &\rightarrow \textit{product}\ \text{“}*\text{”}\ \textit{factor} \\
&\mid \textit{product}\ \text{“/”}\ \textit{factor} \\
&\mid \textit{factor} \\
\textit{factor} &\rightarrow \text{“(”}\textit{expr}\text{“)”} \\
&\mid \textsc{Number} \\
&\mid \textsc{Variable}
\end{aligned}
$$

In this case, the following observations can be made:

1. The start symbol is *expr*, as it is the syntactic variable on the left in the first grammar rule.
2. *expr*, *product*, and *factor* are recognized as syntactic variables because they start with a lowercase letter.
3. The symbols "+", "-", "$*$", and "/" are identified as terminals since they are enclosed in quotes.
4. NUMBER and VARIABLE are terminals, indicated by their initial uppercase letters.

### 6.1.1 Derivations

Next, we aim to identify the language defined by a given grammar $G$. For this purpose, we first define the concept of a derivation step. Consider the following:

1. $G = \langle V, T, R, S \rangle$ is a grammar,
2. $b$ is a syntactic variable in $V$,
3. $\alpha b \gamma$ is a string composed of terminals and syntactic variables from $(V \cup T)^*$, which includes the variable $b$, and
4. $b \rightarrow \delta$ is a rule in $R$.

Under these conditions, the string $\alpha b \gamma$ can undergo a derivation step to transform into the string $\alpha \delta \gamma$. This process involves substituting one occurrence of the syntactic variable $b$ with the right-hand side $\delta$ of the rule $b \rightarrow \delta$. We denote this derivation step as

$$\alpha b \gamma \Rightarrow_G \alpha \delta \gamma.$$

When the grammar $G$ is clear from the context, we may drop the subscript $_G$ and simply use $\Rightarrow$ instead of $\Rightarrow_G$. The transitive and reflexive closure of the relation $\Rightarrow_G$ is indicated by $\Rightarrow_G^*$. To specify that the derivation of string $w$ from the non-terminal $b$ encompasses $n$ derivation steps, we write this as:

$$b \Rightarrow^n w.$$

We illustrate this notation with an example:

$$\begin{aligned}
\textit{expr} &\Rightarrow \textit{expr}\ \text{“+”}\ \textit{product} \\
&\Rightarrow \textit{product}\ \text{“+”}\ \textit{product} \\
&\Rightarrow \textit{product}\ \text{“}*\text{”}\ \textit{factor}\ \text{“+”}\ \textit{product} \\
&\Rightarrow \textit{factor}\ \text{“}*\text{”}\ \textit{factor}\ \text{“+”}\ \textit{product} \\
&\Rightarrow \textsc{Number}\ \text{“}*\text{”}\ \textit{factor}\ \text{“+”}\ \textit{product} \\
&\Rightarrow \textsc{Number}\ \text{“}*\text{”}\ \textsc{Number}\ \text{“+”}\ \textit{product} \\
&\Rightarrow \textsc{Number}\ \text{“}*\text{”}\ \textsc{Number}\ \text{“+”}\ \textit{factor} \\
&\Rightarrow \textsc{Number}\ \text{“}*\text{”}\ \textsc{Number}\ \text{“+”}\ \textsc{Number}
\end{aligned}$$

Hence, we have demonstrated that

$$\textit{expr} \Rightarrow^* \textsc{Number}\ \text{“}*\text{”}\ \textsc{Number}\ \text{“+”}\ \textsc{Number}$$

is valid. To be more precise, we could express this as

$$\textit{expr} \Rightarrow^8 \textsc{Number}\ \text{“}*\text{”}\ \textsc{Number}\ \text{“+”}\ \textsc{Number}$$

since the derivation comprises eight steps. By substituting the terminal NUMBER with distinct numerals, we demonstrate that the string

2*3+4

is an arithmetic expression. Generally, the language $L(G)$ defined by a grammar $G$ is the set of all strings that are exclusively composed of terminals and can be derived from the start symbol $S$ of the grammar; that is, we define

$$L(G) := \{w \in T^* \mid S \Rightarrow^* w\}.$$

**Example**: The language

$$L = \left\{\text{“(”}^n\text{“)”}^n \;\middle|\; n \in \mathbb{N}\right\}$$

is generated by the grammar

$$G = \langle\{s\}, \{\text{“(”}, \text{“)”}\}, R, s\rangle,$$

where the set of rules $R$ is defined as:

$$s \to \text{“(”}s\text{“)”} \mid \lambda.$$

**Exercise 18**: We define $\Sigma = \{\text{“a”}, \text{“b”}\}$ and define the language $L$ as the set of words $w \in \Sigma^*$ in which the letters “a” and “b” occur with the same frequency:

$$L := \left\{w \in \Sigma^* \mid \textit{count}(w, \text{“a”}) = \textit{count}(w, \text{“b”})\right\}$$

Define a grammar $G$ such that $L = L(G)$. ◇

**Exercise 19**: Define $\Sigma := \{\text{“a”}, \text{“b”}\}$. In the previous chapter, we have already defined the reversal of a string $w = c_1c_2\cdots c_{n-1}c_n \in \Sigma^*$ as the string

$$w^R := c_nc_{n-1}\cdots c_2c_1.$$

A string $w \in \Sigma^*$ is called a palindrome if the string is identical to its reversal, i.e. if

$$w = w^R$$

holds true. For example, the strings

$$w_1 = \texttt{ababa} \quad \text{and} \quad w_2 = \texttt{abba}$$

are both palindromes, while the string abb is not a palindrome. The language of palindromes $L_{\text{palindrome}}$

is the set of all strings in $\Sigma^*$ that are palindromes, i.e. we have

$$L_{\text{palindrome}} := \{w \in \Sigma^* \mid w = w^R\}.$$

(a) Prove that the language $L_{\text{palindrome}}$ is a context-free language.

(b) Prove that the language $L_{\text{palindrome}}$ is not a regular language. ◇

### 6.1.2 Parse Trees

Using a grammar $G$, we can not only tell whether a given string $s$ is an element of the language $L(G)$ generated by the grammar, we can also structure the string by building a parse tree. If a grammar is

$$G = \langle V, T, R, S \rangle$$

given, a parse tree for this grammar is a tree satisfying the following conditions:

1. The tree consists of two types of nodes:
   (a) The leaf nodes are those nodes that have no outgoing edges.
   (b) The inner nodes are all those nodes that have outgoing edges.
2. The root is labeled with the start symbol $S$.
3. Each inner node is labeled with a variable.
4. Each leaf node is labeled with a terminal or with a variable.
5. If a leaf node is labeled with a variable $a$, then the grammar contains a rule of the form
   $$a \to \lambda.$$
6. If an inner node is labeled with a variable $a$ and the children of this node are labeled with the symbols $X_1, X_2, \cdots, X_n$, then the grammar $G$ contains a rule of the form
   $$a \to X_1 X_2 \cdots X_n.$$

If we read the leaf nodes of a parse tree from left to right and skip the leaves that are labeled with variables, we obtain a string $w \in L(G)$. Figure 6.2 shows a parse tree for the word "1+2*3". It is derived using the grammar given above for arithmetic expressions.

Since trees of the type shown in Figure 6.2 become too large very quickly we simplify these trees using the following rules:

1. If $n$ is an interior node labeled with the variable $A$ and among the children of this node there is exactly one child labeled with a terminal $o$ then we remove this child and label the node $n$ instead with the terminal $o$.
2. If an inner node has only one child, we replace that node with its child.
3. If a leaf node is labeled with either '(' or ')', then it is dropped.

We call the tree obtained in this way the abstract syntax tree. Figure 6.3 shows an abstract syntax tree that can be used to evaluate the arithmetic expression "2*3+4", because the tree shows us the order for evaluating the operators.

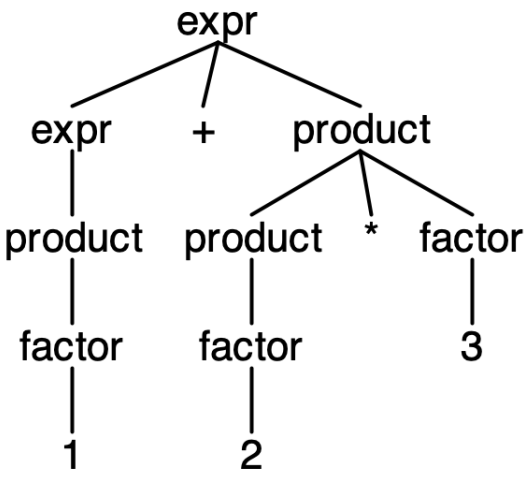



Figure 6.2: A parse tree for the string "1+2*3".

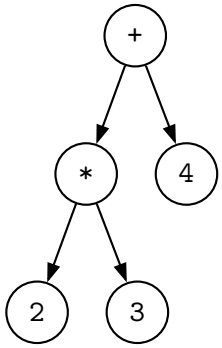



Figure 6.3: The abstract syntax tree for the string "2*3+4".

### 6.1.3 Ambiguous Grammars

The grammar given at the beginning of Section 6.1 to describe arithmetic expressions seems very complicated because it uses three different syntactic categories: *expr*, *product*, and *factor*. We introduce a simpler grammar $G$ which describes the same language:

$$G = \big\langle \{\mathit{expr}\}, \{\textsc{Number}, \textsc{Variable}, \text{“+”}, \text{“-”}, \text{“}*\text{”}, \text{“/”}, \text{“(”}, \text{“)”}\}, R, \mathit{expr} \big\rangle,$$

where the rules $R$ are given as follows:

$$\begin{array}{lcl} \mathit{expr} & \to & \mathit{expr}\ \text{“+”}\ \mathit{expr} \\ & | & \mathit{expr}\ \text{“-”}\ \mathit{expr} \\ & | & \mathit{expr}\ \text{“}*\text{”}\ \mathit{expr} \\ & | & \mathit{expr}\ \text{“/”}\ \mathit{expr} \\ & | & \text{“(”}\ \mathit{expr}\ \text{“)”} \\ & | & \textsc{Number} \\ & | & \textsc{Variable} \end{array}$$

In order to show that the string "2 ∗ 3+4" is in the language generated by this grammar, we give the following derivation:

$$\begin{aligned} \textit{expr} &\Rightarrow \textit{expr} \text{ “+” } \textit{expr} \\ &\Rightarrow \textit{expr} \text{ “}*\text{” } \textit{expr} \text{ “+” } \textit{expr} \\ &\Rightarrow 2 \text{ “}*\text{” } \textit{expr} \text{ “+” } \textit{expr} \\ &\Rightarrow 2 \text{ “}*\text{” } 3 \text{ “+” } \textit{expr} \\ &\Rightarrow 2 \text{ “}*\text{” } 3 \text{ “+” } 4 \end{aligned}$$

This derivation corresponds to the abstract syntax tree shown in Fig. 6.3. However, there is another derivation of the string "2 ∗ 3+4" with this grammar:

$$\begin{aligned} \textit{expr} &\Rightarrow \textit{expr} \text{ “}*\text{” } \textit{expr} \\ &\Rightarrow \textit{expr} \text{ “}*\text{” } \textit{expr} \text{ “+” } \textit{expr} \\ &\Rightarrow 2 \text{ “}*\text{” } \textit{expr} \text{ “+” } \textit{expr} \\ &\Rightarrow 2 \text{ “}*\text{” } 3 \text{ “+” } \textit{expr} \\ &\Rightarrow 2 \text{ “}*\text{” } 3 \text{ “+” } 4 \end{aligned}$$

This derivation corresponds to the abstract syntax tree shown in Fig. 6.4. In this derivation, the string "2 ∗ 3+4" is taken to be a product, which contradicts the convention that the operator "∗" binds stronger than the operator "+". If we were to evaluate the string using the last syntax tree, we would obviously get the wrong result! The reason for this problem is the fact that the last specified grammar is ambiguous.

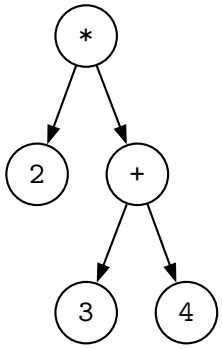



Figure 6.4: Another abstract syntax tree for the string "2 ∗ 3+4".

An ambiguous grammar is unsuitable for parsing. Unfortunately, the question of whether a given grammar is ambiguous is, in general, not decidable: It can be shown that the Post correspondence problem can be reduced to this question. Since Post's correspondence problem has been shown to be undecidable, the question whether a grammar is ambiguous is also undecidable. Proofs of these claims can be found, for example, in the book by Hopcroft, Motwani, and Ullman [HMU06].

## 6.2 Top-Down Parser

In this section, we present a method that can be used to conveniently parse a whole range of grammars. The basic idea is simple: In order to parse a string $w$ using a grammar rule of the form

$$a \to X_1 X_2 \cdots X_n$$

we try to parse an $X_1$ first. In doing so, we decompose the string $w$ into the form $w = w_1 r_1$ such that $w_1 \in L(X_1)$ holds. Then we try to find an $X_2$ in the residual string $r_1$, thus decomposing $r_1$ as

$r_1 = w_2 r_2$ where $w_2 \in L(X_2)$ holds. Continuing this process, we end up with the string $w$ being split as

$$w = w_1 w_2 \cdots w_n \quad \text{with } w_i \in L(X_i) \text{ for all } i = 1, \ldots, n.$$

Unfortunately, this procedure does not work when the grammar is left-recursive, that is, if it contains a rule of the form

$$a \to a\,\beta$$

because then to parse an *a* we would immediately try again to parse an *a* and thus we would be stuck in an infinite loop. There are two ways to deal with this kind of problem:

(a) We can rewrite the grammar so that it no longer is left-recursive.

(b) A simpler method is to extend the notion of a context-free grammar. We will use the notion of a so-called extended Backus-Naur form grammar (abbreviated as EBNF grammar). Theoretically, the expressive power of EBNF grammars is the same as the expressive power of context-free grammars. In practice, however, it turns out that the construction of top-down parsers for EBNF grammars is easier, because in an EBNF grammar the left recursion can often be replaced by iteration in a natural way.

In the rest of this chapter we will discuss the second of these two procedures in more detail using the grammar for arithmetic expressions as an example.

### 6.2.1 Implementing a Recursive Descent Parser that Uses an EBNF Grammar

In this section, we improve the convenience of context-free languages by admitting the regular expression operators "*", "|", and "?" on the right hand side of grammar rules. The meaning of these operators is the same as when these operators are used in the regular expressions of the programming language *Python*. Furthermore, the right hand side of a grammar rule can be structured using parentheses. Grammars of this new type are known as *extended Backus-Naur form* grammars, which is abbreviated as EBNF grammars.

$$\begin{array}{rcl} \mathit{expr} & \to & \mathit{product}\ \big((\text{“+”} \mid \text{“-”})\ \mathit{product}\big)^* \\ \mathit{product} & \to & \mathit{factor}\ \big((\text{“*”} \mid \text{“/”})\ \mathit{factor}\big)^* \\ \mathit{factor} & \to & \text{“(”}\ \mathit{expr}\ \text{“)”} \\ & \mid & \textsc{Number} \end{array}$$

Figure 6.5: EBNF grammar for arithmetical expressions.

It can be shown that the languages described by EBNF grammars are still context-free languages. Therefore, these operators do not change the expressive power of context-free grammars. However, it is often much more convenient to describe a language using an EBNF grammar rather than using a context-free grammar. Figure 6.5 displays an EBNF grammar for arithmetical expressions.
Figure 6.6 shows a recursive descent parser that implements this grammar.

1. The function `parseExpr` recognizes a `product` in line 2. The value of this `product` is stored in the variable `result` together with the list `Rest` of those tokens that have not been consumed yet. If the list `Rest` contains at least two tokens and the first token in this list is either the

operator "+" or the operator "-", then the function `parseExpr` tries to recognize more products. These are added to or subtracted from the `result` computed so far in line 7 or 9. If there are no more products to be parsed, the `while` loop terminates and the function returns the `result` together with the list of the remaining tokens `Rest`.

2. The function `parseProduct` recognizes a `factor` in line 13. The value of this `factor` is stored in the variable `result` together with the list `Rest` of those tokens that have not been consumed yet. If the list `Rest` contains at least two tokens and the first token in this list is either the operator "*" or the operator "/", then the function `parseProduct` tries to recognize more factors. The `result` computed so far is multiplied with or divided by these factors in line 18 or 20. If there are no more factors to be parsed, the `while` loop terminates and the function returns the `result` together with the list `Rest` of tokens that have not been consumed.
3. The function `parseFactor` recognizes a `factor`. This is either an expression in parentheses or a number.
   - If the first token is an opening parenthesis, the function tries to parse an expression next. This expression has to be followed by a closing parenthesis. The tokens following this closing parenthesis are not consumed but rather are returned together with the result of evaluating the expression.
   - Otherwise, the first token has to be a number. It is converted into a floating point number, which is returned together with the list of all those tokens that have not been consumed.

**Exercise 20**: Develop an EBNF grammar for regular expressions and implement a top-down parser for this grammar. The resulting parser should return a nested tuple that represents a regular expression. ◇

**Historical Notes** The language ALGOL [Bac59, NBB+60] was the first programming language whose syntax was defined by a context-free grammar. This grammar was written in Backus-Naur form (BNF), the notation that was later extended to EBNF.

```python
def parseExpr(TL: list[str]) -> tuple[float, list[str]]:
    result, Rest = parseProduct(TL)
    while len(Rest) >= 2 and Rest[0] in {'+', '-'}:
        operator = Rest[0]
        arg, Rest = parseProduct(Rest[1:])
        if operator == '+':
            result += arg
        else:               # operator == '-':
            result -= arg
    return result, Rest

def parseProduct(TL: list[str]) -> tuple[float, list[str]]:
    result, Rest = parseFactor(TL)
    while len(Rest) >= 2 and Rest[0] in {'*', '/'}:
        operator = Rest[0]
        arg, Rest = parseFactor(Rest[1:])
        if operator == '*':
            result *= arg
        else:               # operator == '/':
            result /= arg
    return result, Rest

def parseFactor(TL: list[str]) -> tuple[float, list[str]]:
    if TL[0] == '(':
        expr, Rest = parseExpr(TL[1:])
        assert Rest[0] == ')', \
               f"ERROR: ')' expected, got {Rest[0]}"
        return expr, Rest[1:]
    else:
        return float(TL[0]), TL[1:]
```

Figure 6.6: A recursive descent parser for the grammar in Figure 6.5.

## 6.3 Check your Understanding

(a) Define the concept of a context-free grammar.

(b) Assume that $G$ is a context-free grammar. How is the language $L(G)$ defined?

(c) What is the definition of a parse tree?

(d) How do we transform a parse tree into an abstract syntax tree?

(e) What are ambiguous grammars?

(f) Define the notion of an EBNF grammar.

(g) Implement a top-down parser for arithmetic expressions.

# Chapter 7

# Using Lark as a Parser Generator

*Give a man a parser and he can read one programming language.*
*Give a man a parser generator and he can read all programming languages.*

— Carlos Ben Yussuf

*Build a witch a fire and you keep her warm for one night.*
*Set a witch on fire and you keep her warm for the rest of her life.*

— Pope Hugo The Terrible

Figure 7.1: The Lark parser generator in action.

Many modern programming languages, e.g. C, *Java*, or Go can be parsed using a so-called LALR(1)-

Parser[1]. Therefore, this chapter introduces the parser generator Lark, which can be used to generate a parser for any language that has an Lalr(1) grammar.[2] In Chapter 3 we have already seen how a scanner can be implemented with the help of the module `re`. Lark goes one step further: Both the lexical structure and the syntactical structure of a language are described in a *single* grammar, and Lark generates the scanner *and* the parser from this description. Hence, this chapter no longer has to distinguish between the specification of a scanner and the specification of a parser. We will discuss two examples that use Lark as a parser generator.

1. First, we implement a symbolic calculator.
2. In our second example, we develop a program for symbolic differentiation. This example shows how a parser can be used to construct an abstract syntax tree.

In Chapter 8 we will then use Lark to create an interpreter for a language that is a small fragment of the programming language C.

**Remark**: Traditional parser generators for *Python*, for example Ply, encode every grammar rule as a *Python* function whose document string contains the rule, and the semantic action associated with a rule is the body of this function. Therefore, syntax and semantics are interleaved. Lark takes a different approach: The grammar is written down as an ordinary string using a dialect of the extended Backus-Naur form Ebnf, while the semantics is implemented separately. This separation has two advantages: The grammar remains readable as a grammar, and the same grammar can be reused for different purposes. ◇

## 7.1 A Symbolic Calculator

Figure 7.2 on page 110 shows the grammar of a simple symbolic calculator, i.e. a calculator that supports the use of variables.

In order to generate a symbolic calculator that is based on this grammar, we have to transcribe the grammar shown in Figure 7.2 into the Ebnf dialect that is understood by Lark. Figure 7.3 on page 111 shows the result. Note that there is no separate scanner specification: The terminals are declared inside the very same string that also declares the grammar rules.
Figure 7.3 is discussed line by line.

1. Line 1 imports the class `Lark`, which is responsible for computing the parse table, and the class `Token`. A `Token` represents a leaf of the parse tree, i.e. an object that has been produced by the scanner.
2. Line 3 – 31 define the string `calc_grammar`. Note that this is an ordinary *Python* string: As far as *Python* is concerned, it has no structure at all. Only when this string is handed to Lark in line 33 is it interpreted as a grammar.

   Lark distinguishes syntactical variables from tokens by their spelling: The names of syntactical variables have to be written in *lower case*, while the names of tokens have to be written in *upper case*. Therefore, `expr` is a syntactical variable, while NUMBER is a token.

[1] LALR(1) is an acronym that stands for Look-Ahead Left-to-Right parsing with 1 token of lookahead. We will discuss the theory of Lalr(1) parsing at the end of Chapter 10.

[2] Lark is a pure *Python* library that can be installed via the command

```
pip install lark.
```

Besides Lalr(1) parsing, Lark also implements the Earley algorithm, which is able to parse *every* context-free grammar. We will discuss the Earley algorithm in a later chapter.

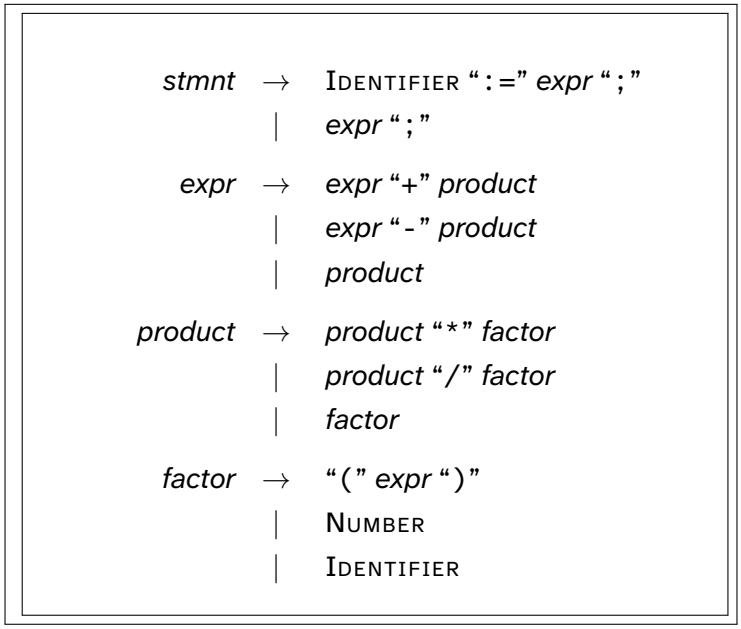



Figure 7.2: A grammar for a symbolic calculator.

3. Line 4 specifies that the syntactical variable `stmnt` is the start symbol of the grammar.

   The prefix "?" that is attached to the variable `start` is an inlining directive: If a rule that is marked with "?" produces a node that has exactly one child, then this node is discarded and its child is used in its place. Without this directive, the parse tree would contain a lot of nodes that have a single child and that carry no information.

4. Line 6 – 7 define the syntactical variable `stmnt`. This definition corresponds to the two grammar rules

   $$\textit{stmnt} \rightarrow \textsc{Identifier}\ \text{":="}\ \textit{expr}\ \text{";"} \quad \text{and} \quad \textit{stmnt} \rightarrow \textit{expr}\ \text{";"}$$

   of the grammar shown in Figure 7.2. Observe that the different alternatives of a rule are separated by the symbol "|", exactly as in the mathematical notation that we have used so far, and that literal strings like ":=" are simply written down as *Python* strings.

   The construct "`-> assign`" is an alias. It specifies the name of the node that is created in the parse tree when this alternative is used. Without an alias, all nodes created by the variable `stmnt` would be called `stmnt` and we would have no easy way to distinguish an assignment from an expression statement. Note further that an alias also suppresses the inlining that is requested by the prefix "?". Therefore, a node of the form `expr_stmnt` is created even though it has just one child.

5. Line 9 – 15 define the variables `expr` and `product`. These rules are literal transcriptions of the corresponding rules of Figure 7.2. In particular, they encode the precedence of the arithmetic operators, and they are left recursive. Left recursion is not a problem here, since LALR(1) parsing is a bottom-up technique.

6. Line 17 – 19 define the variable `factor`. The first alternative in line 17 does not have an alias. The reason is that the tokens "(" and ")" are discarded by LARK and hence this alternative produces a node with a single child. As the variable `factor` is marked with the prefix "?", this node is inlined and the parentheses leave no trace in the parse tree. This is exactly what we want, because the only purpose of the parentheses is to override the precedence of the operators.

```
1  from lark import Lark, Token
2
3  calc_grammar = """
4  ?start: stmnt
5
6  ?stmnt: IDENTIFIER ":=" expr ";" -> assign
7        | expr ";"                 -> expr_stmnt
8
9  ?expr: expr "+" product          -> add
10      | expr "-" product          -> sub
11      | product
12
13 ?product: product "*" factor     -> mul
14         | product "/" factor     -> div
15         | factor
16
17 ?factor: "(" expr ")"
18        | NUMBER                 -> number
19        | IDENTIFIER             -> var
20
21 // Lexical definitions
22 %import common.CNAME -> IDENTIFIER
23 %import common.NUMBER
24 %import common.WS
25
26 ASSIGN: ":="
27 SEMI:   ";"
28
29 // Ignore whitespace
30 %ignore WS
31 """
32
33 parser = Lark(calc_grammar, parser='lalr')
```

Figure 7.3: The grammar of the symbolic calculator, written for LARK.

7. Line 22 – 24 import predefined token definitions from the library `common` that is shipped with LARK. Instead of writing our own regular expressions we reuse the following definitions:
   (a) `CNAME` matches an identifier in the sense of the language C. The construct "`-> IDENTIFIER`" renames this token to `IDENTIFIER`.
   (b) `NUMBER` matches both integers and floating point numbers.
   (c) `WS` matches white space, i.e. blanks, tabulators, and newlines.
8. Line 26 – 27 give explicit names to the tokens "`:=`" and "`;`". These two declarations do not change the language that is accepted: Both of these tokens are still discarded when the parse tree is built. Their only purpose is to ensure that these tokens have readable names instead of

the names that Lark would generate automatically. This is convenient once we start to inspect the Lalr(1) parse table.

9. Line 30 instructs the scanner to discard all white space.

10. Line 33 creates the parser. The keyword argument `parser='lalr'` selects the Lalr(1) algorithm. It is at this point that the parse table is computed. If the given grammar is not an Lalr(1) grammar, then this line raises an exception.

Once the parser has been created, a string `s` can be parsed by evaluating the expression

```
parser.parse(s).
```

This expression returns an object of class `lark.Tree`. A `Tree` has two attributes:

1. `data` is the name of the node. This is either the name of the syntactical variable that has produced this node, or, if an alias has been specified, the name given by this alias.

2. `children` is the list of the subtrees of this node. The elements of this list are either objects of class `Tree` or objects of class `Token`. As the class `Token` is a subclass of `str`, a token can be used like a string.

Note that the parse tree returned by `parser.parse` does not know anything about the *meaning* of the program that has been parsed. Providing this meaning is our job and we do it in two steps:

1. First, we convert the object of class `lark.Tree` into an abstract syntax tree that is represented as a nested tuple.

2. Second, we evaluate this nested tuple.

Admittedly, Lark offers the class `Transformer`, which could be used to evaluate the parse tree directly. However, the two-step approach has the advantage that the abstract syntax tree is a pure *Python* object: It does not depend on Lark in any way and we can inspect it, print it, or store it without having to know anything about the internals of the parser.
Figure 7.4 on page 113 shows the function `ast_to_tuples` that performs the first of these steps. This function is discussed next.

1. Line 2 – 3 deal with the base case: If the given node is a `Token`, then it is a leaf of the parse tree. In this case we return the string that is stored in this token.

2. Line 4 – 13 define the dictionary `operator_map`. This dictionary translates the node names that are used by Lark into the operator symbols that we want to use in our abstract syntax tree. For example, the node name `add`, which has been introduced as an alias in line 9 of Figure 7.3, is translated into the string "+".

3. Line 14 looks up the name of the given node.

4. Line 15 converts all children of the given node recursively.

5. Line 16 returns the resulting nested tuple: The first component is the operator, the remaining components are the converted children.

**Example**: The string

```
"x := 2 * (y + 1);"
```

is converted into the nested tuple

```
(':=', 'x', ('*', ('num', '2'), ('+', ('var', 'y'), ('num', '1')))).
```

Note the following details:

```
1    def ast_to_tuples(node):
2        if isinstance(node, Token):
3            return str(node)
4        operator_map = {
5            'assign':     ':=',
6            'expr_stmnt': 'eval',
7            'add':        '+',
8            'sub':        '-',
9            'mul':        '*',
10           'div':        '/',
11           'number':     'num',
12           'var':        'var'
13       }
14       op       = operator_map[node.data]
15       children = [ast_to_tuples(child) for child in node.children]
16       return (op, *children)
```

Figure 7.4: Converting a parse tree into an abstract syntax tree.

1. The parentheses of the input string have disappeared, since they are already reflected by the structure of the tuple.

2. The identifier x on the left hand side of the assignment operator is represented by the plain string "x", while the identifier y is represented by the tuple `('var', 'y')`. The reason is that x is the *name* of the variable that is assigned to, while y denotes the *value* of a variable and hence has to be looked up.

3. The numbers are still represented as strings. They are converted into floating point numbers only when the abstract syntax tree is evaluated. ◇

Figure 7.5 on page 114 shows the function `evaluate_adt` that implements the semantics of our calculator.

1. Line 1 initializes the dictionary `Names2Values`. For every identifier $x$ that is defined interactively, `Names2Values[`$x$`]` is the value associated with $x$.

2. Line 4 starts a `match` statement. Structural pattern matching is the natural way to work with an abstract syntax tree that is represented as a nested tuple: Every case of the `match` statement corresponds to one of the operators that occur in our abstract syntax tree.

3. Line 5 – 6 evaluate a number. This is the place where the string produced by the scanner is converted into a floating point number.

4. Line 7 – 11 evaluate a variable. If the variable has not been defined previously, an error message is printed and the value $0.0$ is returned.

5. Line 12 – 13 implement an assignment: The expression on the right hand side is evaluated and the resulting value is stored in the dictionary `Names2Values`. Note that an assignment does not return a value.

```
1  Names2Values = {}
2
3  def evaluate_adt(adt):
4      match adt:
5          case ('num', value):
6              return float(value)
7          case ('var', name):
8              if name not in Names2Values:
9                  print(f"Error: variable '{name}' is undefined.")
10                 return 0.0
11             return Names2Values[var_name]
12         case (':=', var_name, expr):
13             Names2Values[var_name] = evaluate_adt(expr)
14         case ('eval', expr):
15             print(evaluate_adt(expr))
16         case (('+' | '-' | '*' | '/') as op, left, right):
17             val_left, val_right = map(evaluate_adt, [left, right])
18             match op:
19                 case '+': return val_left + val_right
20                 case '-': return val_left - val_right
21                 case '*': return val_left * val_right
22                 case '/': return val_left / val_right
23         case _:
24             raise ValueError(f'Unknown ADT structure: {adt}')
```

Figure 7.5: Evaluating the abstract syntax tree.

6. Line 14 – 15 implement an expression statement: The expression is evaluated and the resulting value is printed.

7. Line 16 – 22 evaluate the arithmetic operators. The pattern in line 16 uses an or pattern in order to capture all four binary operators at once. The keyword `as` binds the operator that has actually been found to the variable `op`. Then, both operands are evaluated recursively and the inner `match` statement in line 19 – 23 selects the operation that has to be performed.

8. Line 23 – 24 catch the case that the abstract syntax tree has a shape that is not supported. If this happens, we have made a mistake, either in the grammar or in the function `ast_to_tuples`.

Finally, Figure 7.6 on page 115 shows the function `main`, which is a read-eval-print loop: It reads a string `s` from the command line and then performs the three steps that we have discussed: Line 7 parses the string, line 8 converts the resulting parse tree into an abstract syntax tree, and line 9 evaluates this tree. The loop is terminated as soon as the user enters an empty line.

Observe how Lark reports errors: Instead of calling an error function that has been provided by the user, Lark raises an exception. Both lexical errors and syntax errors are signalled by subclasses of the class `lark.exceptions.UnexpectedInput`. As these exceptions carry the position where the error has occurred, the messages produced by Lark are usually much more informative than the messages that we could produce ourselves with a modest amount of effort. The `try-except`

```
1  def main():
2      while True:
3          try:
4              s = input('calc> ')
5              if s == '':
6                  break
7              raw_tree   = parser.parse(s)
8              adt_tuples = ast_to_tuples(raw_tree)
9              evaluate_adt(adt_tuples)
10         except Exception as e:
11             print(f'Error: {e}')
```

Figure 7.6: A read-eval-print loop for the symbolic calculator.

statement in line 3 and line 10 – 11 catches these exceptions so that a single erroneous input does not terminate the calculator.

The Jupyter notebook `01-Calculator`, which is available at

Python/Chapter-07/01-Calculator.ipynb

implements the symbolic calculator discussed in this section.

**Remark**: The notebook `01-Calculator` also demonstrates how the LALR(1) states that have been computed by LARK can be printed. At this point, these states will not make much sense. However, it is worthwhile to return to this part of the notebook once we have discussed the theory of LALR(1) parsing in Chapter 10. ◇

**Exercise 21**: Extend the grammar shown in Figure 7.3 so that the calculator supports the exponentiation operator "`**`" and the unary minus. The exponentiation operator binds stronger than multiplication and it is right associative, i.e. the expression `2 ** 3 ** 2` has the value $2^{(3^2)} = 512$. Furthermore, extend the functions `ast_to_tuples` and `evaluate_adt` accordingly.

**Hint**: Introduce a new syntactical variable `power` between `product` and `factor` and make this variable right recursive.

## 7.2 Symbolic Differentiation

Our second example shows how a parser can be used to construct an abstract syntax tree that is afterwards transformed by a program. Concretely, we implement symbolic differentiation: Given a string `s` that represents an arithmetic expression containing the variable `x`, we want to compute a string that represents the derivative

$$\frac{\mathrm{d}}{\mathrm{d}x}\mathtt{s}.$$

For example, given the input string `"x*exp(x)"` the program should produce the output string

```
"exp(x) + x*exp(x)"
```

since we have

$$\frac{\mathrm{d}}{\mathrm{d}x}\left(x \cdot \mathrm{e}^x\right) = \mathrm{e}^x + x \cdot \mathrm{e}^x.$$

In contrast to the calculator of the previous section, the abstract syntax tree is no longer evaluated immediately. Instead, it is the *data* that our program works on: It is transformed into a second abstract syntax tree that represents the derivative, this tree is then simplified, and only at the very end is it converted back into a string. This is the reason why building an explicit abstract syntax tree pays off.

Figure 7.7 on page 116 shows the grammar of the arithmetic expressions that our program supports. Besides the four basic arithmetic operators, this grammar supports the exponentiation operator "`**`", the unary minus, and the two functions $\exp$ and $\ln$. For the sake of simplicity, `x` is the only variable.

| | | |
|---|---|---|
| *expr* | → | *expr* "`+`" *product* |
| | \| | *expr* "`-`" *product* |
| | \| | *product* |
| *product* | → | *product* "`*`" *unary* |
| | \| | *product* "`/`" *unary* |
| | \| | *unary* |
| *unary* | → | "`-`" *unary* |
| | \| | *factor* |
| *factor* | → | *base* "`**`" *unary* |
| | \| | *base* |
| *base* | → | "`exp`" "`(`" *expr* "`)`" |
| | \| | "`ln`" "`(`" *expr* "`)`" |
| | \| | "`(`" *expr* "`)`" |
| | \| | NUMBER |
| | \| | "`x`" |

Figure 7.7: A grammar for differentiable arithmetic expressions.

Figure 7.8 on page 117 shows the transcription of this grammar into the EBNF dialect of LARK.
As the general structure of this grammar is the same as the structure of the grammar that we have discussed in the previous section, we only comment on those parts that are new.

1. Line 4 declares the variable `expr` as the start symbol. In contrast to the calculator, this language has no statements: The input is a single expression and there is no terminating semicolon.
2. Line 6 – 12 define the variables `expr` and `product` in the usual way. Note that the second operand of the operators "`*`" and "`/`" is now the variable `unary` instead of the variable `factor`.
3. Line 14 – 15 define the variable `unary`, which implements the unary minus. This rule is right recursive, since a unary operator can be applied repeatedly: The string `"- -x"` is a legal expression.
4. Line 17 – 18 define the variable `factor`, which implements exponentiation. The interplay between `unary` and `factor` deserves a closer look, because it encodes two conventions that

```
from lark import Lark, Token

calc_grammar = """
?start: expr

?expr: expr "+" product   -> add
     | expr "-" product   -> sub
     | product

?product: product "*" unary -> mul
        | product "/" unary -> div
        | unary

?unary: "-" unary           -> neg
      | factor

?factor: base "**" unary    -> pow
       | base

?base: "exp" "(" expr ")"    -> exp
     | "ln" "(" expr ")"     -> ln
     | "(" expr ")"
     | NUMBER                -> number
     | "x"                   -> var

%import common.NUMBER
%import common.WS
%ignore WS
"""

parser = Lark(calc_grammar, parser='lalr')
```

Figure 7.8: The grammar of the differentiator, written for LARK.

every mathematician takes for granted:

(a) The left operand of the operator "`**`" is a `base`, not a `unary` operator. Therefore, the string

```
"-x ** 2"
```

is parsed as $-\left(x^2\right)$ and not as $(-x)^2$, i.e. exponentiation binds stronger than the unary minus.

(b) The right operand of the operator "`**`" is a `unary`, not a `base`. Therefore, the string

```
"x ** -x"
```

is a legal expression and, furthermore, the operator "`**`" is right associative: The string

```
"x ** x ** x"
```

is parsed as the expression $x^{(x^x)}$.

5. Line 20 – 24 define the variable `base`. The strings `"exp"` and `"ln"` in line 20 and line 21 are ordinary literals, i.e. are recognized in exactly the same way as the operator symbols. The nodes `exp` and `ln` have a single child.

6. Line 24 shows that the only variable of our expressions is `x`. Since `x` is written as a literal string, we do not need a token for identifiers at all.

Figure 7.9 on page 118 shows the function `ast_to_tuples` that converts the parse tree produced by Lark into a nested tuple, together with the function `parse` that combines both steps.

```
1   operatorTable = {'add': '+', 'sub': '-',
2                    'mul': '*', 'div': '/', 'pow': '**'}
3
4   def ast_to_tuples(node):
5       if isinstance(node, Token):
6           return str(node)
7       match node.data:
8           case 'number':
9               return int(node.children[0])
10          case 'var':
11              return 'x'
12          case op_name:
13              op_sym = operatorTable.get(op_name, op_name)
14              return (op_sym, *map(ast_to_tuples, node.children))
15
16  def parse(s):
17      return ast_to_tuples(parser.parse(s))
```

Figure 7.9: Converting the parse tree into an abstract syntax tree.

This function serves the same purpose as the function of the same name that we have discussed in the previous section. However, it is organized differently: Instead of providing a separate case for every operator, it is table driven.

1. Line 1 – 2 define the dictionary `operatorTable`. This dictionary maps the names of the nodes that are created by Lark to the operator symbols that we want to use in our abstract syntax tree. For example, the node name `add`, which has been introduced as an alias in line 6 of Figure 7.8, is mapped to the string “+”.

   Observe that the node names `neg`, `exp`, and `ln` do *not* occur in this dictionary. The reason is that we want to use these names unchanged in the abstract syntax tree. Line 13 shows how this is achieved.

2. Line 5 – 6 deal with the base case: If the given node is a `Token`, then it is a leaf of the parse tree and hence we return the string that is stored in this token.

3. Line 7 starts a `match` statement that inspects the attribute `node.data`, i.e. the name of the node. As this name is an ordinary string, all patterns of this `match` statement are patterns for strings.

4. Line 8 – 9 convert a number. In contrast to the calculator of the previous section, the conversion of a string into a number happens already at this point, because the abstract syntax tree is not evaluated later but rather transformed symbolically.

   We will tacitly assume that all numbers are integers.

5. Line 10 – 11 convert the variable `x`. Since the string `"x"` occurs as an anonymous literal in the grammar, it is discarded when the parse tree is built. Therefore, the node `var` has no children at all and we simply return the string "`x`".

6. Line 12 is the most interesting line of this function. The pattern `op_name` is a capture pattern: As `op_name` is a plain variable name, this pattern matches *any* value and binds this value to the variable `op_name`. Hence, this case is the default case that deals with all remaining nodes, i.e. with all nodes that represent an operator.

7. Line 13 translates the name of the node into an operator symbol. The second argument of the method `get` is the value that is returned if the given key is not present in the dictionary. Here, this default value is the name of the node itself. This is the trick that has been announced above: Every node name that is not listed in `operatorTable` is used unchanged. Consequently, the nodes `neg`, `exp`, and `ln` are converted into tuples that start with the strings "`neg`", "`exp`", and "`ln`", respectively.

8. Line 14 converts all children recursively and prepends the operator symbol. The expression

   ```
   map(ast_to_tuples, node.children)
   ```

   applies the function `ast_to_tuples` to every child of the given node, while the operator "`*`" unpacks the resulting iterator into the tuple that is created. Note that this works for unary and for binary operators alike: The number of components of the resulting tuple is determined by the number of children of the node.

9. Line 16 – 17 define the function `parse`, which combines both steps: It parses the string `s` and immediately converts the resulting parse tree into an abstract syntax tree.

**Example**: The string `"ln(x ** x) + exp(x * -x)"` is converted into the nested tuple

`('+', ('ln', ('**', 'x', 'x')), ('exp', ('*', 'x', ('neg', 'x'))))`. ◇

**Remark**: The table driven organization of the function `ast_to_tuples` pays off as soon as the language is extended. In order to support an additional operator or an additional function, it is usually sufficient to extend the grammar: If the alias of the new grammar rule is already the symbol that we want to use in the abstract syntax tree, then the function `ast_to_tuples` does not have to be changed at all. Otherwise, a single entry has to be added to the dictionary `operatorTable`. ◇

Now that we can turn a string into an abstract syntax tree, we are ready to implement the function `diff`. This function takes an abstract syntax tree $e$ and computes an abstract syntax tree that represents the derivative of $e$ with respect to the variable `x`. Figure 7.10 on page 120 shows the implementation. Every case of the `match` statement corresponds to one of the differentiation rules that are taught in any calculus course.
The function `diff` is discussed next.

1. Line 4 – 6 implement the sum rule $\dfrac{\mathrm{d}}{\mathrm{d}x}\big(f(x) + g(x)\big) = \dfrac{\mathrm{d}}{\mathrm{d}x} f(x) + \dfrac{\mathrm{d}}{\mathrm{d}x} g(x)$,
   while line 7 – 9 implement the corresponding rule for the difference of two expressions.

2. Line 10 – 12 implement the product rule

```
def diff(e):
    "differentiate the expression e with respect to the variable x"
    match e:
        case ('+', f, g):
            fs, gs = diff(f), diff(g)
            return ('+', fs, gs)
        case ('-', f, g):
            fs, gs = diff(f), diff(g)
            return ('-', fs, gs)
        case ('*', f, g):
            fs, gs = diff(f), diff(g)
            return ('+', ('*', fs, g), ('*', f, gs))
        case ('/', f, g):
            fs, gs = diff(f), diff(g)
            return ('/', ('-', ('*',fs,g), ('*',f,gs)), ('*',g,g))
        case ('**', f, g):
            return diff(('exp', ('*', g, ('ln', f))))
        case ('neg', f):
            return ('neg', diff(f))
        case ('ln', f):
            fs = diff(f)
            return ('/', fs, f)
        case ('exp', f):
            fs = diff(f)
            return ('*', fs, e)
        case 'x':
            return 1
    return 0
```

Figure 7.10: Symbolic differentiation.

$$\frac{\mathrm{d}}{\mathrm{d}x}\big(f(x)\cdot g(x)\big) = \Big(\frac{\mathrm{d}}{\mathrm{d}x}f(x)\Big)\cdot g(x) + f(x)\cdot\Big(\frac{\mathrm{d}}{\mathrm{d}x}g(x)\Big).$$

3. Line 13 – 15 implement the quotient rule

$$\frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{f(x)}{g(x)}\right) = \frac{\left(\frac{\mathrm{d}}{\mathrm{d}x}f(x)\right)\cdot g(x) - f(x)\cdot\left(\frac{\mathrm{d}}{\mathrm{d}x}g(x)\right)}{g(x)\cdot g(x)}.$$

4. Line 16 – 17 deal with exponentiation. Instead of implementing a separate rule for the expression $f(x)^{g(x)}$, we use the identity

   $$f(x)^{g(x)} = \exp\big(g(x)\cdot\ln(f(x))\big)$$

   and differentiate the resulting expression instead. This is an instance of a general and very useful strategy: Rather than adding a new rule, we rewrite the expression so that it can be handled by the rules that we already have. Note that this identity is only valid if $f(x)$ is positive. Our implementation ignores this side condition.

5. Line 18 – 19 implement the rule $\frac{\mathrm{d}}{\mathrm{d}x}\big(-f(x)\big) = -\frac{\mathrm{d}}{\mathrm{d}x}f(x)$.
6. Line 20 – 22 and line 23 – 25 implement the chain rule for the two functions that our language provides:

   $$\frac{\mathrm{d}}{\mathrm{d}x}\ln\big(f(x)\big) = \frac{\frac{\mathrm{d}}{\mathrm{d}x}f(x)}{f(x)} \qquad \text{and} \qquad \frac{\mathrm{d}}{\mathrm{d}x}\exp\big(f(x)\big) = \Big(\frac{\mathrm{d}}{\mathrm{d}x}f(x)\Big)\cdot\exp\big(f(x)\big).$$

   Observe that line 25 reuses the expression `e` instead of building the tree `('exp', f)` a second time.
7. Line 26 – 27 implement the base case $\frac{\mathrm{d}x}{\mathrm{d}x} = 1$.
8. Line 28 is reached if none of the previous patterns did match. In this case, the expression does not contain the variable `x` and therefore it is a constant. As the derivative of a constant is $0$, the value $0$ is returned.

The abstract syntax tree that is computed by the function `diff` is correct, but it is unreadable. For example, differentiating the string `"x * exp(x)"` yields a tree that corresponds to the string

```
"1*exp(x) + x*1*exp(x)".
```

Therefore, two more functions are needed. Since neither of them has anything to do with parsing, we only describe what they do and refer the interested reader to the notebook for the implementation.

1. The function `simplify` takes an abstract syntax tree $e$ and returns an abstract syntax tree that is mathematically equivalent to $e$ but, in general, smaller. The function works bottom-up: It simplifies all subexpressions first and only then tries to apply a simplification rule to the resulting expression. Some of the rules produce a new expression that can be simplified further. In these cases, the function `simplify` is applied recursively to the result.

   The rules that are implemented are listed below. Here, $a$, $b$, and $f$ denote arbitrary expressions, while $c$ and $d$ denote numbers.

   (a) **Constant folding**: If both operands of one of the operators `+`, `-`, `*`, `/`, or `**` are numbers, then the operation is carried out. A division is only performed if the divisor is different from $0$.

   (b) **Negation**: $-(-a) \to a, \quad -0 \to 0, \quad -c \to d$ where $d$ is the negated number $c$.

   (c) **Addition and subtraction**:
   $a + 0 \to a, \quad 0 + a \to a, \quad a - 0 \to a, \quad 0 - a \to -a, \quad a - a \to 0,$
   $a + (-b) \to a - b, \quad (-a) + b \to b - a, \quad a - (-b) \to a + b.$

   (d) **Multiplication**:
   $a \cdot 0 \to 0, \quad 0 \cdot a \to 0, \quad a \cdot 1 \to a, \quad 1 \cdot a \to a, \quad (-1) \cdot a \to -a, \quad a \cdot (-1) \to -a,$
   $a \cdot \frac{1}{a} \to 1, \quad (-a) \cdot \frac{1}{a} \to -1, \quad a \cdot \Big(-\frac{1}{a}\Big) \to -1.$

   (e) **Division**: $\frac{0}{a} \to 0, \quad \frac{a}{1} \to a, \quad \frac{a}{a} \to 1.$

   (f) **Powers**: $a^0 \to 1, \quad a^1 \to a, \quad 0^a \to 0, \quad 1^a \to 1.$

   (g) **Logarithm and exponential function**:
   $\ln(1) \to 0, \quad \ln\big(\exp(a)\big) \to a, \quad \exp(0) \to 1, \quad \exp\big(\ln(a)\big) \to a,$
   $\exp\big(a \cdot \ln(f)\big) \to f^a.$
   The last of these rules undoes the rewriting that the function `diff` has performed in order to differentiate the operator "`**`".

(h) **Derivatives of powers**: These rules clean up the expressions that are produced when a power is differentiated. For a number $c$ we have

$$a \cdot \frac{1}{f} \cdot f^c \to a \cdot f^{c-1} \qquad \text{and} \qquad \frac{1}{f} \cdot f^c \to f^{c-1}.$$

In the special case $c = 2$ the results are simplified further to $a \cdot f$ and $f$, respectively.

2. The function `toString` takes an abstract syntax tree $e$ and returns a string that represents $e$. The interesting part of this function is that it does not insert parentheses blindly: A subexpression is only enclosed in parentheses if the precedence of its operator is lower than the precedence that is required at this position. To this end, the function uses the precedence table

| | | |
|---|---|---|
| "+", "-" | : | 1, |
| "*", "/" | : | 2, |
| unary minus | : | 3, |
| "**" | : | 4, |
| numbers and x | : | 5. |

Since the operators "-" and "/" are not associative, the right operand of these operators is parenthesized already when its operator has the *same* precedence. For the operator "**" it is the other way round: As this operator is right associative, its right operand needs no parentheses if it is a power itself, while its left operand is parenthesized whenever it is not an atom. Therefore, the expression $x^{(x^x)}$ is printed as `x**x**x`, while $(x+1)^2$ is printed as `(x + 1)**2`.

Putting everything together, the derivative of a string `s` is computed by the expression

```
toString(simplify(diff(parse(s)))).
```

The following table shows the results that are produced for a number of inputs.

| input s | computed derivative |
|---|---|
| `x*x` | `x + x` |
| `x/x` | `0` |
| `x * exp(x)` | `exp(x) + x*exp(x)` |
| `-x ** 2` | `-2*x` |
| `x ** -x` | `(-1 - ln(x))*x**(-x)` |
| `1/(1+exp(-x))` | `exp(-x)/((1 + exp(-x))*(1 + exp(-x)))` |
| `ln(x ** x)` | `(ln(x) + 1)*x**x/x**x` |

**Remark**: The last two rows of this table show the limits of our simplifier. The derivative of `x*x` is computed as `x + x` instead of `2*x`, and the derivative of `ln(x ** x)` is not reduced to `ln(x) + 1`, although the factor $x^x/x^x$ obviously cancels. The reason is that our function `simplify` is nothing but a finite collection of rewrite rules: It does not bring an expression into a normal form and therefore it cannot recognize that two syntactically different expressions are mathematically equal. Computing normal forms of algebraic expressions is a hard problem that is studied in the field of computer algebra. ◇

The Jupyter notebook `02-Differentiator`, which is available at

Python/Chapter-07/02-Differentiator.ipynb

implements the symbolic differentiator that has been discussed in this section.

**Exercise 22**: Extend the differentiator so that it supports the trigonometric functions $\sin$ and $\cos$. You have to

1. add two alternatives to the variable `base` in the grammar shown in Figure 7.8,

2. add the corresponding cases to the function `ast_to_tuples`, and
3. add the rules

$$\frac{\mathrm{d}}{\mathrm{d}x}\sin\big(f(x)\big) = \Big(\frac{\mathrm{d}}{\mathrm{d}x}f(x)\Big)\cdot\cos\big(f(x)\big) \quad \text{and} \quad \frac{\mathrm{d}}{\mathrm{d}x}\cos\big(f(x)\big) = -\Big(\frac{\mathrm{d}}{\mathrm{d}x}f(x)\Big)\cdot\sin\big(f(x)\big)$$

to the function `diff`.

Which changes are necessary in the function `toString`? ◇

## 7.3 Check your Understanding

(a) How does Lark distinguish the names of syntactical variables from the names of tokens?

(b) What is the purpose of the prefix "`?`" that can be attached to a syntactical variable, and under which circumstances does this prefix have no effect?

(c) What is an alias, i.e. what is the effect of the construct "`-> name`" that can be attached to an alternative of a grammar rule?

(d) Which parts of a grammar rule are discarded by Lark when the parse tree is built?

(e) Would you be able to write a Lark grammar for propositional formulas together with a function that converts the resulting parse tree into nested tuples?

(f) How does Lark report lexical errors and syntax errors?

# Chapter 8

# Implementing a Simple Interpreter

Figure 8.2 on page 125 shows the grammar of a simple programming language. In this chapter, we will develop an interpreter for this language. This is our most ambitious example: In contrast to the calculator of Section 7.1 and the differentiator of Section 7.2, the language now has statements and hence a control flow.

Figure 8.1 shows an example program that conforms to this grammar. This program first reads a number which is stored in the variable n. Subsequently, the sum

$$s := \sum_{i=1}^{n^2} i$$

is computed using a `while` loop. Finally, this sum is printed.

```
1  n := read();   // read a number
2  s := 0;
3  i := 0;
4  while (i < n * n) {
5      i := i + 1;
6      s := s + i;
7  }
8  /* print the result */
9  print(s);
```

Figure 8.1: A program to compute the sum $\sum_{i=1}^{n^2} i$.

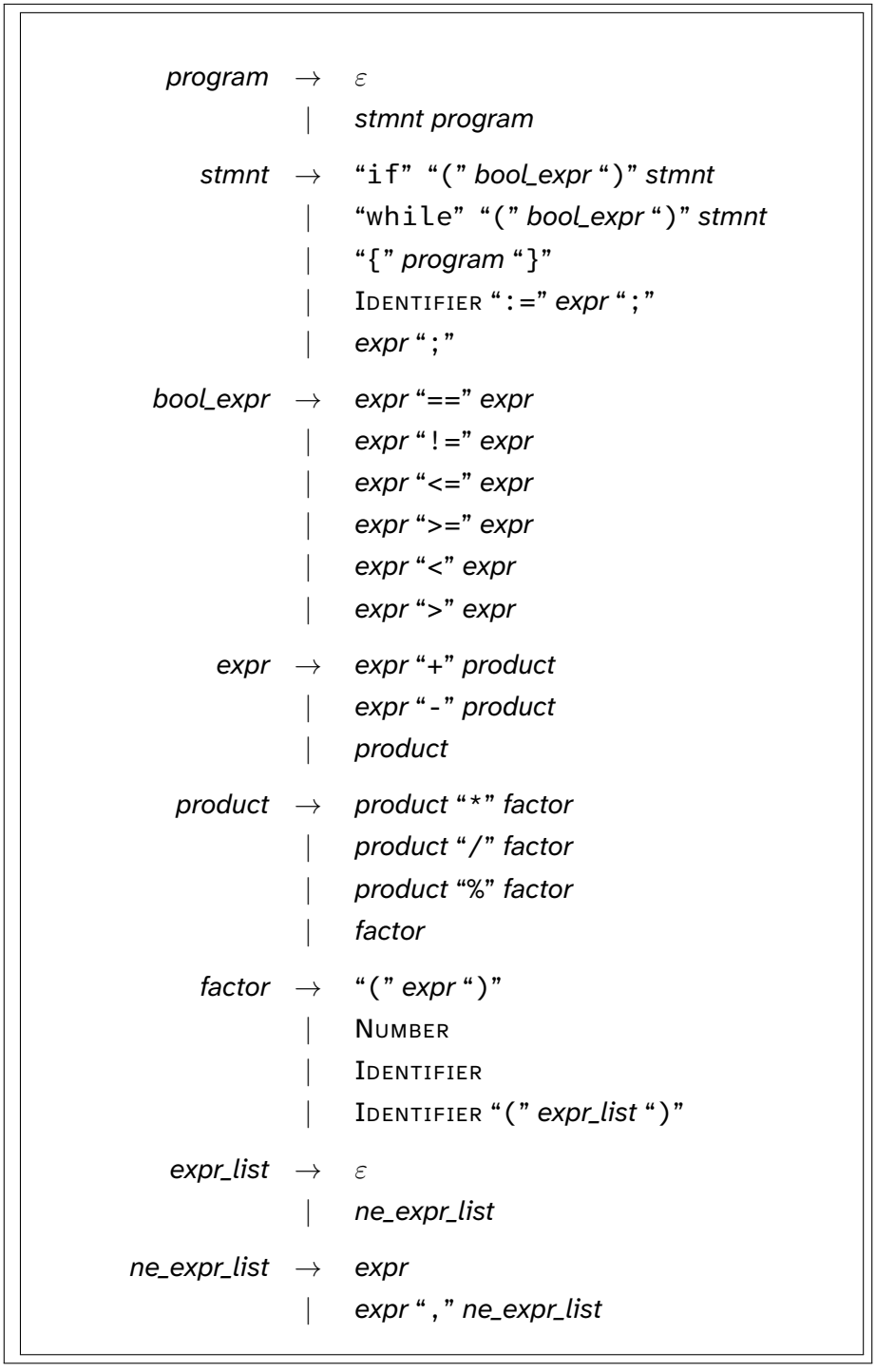
The statements of the language that we want to implement include *assignments*, *function calls*, *`if` statements*, and *`while` loops*. The language supports boolean expressions that can be built with the help of the comparison operators

"==", "!=", "<=", ">=", "<", and ">".

Arithmetic expressions can use the arithmetic operators

"+", "-", "*", "/", and "%".

In the base case, arithmetic expressions are built from numbers, variables, and function calls. Although the language supports the calling of predefined functions like `read` and `print`, it does not support user defined functions.



| | | |
|---|---|---|
| *program* | → | $\varepsilon$ |
| | \| | *stmnt program* |
| *stmnt* | → | “`if`” “`(`” *bool_expr* “`)`” *stmnt* |
| | \| | “`while`” “`(`” *bool_expr* “`)`” *stmnt* |
| | \| | “`{`” *program* “`}`” |
| | \| | IDENTIFIER “`:=`” *expr* “`;`” |
| | \| | *expr* “`;`” |
| *bool_expr* | → | *expr* “`==`” *expr* |
| | \| | *expr* “`!=`” *expr* |
| | \| | *expr* “`<=`” *expr* |
| | \| | *expr* “`>=`” *expr* |
| | \| | *expr* “`<`” *expr* |
| | \| | *expr* “`>`” *expr* |
| *expr* | → | *expr* “`+`” *product* |
| | \| | *expr* “`-`” *product* |
| | \| | *product* |
| *product* | → | *product* “`*`” *factor* |
| | \| | *product* “`/`” *factor* |
| | \| | *product* “`%`” *factor* |
| | \| | *factor* |
| *factor* | → | “`(`” *expr* “`)`” |
| | \| | NUMBER |
| | \| | IDENTIFIER |
| | \| | IDENTIFIER “`(`” *expr_list* “`)`” |
| *expr_list* | → | $\varepsilon$ |
| | \| | *ne_expr_list* |
| *ne_expr_list* | → | *expr* |
| | \| | *expr* “`,`” *ne_expr_list* |

Figure 8.2: A grammar for a simple programming language.

Similar to our program for symbolic differentiation, we represent the individual statements as nested tuples. The program shown in Figure 8.1 is represented by the nested tuple displayed in Figure 8.3. This nested tuple is nothing other than the abstract syntax tree corresponding to the original program. Note that lists of statements are represented as nested tuples that start with a “`.`”.

```
1   ('.',
2       (':=', 'n', ('call', 'read')),
3       (':=', 's', 0),
4       (':=', 'i', 0),
5       ('while', ('<', 'i', ('*', 'n', 'n')),
6           ('.',
7               (':=', 'i', ('+', 'i', 1)),
8               (':=', 's', ('+', 's', 'i'))
9           )
10      ),
11      ('expr', ('call', 'print', 's'))
12  )
```

Figure 8.3: A nested tuple that represents the program from Figure 8.1.

The figures 8.4 and 8.5 on page 127 show the transcription of the grammar of Figure 8.2 into the EBNF dialect of LARK. Recall that there is no separate scanner specification: Both the lexical structure and the syntactical structure of our language are described by the single string `sl_grammar`. Figure 8.4 contains the syntactical part of this string, while Figure 8.5 contains its lexical part.
As the general structure of this grammar specification is already familiar from the previous two sections, we only discuss those parts that are new.

1. Line 3 opens the string `sl_grammar`. Note the prefix "`r`": This string is a raw string. In a raw string, the backslash loses its special meaning, i.e. it does not start an escape sequence but rather denotes itself. This is necessary because the regular expression in line 42 contains the character sequence "`\/`", which is *not* a valid escape sequence of *Python*. Inside an ordinary string, this sequence would provoke a warning.

2. Line 4 declares the variable `program` to be the start symbol.

3. Line 6 defines the variable `program`. Instead of transcribing the two grammar rules

   $$\textit{program} \to \varepsilon \quad \text{and} \quad \textit{program} \to \textit{stmnt program}$$

   literally, we use the Kleene star and write `stmnt*`. This is the first place where the EBNF notation of LARK really pays off: A rule of the form `stmnt*` produces a node whose children are *all* the statements that have been found. Had we used the recursive formulation instead, the parse tree would have been a deeply nested chain of nodes and we would have had to flatten this chain when building the abstract syntax tree.

   Observe that the variable `program` is *not* prefixed with the inlining directive "`?`". The reason is that we always want a node of type `prog`, even for a program that consists of a single statement: This node later becomes the tuple that starts with the string "`.`".

4. Line 8 – 12 define the variable `stmnt`. The five alternatives correspond one to one to the five alternatives of the grammar shown in Figure 8.2 and every alternative is given an alias.

   The keywords `"if"` and `"while"` are written as anonymous literals, i.e. in exactly the same way as the operator symbols. In particular, we do *not* have to declare tokens for them. As the parentheses "`(`" and "`)`", the braces "`{`" and "`}`", the assignment operator "`:=`", and the semicolon "`;`" are anonymous literals as well, they are all discarded when the parse tree is built. Therefore, the node `if_stmnt` has exactly two children: the Boolean expression and the statement.

```
1   from lark import Lark, Token
2
3   sl_grammar = r"""
4   ?start: program
5
6   program: stmnt* -> prog
7
8   ?stmnt: "if" "(" bool_expr ")" stmnt    -> if_stmnt
9         | "while" "(" bool_expr ")" stmnt -> while_stmnt
10        | "{" program "}"                 -> block
11        | IDENTIFIER ":=" expr ";"        -> assign
12        | expr ";"                        -> expr_stmnt
13
14  ?bool_expr: expr "==" expr -> eq
15            | expr "!=" expr -> ne
16            | expr "<=" expr -> le
17            | expr ">=" expr -> ge
18            | expr "<"  expr -> lt
19            | expr ">"  expr -> gt
20
21  ?expr: expr "+" product -> add
22       | expr "-" product -> sub
23       | product
24
25  ?product: product "*" factor -> mul
26          | product "/" factor -> div
27          | product "%" factor -> mod
28          | factor
29
30  ?factor: "(" expr ")"
31         | NUMBER                      -> number
32         | IDENTIFIER                  -> var
33         | IDENTIFIER "(" expr_list ")" -> fct_call
34
35  expr_list: (expr ("," expr)*)? -> explist
```

Figure 8.4: The grammar of our programming language, part I: syntax.

5. Line 14 – 19 define the variable `bool_expr`. Note that every alternative carries an alias and that an alias suppresses the inlining that is requested by the prefix "?". Hence, the prefix "?" has no effect here at all. It is written down only for reasons of uniformity.

6. Line 21 – 28 define the variables `expr` and `product`. These rules encode the precedence of the arithmetic operators in the usual way. In addition to the four basic arithmetic operators, our language supports the modulo operator "%".

7. Line 30 – 33 define the variable `factor`. The alternative in line 30 has no alias and hence the

```
37    // Lexical definitions
38    %import common.CNAME -> IDENTIFIER
39    %import common.INT   -> NUMBER
40    %import common.WS
41
42    CPP_COMMENT: /\/\/[^\n]*/
43    C_COMMENT:   "/*" /(.|\n)*?/ "*/"
44
45    %ignore WS
46    %ignore C_COMMENT
47    %ignore CPP_COMMENT
48    """
49
50    parser = Lark(sl_grammar, parser='lalr')
```

Figure 8.5: The grammar of our programming language, part II: lexical definitions.

parentheses are inlined away, while line 33 describes a function call.

8. Line 35 defines the variable `expr_list`, which describes the list of arguments of a function call. The expression

   ```
   (expr ("," expr)*)?
   ```

   is read as follows: The postfix operator "?" makes the group that precedes it optional, while the Kleene star describes an arbitrary number of repetitions. Therefore, this single line replaces the two grammar rules for *expr_list* and the two grammar rules for *ne_expr_list* that are shown in Figure 8.2. Since the commas are anonymous literals, the resulting node `explist` has exactly one child for every argument.

9. Line 38 – 40 import three token definitions from the library `common`. In contrast to the calculator of Section 7.1, we import the token `INT` and rename it to `NUMBER`, because the language that we implement only supports integers.

10. Line 42 – 43 define the tokens `CPP_COMMENT` and `C_COMMENT`. In Lark, a regular expression is written between two slashes, exactly as in the language *JavaScript*. Our language supports two kinds of comments.

    (a) Line 42 describes one line comments that start with the character sequence "`//`" and extend to the end of the line. Note that the two slashes that start the comment have to be escaped inside the regular expression, since an unescaped slash would terminate it. This is the reason why `sl_grammar` has to be a raw string.

    (b) Line 43 describes multiline comments, i.e. text that is surrounded by "`/*`" and "`*/`". Here, we have to use the so-called *non-greedy* version of the Kleene operator "`*`". The non-greedy version of the operator "`*`" is written as "`*?`" and matches as little as possible. Therefore, the regular expression

    ```
    "/*" /(.|\n)*?/ "*/"
    ```

    represents a string that starts with the character sequence "`/*`", ends with the character

sequence "`*/`", and is as short as possible. This ensures that in a line like

```
/* Hugo */ i := i + 1; /* Anton */
```

two separate comments are recognized.

11. Line 45 – 47 instruct the scanner to discard white space and both kinds of comments.
12. Line 50 creates the LALR(1) parser.

**Remark**: The treatment of the keywords "`if`" and "`while`" deserves a closer look. Both of these strings also match the token `IDENTIFIER`, which has been imported as `common.CNAME`. A scanner that is generated with the module `re` would therefore have to recognize these keywords as identifiers first and then look them up in a dictionary of keywords in order to change the type of the token. With LARK, none of this is necessary: Tokens that are given as literal strings automatically have a higher priority than tokens that are given as regular expressions. Consequently, the string "`while`" is recognized as the keyword, while the string "`whilepig`" is recognized as an identifier. ◇

**Remark**: The library `common` already provides the definitions C_COMMENT and CPP_COMMENT. Hence, line 42 – 43 of Figure 8.5 could be replaced with two `%import` directives. We have written these tokens ourselves in order to show how a regular expression is specified in LARK. ◇

Figure 8.6 on page 130 shows both the dictionary SYMBOL_MAP, which maps the aliases given in the grammar to the node used in the nested tuples as well as the function `ast_to_tuples`, which converts the parse tree that is returned by LARK into the nested tuples that we have seen in Figure 8.3. As in Section 7.2, this conversion is organized in a table driven way: Most of the work is done by a dictionary, while the `match` statement only has to deal with those nodes that need special treatment. The function `ast_to_tuples` is discussed next.

1. Line 1 – 11 define the dictionary SYMBOL_MAP. This dictionary maps the aliases that have been introduced in Figure 8.4 to the symbols that we want to use in our abstract syntax tree. For example, the alias "`add`" of line 21 of Figure 8.4 is mapped to the string "+", while the alias "`prog`" of line 6 is mapped to the string "`.`".

   The aliases that are collected in this dictionary have one thing in common: For all of them, the abstract syntax tree is built by simply replacing the name of the node with the corresponding symbol and converting all children recursively. The remaining aliases are precisely those that need a different treatment.

2. Line 15 – 18 deal with the base case: If the given node is a `Token`, then it is a leaf of the parse tree. In contrast to the previous two sections, the conversion of the string that is stored in the token happens already at this point: If the token has the type `NUMBER`, the string is converted into an integer, while all other tokens are converted into plain strings.

   This is the reason why numbers occur as *Python* integers in Figure 8.3, while variables occur as strings.

3. Line 19 – 22 implement the table driven part of the conversion. If the name of the node is a key of the dictionary SYMBOL_MAP, then the corresponding symbol is looked up, all children are converted recursively, and the results are combined into a single tuple. The operator "`*`" unpacks the list of the converted children into the tuple that is created.

   Note that this works for nodes with any number of children: The node `prog` has one child per statement, the nodes `if_stmnt` and `while_stmnt` have two children, and the node `expr_stmnt` has a single child. This is the reason why a program is converted into a tuple of the form `('.', `$s_1$`, ···, `$s_n$`)` without any further effort on our part.

```
SYMBOL_MAP = {
    # Program and Statements
    'prog': '.',
    'if_stmnt': 'if',
    'while_stmnt': 'while',
    'expr_stmnt': 'expr',
    # Boolean Operations
    'eq': '==', 'ne': '!=', 'le': '<=', 'ge': '>=',
    'lt': '<', 'gt': '>',
    # Arithmetic Operations
    'add': '+', 'sub': '-', 'mul': '*', 'div': '/', 'mod': '%'
}
def ast_to_tuples(node):
    # Base case: Leaf node (Token)
    if isinstance(node, Token):
        if node.type == 'NUMBER':
            return int(node.value)
        return str(node.value)
    if node.data in SYMBOL_MAP:
        symbol = SYMBOL_MAP[node.data]
        return (symbol,
                *[ast_to_tuples(child) for child in node.children])
    match node.data:
        case 'block':
            return ast_to_tuples(node.children[0])
        case 'assign':
            return (':=', str(node.children[0]),
                          ast_to_tuples(node.children[1]))
        case 'number':
            return int(node.children[0])
        case 'var':
            return str(node.children[0])
        case 'fct_call':
            f_name = str(node.children[0])
            args = ast_to_tuples(node.children[1])
            return ('call', f_name, *args)
        case 'explist':
            return tuple([ast_to_tuples(child)
                          for child in node.children])
        case _: raise ValueError(f"Unknown node: {node.data}")
```

Figure 8.6: Converting the parse tree into an abstract syntax tree.

4. Line 23 starts a `match` statement that deals with those nodes that are not covered by the dictionary.

5. Line 24 – 25 convert a block, i.e. a list of statements that is enclosed in curly braces. As the

braces serve no purpose other than grouping, the node `block` has a single child, which is a node of type `prog`. Hence, we simply convert this child and return the result. Therefore, a block is represented by a tuple that starts with the string "`.`", exactly like the program itself. This is the reason why the body of the `while` loop in Figure 8.3 is a tuple starting with a dot.

6. Line 26 – 28 convert an assignment. Note the asymmetry between the two children: The first child is the *name* of the variable that is assigned to and therefore it is converted into a plain string, while the second child is an expression that has to be converted recursively.

7. Line 29 – 32 convert the leaves of an expression: A number is represented by itself, while a variable is represented by its name, i.e. the variable `x` is represented by the string "`x`". These two nodes can not be handled by the dictionary, because the tuple that the dictionary would produce, for example `('num', 3)`, would carry an operator symbol that we do not want: A number is supposed to be represented by itself and not by a tuple.

8. Line 33 – 36 convert a function call of the form

   $$f(a_1, \cdots, a_n)$$

   into the tuple

   $$\texttt{('call', } f\texttt{, } a_1\texttt{, } \cdots\texttt{, } a_n\texttt{)}.$$

   In line 33, the argument list is converted into a tuple of expressions and in line 34 this tuple is unpacked into the tuple that is created. Without this unpacking, the arguments would form a nested tuple of their own.

9. Line 37 – 39 convert the list of arguments into a tuple. Note that this tuple does *not* start with an operator symbol: It is an auxiliary object that is consumed by the case `'fct_call'` immediately.

10. Line 40 catches the case that the parse tree contains a node that we have forgotten to handle. If this happens, we have made a mistake, either in the grammar or in the function `ast_to_tuples`.

**Example**: The string

```
"x := f(1, 2+3, y);"
```

is converted into the nested tuple

```
('.', (':=', 'x', ('call', 'f', 1, ('+', 2, 3), 'y'))).
```
◇

Figure 8.7 shows the function `main`, which receives the name of a file containing a program in our simple programming language as its argument.

```
1  def main(file):
2      with open(file, 'r') as handle:
3          program = handle.read()
4      tree = parser.parse(program)
5      ast  = ast_to_tuples(tree)
6      display(tuple2dot(ast))
7      Values = {}
8      execute(ast, Values)
```

Figure 8.7: Specification of the function `main`.

The function `main` performs the three steps that are by now familiar.

1. Line 2 – 3 read the program into the string `program`.
2. Line 4 parses this string. The result is an object of class `lark.Tree`.
3. Line 5 converts this parse tree into an abstract syntax tree that is represented as a nested tuple.
4. Line 6 displays this abstract syntax tree graphically. The function `tuple2dot` is defined in the auxiliary notebook `AST2Dot.ipynb`, which renders a nested tuple with the help of the *Graphviz* library. This function has nothing to do with parsing, but it is very useful when debugging a grammar: It makes the structure of the abstract syntax tree immediately visible.

   The notebook also provides the function `parse`, which performs the first three of these steps and displays both the program text and the resulting nested tuple. This function is handy if we want to inspect an abstract syntax tree without executing the corresponding program.
5. Line 7 – 8 execute the abstract syntax tree. In order to do so, the function `execute` needs to know the values of all variables. These are stored in the dictionary `Values`, which initially is empty. Every time a variable is assigned, the variable and the corresponding value are stored in this dictionary.

```
1   def execute(stmnt, Values):
2       match stmnt:
3           case ('.', *SL):
4               execute_tuple(tuple(SL), Values)
5           case (':=', var, value):
6               Values[var] = evaluate(value, Values)
7           case ('expr', expr):
8               evaluate(expr, Values)
9           case ('if', test, stmnt):
10              if evaluate_bool(test, Values):
11                  execute(stmnt, Values)
12          case ('while', test, stmnt):
13              while evaluate_bool(test, Values):
14                  execute(stmnt, Values)
15          case _:
16              assert False, f'{stmnt} unexpected'
17
18  def execute_tuple(StatementList, Values={}):
19      for stmnt in StatementList:
20          execute(stmnt, Values)
```

Figure 8.8: The function `execute`.

Figure 8.8 shows the implementation of the function `execute`. This implementation consists mainly of a large case distinction based on the type of the statement to be executed. Note that this function does not refer to Lark in any way: It operates on the pure *Python* object that has been built by the function `ast_to_tuples`.

1. First, we check if `stmnt` is a list of statements. A statement is a list of statements if the first element of the tuple representing this statement is the string "`.`".

   In this case, the statements following the string "`.`" are executed via the auxiliary function `execute_tuple`. Observe that the pattern in line 3 uses a star pattern: The variable SL is bound to the list of all remaining components of the tuple. This is necessary because a list of statements can have any length.

2. If the statement is an assignment of the form

   ```
   (':=', var, value)
   ```

   then the value of the expression `value` is computed using the function `evaluate`. This value is then stored in the dictionary `Values` under the key `var`.

3. If the statement is an expression statement of the form

   ```
   ('expr', expr)
   ```

   then the corresponding `expr` is evaluated using the function `evaluate`.

4. If the statement is of the form

   ```
   ('if', test, stmnt)
   ```

   then `test` is a nested tuple representing a Boolean expression, while `stmnt` is a statement. In this case, the expression `test` is first evaluated using the function `evaluate_bool`. If this evaluation yields the value `True`, then `stmnt` is executed.

5. If the statement is of the form

   ```
   ('while', test, stmnt)
   ```

   then `test` is a Boolean expression and `stmnt` is a statement. In this case, the expression `test` is first evaluated using the function `evaluate_bool`. If this evaluation yields the value `True`, then `stmnt` is executed. Then the expression `test` is evaluated again. If the result is `False`, then the execution of the while loop finishes. Otherwise, `stmnt` is executed as long as the evaluation of `test` yields `True`.

Figure 8.9 shows the implementation of the function `evaluate_bool`. This function receives a boolean expression and a dictionary storing the current values of the variables as input.

(a) If the expression to be evaluated is a Boolean expression of the form

```
('==', lhs, rhs)
```

we recursively evaluate the expressions `lhs` and `rhs` and return `True` if and only if both expressions yield the same value.

(b) If the expression to be evaluated is a Boolean expression of the form

```
('<', lhs, rhs)
```

we recursively evaluate the expressions `lhs` and `rhs` and return `True` if and only if the value obtained from the evaluation of `lhs` is less than the value obtained from the evaluation of `rhs`.

The remaining cases are similar.

Figure 8.10 shows the implementation of the function `evaluate`. This function receives an arithmetic expression and a dictionary storing the values of the variables as input.

```
1  def evaluate_bool(expr, Values):
2      match expr:
3          case ('==', lhs, rhs):
4              return evaluate(lhs, Values) == evaluate(rhs, Values)
5          case ('!=', lhs, rhs):
6              return evaluate(lhs, Values) != evaluate(rhs, Values)
7          case ('<=', lhs, rhs):
8              return evaluate(lhs, Values) <= evaluate(rhs, Values)
9          case ('>=', lhs, rhs):
10             return evaluate(lhs, Values) >= evaluate(rhs, Values)
11         case ('<', lhs, rhs):
12             return evaluate(lhs, Values) <  evaluate(rhs, Values)
13         case ('>', lhs, rhs):
14             return evaluate(lhs, Values) >  evaluate(rhs, Values)
15         case _:
16             assert False, f'{expr} unexpected'
```

Figure 8.9: The evaluation of boolean expressions.

(a) If the expression to be evaluated is a number, we return this number as the result. Recall that the conversion of the string produced by the scanner into an integer has already been performed by the function `ast_to_tuples`.

(b) If the expression to be evaluated is a variable, we look up the value of this variable in the dictionary `Values` and return this value as the result.

(c) If the expression to be evaluated is a call to the function `read`, we prompt the user to enter a natural number. We then convert the string entered by the user into an integer.

Note that the pattern in line 7 is a tuple of length two: An empty argument list produces no children at all and therefore the tuple representing the call `read()` is `('call', 'read')`.

(d) If the expression to be evaluated is a call to the function `print`, we evaluate the argument and print it. Furthermore, we return `0` since the evaluation of an expression always has to return a value.

(e) If the expression to be evaluated is a sum of the form

```
('+', lhs, rhs)
```

we recursively evaluate the expressions `lhs` and `rhs` and return the sum of these values.

(f) The evaluation of the arithmetic operators `'-'`, `'*'`, `'/'`, and `'%'` is analogous to the evaluation of the operator `'+'`.

```
def evaluate(expr, Values):
    match expr:
        case int():
            return expr
        case str():
            return Values[expr]
        case ('call', 'read'):
            return int(input('Please enter a natural number: '))
        case ('call', 'print', expr):
            print(evaluate(expr, Values))
            return 0
        case ('+', lhs, rhs):
            return evaluate(lhs, Values) + evaluate(rhs, Values)
        case ('-', lhs, rhs):
            return evaluate(lhs, Values) - evaluate(rhs, Values)
        case ('*', lhs, rhs):
            return evaluate(lhs, Values) * evaluate(rhs, Values)
        case ('/', lhs, rhs):
            return evaluate(lhs, Values) / evaluate(rhs, Values)
        case ('%', lhs, rhs):
            return evaluate(lhs, Values) % evaluate(rhs, Values)
        case _:
            assert False, f'{expr} unexpected'
```

Figure 8.10: The evaluation of arithmetic expressions.

The Jupyter notebook `03-Interpreter`, which is available at

Python/Chapter-08/03-Interpreter.ipynb

implements the interpreter that has been discussed in this chapter.

**Exercise 23**:

(a) Add `for` loops to the interpreter.

(b) Expand the interpreter to include logical operators "&&" for logical *and*, "||" for logical *or*, and "!" for *negation*. The operator "!" should bind strongest and the operator "||" should bind weakest.

(c) Enhance the interpreter to allow for user-defined functions.

- A function should only have access to its parameters and those variables that are defined locally inside the function.
- A function should always return a value using a `return` statement. In order to facilitate that a function can contain multiple `return` statements and that a `return` statement can occur anywhere in the function, use an exception to communicate the return value and to transfer the control flow out of the function.

Figure 8.11 on page 136 shows a simple program that implements a function to compute the factorial function. Your interpreter should be able to execute this program. Use the Jupyter notebook

`Interpreter-Frame`, which is available at

Python/Chapter-08/Interpreter-Frame.ipynb

as a starting point for your solution.

**Hint**: Both the `for` loop and the logical operators can be added by extending the grammar shown in Figure 8.4 and by adding the corresponding cases to the functions `execute` and `evaluate_bool`. As long as the new grammar rules are given aliases, the function `ast_to_tuples` only needs new entries in the dictionary SYMBOL_MAP. For the logical operators, introduce the syntactical variables `disjunction`, `conjunction`, and `negation` in order to encode the precedences. ◇

```
function factorial(n) {
    if (n == 0) {
        return 1;
    }
    return n * factorial(n - 1);
}

for (i := 2; i <= 25; i := i + 1) {
    print(factorial(i));
}
```

Figure 8.11: A program to compute the factorial function.

## 8.1 Check your Understanding

(a) How can a grammar rule of the form

$$v \to \varepsilon \mid w\, v$$

be expressed more concisely in the EBNF dialect of LARK, and why is the resulting parse tree easier to work with?

(b) Why is the string that contains the grammar of our interpreter written as a raw string?

(c) Why do the keywords of a programming language not have to be declared as tokens when the scanner is generated by LARK? How would this problem be solved if the scanner were implemented with the module `re`?

(d) Why do we have to use the *non-greedy* version of the Kleene operator when specifying multiline comments?

(e) Are you able to extend the interpreter that has been discussed in this chapter?

(f) Assume that we develop an interpreter that supports user defined functions. How can we accommodate the fact that a `return` statement can occur anywhere in a function?

# Chapter 9

# Earley Parser

Figure 9.1: The seventh Earl of Eye in Suffolk.

In this chapter we will discuss an efficient algorithm that takes two inputs:

1. A context-free grammar $G = \langle V, \Sigma, R, s \rangle$ and
2. a string $w \in \Sigma^*$.

The algorithm decides, whether $w \in L(G)$ holds, i.e. it checks, whether the string is a member of the language generated by the CFG $G$.

The algorithm that is presented next has been published in 1970 by Jay Earley [Ear70]. There is another algorithm solving the same problem, namely the Cocke-Younger-Kasami algorithm, which is also known as the CYK algorithm. It has been discovered independently by John Cocke [CS70], Daniel H. Younger [You67], and Tadao Kasami [Kas65]. The CYK algorithm can only be used when the grammar

has a special form, namely it has to be in Chomsky normal form. As it is quite tedious to transform a given grammar into Chomsky normal form, this algorithm is not used in practical applications. In contrast, the Earley algorithm works for arbitrary context-free grammars and hence is used in practical applications. In the general case, the Earley algorithm has a complexity of $\mathcal{O}(n^3)$ where $n$ is the length of the string that is to be parsed. However, if the grammar is not ambiguous, the complexity is only $\mathcal{O}(n^2)$. Skillful implementations of Earley's algorithm even achieve a linear runtime for many practically relevant grammars. For example, Earley's algorithm has a linear complexity for both $LL(k)$ grammars and also for $LR(1)$ grammars. We will discuss $LR(1)$ grammars in a later Chapter. On the contrary, the CYK algorithm always has the complexity $\mathcal{O}(n^3)$, which is prohibitive for practical applications.

This chapter is structured as follows:

(a) First, we sketch the theory of Earley's algorithm.

(b) Next, we show how this algorithm can be implemented in *Python*.

## 9.1 The Earley Algorithm

The central notion that is needed to understand Earley's algorithm is the notion of an Earley object. This notion is defined below.

**Definition 22 (Earley Object)** Assume that $G = \langle V, \Sigma, R, s\rangle$ is a context-free grammar and $w = x_1 x_2 \cdots x_n \in \Sigma^*$ is a string of length $n$. A pair of the form

$$\langle a \to \beta \bullet \gamma, k\rangle$$

is called an Earley object if and only if

(a) $(a \to \beta\gamma) \in R$ and

(b) $k \in \{0, 1, \cdots, n\}$. ◇

**Explanation**: An Earley object describes a possible state of a parser. If a parser has to parse a string of the form $x_1 \cdots x_n$, then this parser will maintain $n+1$ sets of Earley objects. These sets are denoted as

$$Q_0, Q_1, \cdots, Q_n.$$

If $i \in \{0, 1, \cdots, n\}$, then the state $Q_i$ contains those Earley objects that describe the state of the parser after it has parsed the tokens $x_1$, $\cdots$, $x_i$. The interpretation of

$$\langle a \to \beta \bullet \gamma, k\rangle \in Q_j \quad \text{where } j \geq k$$

is as follows:

1. The parser tries to use the grammar rule $a \to \beta\gamma$ to parse the variable $a$ at the beginning of the substring $x_{k+1} \cdots x_n$.
2. The parser has already parsed $\beta$ in the substring $x_{k+1} \cdots x_j$, we have
$$\beta \Rightarrow^* x_{k+1} \cdots x_j.$$
3. Hence, in order to parse the variable $a$ the parser only needs to recognize $\gamma$ at the beginning of the substring $x_{j+1} \cdots x_n$.

Earley's algorithm manages the sets $Q_0$, $Q_1$, $\cdots$, $Q_n$ of Earley objects. The set $Q_j$ contains those Earley objects that contain all those states the parser could be in when it has read the prefix $x_1 \cdots x_j$.

At the beginning of the algorithm we add a new start variable $\widehat{s}$ to the grammar. Furthermore the rule $\widehat{s} \to s$ is added to the grammar. Hence, the grammar $G = \langle V, \Sigma, R, s\rangle$ is transformed into the

augmented grammar

$$\widehat{G} = \langle V \cup \{\widehat{s}\}, \Sigma, R \cup \{\widehat{s} \to s\}, \widehat{s}\rangle.$$

Next, the set $Q_0$ is defined as

$$Q_0 := \big\{\langle \widehat{s} \to \bullet s, 0\rangle\big\}.$$

The reason is that the parser should recognize the start symbol $s$ at the beginning of the string $x_1 \cdots x_n$. The remaining sets $Q_j$ are initially empty for $j = 1, \cdots, n$. These sets are extended by the following three operations:

1. *Reading*

   If the set $Q_j$ contains an Earley object of the form $\langle a \to \beta \bullet T\gamma, k\rangle$ and $T$ is a terminal, then the parser tries to parse the right hand side of the grammar rule $a \to \beta T\gamma$ and, after reading $x_{k+1} \cdots x_j$ it has already recognized $\beta$. If this $\beta$ is now followed by the token $T$, then in order to recognize $a$ with the grammar rule $a \to \beta T\gamma$, the parser only has to recognize $\gamma$ in the substring $x_{j+2} \cdots x_n$. Hence, in this case we add the Earley object

   $$\langle a \to \beta T \bullet \gamma, k\rangle$$

   to the set $Q_{j+1}$:

   $$\langle a \to \beta \bullet T\gamma, k\rangle \in Q_j \wedge x_{j+1} = T \;\Rightarrow\; Q_{j+1} := Q_{j+1} \cup \big\{\langle a \to \beta T \bullet \gamma, k\rangle\big\}.$$

2. *Prediction*

   If the set $Q_j$ contains an Earley object of the form $\langle a \to \beta \bullet c\delta, k\rangle$ such that $c$ is a variable, then the parser tries to recognize the substring $c\delta$ after having parsed the substring $x_{k+1} \cdots x_j$ as $\beta$. Hence the parser now has to recognize the variable $c$. Therefore, for every rule of the form $c \to \gamma$ that is contained in the grammar $G$ the Earley object $\langle c \to \bullet\gamma, j\rangle$ is added to the set $Q_j$:

   $$\langle a \to \beta \bullet c\delta, k\rangle \in Q_j \wedge (c \to \gamma) \in R \;\Rightarrow\; Q_j := Q_j \cup \big\{\langle c \to \bullet\gamma, j\rangle\big\}.$$

3. *Completion*

   If the set $Q_i$ contains an Earley object of the form $\langle c \to \gamma\bullet, j\rangle$ and the set $Q_j$ contains an Earley object of the form $\langle a \to \beta \bullet c\delta, k\rangle$, then the parser has tried to parse the variable $c$ after it had read the substring $x_1 \cdots x_j$ and after reading the substring $x_{j+1} \cdots x_i$ it has recognized the variable $c$. Therefore, the Earley object $\langle a \to \beta c \bullet \delta, k\rangle$ is now added to the set $Q_i$:

   $$\langle c \to \gamma\bullet, j\rangle \in Q_i \wedge \langle a \to \beta \bullet c\delta, k\rangle \in Q_j \;\Rightarrow\; Q_i := Q_i \cup \big\{\langle a \to \beta c \bullet \delta, k\rangle\big\}.$$

When trying to parse the string $w = x_1 \cdots x_n$ with the grammar $G = \langle V, \Sigma, R, s\rangle$ Earley's algorithm works as follows:

1. The sets $Q_i$ are initialized as follows:

   $$Q_0 := \big\{\langle \widehat{s} \to \bullet s, 0\rangle\big\},$$
   $$Q_i := \big\{\big\} \quad \text{for } i = 1, \cdots, n.$$

2. After that we iterate from $i = 0$ to $i = n$ and perform the following operations:

   (a) The set $Q_i$ is enlarged using completion until we find no further Earley objects.

   (b) Next we use prediction to enlarge the set $Q_i$. Again, this operation is performed until no new Earley objects are found.

   (c) If $i < n$, we read the next token and use it to initialize $Q_{i+1}$.

If the grammar $G$ has $\lambda$-rules, i.e. if it has rules with empty right hand side of the form

$$c \to \lambda,$$

then it might happen that after applying prediction in the set $Q_i$ we can afterwards immediately apply completion in $Q_i$. In this case prediction and completion have to be iterated until no further Earley objects are generated for $Q_i$.

3. If after termination of the algorithm the set $Q_n$ contains the Earley object $\langle \widehat{s} \to s\bullet, 0\rangle$, then parsing is successful and we have shown that the string $w = x_1 \cdots x_n$ is an element of $L(G)$.

**Example**: Figure 9.2 shows a simplified grammar for arithmetic expressions, consisting only of the numbers "1", "2", and "3" and the two operator symbols "+" and "*". The set $T$ of terminals for this grammar is therefore given by

$$T = \{\text{ "1" , "2" , "3" , "+" , "*" }\}.$$

We demonstrate how the string "1+2*3" can be parsed using this grammar and Earley's algorithm. In the following representation, we will abbreviate the syntactic variable `expr` with the letter $e$, write $p$ for `prod`, and use $f$ as an abbreviation for `fact`.

```
1   expr : expr '+' prod
2        | prod
3        ;
4
5   prod : prod '*' fact
6        | fact
7        ;
8
9   fact : '1'
10       | '2'
11       | '3'
12       ;
```

Figure 9.2: A simplified grammar for arithmetic expressions.

1. We initialize $Q_0$ as

$$Q_0 = \{\langle \widehat{s} \to \bullet\, e, 0\rangle\}.$$

The set $Q_1$, $Q_2$, $Q_3$, $Q_4$ and $Q_5$ are initially empty. If we apply the completion operation to the set $Q_0$, we find no new Earley objects.

Next, we apply the prediction operation to the Earley object $\langle \widehat{s} \to \bullet\, e, 0\rangle$. This initially adds the following two Earley objects to the set $Q_0$:

$$\langle e \to \bullet\, e \text{"+"} p, 0\rangle \quad \text{and} \quad \langle e \to \bullet\, p, 0\rangle$$

We can apply the prediction operation once more to the Earley object $\langle e \to \bullet\, p, 0\rangle$, which results in two new Earley objects:

$$\langle p \to \bullet\, p \text{"*"} f, 0\rangle \quad \text{and} \quad \langle p \to \bullet\, f, 0\rangle$$

Applying the prediction operation to the Earley object $\langle p \to \bullet\ f, 0\rangle$, we finally add the following Earley objects to $Q_0$:

$$\langle f \to \bullet\text{“1”}, 0\rangle, \quad \langle f \to \bullet\text{“2”}, 0\rangle, \quad \text{and} \quad \langle f \to \bullet\text{“3”}, 0\rangle$$

In total, $Q_0$ now contains the following Earley objects:

(a) $\langle \widehat{s} \to \bullet\ e, 0\rangle$,
(b) $\langle e \to \bullet\ e\text{“+”}p, 0\rangle$,
(c) $\langle e \to \bullet\ p, 0\rangle$,
(d) $\langle p \to \bullet\ p\text{“*”}f, 0\rangle$,
(e) $\langle p \to \bullet\ f, 0\rangle$,
(f) $\langle f \to \bullet\text{“1”}, 0\rangle$,
(g) $\langle f \to \bullet\text{“2”}, 0\rangle$,
(h) $\langle f \to \bullet\text{“3”}, 0\rangle$.

Next, we apply the scanning operation to $Q_0$. Since the first character of the string to be parsed is “1,” the set $Q_1$ takes the following form:

$$Q_1 = \{\langle f \to \text{“1”}\bullet, 0\rangle\}$$

2. Now, we set $i = 1$ and first apply the completion operation to $Q_1$. Based on the Earley object $\langle f \to \text{“1”}\bullet, 0\rangle$ in $Q_1$, we look for an Earley object in $Q_0$ where the marker “•” is positioned before the variable $f$. We find the Earley object $\langle p \to \bullet\ f, 0\rangle$. Therefore, we add the following Earley object to $Q_1$:

$$\langle p \to f\ \bullet, 0\rangle$$

We can then apply the completion operation again. After repeated applications, $Q_1$ contains the following Earley objects:

(a) $\langle p \to f\ \bullet, 0\rangle$,
(b) $\langle p \to p\ \bullet\text{“*”}f, 0\rangle$,
(c) $\langle e \to p\ \bullet, 0\rangle$,
(d) $\langle e \to e\ \bullet\text{“+”}p, 0\rangle$,
(e) $\langle \widehat{s} \to e\ \bullet, 0\rangle$.

Next, we apply the prediction operation to these Earley objects. However, since the marker “•” in none of the Earley objects in $Q_i$ precedes a variable, no new Earley objects are generated.

Finally, we apply the scanning operation to $Q_1$. Since the character “+” is at position 2 in the string “1+2*3” and $Q_1$ contains the Earley object

$$\langle e \to e\ \bullet\text{“+”}p, 0\rangle,$$

we add the following Earley object to $Q_2$:

$$\langle e \to e\text{“+”}\bullet\ p, 0\rangle$$

3. Next, we set $i = 2$ and first apply the completion operation to $Q_2$. At this point, we have

   $$Q_2 = \{\langle e \to e\text{“+”} \bullet\ p, 0\rangle\}.$$

   Since the marker “•” in the only Earley object present here is not at the end of the grammar rule, the completion operation does not yield any new Earley objects in this step.

   Next, we apply the prediction operation to $Q_2$. Since the marker precedes the variable $p$, we first find the following two Earley objects:

   $$\langle p \to \bullet\ f, 2\rangle \quad \text{and} \quad \langle p \to \bullet\ p\text{“*”}f, 2\rangle.$$

   Since in the first Earley object the marker precedes the variable $f$, the prediction operation can be applied once more, yielding the following additional Earley objects:

   (a) $\langle f \to \bullet\text{“1”}, 2\rangle$,
   (b) $\langle f \to \bullet\text{“2”}, 2\rangle$,
   (c) $\langle f \to \bullet\text{“3”}, 2\rangle$.

   Finally, we apply the scanning operation to $Q_2$. Since the third character of the string “1+2*3” is the digit “2,” $Q_3$ now takes the form

   $$Q_3 = \{\langle f \to \text{“2”}\bullet, 2\rangle\}.$$

4. We set $i = 3$ and apply the completion operation to $Q_3$. This adds

   $$\langle p \to f\ \bullet, 2\rangle$$

   to $Q_3$. Here, we can apply the completion operation once more. Through iterative application of the completion operation, we additionally obtain the following Earley objects:

   (a) $\langle p \to p \bullet \text{“*”}f, 2\rangle$,
   (b) $\langle e \to e\text{“+”}p \bullet, 0\rangle$,
   (c) $\langle e \to e\ \bullet \text{“+”}p, 0\rangle$,
   (d) $\langle \widehat{s} \to e \bullet, 0\rangle$.

   Finally, we apply the scanning operation. Since the next character to be read is the symbol “*,” we get

   $$Q_4 = \{\langle p \to p\text{“*”} \bullet\ f, 2\rangle\}.$$

5. We set $i = 4$. The completion operation does not yield any new Earley objects. The prediction operation provides the following Earley objects:

   (a) $\langle f \to \bullet\text{“1”}, 4\rangle$,
   (b) $\langle f \to \bullet\text{“2”}, 4\rangle$,
   (c) $\langle f \to \bullet\text{“3”}, 4\rangle$.

   Since the next character is the digit “3,” the scanning operation for $Q_5$ results in:

   $$Q_5 = \{\langle f \to \text{“3”}\bullet, 4\rangle\}.$$

6. We set $i = 5$. The completion operation sequentially provides the following Earley objects:

(a) $\langle p \rightarrow p \text{“*”} f \bullet, 2\rangle$,
(b) $\langle e \rightarrow e \text{“+”} p \bullet, 0\rangle$,
(c) $\langle p \rightarrow p \bullet \text{“*”} f, 2\rangle$,
(d) $\langle e \rightarrow e \bullet \text{“+”} p, 0\rangle$
(e) $\langle \widehat{s} \rightarrow e \bullet, 0\rangle$.

Since the set $Q_5$ contains the Earley object $\langle \widehat{s} \rightarrow e \bullet, 0\rangle$, we can conclude that the string "1+2*3" indeed belongs to the language generated by the given grammar.

**Exercise 24**: Use Earley's algorithm to show that the string "1*2+3" belongs to the language of the grammar shown in Figure 9.2. ◇

## 9.2 Implementing Earley's Algorithm in *Python*

The *Jupyter* notebook

Python/Chapter-09/Earley-Parser.ipynb

contains an implementation of Earley's algorithm.

## 9.3 Check Your Understanding

(a) How is an Earley object defined, and how are the components of an Earley object interpreted?

(b) How does the scanning operation work in Earley's algorithm?

(c) How does the prediction operation work in Earley's algorithm?

(d) How does the completion operation work in Earley's algorithm?

(e) Outline Earley's algorithm for the case where the grammar contains no $\varepsilon$-rules.

(f) What is the complexity of Earley's algorithm in the general case?

(g) What is the complexity of Earley's algorithm in the case that the grammar is not ambiguous?

# Chapter 10

# Bottom-up Parsers

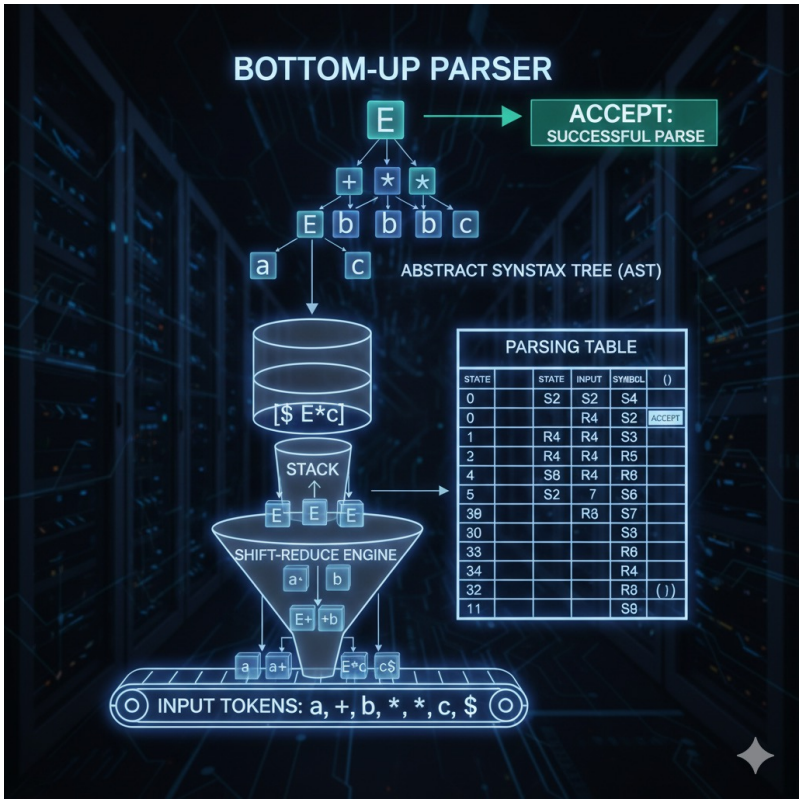



Figure 10.1: An artist's rendition of a Bottom-Up Parser.

In constructing a parser, there are generally two approaches: top-down and bottom-up. We have already discussed the top-down approach. In this chapter, we will explain the bottom-up approach.

(a) In the next section, we present the general concept underlying a bottom-up parser.

(b) In the following section, we show how bottom-up parsers can be implemented and introduce shift-reduce parsers as one possible implementation.

A *shift-reduce parser* operates with a table that specifies how the parser should process inputs in a given state.

(c) We then develop the theory of how such a table can be meaningfully filled with information in the following section: First, we discuss the SLR parsers (simple LR parsers), which are the simplest

class of shift-reduce parsers.

(d) Unfortunately, the concept of SLR parsers is not powerful enough for practical use. Therefore, we refine this concept to obtain the class of canonical LR parsers.

(e) As the tables for LR parsers often become large, we simplify these tables slightly and then obtain the concept of LALR parsers, which in terms of power lies between the concept of SLR parsers and that of LR parsers.

In the following chapter, we will discuss the parser generator PLY, which is an LALR parser.

## 10.1 Bottom-Up-Parser

The parsers that we have implemented in Chapter 6 without the help of any tool are known as top-down parsers: starting from the start symbol of the grammar, they attempt to parse a given input by applying various grammar rules. In contrast, the parsers generated by PLY are bottom-up parsers. In such parsers, the idea is to start with the string to be parsed and group terminals based on the right-hand sides of the grammar rules.

As an example, consider parsing the string "1 + 2 * 3" using the grammar defined by the following rules:

$$
\begin{array}{lcl}
e & \rightarrow & e \text{ "+" } p \mid p \\
p & \rightarrow & p \text{ "*" } f \mid f \\
f & \rightarrow & \text{"1"} \mid \text{"2"} \mid \text{"3"}
\end{array}
$$

To parse this string, we look for substrings that correspond to the right-hand sides of grammar rules, scanning the string from left to right. This way, we try to construct a parse tree in reverse from bottom to top:

$$
\begin{array}{lcll}
\texttt{1 + 2 * 3} & \Leftarrow & f \texttt{ + 2 * 3} & (\text{rule: } f \rightarrow \text{"1"}) \\
& \Leftarrow & p \texttt{ + 2 * 3} & (\text{rule: } p \rightarrow f) \\
& \Leftarrow & e \texttt{ + 2 * 3} & (\text{rule: } e \rightarrow p) \\
& \Leftarrow & e \texttt{ + } f \texttt{ * 3} & (\text{rule: } f \rightarrow \text{"2"}) \\
& \Leftarrow & e \texttt{ + } p \texttt{ * 3} & (\text{rule: } p \rightarrow f) \\
& \Leftarrow & e \texttt{ + } p \texttt{ * } f & (\text{rule: } f \rightarrow \text{"3"}) \\
& \Leftarrow & e \texttt{ + } p & (\text{rule: } p \rightarrow p \text{ "*" } f) \\
& \Leftarrow & e & (\text{rule: } e \rightarrow e \text{ "+" } P)
\end{array}
$$

In the first step, we used the grammar rule $f \rightarrow$ "1" to replace the string "1" with $f$, thus obtaining the string "f + 2 * 3". In the second step, we used the rule $p \rightarrow f$ to replace $f$ with $p$. In this manner, we ultimately traced the original string "1 + 2 * 3" back to $e$. At this point, we can make two observations:

(a) In our approach, we always replace the leftmost substring that can be replaced when we want to trace the initially given string back to the start symbol of the grammar.

(b) If we write down the derivation that we have constructed backwards, we obtain:

$$
\begin{aligned}
e &\Rightarrow e + p \\
&\Rightarrow e + p * f \\
&\Rightarrow e + p * 3 \\
&\Rightarrow e + f * 3 \\
&\Rightarrow e + 2 * 3 \\
&\Rightarrow p + 2 * 3 \\
&\Rightarrow f + 2 * 3 \\
&\Rightarrow 1 + 2 * 3
\end{aligned}
$$

We see here that in this derivation, the rightmost syntactic variable is always the one being replaced. Such a derivation is referred to as a rightmost derivation.

In contrast, the derivations produced by a top-down parser are exactly the opposite: there, the leftmost syntactic variable is always replaced. Therefore, the derivations created with such a parser are called leftmost derivations.

The above two observations are the reason why the parsers discussed in this chapter are referred to as LR Parsers. The L stands for left to right and describes the approach of always scanning the string from left to right, while the R stands for reverse rightmost derivation and indicates that such parsers construct a rightmost derivation in reverse.

In the implementation of an LR parser, two questions arise:

1. Which substring do we replace?
2. Which grammar rules should we use in the process?

Answering these questions is generally non-trivial. Although we always scan the strings from left to right, it is not immediately clear which substring we should replace, as the potentially replaceable substrings can overlap. Consider, for example, the intermediate result

$$e + p * 3,$$

that we obtained in the fifth step above. Here, we could replace the substring “p” using the rule

$$e \to p$$

with “e”. Then, we would obtain the string

$$e + e * 3.$$

The only reductions we can now perform, via the intermediate results $e + e * f$ and $e + e * p$, lead to the string

$$e + e * e,$$

which then cannot be reduced any further with the grammar given above. The answers to the above questions, which substring we replace and which rule we use, require a fair amount of theory, which we will develop in the following sections.

## 10.2 Shift-Reduce-Parser

Shift-reduce parsing is one way to implement bottom up parsing. Assume a grammar $G = \langle V, T, R, s \rangle$ is given. A shift-reduce parser is defined as a 4-tuple

$$P = \langle Q, q_0, \mathit{action}, \mathit{goto} \rangle$$

where

1. $Q$ is the set of states of the shift-reduce parser.

   At first, states are purely abstract.

2. $q_0 \in Q$ is the start state.

3. *action* is a function taking two arguments. The first argument is a state $q \in Q$ and the second argument is a terminal $t \in T$. The result of this function is an element from the set

   $$\mathit{Action} := \{\langle \texttt{shift}, q\rangle \mid q \in Q\} \cup \{\langle \texttt{reduce}, r\rangle \mid r \in R\} \cup \{\texttt{accept}\} \cup \{\texttt{error}\}.$$

   Here `shift`, `reduce`, `accept`, and `error` are strings that serve to distinguish the different kinds of result of the function *action*. Then the signature of the function *action* is

   $$\mathit{action} : Q \times T \to \mathit{Action}.$$

4. *goto* is a function that takes a state $q \in Q$ and a syntactical variable $v \in V$ and computes a new state. The signature of *goto* is

   $$\mathit{goto} : Q \times V \to Q.$$

To describe the algorithm implemented in a shift-reduce parser, we first describe the data structures that are used by this algorithm:

(a) *States* is a stack of states from the set $Q$:

$$\mathit{States} \in \mathit{Stack}[Q].$$

(b) *Symbols* is a stack of grammar symbols, i.e. this stack contains both terminals and syntactical variables:

$$\mathit{Symbols} \in \mathit{Stack}[T \cup V].$$

In order to simplify the exposition of shift-reduce parsing we assume that the set $T$ of terminals contains the special symbol "EOF" (short for end of file). This symbol is assumed to occur at the end of the input string but does not occur elsewhere.

In order to understand how a shift-reduce parser works we introduce the notion of a parser configuration. A parser configuration is a triple of the form

$$\langle \mathit{States}, \mathit{Symbols}, \mathit{Tokens}\rangle$$

where *States* and *Symbols* are the aforementioned stacks of states and grammar symbols, while *Tokens* is the rest of the tokens from the input string that have not been processed. The stack *States* starts with the start state $q_0$, i.e. initially we have

$$\mathit{States} = [q_0].$$

The length of *States* is always the length of the stack *Symbols* plus one:

$$\texttt{length}(\mathit{States}) = \texttt{length}(\mathit{Symbols}) + 1.$$

If the input string that is to be parsed has the form

$$[t_1, \cdots, t_n]$$

and we have already processed the first part $[t_1, \cdots, t_k]$ of the input string to produce the symbols

$$[X_1, \cdots, X_m],$$

while $[t_{k+1}, \cdots, t_n]$ is the part of the input string that still needs to be processed, then we know that by construction of our parser we have

$$X_1 \cdots X_m \Rightarrow^* t_1 \cdots t_k$$

and, furthermore, the parts of the parser configuration have the form

$$\mathit{States} = [q_0, q_1 \cdots, q_m], \quad \mathit{Symbols} = [X_1, \cdots, X_m], \quad \text{and} \quad \mathit{Tokens} = [t_{k+1}, \cdots, t_n, \textsc{Eof}],$$

and the parser configuration $\langle \mathit{States}, \mathit{Symbols}, \mathit{Tokens} \rangle$ is written as

$$q_0, q_1, \cdots, q_m \mid X_1, \cdots, X_m \mid t_{k+1}, \cdots, t_n, \textsc{Eof}.$$

Shift-reduce parsing starts out with the configuration

$$q_0 \mid \mid t_1, \cdots, t_n, \textsc{Eof}.$$

Therefore, initially the stack *States* contains only the start symbol, the stack *Symbols* is empty, and the list `Tokens` consists of the sequence of tokens that need to be parsed. Then, parsing proceeds iteratively. If the current configuration is

$$q_0, q_1, \cdots, q_m \mid X_1, \cdots, X_m \mid t_{k+1}, \cdots, t_n, \textsc{Eof},$$

then there is a case distinction according to the value of $\mathit{action}(q_m, t_{k+1})$.

(a) If $\mathit{action}(q_m, t_{k+1}) = \mathit{error}$, then we know that the given string $t_1 \cdots, t_n$ is not generated by the given grammar and parsing is aborted with an error message.

(b) If $\mathit{action}(q_m, t_{k+1}) = \mathit{accept}$, then we must have $k = n$ and hence $t_{k+1} = t_{n+1} = \textsc{Eof}$ and we also must have $X_1, \cdots, X_m = S$. In this case, we have reduced the string $t_1 \cdots t_n$ to the start symbol $S$ of the given grammar and parsing finishes with success.

(c) If $\mathit{action}(q_m, t_{k+1}) = \langle \mathit{shift}, q \rangle$, then the current configuration is changed into the new configuration

$$q_0, q_1, \cdots, q_m, q \mid X_1, \cdots, X_m, t_{k+1} \mid t_{k+2}, \cdots, t_n, \textsc{Eof},$$

i.e. the next token $t_{k+1}$ is moved from the unread input to the top of the symbol stack and the new state $q$ is pushed onto the stack *States*.

(d) If $\mathit{action}(q_m, t_{k+1}) = \langle \mathit{reduce}, r \rangle$, where $r$ is a grammar rule, then the grammar rule $r$ must have the form

$$a \to X_{m-l} \cdots X_m,$$

i.e. the right hand side of the grammar rule matches the end of the stack *Symbols*. In this case, the symbols stack is reduced with this grammar rule, i.e. the symbols $X_{m-l}, \cdots, X_m$ are replaced by $a$. Furthermore, in the stack *States* the states $q_{m-l} \cdots q_m$ are replaced by the state $\mathit{goto}(q_{m-l-1}, a)$. Therefore, the configuration changes as follows:

$$q_0, q_1, \cdots, q_{m-l-1}, \mathit{goto}(q_{m-l-1}, a) \mid X_1, \cdots, X_{m-l-1}, a \mid t_{k+1}, \cdots, t_n, \textsc{Eof}.$$

```
1  class ShiftReduceParser():
2      def __init__(self, actionTable, gotoTable):
3          self.mActionTable = actionTable
4          self.mGotoTable   = gotoTable
```

Figure 10.2: Implementation of a shift-reduce parser in *Python*

Figure 10.2 on page 149 shows the class `ShiftReduceParser`, which maintains two dictionaries.

(a) `mActionTable` stores the function $\mathit{action} : Q \times T \to \mathit{Action}$.

```
1   def parse(self, TL):
2       index   = 0        # points to next token
3       Symbols = []       # stack of symbols
4       States  = ['s0']   # stack of states, s0 is start state
5       TL     += ['EOF']  # token list
6       while True:
7           q = States[-1]
8           t = TL[index]
9           # undefined table entries are interpreted as 'error'
10          match self.mActionTable.get((q, t), 'error'):
11              case 'error':
12                  return False
13              case 'accept':
14                  return True
15              case 'shift', s:
16                  Symbols += [t]
17                  States  += [s]
18                  index   += 1
19              case 'reduce', rule:
20                  head, body = rule
21                  n       = len(body)
22                  Symbols = Symbols[:-n]
23                  States  = States [:-n]
24                  Symbols = Symbols + [head]
25                  state   = States[-1]
26                  States += [ self.mGotoTable[state, head] ]
```

Figure 10.3: Implementation of a shift-reduce parser in *Python*

(b) `mGotoTable` stores the function *goto* : $Q \times V \to Q$.

Figure 10.3 on page 150 shows the implementation of the method `parse` that implements shift-reduce parsing. This method assumes that the function *action* is coded as a dictionary that is stored in the member variable `mActionTable`. The function *goto* is also represented as a dictionary. It is stored in the member variable `mGotoTable`. The method `parse` is called with one argument *TL*. This is the list of tokens that have to be parsed. We append the special token "EOF" at the end of this list. The invocation `parse`(*TL*) returns `True` if the token list *TL* can be parsed successfully and `False` otherwise. The implementation of `parse` works as follows:

1. The variable *index* points to the next token in the token list that is to be read. Therefore, this variable is initialized to 0.

2. The variable *Symbols* stores the stack of symbols. The top of this stack is at the end of this list. Initially, the stack of symbols is empty.

3. The variable *States* is the stack of states. The start state is assumed to be the state "s0". Therefore this stack is initialized to contain only this state.

4. The main loop of the parser

- sets the variable $q$ to the current state,
- initializes $t$ to the next token, and then
- looks up the appropriate action in the action table.

What happens next depends on this value of *action*$(q, t)$.

(a) *action*$(q, t) =$ *error*.
In this case the parser has found a syntax error and returns `False`.

(b) *action*$(q, t) =$ `accept`.
In this case parsing is successful and therefore the function returns `True`.

(c) *action*$(q, t) = \langle$`shift`$, s\rangle$.
In this case, the token $t$ is pushed onto the symbol stack in line 16, while the state $s$ is pushed onto the stack of states. Furthermore, the variable *index* is incremented to point to the next unread token.

(d) *action*$(q, t) = \langle$`reduce`$, A \to X_1 \cdots X_n\rangle$.
In this case, we use the grammar rule

$$r = (A \to X_1 \cdots X_n)$$

to reduce the symbol stack. The variable *head* represents the left hand side $A$ of this rule, while the list $[X_1, \cdots, X_n]$ is represented by the variable *body*.
In this case, it can be shown that the symbols $X_1, \cdots, X_n$ are on top of the symbol stack. As we are going to reduce the symbol stack with the rule $r$, we remove these $n$ symbols from the symbol stack and replace them with the variable $A$.
Furthermore, we have to remove $n$ states from the stack of states. After that, we set the variable *state* to the state that is on top of the stack of states after we have removed $n$ states. Next, the new state *goto*$(state, A)$ is put on top of the stack of states in line 26.

In order to make the method `parse` work we have to provide an implementation of the functions *action* and *goto*. The Tables 10.1 and 10.2 show these functions for the grammar given in Figure 10.4. For this grammar, there are 16 different states, which have been baptized as $s_0$, $s_1$, $\cdots$, $s_{15}$. The tables use two different abbreviations:

1. $\langle$*shft*$, s_i\rangle$ is short for $\langle$`shift`$, s_i\rangle$.
2. $\langle$*rdc*$, r_i\rangle$ is short for $\langle$`reduce`$, r_i\rangle$, where $r_i$ denotes the grammar rule number $i$. Here, we have numbered the rules as follows:
   (a) $r_1 = ($*expr* $\to$ *expr* "+" *product*$)$
   (b) $r_2 = ($*expr* $\to$ *expr* "-" *product*$)$
   (c) $r_3 = ($*expr* $\to$ *product*$)$
   (d) $r_4 = ($*product* $\to$ *product* "*" *factor*$)$
   (e) $r_5 = ($*product* $\to$ *product* "/" *factor*$)$
   (f) $r_6 = ($*product* $\to$ *factor*$)$
   (g) $r_7 = ($*factor* $\to$ "(" *expr* ")" $)$
   (h) $r_8 = ($*factor* $\to$ NUMBER$)$

The corresponding grammar is shown in Figure 10.4. The definition of the grammar rules and the coding of the functions *action* and *goto* is shown in the Figures 10.5, 10.7, and 10.6 on the following pages. Of course, at present we do not have any idea how the functions *action* and *goto* are computed. This requires some theory that will be presented in the next section.

$$
\begin{array}{rcl}
\textit{expr} & \rightarrow & \textit{expr}\ \text{“+”}\ \textit{product} \\
& | & \textit{expr}\ \text{“-”}\ \textit{product} \\
& | & \textit{product} \\
\textit{product} & \rightarrow & \textit{product}\ \text{“*”}\ \textit{factor} \\
& | & \textit{product}\ \text{“/”}\ \textit{factor} \\
& | & \textit{factor} \\
\textit{factor} & \rightarrow & \text{“(”}\ \textit{expr}\ \text{“)”} \\
& | & \textsc{Number}
\end{array}
$$

Figure 10.4: A grammar for arithmetical expressions.

| State | Eof | + | - | * | / | ( | ) | Number |
|---|---|---|---|---|---|---|---|---|
| $s_0$ | | | | | | $\langle \textit{shft}, s_5 \rangle$ | | $\langle \textit{shft}, s_2 \rangle$ |
| $s_1$ | $\langle \textit{rdc}, r_6 \rangle$ | $\langle \textit{rdc}, r_6 \rangle$ | $\langle \textit{rdc}, r_6 \rangle$ | $\langle \textit{rdc}, r_6 \rangle$ | $\langle \textit{rdc}, r_6 \rangle$ | | $\langle \textit{rdc}, r_6 \rangle$ | |
| $s_2$ | $\langle \textit{rdc}, r_8 \rangle$ | $\langle \textit{rdc}, r_8 \rangle$ | $\langle \textit{rdc}, r_8 \rangle$ | $\langle \textit{rdc}, r_8 \rangle$ | $\langle \textit{rdc}, r_8 \rangle$ | | $\langle \textit{rdc}, r_8 \rangle$ | |
| $s_3$ | $\langle \textit{rdc}, r_3 \rangle$ | $\langle \textit{rdc}, r_3 \rangle$ | $\langle \textit{rdc}, r_3 \rangle$ | $\langle \textit{shft}, s_{12} \rangle$ | $\langle \textit{shft}, s_{11} \rangle$ | | $\langle \textit{rdc}, r_3 \rangle$ | |
| $s_4$ | *accept* | $\langle \textit{shft}, s_8 \rangle$ | $\langle \textit{shft}, s_9 \rangle$ | | | | | |
| $s_5$ | | | | | | $\langle \textit{shft}, s_5 \rangle$ | | $\langle \textit{shft}, s_2 \rangle$ |
| $s_6$ | | $\langle \textit{shft}, s_8 \rangle$ | $\langle \textit{shft}, s_9 \rangle$ | | | | $\langle \textit{shft}, s_7 \rangle$ | |
| $s_7$ | $\langle \textit{rdc}, r_7 \rangle$ | $\langle \textit{rdc}, r_7 \rangle$ | $\langle \textit{rdc}, r_7 \rangle$ | $\langle \textit{rdc}, r_7 \rangle$ | $\langle \textit{rdc}, r_7 \rangle$ | | $\langle \textit{rdc}, r_7 \rangle$ | |
| $s_8$ | | | | | | $\langle \textit{shft}, s_5 \rangle$ | | $\langle \textit{shft}, s_2 \rangle$ |
| $s_9$ | | | | | | $\langle \textit{shft}, s_5 \rangle$ | | $\langle \textit{shft}, s_2 \rangle$ |
| $s_{10}$ | $\langle \textit{rdc}, r_2 \rangle$ | $\langle \textit{rdc}, r_2 \rangle$ | $\langle \textit{rdc}, r_2 \rangle$ | $\langle \textit{shft}, s_{12} \rangle$ | $\langle \textit{shft}, s_{11} \rangle$ | | $\langle \textit{rdc}, r_2 \rangle$ | |
| $s_{11}$ | | | | | | $\langle \textit{shft}, s_5 \rangle$ | | $\langle \textit{shft}, s_2 \rangle$ |
| $s_{12}$ | | | | | | $\langle \textit{shft}, s_5 \rangle$ | | $\langle \textit{shft}, s_2 \rangle$ |
| $s_{13}$ | $\langle \textit{rdc}, r_4 \rangle$ | $\langle \textit{rdc}, r_4 \rangle$ | $\langle \textit{rdc}, r_4 \rangle$ | $\langle \textit{rdc}, r_4 \rangle$ | $\langle \textit{rdc}, r_4 \rangle$ | | $\langle \textit{rdc}, r_4 \rangle$ | |
| $s_{14}$ | $\langle \textit{rdc}, r_5 \rangle$ | $\langle \textit{rdc}, r_5 \rangle$ | $\langle \textit{rdc}, r_5 \rangle$ | $\langle \textit{rdc}, r_5 \rangle$ | $\langle \textit{rdc}, r_5 \rangle$ | | $\langle \textit{rdc}, r_5 \rangle$ | |
| $s_{15}$ | $\langle \textit{rdc}, r_1 \rangle$ | $\langle \textit{rdc}, r_1 \rangle$ | $\langle \textit{rdc}, r_1 \rangle$ | $\langle \textit{shft}, s_{12} \rangle$ | $\langle \textit{shft}, s_{11} \rangle$ | | $\langle \textit{rdc}, r_1 \rangle$ | |

Table 10.1: The function *action*().

```
1   r1 = ('e', ('e', '+', 'p'))
2   r2 = ('e', ('e', '-', 'p'))
3   r3 = ('e', ('p'))
4
5   r4 = ('p', ('p', '*', 'f'))
6   r5 = ('p', ('p', '/', 'f'))
7   r6 = ('p', ('f'))
8
9   r7 = ('f', ('(', 'e', ')'))
10  r8 = ('f', ('NUMBER',))
```

Figure 10.5: Grammar rules coded in *Python*.

| State | *expr* | *product* | *factor* |
|---|---|---|---|
| $s_0$ | $s_4$ | $s_3$ | $s_1$ |
| $s_1$ | | | |
| $s_2$ | | | |
| $s_3$ | | | |
| $s_4$ | | | |
| $s_5$ | $s_6$ | $s_3$ | $s_1$ |
| $s_6$ | | | |
| $s_7$ | | | |
| $s_8$ | | $s_{15}$ | $s_1$ |
| $s_9$ | | $s_{10}$ | $s_1$ |
| $s_{10}$ | | | |
| $s_{11}$ | | | $s_{14}$ |
| $s_{12}$ | | | $s_{13}$ |
| $s_{13}$ | | | |
| $s_{14}$ | | | |
| $s_{15}$ | | | |

Table 10.2: The function *goto*().

```
1   gotoTable   := {};
2
3   gotoTable["s0", "e"] := "s4";
4   gotoTable["s0", "p"] := "s3";
5   gotoTable["s0", "f"] := "s1";
6
7   gotoTable["s5", "e"] := "s6";
8   gotoTable["s5", "p"] := "s3";
9   gotoTable["s5", "f"] := "s1";
10
11  gotoTable["s8", "p"] := "s15";
12  gotoTable["s8", "f"] := "s1";
13
14  gotoTable["s9", "p"] := "s10";
15  gotoTable["s9", "f"] := "s1";
16
17  gotoTable["s11", "f"] := "s14";
18  gotoTable["s12", "f"] := "s13";
```

Figure 10.6: Goto table coded in PYTHON.

```
actionTable = {}

actionTable[s0, '('] = (shft, s5)  actionTable[s8, '('] = (shft, s5)
actionTable[s0, NBR] = (shft, s2)  actionTable[s8, NBR] = (shft, s2)

actionTable[s1, EOF] = (rdc, r6)   actionTable[s9, '('] = (shft, s5)
actionTable[s1, '+'] = (rdc, r6)   actionTable[s9, NBR] = (shft, s2)
actionTable[s1, '-'] = (rdc, r6)
actionTable[s1, '*'] = (rdc, r6)   actionTable[s0, EOF] = (rdc, r2)
actionTable[s1, '/'] = (rdc, r6)   actionTable[s0, '+'] = (rdc, r2)
actionTable[s1, ')'] = (rdc, r6)   actionTable[s0, '-'] = (rdc, r2)
                                   actionTable[s0, '*'] = (shft, s2)
actionTable[s2, EOF] = (rdc, r8)   actionTable[s10, '/'] = (shft, s11)
actionTable[s2, '+'] = (rdc, r8)   actionTable[s10, ')'] = (rdc, r2)
actionTable[s2, '-'] = (rdc, r8)
actionTable[s2, '*'] = (rdc, r8)   actionTable[s11, '('] = (shft, s5)
actionTable[s2, '/'] = (rdc, r8)   actionTable[s11, NBR] = (shft, s2)
actionTable[s2, ')'] = (rdc, r8)
                                   actionTable[s12, '('] = (shft, s5)
actionTable[s3, EOF] = (rdc, r3)   actionTable[s12, NBR] = (shft, s2)
actionTable[s3, '+'] = (rdc, r3)
actionTable[s3, '-'] = (rdc, r3)   actionTable[s13, EOF] = (rdc, r4)
actionTable[s3, '*'] = (shft, s12) actionTable[s13, '+'] = (rdc, r4)
actionTable[s3, '/'] = (shft, s11) actionTable[s13, '-'] = (rdc, r4)
actionTable[s3, ')'] = (rdc, r3)   actionTable[s13, '*'] = (rdc, r4)
                                   actionTable[s13, '/'] = (rdc, r4)
actionTable[s4, EOF] = 'accept'    actionTable[s13, ')'] = (rdc, r4)
actionTable[s4, '+'] = (shft, s8)
actionTable[s4, '-'] = (shft, s9)  actionTable[s14, EOF] = (rdc, r5)
                                   actionTable[s14, '+'] = (rdc, r5)
actionTable[s5, '('] = (shft, s5)  actionTable[s14, '-'] = (rdc, r5)
actionTable[s5, NBR] = (shft, s2)  actionTable[s14, '*'] = (rdc, r5)
                                   actionTable[s14, '/'] = (rdc, r5)
actionTable[s6, '+'] = (shft, s8)  actionTable[s14, ')'] = (rdc, r5)
actionTable[s6, '-'] = (shft, s9)
actionTable[s6, ')'] = (shft, s7)  actionTable[s15, EOF] = (rdc, r1)
                                   actionTable[s15, '+'] = (rdc, r1)
actionTable[s7, EOF] = (rdc, r7)   actionTable[s15, '-'] = (rdc, r1)
actionTable[s7, '+'] = (rdc, r7)   actionTable[s15, '*'] = (shft, s12)
actionTable[s7, '-'] = (rdc, r7)   actionTable[s15, '/'] = (shft, s11)
actionTable[s7, '*'] = (rdc, r7)   actionTable[s15, ')'] = (rdc, r1)
actionTable[s7, '/'] = (rdc, r7)
actionTable[s7, ')'] = (rdc, r7)
```

Figure 10.7: The action table for our shift/reduce parser.

## 10.3 SLR-Parser

In this section, we demonstrate how to compute the functions

$$\mathit{action} : Q \times T \to \mathit{Action} \quad \text{and} \quad \mathit{goto} : Q \times V \to Q$$

that have been used in the last section to build a shift-reduce parser for a given context-free grammar $G$. For this purpose, we first clarify what information the states contained in the set $Q$ should hold. We will define these states in such a way that they contain information about which rule the shift-reduce parser is attempting to apply, which part of the right hand side of a grammar rule has already been recognized, and what are the symbols that are still expected. To this end, we define the concept of a marked rule. Unfortunately, the original English literature [Knu65] uses the rather meaningless term "item" instead of the notion of a *marked rule*.

**Definition 23 (Marked Rule)** A marked rule of a context-free grammar $G = \langle V, T, R, s\rangle$ is a triple

$$\langle a, \beta, \gamma\rangle,$$

such that

$$(a \to \beta\gamma) \in R.$$

A marked rule of the form $\langle a, \beta, \gamma\rangle$ is written as

$$a \to \beta \bullet \gamma.$$ ◇

The marked rule $a \to \beta \bullet \gamma$ indicates that the parser is trying to parse an $a$ with the rule $a \to \beta\gamma$, having already seen $\beta$ and is next attempting to recognize $\gamma$. The symbol $\bullet$ thus marks the part of the right side of the rule that we have already read. Now the idea is to represent a state of an SLR parser as a set of marked rules. To illustrate this idea, let's consider a specific example: we start with the grammar for arithmetic expressions shown in Figure 10.4 on page 152. We extend this grammar by a new start symbol $\widehat{s}$ and the rule

$$\widehat{s} \to \mathit{expr}\ \$$$

where the symbol "$" is used to denote EOF, i.e. the end of the input. The start state clearly contains the marked rule

$$\widehat{s} \to \lambda \bullet \mathit{expr}\ \$,$$

since at the beginning we are trying to derive the start symbol $\widehat{s}$. The component $\lambda$ indicates that we have not yet processed anything. In addition to this marked rule, the start state must also contain the marked rules

1. $\mathit{expr} \to \lambda \bullet \mathit{expr}$ '+' $\mathit{product}$,
2. $\mathit{expr} \to \lambda \bullet \mathit{expr}$ '-' $\mathit{product}$ and
3. $\mathit{expr} \to \lambda \bullet \mathit{product}$

because, for example, it may be necessary to use the rule

$$\mathit{expr} \to \mathit{expr}\ \text{'+'}\ \mathit{product}$$

to derive the sought *expr*. Similarly, it could be that instead we need to use the rule

$$\mathit{expr} \to \mathit{product}$$

This explains why we must include the marked rule

$$\mathit{expr} \to \lambda \bullet \mathit{product}$$

in the start state, because at the beginning we cannot yet know which rule we will need, so the start state must contain all these rules. Once we have added the marked rule

$$\textit{expr} \to \lambda \bullet \textit{product}$$

to the start state, we see that we might need to read a *product* next. Therefore, the start state also contains the following marked rules[1]:

4. *product* $\to \bullet$ *product* '*' *factor*,
5. *product* $\to \bullet$ *product* '/' *factor*,
6. *product* $\to \bullet$ *factor*,

   Now, the sixth rule indicates that we might first read a *factor*. Therefore, we add the following two marked rules to the start state:

7. *factor* $\to \bullet$ '(' *expr* ')' ,
8. *factor* $\to \bullet$ NUMBER.

Overall, we see that the start state consists of a set of 8 marked rules. The system shown above, for deriving further rules from a given rule, is formalized in the concept of the closure of a set of marked rules.

**Definition 24 (*closure*$(\mathcal{M})$)** Let $\mathcal{M}$ be a set of marked rules. Then we define the closure of this set as the smallest set $\mathcal{K}$ of marked rules such that:

1. $\mathcal{M} \subseteq \mathcal{K}$,

   i.e. the closure includes the original set of rules.

2. If on the one hand

   $$a \to \beta \bullet c\,\delta$$

   is a marked rule from the set $\mathcal{K}$, where $c$ is a syntactic variable, and on the other hand

   $$c \to \gamma$$

   is a grammar rule of the underlying grammar $G$, then the marked rule

   $$c \to \bullet\, \gamma$$

   is also an element of the set $\mathcal{K}$. Formally, this is written as follows:

   $$(a \to \beta \bullet c\,\delta) \in \mathcal{K} \;\wedge\; (c \to \gamma) \in R \;\Rightarrow\; (c \to \bullet\, \gamma) \in \mathcal{K}$$

The set $\mathcal{K}$ defined in this way is uniquely determined and is referred to as *closure*$(\mathcal{M})$ in the following. ◇

**Remark**: If you recall the Earley algorithm, you will notice that the calculation of the closure is the same as the *prediction operation* in the Earley algorithm. ◇

For a given set $\mathcal{M}$ of marked rules, the computation of $\mathcal{K} := \textit{closure}(\mathcal{M})$ can be done iteratively:

1. Initially, set $\mathcal{K} := \mathcal{M}$.

[1] In order to be more concise, we drop the empty string $\lambda$ in the following rules.

2. Then, find all rules of the form

   $$a \to \beta \bullet c\,\delta$$

   within the set $\mathcal{K}$, where $c$ is a syntactic variable. Subsequently, for all rules of the form $c \to \gamma$, add the new marked rule

   $$c \to \bullet\, \gamma$$

   into the set $\mathcal{K}$. Repeat this step until no new rules are found.

**Example**: Starting from the grammar for arithmetic expressions shown in Figure 10.4 on page 152, let's consider the set

$$\mathcal{M} := \{\mathit{product} \to \mathit{product}\ \texttt{'*'} \bullet \mathit{factor}\}$$

For the set $\mathit{closure}(\mathcal{M})$, we then find

$$\begin{array}{rcll} \mathit{closure}(\mathcal{M}) & = & \{ & \mathit{product} \to \mathit{product}\ \texttt{'*'} \bullet \mathit{factor}, \\ & & & \mathit{factor} \to \bullet\ \texttt{'('}\ \mathit{expr}\ \texttt{')'}, \\ & & & \mathit{factor} \to \bullet\ \textsc{Number} \\ & & \}. & \end{array}$$

◇

Our goal is to define a Shift-Reduce-Parser

$$P = \langle Q, q_0, \mathit{action}, \mathit{goto} \rangle$$

for a given context-free grammar $G = \langle V, T, R, s \rangle$. To achieve this goal, we first need to determine how to define the states in the set $Q$, as this will almost automatically define the remaining components. As mentioned earlier, we define the states as sets of marked rules. Let's first define

$$\Gamma := \{a \to \beta \bullet \gamma \mid (a \to \beta\gamma) \in R\}$$

as the set of all marked rules of the grammar. However, it is not sensible to allow arbitrary subsets of $\Gamma$ as states: A subset $\mathcal{M} \subseteq \Gamma$ is only considered as a state if the set $\mathcal{M}$ is closed under the function $\mathit{closure}()$, meaning $\mathit{closure}(\mathcal{M}) = \mathcal{M}$. Therefore, we define

$$Q := \{\mathcal{M} \in 2^{\Gamma} \mid \mathit{closure}(\mathcal{M}) = \mathcal{M}\}.$$

The interpretation of the sets $\mathcal{M} \in Q$ is that a state $\mathcal{M}$ contains exactly those marked rules that can be applied in the situation described by the state.

To simplify the following constructions, we extend the grammar $G = \langle V, T, R, s \rangle$ by introducing a new start symbol $\widehat{s}$ and a new token \$ to the grammar

$$\widehat{G} = \Big\langle V \cup \{\widehat{s}\}, T \cup \{\$\}, R \cup \{\widehat{s} \to s\,\$\}, \widehat{s} \Big\rangle.$$

We refer to the grammar $\widehat{G}$ as the augmented grammar. The use of the augmented grammar enables the following definition of the start state. We set:

$$q_0 := \mathit{closure}\Big(\{\widehat{s} \to \bullet s\,\$\}\Big).$$

Next, we construct the function $\mathit{goto}()$. The definition is as follows:

$$\mathit{goto}(\mathcal{M}, c) := \mathit{closure}\Big(\{a \to \beta\,c \bullet \delta \mid (a \to \beta \bullet c\,\delta) \in \mathcal{M}\}\Big).$$

To understand this definition, assume that the parser is in a state where it tries to parse an $a$ using the rule $a \to \beta\,c\,\delta$ and, furthermore, assume that the substring $\beta$ has already been recognized. This state must contain the marked rule

$$a \to \beta \bullet c\,\delta$$

If a $c$ is recognized in this state, the parser can transition from this state to a state containing the rule

$$a \to \beta\, c \bullet \delta.$$

Therefore, we get the above definition of the function *goto*$(\mathcal{M}, c)$. For the subsequent definition of the function *action*$(\mathcal{M}, t)$, it is useful to extend the definition of the function *goto* to terminals. For terminals $t \in T$ we set:

$$\textit{goto}(\mathcal{M}, t) := \textit{closure}\Big(\big\{a \to \beta\, t \bullet \delta \mid (a \to \beta \bullet t\, \delta) \in \mathcal{M}\big\}\Big).$$

Before we can compute the function *action*, we need to define two auxiliary functions, the functions *First* and *Follow*.

### 10.3.1 The Functions *First* and *Follow*

In this section, we define the two functions *First* and *Follow*. These functions are needed to compute the function *action*. We consider a given context-free grammar $G = \langle V, T, R, s\rangle$.

(a) For a syntactic variable $a$, *First*$(a)$ calculates the set of all tokens with which a string $w$ can begin, which is derived from the variable $a$, for which $a \Rightarrow_G^* w$ holds.

(b) For a syntactic variable $a$, *Follow*$(a)$ calculates the set of tokens that can follow an $a$ in a string derived from $s$, i.e., $t \in$ *Follow*$(a)$ if there is a derivation of the following form:

$$s \Rightarrow_G^* \beta\, a\, t\, \gamma.$$

Here, $\beta$ and $\gamma$ are strings consisting of tokens and variables, while $t$ denotes a single token.

**Definition 25 ($\lambda$-generating)** Let $G = \langle V, T, R, s\rangle$ be a context-free grammar and *a* be a syntactic variable, i.e., *a* $\in V$. The variable *a* is called $\lambda$-generating precisely when

$$a \Rightarrow^* \lambda$$

holds, which means that the empty word can be derived from the variable *a*. We write *nullable*$(a)$ if the variable *a* is $\lambda$-generating. ◇

**Example**: Suppose the grammar $G$ contains the following rules:

$$S \to a\ b\ c$$

$$a \to \text{'X'}\ b \mid a\ \text{'Y'} \mid b\ c$$

$$b \to \text{'X'}\ b \mid a\ \text{'Y'} \mid c\ c$$

$$c \to a\ b\ c \mid \lambda$$

Initially, it's clear that the variable *c* is $\lambda$-generating. Then, due to the rule $b \to c\ c$, *b* is also $\lambda$-generating, and from the rule $a \to b\ c$, *a* becomes $\lambda$-generating as well. Finally, $S$ is recognized as $\lambda$-generating, since the first rule states

$$S \to a\ b\ c$$

and here, all variables on the right side of the rule have already been proven to be $\lambda$-generating variables. ◇

**Definition 26 (*First*())** Let $G = \langle V, T, R, s\rangle$ be a context-free grammar and $a \in V$. We define *First*$(a)$ as the set of all tokens $t$ with which a word derived from *a* can begin:

$$\textit{First}(a) := \{t \in T \mid \exists \gamma \in (V \cup T)^* : a \Rightarrow^* t\, \gamma\}.$$

The definition of the function *First*() can be extended to strings in $(V \cup T)^*$ as follows:

1. $\mathit{First}(\lambda) = \{\}$.
2. $\mathit{First}(t\beta) = \{t\}$ if $t \in T$.
3. $\mathit{First}(a\beta) = \begin{cases} \mathit{First}(a) \cup \mathit{First}(\beta) & \text{if } a \Rightarrow^* \lambda; \\ \mathit{First}(a) & \text{otherwise.} \end{cases}$

If $a$ is a variable of $G$ and the rules defining $a$ are given as

$$a \to \alpha_1 \mid \cdots \mid \alpha_n,$$

then we have

$$\mathit{First}(a) = \bigcup_{i=1}^{n} \mathit{First}(\alpha_i). \qquad \diamond$$

**Remark**: Note that the definitions of the function $\mathit{First}(a)$ for variables $a \in V$ and the function $\mathit{First}(\alpha)$ for strings $\alpha \in (V \cup T)^*$ are mutually recursive. The computation of $\mathit{First}(a)$ is best done via a fixpoint computation: Start by setting $\mathit{First}(a) := \{\}$ for all variables $a \in V$ and then continue to iterate the equations defining $\mathit{First}(a)$ until none of the sets $\mathit{First}(a)$ changes any more. The next example clarifies this idea.

| | | |
|---|---|---|
| *expr* | → | *product exprRest* |
| *exprRest* | → | "+" *product exprRest* |
| | \| | "-" *product exprRest* |
| | \| | $\lambda$ |
| *product* | → | *factor productRest* |
| *productRest* | → | "·" *factor productRest* |
| | \| | "/" *factor productRest* |
| | \| | $\lambda$ |
| *factor* | → | "(" *expr* ")" |
| | \| | NUMBER |

Figure 10.8: Grammar for arithmetic expressions.

**Example**: We can iteratively compute the sets $\mathit{First}(a)$ for the variables $a$ of the grammar shown in Figure 10.8. It's best to compute the function $\mathit{First}(a)$ for the individual variables $a$ starting with those at the bottom of the hierarchy.

1. First, the rules

   *factor* → '(' *expr* ')' | NUMBER

   imply that every string derived from *factor* either starts with an opening parenthesis or a number:

   $\mathit{First}(\mathit{factor}) = \{$ '(' , NUMBER $\}$.

2. Similarly, from the rules

   *productRest* → '*' *factor productRest* | '/' *factor productRest* | $\lambda$

we find that a *productRest* either starts with the character "*" or "/":

$$\mathit{First}(\mathit{productRest}) = \{\ \texttt{'*'}\ ,\ \texttt{'/'}\ \}.$$

3. The rule for the variable *product* is

   $$\mathit{product} \rightarrow \mathit{factor}\ \ \mathit{productRest}.$$

   Since the variable *factor* is not $\lambda$-generating, we see that the set $\mathit{First}(\mathit{product})$ matches the set $\mathit{First}(\mathit{factor})$:

   $$\mathit{First}(\mathit{product}) = \{\ \texttt{'('}\ , \textsc{Number}\ \}.$$

4. From the rules

   $$\mathit{exprRest} \rightarrow\ \texttt{'+'}\ \mathit{product}\ \ \mathit{exprRest} \mid\ \texttt{'-'}\ \mathit{product}\ \ \mathit{exprRest} \mid \lambda,$$

   we can compute $\mathit{First}(\mathit{exprRest})$ as follows:

   $$\mathit{First}(\mathit{exprRest}) = \{\ \texttt{'+'}\ ,\ \texttt{'-'}\ \}.$$

5. Further, from the rule

   $$\mathit{expr} \rightarrow \mathit{product}\ \ \mathit{exprRest},$$

   and the fact that the variable *product* is not $\lambda$-generating, the set $\mathit{First}(\mathit{expr})$ matches the set $\mathit{First}(\mathit{product})$:

   $$\mathit{First}(\mathit{expr}) = \{\ \texttt{'('}\ , \textsc{Number}\ \}.$$

Since we have computed the sets $\mathit{First}(a)$ in a clever order, we did not have to perform a proper fixpoint iteration in this example

**Definition 27 (*Follow*())** Let $G = \langle V, T, R, s\rangle$ be a context-free grammar and let

$$\widehat{G} = \left\langle V \cup \{\widehat{s}\}, T \cup \{\$\}, R \cup \{\widehat{s} \rightarrow s\,\$\}, \widehat{s}\right\rangle$$

be the associated augmented grammar. For a variable $a \in V$ we define $\mathit{Follow}(a)$ as the set of all tokens $t$ that can follow the variable $a$ in a derivation:

$$\mathit{Follow}(a) := \left\{t \in T \cup \{\$\} \;\middle|\; \exists \beta, \gamma \in (V \cup T \cup \{\$\})^* : \widehat{s} \Rightarrow^* \beta\, a\, t\, \gamma\right\}.$$

If the start symbol $\widehat{s}$ can somehow derive a string $\beta\, a\, t\, \gamma$ in which the token $t$ follows the variable $a$, then $t$ is an element of the set $\mathit{Follow}(a)$. ◇

**Example**: Let's reexamine the grammar for arithmetic expressions shown in Figure 10.8.

1. Due to the newly added rule

   $$\widehat{S} \rightarrow \mathit{expr}\ \$$$

   the set $\mathit{Follow}(\mathit{expr})$ must contain the symbol $\$$. Furthermore, due to the rule

   $$\mathit{factor} \rightarrow\ \texttt{'('}\ \mathit{expr}\ \texttt{')'}$$

   the set $\mathit{Follow}(\mathit{expr})$ must also contain the symbol `')'`. Thus, we have in total

   $$\mathit{Follow}(\mathit{expr}) = \{\$,\ \texttt{')'}\ \}.$$

2. Due to the rule

   $$\mathit{expr} \rightarrow \mathit{product}\ \ \mathit{exprRest}$$

   we know that all terminals that can follow an *expr* can also follow an *exprRest*. Thus, we already

know that *Follow*(*exprRest*) contains the tokens \$ and ')' . Since *exprRest* otherwise only appears at the end of its defining rules, these are all the tokens that can follow *exprRest*, and we have

$$\textit{Follow}(\textit{exprRest}) = \{\$,\ \text{')'}\}.$$

3. The rules

   $$\textit{exprRest} \to \text{'+'}\ \textit{product}\ \ \textit{exprRest} \mid \text{'-'}\ \textit{product}\ \ \textit{exprRest}$$

   show that all elements from *First*(*exprRest*) can follow a *product*. However, that's not all: since the variable *exprRest* is $\lambda$-generating, additionally, all tokens that can follow *exprRest* can also follow *product*. Thus, we have in total

   $$\textit{Follow}(\textit{product}) = \{\ \text{'+'}\ ,\ \text{'-'}\ ,\$,\ \text{')'}\ \}.$$

4. The rule

   $$\textit{product} \to \textit{factor}\ \ \textit{productRest}$$

   shows that all terminals that can follow a *product* can also follow a *productRest*. Since *productRest* otherwise only appears at the end of its defining rules, these are all the tokens that can follow *productRest*, and we have in total

   $$\textit{Follow}(\textit{productRest}) = \{\ \text{'+'}\ ,\ \text{'-'}\ ,\$,\ \text{')'}\ \}.$$

5. The rules

   $$\textit{productRest} \to \text{'*'}\ \textit{factor}\ \ \textit{productRest} \mid \text{'/'}\ \textit{factor}\ \ \textit{productRest}$$

   show that all elements from *First*(*productRest*) can follow a *factor*. However, that's not all: since the variable *productRest* is $\lambda$-generating, additionally, all tokens that can follow *productRest* can also follow *factor*. Thus, we have in total

   $$\textit{Follow}(\textit{factor}) = \{\ \text{'*'}\ ,\ \text{'/'}\ ,\ \text{'+'}\ ,\ \text{'-'}\ ,\$,\ \text{')'}\ \}.$$ ◇

The last example demonstrates that the computation of the *nullable*() predicate and the computation of the sets *First*($a$) and *Follow*($a$) for a syntactic variable *a* are closely interconnected. Suppose we have a grammar rule:

$$a \to Y_1 Y_2 \cdots Y_k$$

Then the following relationships exist between the *nullable*() predicate and the *First*() and *Follow*() functions:

1. $\forall t \in T : \neg\, \textit{nullable}(t)$.
2. $k = 0 \Rightarrow \textit{nullable}(a)$.
3. $\big(\forall i \in \{1, \cdots, k\} : \textit{nullable}(Y_i)\big) \Rightarrow \textit{nullable}(a)$.
   Setting $k = 0$ here, we see that 2. is a special case of 3.
4. $\textit{First}(Y_1) \subseteq \textit{First}(a)$.
5. $\big(\forall j \in \{1, \cdots, i-1\} : \textit{nullable}(Y_j)\big) \Rightarrow \textit{First}(Y_i) \subseteq \textit{First}(a)$.
   Setting $i = 1$ above, we see that 4. is a special case of 5.
6. $\textit{Follow}(a) \subseteq \textit{Follow}(Y_k)$.
7. $\big(\forall j \in \{i+1, \cdots, k\} : \textit{nullable}(Y_j)\big) \Rightarrow \textit{Follow}(a) \subseteq \textit{Follow}(Y_i)$.
   Setting $i = k$ here, we see that 6. is a special case of 7.

8. $\forall i \in \{1, \cdots, k-1\} : \mathit{First}(Y_{i+1}) \subseteq \mathit{Follow}(Y_i)$.
9. $\big(\forall j \in \{i+1, \cdots, l-1\} : \mathit{nullable}(Y_j)\big) \Rightarrow \mathit{First}(Y_l) \subseteq \mathit{Follow}(Y_i)$.
   Setting $l = i+1$ here, we see that 8. is a special case of 9.

Using these relationships, *nullable*(), *First*(), and *Follow*() can be computed iteratively through a fixpoint iteration:

1. Initially, the functions *First*($a$) and *Follow*($a$) for each syntactic variable *a* are initialized with the empty set. The predicate *nullable*($a$) is set to `false` for each syntactic variable.
2. Subsequently, the rules outlined above are applied repeatedly until no further changes occur from their application.

### 10.3.2 Computing the Function *action*

Finally, we specify how the function *action*$(\mathcal{M}, t)$ is computed for a set of marked rules $\mathcal{M}$ and a token $t$. When defining *action*$(\mathcal{M}, t)$, we distinguish four cases.

1. If $\mathcal{M}$ contains a marked rule of the form $a \to \beta \bullet t\, \delta$ and $t \neq \$$, then we set

   $$\mathit{action}(\mathcal{M}, t) := \langle \texttt{shift}, \mathit{goto}(\mathcal{M}, t) \rangle,$$

   because in this case the parser is trying to recognize an $a$ using the rule $a \to \beta\, t \delta$ and has already recognized $\beta$. If the next token in the input string is indeed the token $t$, the parser can read this $t$ and transitions from the state $a \to \beta \bullet t\, \delta$ to the state $a \to \beta\, t \bullet \delta$, which is computed by the function *goto*$(\mathcal{M}, t)$. So, we have

   $$\mathit{action}(\mathcal{M}, t) := \langle \texttt{shift}, \mathit{goto}(\mathcal{M}, t) \rangle \quad \text{if} \quad (a \to \beta \bullet t\, \delta) \in \mathcal{M} \text{ and } t \neq \$.$$

2. If $\mathcal{M}$ contains a marked rule of the form $a \to \beta\bullet$ and additionally $t \in \mathit{Follow}(a)$, then we set

   $$\mathit{action}(\mathcal{M}, t) := \langle \texttt{reduce}, a \to \beta \rangle,$$

   because in this case the parser is trying to recognize an $a$ using the rule $a \to \beta$ and has already recognized $\beta$. If the next token in the input string is the token $t$ and $t$ is a token that can follow $a$, i.e., $t \in \mathit{Follow}(a)$, then the parser can apply the rule $a \to \beta$ and reduce the symbol stack with this rule. Therefore,

   $$\mathit{action}(\mathcal{M}, t) := \langle \texttt{reduce}, a \to \beta \rangle \quad \text{if } (a \to \beta\bullet) \in \mathcal{M},\ a \neq \widehat{s} \text{ and } t \in \mathit{Follow}(a).$$

3. If $\mathcal{M}$ contains the marked rule $\widehat{s} \to s \bullet \$$ and we have completely read the string to be parsed, we set

   $$\mathit{action}(\mathcal{M}, \$) := \texttt{accept},$$

   because in this case the parser is trying to recognize $\widehat{s}$ using the rule $\widehat{s} \to s\, \$$ and has already recognized $s$. If the next token in the input string is the end-of-file character \$, then the string being parsed belongs to the language specified by the grammar $G$, $L(G)$. Therefore, we have

   $$\mathit{action}(\mathcal{M}, \$) := \texttt{accept}, \quad \text{if } (\widehat{s} \to s \bullet \$) \in \mathcal{M}.$$

4. In all other cases, we set

   $$\mathit{action}(\mathcal{M}, t) := \texttt{error}.$$

Conflicts can arise between the first two rules. We distinguish between two types of conflicts.

1. A shift-reduce conflict occurs when both the first and second cases apply. In this situation, the set $\mathcal{M}$ contains, on one hand, a marked rule of the form

   $$a \to \beta \bullet t\, \gamma,$$

   and on the other hand, $\mathcal{M}$ contains a rule of the form

   $$c \to \delta\bullet \quad \text{with } t \in \mathit{Follow}(c).$$

   If the next token is $t$, it is unclear whether this token should be pushed onto the symbol stack and the parser transition to a state with the marked rule $a \to \beta\, t \bullet \gamma$, or whether the symbol stack should be reduced using the rule $c \to \delta$ instead.

2. A reduce-reduce conflict exists if the set $\mathcal{M}$ contains two different marked rules of the form

   $$c_1 \to \gamma_1\bullet \quad \text{and} \quad c_2 \to \gamma_2\bullet$$

   and if at the same time $t \in \mathit{Follow}(c_1) \cap \mathit{Follow}(c_2)$, because then it is not clear which of the two rules the parser should apply when the next token to be read is $t$.

If either of these conflicts occurs, we say that the grammar is not an SLR grammar. Such a grammar cannot be parsed using an SLR parser. We will later provide examples of both types of conflicts, but first, we want to examine a grammar where no conflicts occur and actually compute the functions *goto*() and *action*() for it. We use the grammar for arithmetic expressions that is shown in Figure 10.4 as a basis.

Since the syntactic variable *expr* appears on the right side of grammar rules, we define *start* as a new start symbol and add the rule

$$\mathit{start} \to \mathit{expr}\ \$$$

to the grammar. This step corresponds to the previously discussed process of augmenting the grammar.

First, we compute the set of states $Q$. We had previously provided the following formula for this:

$$Q := \{\mathcal{M} \in 2^{\Gamma} \mid \mathit{closure}(\mathcal{M}) = \mathcal{M}\}.$$

However, this set also contains states that cannot be reached from the start state via the function *goto*(). Therefore, we only compute the states that can actually be reached from the start state using the function *goto*(). To avoid making the calculation too wordy, we introduce the following abbreviations:

$$s := \mathit{start},\ e := \mathit{expr},\ p := \mathit{product},\ f := \mathit{factor}, \text{ and } N := \textsc{Number}.$$

We begin with the start state:

1. $$\begin{aligned} s_0 &:= \mathit{closure}(\{\ s \to \bullet\, e\,\$\ \}) \\ &= \{\ s \to \bullet\, e\,\$, \\ &\quad\ e \to \bullet\, e\ \text{'+'}\ p,\ e \to \bullet\, e\ \text{'-'}\ p,\ e \to \bullet\, p, \\ &\quad\ p \to \bullet\, p\ \text{'*'}\ f,\ p \to \bullet\, p\ \text{'/'}\ f,\ p \to \bullet\, f, \\ &\quad\ f \to \bullet\,\text{'('}\, e\,\text{')'}\ ,\ f \to \bullet\, N \qquad\qquad \}. \end{aligned}$$

2. $$\begin{aligned} s_1 &:= \mathit{goto}(s_0, f) \\ &= \mathit{closure}(\{p \to f\bullet\}) \\ &= \{p \to f\bullet\}. \end{aligned}$$

3. $$\begin{aligned} s_2 &:= \mathit{goto}(s_0, N) \\ &= \mathit{closure}(\{f \to N\bullet\}) \\ &= \{f \to N\bullet\}. \end{aligned}$$

4. $s_3 := \mathit{goto}(s_0, p)$
$= \mathit{closure}(\{p \to p \bullet \text{'*'}\ f,\ p \to p \bullet \text{'/'}\ f,\ e \to p\bullet\})$
$= \{p \to p \bullet \text{'*'}\ f,\ p \to p \bullet \text{'/'}\ f, e \to p\bullet\}.$

5. $s_4 := \mathit{goto}(s_0, e)$
$= \mathit{closure}(\{s \to e \bullet \$,\ e \to e \bullet \text{'+'}\ p,\ e \to e \bullet \text{'-'}\ p\ \})$
$= \{\ s \to e \bullet \$,\ e \to e \bullet \text{'+'}\ p,\ e \to e \bullet \text{'-'}\ p\ \}.$

6. $s_5 := \mathit{goto}(s_0, \text{'('}\ )$
$= \mathit{closure}(\{f \to \text{'('}\ \bullet e\ \text{')'}\ \})$
$= \{\ f \to \text{'('}\ \bullet e\ \text{')'}$
$e \to \bullet e\ \text{'+'}\ p,\ e \to \bullet e\ \text{'-'}\ p,\ e \to \bullet p,$
$p \to \bullet p\ \text{'*'}\ f,\ p \to \bullet p\ \text{'/'}\ f,\ p \to \bullet f,$
$f \to \bullet \text{'('} e \text{')'}\ ,\ f \to \bullet N \qquad \}.$

7. $s_6 := \mathit{goto}(s_5, e)$
$= \mathit{closure}(\{\ f \to \text{'('}\ e \bullet \text{')'}\ ,\ e \to e \bullet \text{'+'}\ p,\ e \to e \bullet \text{'-'}\ p\ \})$
$= \{\ f \to \text{'('}\ e \bullet \text{')'}\ ,\ e \to e \bullet \text{'+'}\ p,\ e \to e \bullet \text{'-'}\ p.\ \}$

8. $s_7 := \mathit{goto}(s_6, \text{')'}\ )$
$= \mathit{closure}(\{\ f \to \text{'('}\ e\ \text{')'} \bullet\ \})$
$= \{\ f \to \text{'('}\ e\ \text{')'} \bullet\ \}.$

9. $s_8 := \mathit{goto}(s_4, \text{'+'}\ )$
$= \mathit{closure}(\{e \to e\ \text{'+'}\ \bullet p\})$
$= \{\ e \to e\ \text{'+'}\ \bullet p$
$p \to \bullet p\ \text{'*'}\ f,\ p \to \bullet p\ \text{'/'}\ f,\ p \to \bullet f,$
$f \to \bullet \text{'('} e \text{')'}\ ,\ f \to \bullet N \qquad \}.$

10. $s_9 := \mathit{goto}(s_4, \text{'-'}\ )$
$= \mathit{closure}(\{e \to e\ \text{'-'}\ \bullet p\})$
$= \{\ e \to e\ \text{'-'}\ \bullet p$
$p \to \bullet p\ \text{'*'}\ f,\ p \to \bullet p\ \text{'/'}\ f,\ p \to \bullet f,$
$f \to \bullet \text{'('} e \text{')'}\ ,\ f \to \bullet N \qquad \}.$

11. $s_{10} := \mathit{goto}(s_9, p)$
$= \mathit{closure}(\{e \to e\ \text{'-'}\ p\bullet,\ p \to p \bullet \text{'*'}\ f,\ p \to p \bullet \text{'/'}\ f\ \})$
$= \{\ e \to e\ \text{'-'}\ p\bullet,\ p \to p \bullet \text{'*'}\ f,\ p \to p \bullet \text{'/'}\ f\ \}.$

12. $s_{11} := \mathit{goto}(s_3, \text{'/'}\ )$
$= \mathit{closure}(\{p \to p\ \text{'/'}\ \bullet f\ \})$
$= \{\ p \to p\ \text{'/'}\ \bullet f,\ f \to \bullet \text{'('} e \text{')'},\ f \to \bullet N\ \}.$

13. $s_{12} := \mathit{goto}(s_3, \text{'*'}\ )$
$= \mathit{closure}(\{p \to p\ \text{'*'}\ \bullet f\ \})$
$= \{\ p \to p\ \text{'*'}\ \bullet f,\ f \to \bullet \text{'('} e \text{')'},\ f \to \bullet N\ \}.$

14. $s_{13} := \mathit{goto}(s_{12}, f)$
$= \mathit{closure}(\{p \to p\ \text{'*'}\ f\bullet\ \})$
$= \{\ p \to p\ \text{'*'}\ f\bullet\ \}.$

15. $s_{14} := \mathit{goto}(s_{11}, f)$
$= \mathit{closure}(\{p \to p\ \text{'/'}\ f\bullet\ \})$
$= \{\ p \to p\ \text{'/'}\ f\bullet\ \}.$

16. $s_{15} := \mathit{goto}(s_8, p)$
$= \mathit{closure}(\{e \to e\ \text{'+'}\ p\bullet,\ p \to p\bullet\ \text{'*'}\ f,\ p \to p\bullet\ \text{'/'}\ f\ \})$
$= \{\ e \to e\ \text{'+'}\ p\bullet,\ p \to p\bullet\ \text{'*'}\ f,\ p \to p\bullet\ \text{'/'}\ f\ \}.$

Further calculations do not yield any new states. For instance, when we compute $\mathit{goto}(s_8,\ \text{'('}\ )$, we find:

$$
\begin{aligned}
& \mathit{goto}(s_8,\ \text{'('}\ ) \\
= \ & \mathit{closure}(\{f \to\ \text{'('}\ \bullet e\ \text{')'}\ \}) \\
= \ & \{\ f \to\ \text{'('}\ \bullet e\ \text{')'} \\
& \quad e \to \bullet e\ \text{'+'}\ p,\ e \to \bullet e\ \text{'-'}\ p,\ e \to \bullet p, \\
& \quad p \to \bullet p\ \text{'*'}\ f,\ p \to \bullet p\ \text{'/'}\ f,\ p \to \bullet f, \\
& \quad f \to \bullet\ \text{'('}e\ \text{')'}\ ,\ f \to \bullet N \qquad\qquad \} \\
= \ & s_5.
\end{aligned}
$$

Therefore, the set of states for the Shift-Reduce parser is given by

$$Q := \{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8, s_9, s_{10}, s_{11}, s_{12}, s_{13}, s_{14}, s_{15}\}$$

Next, we investigate whether there are any conflicts, and we look specifically at the set $s_{15}$. Due to the marked rule

$$p \to p\bullet\ \text{'*'}\ f$$

a shift must occur in state $s_{15}$ if the next token is '*'. On the other hand, the state $s_{15}$ includes the rule

$$e \to e\ \text{'+'}\ p\bullet.$$

This rule indicates that the symbol stack must be reduced with the grammar rule $e \to e$ '+' $p$ if a character from the set $\mathit{Follow}(e)$ appears in the input. If '*' were in $\mathit{Follow}(e)$, then we would have a Shift-Reduce conflict. However,

$$\mathit{Follow}(e) = \{\ \text{'+'}\ ,\ \text{'-'}\ ,\ \text{')'}\ ,\ \text{'\$'}\ \},$$

and therefore '*' $\notin \mathit{Follow}(e)$, so there is no Shift-Reduce conflict here. An examination of the other sets shows that there are also no Shift-Reduce or Reduce-Reduce conflicts in any of them.

Next, we calculate the *action* function. We consider two cases as examples.

1. First, we compute $\mathit{action}(s_1,\ \text{'+'}\ )$. It holds that

$$
\begin{aligned}
\mathit{action}(s_1,\ \text{'+'}\ ) &= \mathit{action}(\{p \to f\bullet\},\ \text{'+'}\ ) \\
&= \langle \mathit{reduce}, p \to f\rangle,
\end{aligned}
$$

since we have '+' $\in \mathit{Follow}(p)$.

2. Next, we compute $\mathit{action}(s_4,\ \text{'+'}\ )$. It holds that

$$
\begin{aligned}
\mathit{action}(s_4,\ \text{'+'}\ ) &= \mathit{action}(\{s \to e\bullet\ \$,\ e \to e\bullet\ \text{'+'}\ p,\ e \to e\bullet\ \text{'-'}\ p\ \},\ \text{'+'}\ ) \\
&= \langle \mathit{shift}, \mathit{closure}(\{e \to e\ \text{'+'}\ \bullet p\})\rangle \\
&= \langle \mathit{shift}, s_8\rangle.
\end{aligned}
$$

If we were to continue these calculations, we would obtain Table 10.1, as we have chosen the names of the states to match those of the corresponding states in Tables 10.1 and 10.2.

**Exercise 25**: Figure 10.9 shows a grammar for boolean expressions.

(a) Provide the sets *First*$(v)$ for all syntactic variables $v$.

(b) Provide the sets *Follow*$(v)$ for all syntactic variables $v$.

(c) Calculate the set of SLR states.

(d) Check whether there are any conflicts.

(e) Specify the function *action* for the first four states.

Abbreviate the names of the syntactic variables and tokens with $b$, $c$, $s$, and $I$. ◇

$$
\begin{array}{rcl}
\textit{bool} & \to & \textit{bool}\ \text{'}\vee\text{'}\ \textit{conjunction} \\
& \mid & \textit{conjunction} \\
\textit{conjunction} & \to & \textit{conjunction}\ \text{'}\wedge\text{'}\ \textit{simple} \\
& \mid & \textit{simple} \\
\textit{simple} & \to & \text{'(' }\textit{bool}\text{ ')'} \\
& \mid & \text{'}\neg\text{'}\ \textsc{Identifier} \\
& \mid & \textsc{Identifier}
\end{array}
$$

Figure 10.9: A grammar for Boolean expression in conjunctive normal form.

### 10.3.3 Shift/Reduce and Reduce/Reduce Conflicts

In this section, we examine shift-reduce and reduce-reduce conflicts in more detail and consider two examples. The first example shows a shift-reduce conflict. The grammar shown in Figure 10.10 is ambiguous because it does not specify whether the operator '+' binds stronger or weaker than the operator '*'. If we interpret the non-terminal $N$ as an abbreviation for NUMBER, this grammar allows us to interpret the expression $1 + 2 * 3$ either as

$$(1 + 2) * 3 \quad \text{or as} \quad 1 + (2 * 3).$$

$$
\begin{array}{rcl}
e & \to & e\ \text{'+'}\ e \\
& \mid & e\ \text{'*'}\ e \\
& \mid & N
\end{array}
$$

Figure 10.10: A grammar with shift/reduce conflicts.

We augment the grammar and calculate the start state $s_0$:

$$
\begin{array}{rcl}
s_0 & = & \textit{closure}\big(\{s \to \bullet\, e\, \$\}\big) \\
& = & \big\{s \to \bullet\, e\, \$,\ e \to \bullet\, e\text{'+'}e,\ e \to \bullet\, e\text{'*'}e,\ e \to \bullet\, N\ \big\}.
\end{array}
$$

Next, we calculate $s_1 := \textit{goto}(s_0, e)$:

$$
\begin{array}{rcl}
s_1 & = & \textit{goto}(s_0, e) \\
& = & \textit{closure}\big(\{s \to e \bullet\ \$,\ e \to e \bullet\ \text{'+'}\ e,\ e \to e \bullet\ \text{'*'}\ e\ \}\big) \\
& = & \{s \to e \bullet\ \$,\ e \to e \bullet\ \text{'+'}\ e,\ e \to e \bullet\ \text{'*'}\ e\ \}
\end{array}
$$

Now we calculate $s_2 := \mathit{goto}(s_1, \text{'+'})$:

$$\begin{array}{lcl} s_2 & = & \mathit{goto}(s_1, \text{'+'}) \\ & = & \mathit{closure}\big(\{e \to e\,\text{'+'} \bullet e,\ \}\big) \\ & = & \{e \to e\,\text{'+'} \bullet e,\ e \to \bullet\, e\,\text{'+'}e,\ e \to \bullet\, e\,\text{'*'}e,\ e \to \bullet\, N\ \} \end{array}$$

Next, we calculate $s_3 := \mathit{goto}(s_2, e)$:

$$\begin{array}{lcl} s_3 & = & \mathit{goto}(s_2, e) \\ & = & \mathit{closure}\big(\{e \to e\,\text{'+'}e\bullet,\ e \to e \bullet \text{'+'}\ e,\ e \to e \bullet \text{'*'}\ e\}\big) \\ & = & \{e \to e\,\text{'+'}e\bullet,\ e \to e \bullet \text{'+'}\ e,\ e \to e \bullet \text{'*'}\ e\ \} \end{array}$$

Here, a shift-reduce conflict arises when calculating $\mathit{action}(s_3, \text{'*'})$, because on one hand, the marked rule

$$e \to e \bullet \text{'*'}\ e,$$

demands that the token '*' is pushed onto the stack, but on the other hand, we have

$$\mathit{Follow}(e) = \{\ \text{'+'},\ \text{'*'},\ \text{'\$'}\ \},$$

so that, if the next token to be read has the value '*', the symbol stack should be reduced with the rule

$$e \to e\,\text{'+'}\,e\,\bullet.$$

**Remark**: It is not surprising that we found a conflict in the grammar specified above, as this grammar is ambiguous. On the other hand, it can be shown that every SLR grammar must be unambiguous. Consequently, an ambiguous grammar is never an SLR grammar. However, the converse of this statement is not true, as we will see in the next example. ◇

$$\begin{array}{lcl} s & \to & a\ \text{'x'}\ a\ \text{'y'} \\ & | & b\ \text{'y'}\ b\ \text{'x'} \\ a & \to & \lambda \\ b & \to & \lambda \end{array}$$

Figure 10.11: A grammar containing a reduce/reduce conflict.

Next, we examine a grammar that is not an SLR grammar because it contains reduce-reduce conflicts. We consider the grammar shown in Figure 10.11. This grammar is unambiguous, as we have

$$L(s) = \{\ \text{'xy'},\ \text{'yx'}\ \}$$

and the string 'xy' can only be derived using the rule $s \to a$ 'x' $a$ 'y' , while the string 'yx' can only be generated using the rule $s \to b$ 'y' $b$ 'x' . To demonstrate that this grammar contains shift-reduce conflicts, we calculate the start state of an SLR parser for this grammar.

$$\begin{array}{lcl} s_0 & = & \mathit{closure}\big(\{\widehat{s} \to \bullet\, s\,\$\}\big) \\ & = & \big\{\widehat{s} \to \bullet\, s\,\$,\ s \to \bullet\, a\ \text{'x'}\ a\ \text{'y'}\ ,\ s \to \bullet\, b\ \text{'y'}\ b\ \text{'x'}\ ,\ a \to \bullet\, \lambda,\ b \to \bullet\, \lambda\ \big\} \\ & = & \big\{\widehat{s} \to \bullet\, s\,\$,\ s \to \bullet\, a\ \text{'x'}\ a\ \text{'y'}\ ,\ s \to \bullet\, b\ \text{'y'}\ b\ \text{'x'}\ ,\ a \to \lambda\bullet,\ b \to \lambda\bullet\ \big\}, \end{array}$$

since $a \to \bullet\, \lambda$ is the same as $a \to \lambda\bullet$. In this state, there is a reduce-reduce conflict between the two marked rules

$$a \to \lambda \bullet \quad \text{and} \quad b \to \lambda \bullet.$$

This conflict arises when calculating

$$\mathit{action}(s_0,\ \text{'x'}\ )$$

because we have

$$\mathit{Follow}(a) = \{\text{'x'}, \text{'y'}\} = \mathit{Follow}(b).$$

Thus, it is unclear which of these rules the parser should use to reduce the input in state $s_0$ when the next token read has the value 'x', as this token is both an element of the set $\mathit{Follow}(a)$ and the set $\mathit{Follow}(b)$.

**Remark**: As part of the resources provided with this lecture, the file

Python/Chapter-10/SLR-Table-Generator.ipynb

contains a *Python* program that checks whether a given grammar is an SLR grammar. This program computes the states as well as the action table of a given grammar. ◇

## 10.4 Canonical LR Parsers

The reduce-reduce conflict that occurs in the grammar shown in Figure 10.11 can be resolved as follows: In the state

$$\begin{array}{lcl} s_0 & = & \mathit{closure}\big(\{\widehat{s} \to \bullet\, s\, \$\}\big) \\ & = & \big\{\widehat{s} \to \bullet\, s\, \$,\ s \to \bullet\, a\ \text{'x'}\ a\ \text{'y'}\ ,\ s \to \bullet\, b\ \text{'y'}\ b\ \text{'x'}\ ,\ a \to \lambda\bullet,\ b \to \lambda\bullet\ \big\} \end{array}$$

the marked rules $a \to \lambda\bullet$ and $b \to \lambda\bullet$ arise from the closure calculation of the rules

$$s \to \bullet\, a\ \text{'x'}\ a\ \text{'y'} \quad \text{and} \quad s \to \bullet\, b\ \text{'y'}\ b\ \text{'x'}\ .$$

For the first rule, it is clear that the first $a$ must be followed by a 'x', and for the second rule, we see that the first $b$ must be followed by a 'y'. This information goes beyond what is contained in the sets $\mathit{Follow}(a)$ and $\mathit{Follow}(b)$, as we now consider the context in which the syntactic variable appears. This allows us to define the function $\mathit{action}(s_0, \text{'x'})$ and $\mathit{action}(s_0, \text{'y'})$ as follows:

$$\mathit{action}(s_0, \text{'x'}) = \langle \texttt{reduce}, a \to \lambda \rangle \quad \text{and} \quad \mathit{action}(s_0, \text{'y'}) = \langle \texttt{reduce}, b \to \lambda \rangle.$$

This definition resolves the reduce-reduce conflict. The central idea is to include the context in which a rule appears in the closure calculation. To do this, we expand the definition of a marked rule next.

**Definition 28 (Extended Marked Rule)**
An extended marked rule (abbreviated: e.m.R.) of a grammar $G = \langle V, T, R, s \rangle$ is a quadruple

$$\langle a, \beta, \gamma, L \rangle,$$

where:

1. $(a \to \beta\gamma) \in R$.
2. $L \subseteq T$.

We write the extended marked rule $\langle a, \beta, \gamma, L \rangle$ as

$$a \to \beta \bullet \gamma : L.$$

If $L$ consists of only one element $t$, that is if $L = \{t\}$, we omit the set brackets and write the rule as

$$a \to \beta \bullet \gamma : t.$$ ◇

Intuitively, we interpret the e.m.R. $a \to \beta \bullet \gamma : L$ as a state in which the following applies:

(a) The parser attempts to recognize an $a$ using the grammar rule $a \to \beta\gamma$.

(b) So far, $\beta$ has been recognized. For the rule $a \to \beta\gamma$ to be applied, $\gamma$ must now be recognized.

(c) Additionally, we know that a token from the set $L$ must follow the syntactic variable $a$.

Therefore, we refer to the set $L$ as the set of Follow Tokens.

Working with extended marked rules is quite similar to working with marked rules, however, we need to modify the definitions of the functions *closure*, *goto*, and *action* slightly. We begin with the function *closure*.

**Definition 29 (*closure*$(\mathcal{M})$)** Let $\mathcal{M}$ be a set of extended marked rules. We define the closure of $\mathcal{M}$ as the smallest set $\mathcal{K}$ of marked rules for which the following holds:

1. $\mathcal{M} \subseteq \mathcal{K}$,

   thus the closure encompasses the original set of rules.

2. On one hand, if

   $$a \to \beta \bullet c\,\delta : L$$

   is an e.m.R. from the set $\mathcal{K}$, where $c$ is a syntactic variable, and on the other hand, if

   $$c \to \gamma$$

   is a grammar rule of the underlying grammar $G$, then the e.m.R.

   $$c \to \bullet\,\gamma : \bigcup\{\mathit{First}(\delta\,t) \mid t \in L\}$$

   is also an element of the set $\mathcal{K}$. The function *First*$(\alpha)$ computes for a string $\alpha \in (T \cup V)^*$ the set of all tokens $t$ with which a string that has been derived from $\alpha$ can begin.

The uniquely determined set $\mathcal{K}$ defined in this way is again denoted by *closure*$(\mathcal{M})$. ◇

**Remark**: Compared to the old definition, only the computation of the set of follow tokens has been added. The context in which the variable $c$, to be recognized by the rule $c \to \gamma$, appears is initially given by the string $\delta$ that follows $c$ in the rule $a \to \beta \bullet c\,\delta : L$. Now it is possible that $\delta$ derives the empty string $\lambda$. In this case, the follow tokens from the set $L$ also play a role, because if $\delta \Rightarrow^* \lambda$ is true, then a follow token $t$ from the set $L$ can also follow the $c$. ◇

For a given set of e.m.R.s $\mathcal{M}$, the computation of $\mathcal{K} := \mathit{closure}(\mathcal{M})$ can be done iteratively. Figure 10.12 shows the computation of *closure*$(\mathcal{M})$. The main difference compared to the earlier computation of *closure*$()$ is that in the e.m.R.s that we include for a variable $c$ in *closure*$(\mathcal{M})$, we consider the context in which $c$ appears in the set of Follow Tokens. This makes it possible to describe the states of the parser more precisely than with marked rules.

**Remark**: The expression $\bigcup\{\mathit{First}(\delta\,t) \mid t \in L\}$ looks more complicated than it actually is. To compute this expression, it is useful to make a distinction based on whether $\delta$ can derive the empty string $\lambda$ or not, because it holds that

$$\bigcup\{\mathit{First}(\delta\,t) \mid t \in L\} = \begin{cases} \mathit{First}(\delta) \cup L & \text{if } \delta \Rightarrow^* \lambda; \\ \mathit{First}(\delta) & \text{otherwise.} \end{cases}$$

The computation of *goto*$(\mathcal{M}, t)$ for a set $\mathcal{M}$ of extended rules and a symbol $x$ changes from the computation in the case of simple marked rules only by appending the set of follow tokens, which itself remains unchanged:

$$\mathit{goto}(\mathcal{M}, x) := \mathit{closure}\Big(\big\{a \to \beta\,x \bullet \delta : L \mid (a \to \beta \bullet x\,\delta : L) \in \mathcal{M}\big\}\Big).$$

Similar to the theory of SLR parsers, we augment our grammar $G$ by adding a new start variable $\widehat{s}$ to the set of variables and the new rule $\widehat{s} \to s$ to the set of rules. Further, we add the symbol \$ to the

```
1   function closure(𝓜) {
2       𝓚  := 𝓜;
3       𝓚⁻ := {};
4       while (𝓚⁻ ≠ 𝓚) {
5           𝓚⁻ := 𝓚;
6           𝓚  := 𝓚 ∪ {(c → •γ : ⋃{First(δ t) | t ∈ L}) |
7                                (a → β • c δ : L) ∈ 𝓚 ∧ (c → γ) ∈ R};
8       }
9       return 𝓚;
10  }
```

Figure 10.12: The computation of *closure*$(\mathcal{M})$.

tokens. Then the start state has the form

$$q_0 := \mathit{closure}\big(\{\widehat{s} \to \bullet\, s : \$\}\big),$$

because the file end symbol "$" must follow the start symbol. Finally, we show how the definition of the function *action*() must be changed. We specify the computation of this function through the following conditional equations.

1. $(a \to \beta \bullet t\,\delta : L) \in \mathcal{M} \;\Rightarrow\; \mathit{action}(\mathcal{M}, t) := \langle \texttt{shift}, \mathit{goto}(\mathcal{M}, t)\rangle$.
2. $(a \to \beta\bullet : L) \in \mathcal{M} \;\wedge\; a \neq \widehat{s} \;\wedge\; t \in L \;\Rightarrow\; \mathit{action}(\mathcal{M}, t) := \langle \texttt{reduce}, a \to \beta\rangle$.
3. $(\widehat{s} \to s\bullet : \$) \in \mathcal{M} \;\Rightarrow\; \mathit{action}(\mathcal{M}, \$) := \texttt{accept}$.
4. Otherwise: $\mathit{action}(\mathcal{M}, t) := \texttt{error}$.

If a conflict occurs in these equations, because both the condition of the first equation and the condition of the second equation are met, then we speak again of a shift-reduce conflict. A shift-reduce conflict thus occurs in the computation of $\mathit{action}(\mathcal{M}, t)$ when there are two e.m.R.s

$$(a \to \beta \bullet t\,\delta : L_1) \in \mathcal{M} \quad \text{and} \quad (c \to \gamma\bullet : L_2) \in \mathcal{M} \quad \text{with } t \in L_2$$

because then it is unclear whether in state $\mathcal{M}$ the token $t$ should be pushed onto the stack, or whether instead the symbol stack should be reduced with the rule $c \to \gamma$.

**Remark**: Compared to an SLR parser, the possibility of shift-reduce conflicts is reduced, because in an SLR parser, a shift-reduce conflict already occurs if $t \in \mathit{Follow}(c)$ and the set $L_2$ is usually smaller than the set $\mathit{Follow}(c)$. ◇

A reduce-reduce conflict occurs when there are two e.m.R.s

$$(a \to \beta \bullet : L_1) \in \mathcal{M} \quad \text{and} \quad (c \to \delta \bullet : L_2) \in \mathcal{M} \quad \text{with} \quad L_1 \cap L_2 \neq \{\}$$

because then it is unclear which of these two rules should be used to reduce the symbol stack when the next token is an element of the intersection $L_1 \cap L_2$.

**Remark**: Compared to an SLR parser, the possibility of reduce-reduce conflicts is reduced, because in an SLR parser, a reduce-reduce conflict already occurs if there exists a $t$ in the set $\mathit{Follow}(a) \cap \mathit{Follow}(c)$ and the *Follow* sets are often larger than the sets $L_1$ and $L_2$. ◇

**Example**: We revisit the example of the grammar shown in Figure 10.11 and first calculate the set of all states.

1. $s_0 := \mathit{closure}\Big(\{\widehat{s} \to \bullet\, s : \$\}\Big)$
   $= \{\widehat{s} \to \bullet\, s : \$, s \to \bullet\, a\text{'x'}a\text{'y'} : \$, s \to \bullet\, b\text{'y'}b\text{'x'} : \$, a \to \bullet\ : \text{'x'}, b \to \bullet\ : \text{'y'}\}.$

2. $s_1 := \mathit{goto}(s_0, a)$
   $= \mathit{closure}(\{s \to a \bullet \text{'x'}a\text{'y'} : \$\})$
   $= \{s \to a \bullet \text{'x'}a\text{'y'} : \$\}.$

3. $s_2 := \mathit{goto}(s_0, s)$
   $= \mathit{closure}(\{\widehat{s} \to s\bullet : \$\})$
   $= \{\widehat{s} \to s\bullet : \$\}.$

4. $s_3 := \mathit{goto}(s_0, b)$
   $= \mathit{closure}(\{s \to b \bullet \text{'y'}b\text{'x'} : \$\})$
   $= \{s \to b \bullet \text{'y'}b\text{'x'} : \$\}.$

5. $s_4 := \mathit{goto}(s_3, \text{'y'})$
   $= \mathit{closure}(\{s \to b\text{'y'} \bullet b\text{'x'} : \$\})$
   $= \{s \to b\text{'y'} \bullet b\text{'x'} : \$, b \to \bullet\ : \text{'x'}\}.$

6. $s_5 := \mathit{goto}(s_4, b)$
   $= \mathit{closure}(\{s \to b\text{'y'}b \bullet \text{'x'} : \$\})$
   $= \{s \to b\text{'y'}b \bullet \text{'x'} : \$\}.$

7. $s_6 := \mathit{goto}(s_5, \text{'x'})$
   $= \mathit{closure}(\{s \to b\text{'y'}b\text{'x'}\bullet : \$\})$
   $= \{s \to b\text{'y'}b\text{'x'}\bullet : \$\}.$

8. $s_7 := \mathit{goto}(s_1, \text{'x'})$
   $= \mathit{closure}(\{s \to a\text{'x'} \bullet a\text{'y'} : \$\})$
   $= \{s \to a\text{'x'} \bullet a\text{'y'} : \$, a \to \bullet\ : \text{'y'}\}.$

9. $s_8 := \mathit{goto}(s_7, a)$
   $= \mathit{closure}(\{s \to a\text{'x'}a \bullet \text{'y'} : \$\})$
   $= \{s \to a\text{'x'}a \bullet \text{'y'} : \$\}.$

10. $s_9 := \mathit{goto}(s_8, \text{'y'})$
    $= \mathit{closure}(\{s \to a\text{'x'}a\text{'y'}\bullet : \$\})$
    $= \{s \to a\text{'x'}a\text{'y'}\bullet : \$\}.$

Next, we examine whether there are any conflicts in the states. In the last section, we found a Reduce-Reduce conflict in the start state $s_0$ between the two rules $a \to \lambda$ and $b \to \lambda$, because

$$\mathit{Follow}(a) \cap \mathit{Follow}(b) = \{\text{'x'}, \text{'y'}\} \neq \{\}$$

holds. This conflict has now disappeared, as there is no conflict between the e.m.R.s

$$a \to \bullet\ : \text{'x'} \quad \text{and} \quad b \to \bullet\ : \text{'y'}$$

due to 'x' $\neq$ 'y'. It is easy to see that no conflicts arise in the other states either.

**Exercise 26**: Calculate the set of states for an LR parser for the following grammar:

$$
\begin{array}{lcl}
e & \to & e \text{ '+' } p \\
  & \mid & p \\
p & \to & p \text{ '*' } f \\
  & \mid & f \\
f & \to & \text{'('} e \text{ ')'} \\
  & \mid & N
\end{array}
$$

Additionally, investigate whether this grammar has shift-reduce conflicts or reduce-reduce conflicts.

**Remark**: As part of the resources provided with this lecture, the file

Python/Chapter-10/LR-Table-Generator.ipynb

contains a *Python* program that checks whether a given grammar qualifies as a canonical LR grammar. This program computes the LR-states as well as the action table for a given grammar. ◇

**Remark**: The theory of LR-parsing has been developed by Donald E. Knuth [Knu65]. His theory is described in the paper "On the translation of languages from left to right". ◇

## 10.5 LALR-Parser*

The number of states in an LR parser is often significantly larger than the number of states in an SLR parser for the same grammar. For instance, an SLR parser for the C grammar manages with 349 states. However, since the language C is not an SLR language, creating an SLR parse table for C results in a number of conflicts, so that an SLR parser for the language C does not work. In contrast, an LR parser for the language C has 1572 states, as you can see here. In the seventies, when the available main memory of most computers was still modestly sized compared to today, LR parsers therefore had a size that was problematic in practice. A detailed analysis of the set of states of LR parsers showed that it is often possible to combine certain states. This can significantly reduce the number of states in most cases. We illustrate the concept with an example and consider the grammar shown in Figure 10.13, which I have taken from the Dragon Book [ASUL06]. (The "Dragon Book" is the standard work in the field of compiler construction.)

$$
\begin{array}{lcl}
\widehat{s} & \to & s \\
s & \to & c\ c \\
c & \to & \text{'x'}\ c \\
  & \mid & \text{'y'}
\end{array}
$$

Figure 10.13: A grammar from the Dragon Book.

Figure 10.14 shows the so-called LR-goto-graph for this grammar. The nodes of this graph are the states. Looking at the LR-goto-graph, we find that the states $s_6$ and $s_3$ differ only in the sets of follow tokens, as on one hand, we have

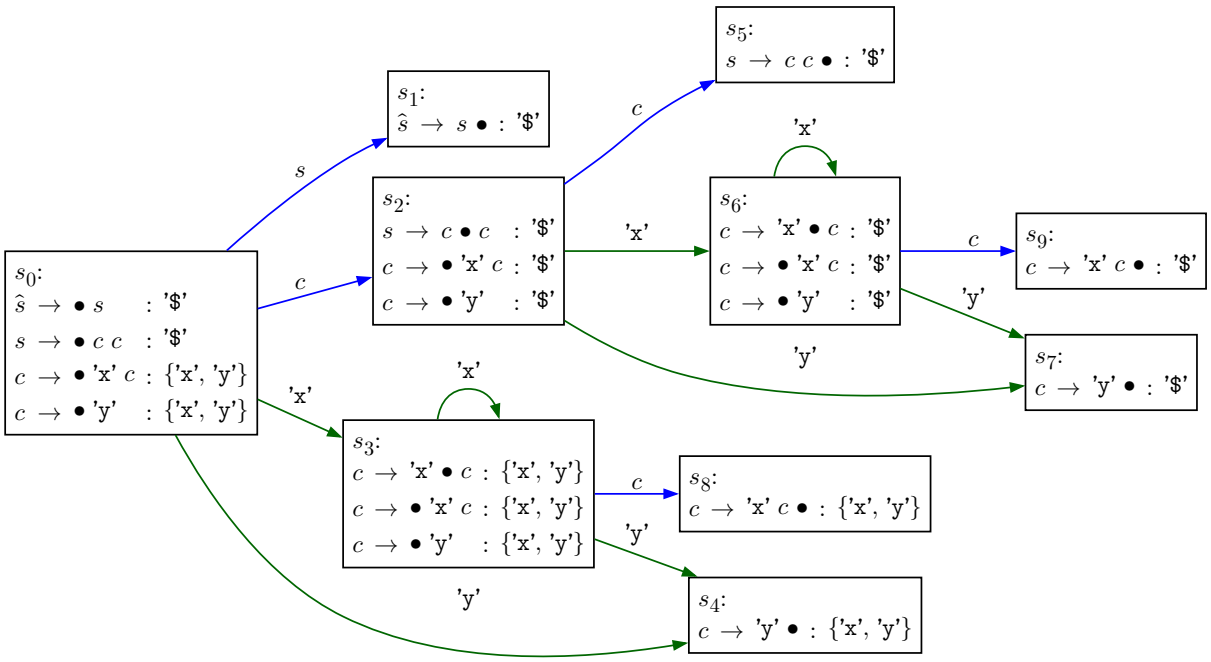



Figure 10.14: LR-goto-graph for the grammar of Figure 10.13.

$$s_6 = \Big\{c \to \text{'x'} \bullet c : \text{'\$'}, c \to \bullet\ \text{'x'}\ c : \text{'\$'}, c \to \bullet\ \text{'y'}\ : \text{'\$'}\Big\},$$

and on the other hand, we have

$$s_3 = \Big\{c \to \text{'x'} \bullet c : \{\text{'x'}, \text{'y'}\}, c \to \bullet\ \text{'x'}\ c : \{\text{'x'}, \text{'y'}\}, c \to \bullet\ \text{'y'}\ : \{\text{'x'}, \text{'y'}\}\Big\}.$$

Clearly, the set $s_3$ is derived from the set $s_6$ by replacing everywhere '$' with the set {'x', 'y'}. Similarly, the set $s_7$ can be transformed into $s_4$, and $s_9$ into $s_8$. The crucial realization is that the function *goto*() is invariant under this kind of transformation, because in the definition of this function, the set of follow tokens is irrelevant. For example, we see that

$$\mathit{goto}(s_3, c) = s_8 \quad \text{and} \quad \mathit{goto}(s_6, c) = s_9$$

hold and that, on the other hand, the state $s_9$ changes into the state $s_8$ when we replace everywhere in $s_9$ the terminal '$' with the set {'x', 'y'}. If we define the core of a set of extended marked rules by omitting the set of follow tokens in each rule, and then combine states with the same core, we obtain the goto-graph shown in Figure 10.15.

To generalize and formalize the observations made while examining the grammar shown in Figure 10.13, we define a function *core*(), which calculates the core of a set of e.m.R.s (extended marked rules) and thus transforms this set into a set of marked rules:

$$\mathit{core}(\mathcal{M}) := \{a \to \beta \bullet \gamma \mid (a \to \beta \bullet \gamma : L) \in \mathcal{M}\}.$$

Thus, the function *core*() simply removes the set of follow tokens from the e.m.R.s. We had defined the function *goto*() for a set $\mathcal{M}$ of extended marked rules and a symbol $x$ as

$$\mathit{goto}(\mathcal{M}, x) := \mathit{closure}\Big(\big\{a \to \beta\, x \bullet \gamma : L \mid (a \to \beta \bullet x\, \gamma : L) \in \mathcal{M}\big\}\Big).$$

Clearly, the set of follow tokens plays no role in the calculation of $\mathit{goto}(\mathcal{M}, x)$, formally for two e.m.R.-sets $\mathcal{M}_1$ and $\mathcal{M}_2$ and a symbol $x$, the formula holds:

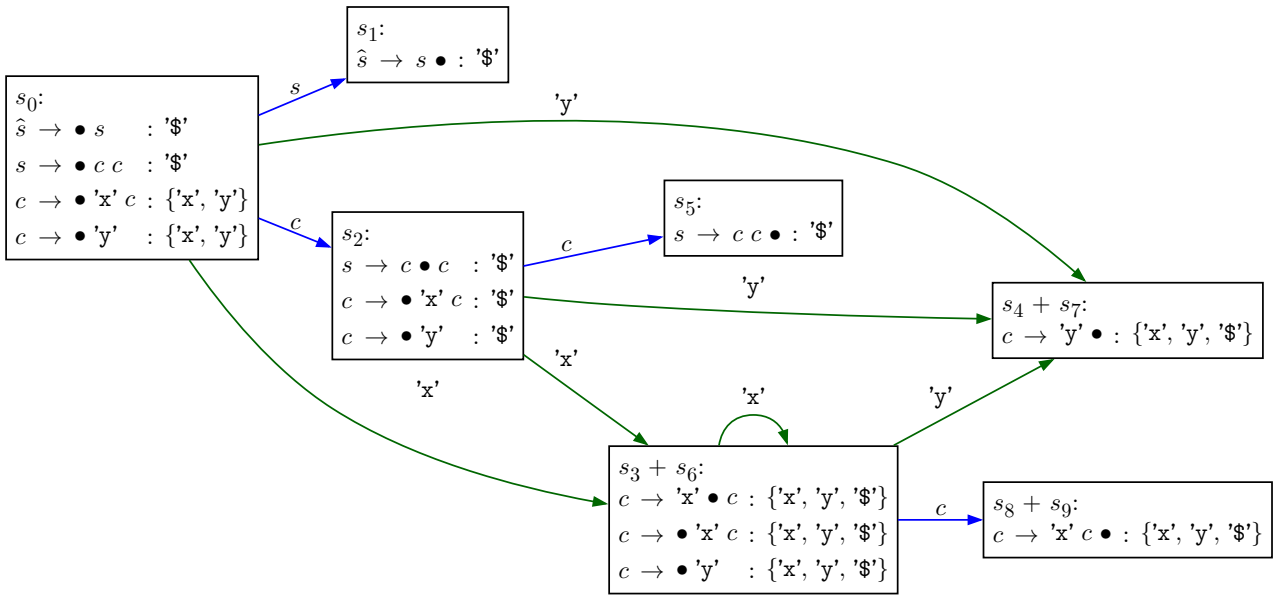



Figure 10.15: The LALR-goto-graph for the grammar from Figure 10.13.

$$\mathit{core}(\mathcal{M}_1) = \mathit{core}(\mathcal{M}_2) \;\Rightarrow\; \mathit{core}(\mathit{goto}(\mathcal{M}_1, x)) = \mathit{core}(\mathit{goto}(\mathcal{M}_2, x)).$$

For two sets of e.m.R.s (extended marked rules) $\mathcal{M}$ and $\mathcal{N}$ that have the same core, we define the extended union $\mathcal{M} \uplus \mathcal{N}$ of $\mathcal{M}$ and $\mathcal{N}$ as

$$\mathcal{M} \uplus \mathcal{N} := \{a \to \beta \bullet \gamma : K \cup L \mid (a \to \beta \bullet \gamma : K) \in \mathcal{M} \;\wedge\; (a \to \beta \bullet \gamma : L) \in \mathcal{N}\}.$$

We generalize this definition to an operation $\biguplus$, defined on a set of sets of e.m.R.s: If $\mathfrak{I}$ is a set of sets of e.m.R.s, all having the same core, i.e.,

$$\mathfrak{I} = \{\mathcal{M}_1, \cdots, \mathcal{M}_k\} \quad \text{with} \quad \mathit{core}(\mathcal{M}_i) = \mathit{core}(\mathcal{M}_j) \quad \text{for all } i, j \in \{1, \cdots, k\},$$

then we define

$$\biguplus \mathfrak{I} := \mathcal{M}_1 \uplus \cdots \uplus \mathcal{M}_k.$$

Let $\Delta$ be the set of all states of an LR parser. Then, the set of states of the corresponding LALR parser is given by the extended union of all subsets of $\Delta$ whose elements have the same core:

$$\mathfrak{Q} := \left\{ \biguplus \mathfrak{I} \mid \mathfrak{I} \in 2^{\Delta} \wedge \forall \mathcal{M}, \mathcal{N} \in \mathfrak{I} : \mathit{core}(\mathcal{M}) = \mathit{core}(\mathcal{N}) \wedge \text{and } \mathfrak{I} \text{ is maximal} \right\}.$$

The requirement "$\mathfrak{I}$ maximal" in the above definition expresses that in $\mathfrak{I}$, indeed all sets from $\Delta$ are combined that have the same core. The thus defined set $\mathfrak{Q}$ is the set of LALR states.

Next, we consider how the computation of $\mathit{goto}(\mathcal{M}, X)$ must change if $\mathcal{M}$ is an element of the set $\mathfrak{Q}$ of LALR states. To compute $\mathit{goto}(\mathcal{M}, X)$, we first calculate the set

$$\mathit{closure}\Big(\{a \to \alpha X \bullet \beta : L \mid (a \to \alpha \bullet X \beta : L) \in \mathcal{M}\}\Big).$$

The problem is that this set is generally not an element of the set $\mathfrak{Q}$, as the states in $\mathfrak{Q}$ arise from the combination of several LR states. However, the states that are combined in the computation of $\mathfrak{Q}$ all have the same core. Therefore, the set

$$\Big\{q \in \mathfrak{Q} \mid \mathit{core}(q) = \mathit{core}\big(\mathit{closure}\big(\{a \to \beta\, X \bullet \gamma : L \mid (a \to \beta \bullet X\, \gamma : L) \in \mathcal{M}\}\big)\big)\Big\}$$

contains exactly one element and this element is the value of $\mathit{goto}(\mathcal{M}, X)$. Thus, we can set

$$\mathit{goto}(\mathcal{M}, X) := \mathit{arb}\Big(\Big\{q \in \mathfrak{Q} \mid \mathit{core}(q) = \mathit{core}\Big(\mathit{closure}\Big(\big\{a \to \beta\, X \bullet \gamma : L \mid (a \to \beta \bullet X\, \gamma : L) \in \mathcal{M}\big\}\Big)\Big)\Big\}\Big)$$

The function $\mathit{arb}()$ used here serves to extract an arbitrary element from a set. Since the set from which the element is extracted contains exactly one element, $\mathit{goto}(\mathcal{M}, x)$ is well-defined. The computation of the expression $\mathit{action}(\mathcal{M}, t)$ does not change compared to the computation for an LR parser.

## 10.6 Comparison of SLR, LR, and LALR Parsers

We now want to compare the different methods with which we have constructed Shift-Reduce parsers in this chapter. We call a language $\mathcal{L}$ an SLR language if $\mathcal{L}$ can be recognized by an SLR parser. The terms canonical LR language and LALR language are defined analogously. The following relationships exist between these languages:

$$\text{SLR language} \subsetneq \text{LALR language} \subsetneq \text{canonical LR language} \qquad (\star)$$

These inclusions are easy to understand: In defining the LR parsers, we added sets of follow tokens to the marked rules. This made it possible to avoid Shift-Reduce and Reduce-Reduce conflicts in certain cases. As the state sets of canonical LR parsers can become very large, we then combined sets of extended marked rules that have the same core. This is how we obtained the LALR parsers. However, by combining rule sets, we can in some cases introduce Reduce-Reduce conflicts, so the set of LALR languages is a subset of the canonical LR languages.

We will show in the following subsections that the inclusions in $(\star)$ are proper.

### 10.6.1 SLR Language $\subsetneq$ LALR Language

The states of an LALR parser, as compared to those of an SLR parser, include sets of follow tokens. This makes LALR parsers at least as powerful as SLR parsers. We will now demonstrate that LALR parsers are indeed more powerful than SLR parsers. To support this claim, we present a grammar for which there exists an LALR parser, but no SLR parser. We had seen on page 167 that the grammar

$$\begin{array}{lcl} s & \to & a\text{'x'}\, a\text{'y'} \mid b\text{'y'}\, b\text{'x'} \\ a & \to & \lambda \\ b & \to & \lambda \end{array}$$

is not an SLR grammar. Later, we saw that this grammar can be parsed by a canonical LR parser. We will now show that this grammar can also be parsed by an LALR parser. To do this, we calculate the set of LALR states. This requires first calculating the set of canonical LR states. We had already carried out this calculation earlier and obtained the following states:

1. $s_0 = \big\{\widehat{s} \to \bullet\, s : \$, s \to \bullet\, a\text{'x'}a\text{'y'} : \$, s \to \bullet\, b\text{'y'}b\text{'x'} : \$, a \to \bullet\ : \text{'x'}, b \to \bullet\ : \text{'y'}\big\}$,
2. $s_1 = \big\{s \to a \bullet\, \text{'x'}a\text{'y'} : \$\big\}$,
3. $s_2 = \big\{\widehat{s} \to s\bullet : \$\big\}$,
4. $s_3 = \big\{s \to b \bullet\, \text{'y'}b\text{'x'} : \$\big\}$,
5. $s_4 = \big\{s \to b\text{'y'} \bullet\, b\text{'x'} : \$, b \to \bullet\ : \text{'x'}\big\}$,
6. $s_5 = \big\{s \to b\text{'y'}b \bullet\, \text{'x'} : \$\big\}$,
7. $s_6 = \big\{s \to b\text{'y'}b\text{'x'}\bullet : \$\big\}$,
8. $s_7 = \big\{s \to a\text{'x'} \bullet\, a\text{'y'} : \$, a \to \bullet\ : \text{'y'}\big\}$,

9. $s_8 = \{s \to a\text{'x'}a \bullet \text{'y'} : \$\}$,
10. $s_9 = \{s \to a\text{'x'}a\text{'y'}\bullet : \$\}$.

We observe that the cores of all the states listed here are different. Therefore, for this grammar, the set of states of the LALR parser coincides with the set of states of the canonical LR parser. This implies that there are no conflicts in the LALR states, because in the transition from canonical LR parsers to LALR parsers, we have only combined states with the same core, and the definition of the functions *goto*() and *action*() remained unchanged.

### 10.6.2 LALR Language $\subsetneq$ Canonical LR Language

We have defined LALR parsers by combining different states of a canonical LR parser. Therefore, it is clear that canonical LR parsers are at least as powerful as LALR parsers. To demonstrate that canonical LR parsers are indeed more powerful than LALR parsers, we need a grammar for which a canonical LR parser can be generated, but not an LALR parser. Figure 10.16 shows such a grammar, which I have taken from the Dragon Book.

$$
\begin{array}{rcl}
s & \to & \text{'v'}\ a\ \text{'y'} \\
 & | & \text{'w'}\ b\ \text{'y'} \\
 & | & \text{'v'}\ b\ \text{'z'} \\
 & | & \text{'w'}\ a\ \text{'z'} \\
a & \to & \text{'x'} \\
b & \to & \text{'x'}
\end{array}
$$

Figure 10.16: A canonical LR grammar that is not an LALR grammar.

We first calculate the set of states for a canonical LR parser for this grammar. We obtain the following sets of extended marked rules:

1. $s_0 = \mathit{closure}(\widehat{s} \to \bullet\ s : \$) = \{\ \widehat{s} \to \bullet\ s : \$,$
$s \to \bullet\text{'v'}a\text{'y'} : \$,$
$s \to \bullet\text{'v'}b\text{'z'} : \$,$
$s \to \bullet\text{'w'}a\text{'z'} : \$,$
$s \to \bullet\text{'w'}b\text{'y'} : \$\ \}$,
2. $s_1 = \mathit{goto}(s_0, s) = \{\widehat{s} \to s\bullet : \$\}$
3. $s_2 = \mathit{goto}(s_0,\ \text{'v'}\ ) = \{\ s \to \text{'v'} \bullet b\text{'z'} : \$,$
$s \to \text{'v'} \bullet a\text{'y'} : \$,$
$a \to \bullet\text{'x'} : \text{'y'},$
$b \to \bullet\text{'x'} : \text{'z'}\ \}$,
4. $s_3 = \mathit{goto}(s_0,\ \text{'w'}\ ) = \{\ s \to \text{'w'} \bullet a\text{'z'} : \$,$
$s \to \text{'w'} \bullet b\text{'y'} : \$,$
$a \to \bullet\text{'x'} : \text{'z'},$
$b \to \bullet\text{'x'} : \text{'y'}\ \}$,
5. $s_4 = \mathit{goto}(s_2,\ \text{'x'}\ ) = \{a \to \text{'x'}\bullet : \text{'y'},\ b \to \text{'x'}\bullet : \text{'z'}\}$,

6. $s_5 = \mathit{goto}(s_3, \text{'x'}) = \{a \to \text{'x'}\bullet : \text{'z'},\ b \to \text{'x'}\bullet : \text{'y'}\}$,
7. $s_6 = \mathit{goto}(s_2, a) = \{s \to \text{'v'}a \bullet \text{'y'} : \$\}$,
8. $s_7 = \mathit{goto}(s_6, \text{'y'}) = \{s \to \text{'v'}a\text{'y'}\bullet : \$\}$,
9. $s_8 = \mathit{goto}(s_2, b) = \{s \to \text{'v'}b \bullet \text{'z'} : \$\}$,
10. $s_9 = \mathit{goto}(s_8, \text{'z'}) = \{s \to \text{'v'}b\text{'z'}\bullet : \$\}$,
11. $s_{10} = \mathit{goto}(s_3, a) = \{s \to \text{'w'}a \bullet \text{'z'} : \$\}$,
12. $s_{11} = \mathit{goto}(s_{10}, \text{'z'}) = \{s \to \text{'w'}a\text{'z'}\bullet : \$\}$,
13. $s_{12} = \mathit{goto}(s_3, b) = \{s \to \text{'w'}b \bullet \text{'y'} : \$\}$,
14. $s_{13} = \mathit{goto}(s_{12}, \text{'y'}) = \{s \to \text{'w'}b\text{'y'}\bullet : \$\}$.

The only states where conflicts could occur are the sets $s_4$ and $s_5$, as in these states, reductions with both rules

$$a \to \text{'x'} \quad \text{and} \quad b \to \text{'x'}$$

are potentially possible. However, since the sets of follow tokens have an empty intersection, there is actually no conflict, and the grammar is a canonical LR grammar.

Next, we calculate the LALR states for the grammar mentioned above. The only states that have a common core are $s_4$ and $s_5$, as

$$\mathit{core}(s_4) = \{a \to \text{'x'}\bullet,\ b \to \text{'x'}\bullet\} = \mathit{core}(s_5).$$

In calculating the LALR states, these two states are combined into one state $s_{\{4,5\}}$. This new state takes the form

$$s_{\{4,5\}} = \{A \to \text{'x'}\bullet : \{\text{'y'}, \text{'z'}\},\ B \to \text{'x'}\bullet : \{\text{'y'}, \text{'z'}\}\}.$$

Here, there is obviously a reduce-reduce conflict, as on one hand we have

$$\mathit{action}(s_{\{4,5\}}, \text{'y'}) = \langle \mathit{reduce}, A \to \text{'x'}\rangle,$$

but on the other hand, it also holds that

$$\mathit{action}(s_{\{4,5\}}, \text{'y'}) = \langle \mathit{reduce}, B \to \text{'x'}\rangle.$$

**Exercise 27**: Let $G = \langle V, T, R, s\rangle$ be an LR grammar and $\mathcal{N}$ be the set of LALR states of the grammar. Explain why there can be no shift-reduce conflicts in the set $\mathcal{N}$. ◇

**Historical Notes** The theory of LALR parsing is due to Franklin L. DeRemer [DeR71]. At the time of its invention, the space savings of LALR parsing in comparison to LR parsing were crucial.

### 10.6.3 Evaluation of the Different Methods

In practice, SLR parsers are not sufficient, as there are a number of practically relevant language constructs for which no SLR parser can be generated. Canonical LR parsers are much more powerful but often require significantly more states. LALR parsers represent a compromise here: on the one hand, LALR languages are almost as expressive as canonical LR languages, but on the other hand, the memory requirements of LALR parsers are in the same order of magnitude as those of SLR parsers. For example, the SLR parse table for the C language has a total of 349 states, the corresponding LR parse table has 1572 states, while the LALR parser manages with 350 states, thus only one state more

than the SLR parser. In the main memories typically available today, however, canonical LR parsers can usually be accommodated without difficulty, so there is actually no compelling reason to use an LALR parser instead of an LR parser.

On the other hand, no one will want to program an LALR parser or a canonical LR parser by hand. Instead, you will later use a parser generator like PLY for *Python*, *Bison* for C, or *JavaCup* for *Java*. While PLY and *JavaCup* can only generate an LALR parser, *Bison* can also generate an LR parser.

## 10.7 Check your Understanding

(a) What is the definition of a shift-reduce parser and how is the set *Action* defined?

(b) How does a shift-reduce parser work?

(c) How is the concept of a marked rule defined? How is a marked rule interpreted?

(d) Can you compute the closure of a set of marked rules?

(e) Are you able to define the functions `First` and `Follow` for an SLR-parser?

(f) Are you able to compute the states of an SLR-parser?

(g) Given the set of all states of an SLR-parser, can you compute both the action table and the goto table?

(h) How is the concept of an extended marked rule defined?

(i) Are you able to compute the closure of a set of extended marked rules?

(j) Define the notion of shift/reduce and reduce/reduce conflicts for both SLR-parsers and for LR-parsers.

(k) Why are LR-parser-generators more powerful than SLR-parser-generators?

(l) How is the set of states reduced when we transform an LR-parser into an LALR-parser?

# Chapter 11

# Advanced Features of Lark

Figure 11.1: Larkhall in South Lanarkshire, Scotland.

When we discussed the parser generator Lark in chapter 7, we were not able to discuss all the features of Lark, since the underlying theory, which is the theory of Lalr parsing, had not been developed. Since we have acquainted ourselves with this theory in the previous chapter, we are finally able to discuss some of the more advanced features of Lark.

1. First, we show how shift-reduce and reduce-reduce conflicts are reported by Lark and how the Lalr states that Lark computes can be inspected.

2. Then we discuss how these conflicts can be resolved. As Lark resolves every shift-reduce conflict in favour of shifting and offers no way to change this decision, the precedences and associativities of the operators have to be encoded into the grammar itself.

All examples of this chapter are available as Jupyter notebooks in the following directory:

`Python/Chapter-11`.

| notebook | topic |
|---|---|
| 01-Conflicts.ipynb | shift-reduce conflicts and how to inspect the states |
| 02-Conflicts-Resolved.ipynb | encoding operator precedences into the grammar |
| 03-Look-Ahead.ipynb | a reduce-reduce conflict caused by insufficient look-ahead |
| 04-Look-Ahead-Solved.ipynb | the same grammar, rewritten so that it is LALR(1) |
| 05-Mysterious-Conflicts.ipynb | a mysterious reduce-reduce conflict |

## 11.1 Shift-Reduce and Reduce-Reduce Conflicts

In this section we show how shift-reduce and reduce-reduce conflicts are dealt with in Lark. Figure 11.2 on page 180 shows a grammar for arithmetical expressions that is ambiguous because it does not specify the precedence of the different arithmetical operators.

```
1  expr : expr "+" expr   -> add
2       | expr "*" expr   -> mul
3       | NUMBER          -> number
4
5  NUMBER : /0|[1-9][0-9]*/
6
7  %import common.WS
8  %ignore WS
```

Figure 11.2: An ambiguous grammar for arithmetical expressions.

Note that in a Lark grammar the syntactical variables are written in lower case, while the names of the token classes are written in upper case. The operators "+" and "*" occur as anonymous terminals: they are given as string literals inside the grammar rules and Lark invents the names PLUS and STAR for them. These invented names show up in the messages that we discuss below. The three alternatives additionally carry the aliases `add`, `mul` and `number`, which are attached with the operator "->". These aliases only determine the names of the nodes of the parse tree, they have no influence on the parse table. They are discussed in the remark on page 185.

This grammar does not specify whether the string

"1 + 2 * 3" is interpreted as "(1 + 2) * 3" or as "1 + (2 * 3)".

Since every LALR grammar is unambiguous, but the grammar shown in Figure 11.2 is ambiguous, it has to have at least one shift-reduce or reduce-reduce conflict. This grammar is part of the jupyter notebook

Python/Chapter-11/01-Conflicts.ipynb.

### 11.1.1 Making Lark Talk About Its Conflicts

By default, Lark is remarkably tight-lipped about shift-reduce conflicts: the parser is built, the conflicts are resolved silently, and nothing is printed. There are two reasons for this silence.

1. The conflict messages are written to the logger object `lark.logger`, and the level of this logger is set to `logging.CRITICAL` when Lark is imported. Therefor only critical errors are printed but warnings are discarded.

2. The messages are only issued with the severity `warning` if the keyword argument `debug=True` is given when the parser is constructed. Otherwise they are issued with the severity `debug`.

Therefore, in order to see the conflicts of the grammar given in Figure 11.2, we have to create the parser as shown in Figure 11.3 on page 181.

```
import logging
from lark import Lark, logger

logger.setLevel(logging.DEBUG)

parser = Lark(grammar, start='expr', parser='lalr', debug=True)
```

Figure 11.3: Creating a parser so that conflicts are reported.

When we run this code with the grammar of Figure 11.2, we get the output that is shown in Figure 11.4 on page 181.

```
Shift/Reduce conflict for terminal PLUS: (resolving as shift)
 * <expr : expr PLUS expr>
Shift/Reduce conflict for terminal STAR: (resolving as shift)
 * <expr : expr PLUS expr>
Shift/Reduce conflict for terminal PLUS: (resolving as shift)
 * <expr : expr STAR expr>
Shift/Reduce conflict for terminal STAR: (resolving as shift)
 * <expr : expr STAR expr>
```

Figure 11.4: The conflicts reported for the grammar of Figure 11.2.

Hence there are four shift-reduce conflicts. Every one of these messages names

1. the terminal $o$ for which the entry *action*$(s, o)$ of the action table is ambiguous, and

2. the grammar rule $r$ that could have been used to reduce the symbol stack in this situation.

Unfortunately, the message does not name the state $s$ in which the conflict occurs, and it does not show the follow sets of the rules involved. We will remedy this in the next subsection.

The important observation is the parenthesised remark "`(resolving as shift)`": Lark resolves *every* shift-reduce conflict in favour of shifting. There is no way to change this decision for an

individual conflict. The only choice we have is to declare that we do not want to tolerate conflicts at all. If the parser is created with the keyword argument `strict=True`, then the first shift-reduce conflict raises an exception of class `GrammarError` whose message is

```
Shift/Reduce conflict for terminal PLUS. [strict-mode]
 * <expr : expr PLUS expr>
```

It is a good idea to develop a grammar with `strict=True`: this way, a conflict cannot be overlooked. Note that `strict=True` additionally checks the terminals for collisions and that this check needs the package `interegular`, which is not installed together with Lark by default. If it is missing, an exception of class `LexError` is raised as soon as the grammar is free of conflicts.

Reduce-reduce conflicts are treated much more severely. A reduce-reduce conflict always raises an exception of class `GrammarError`, regardless of the value of `strict`. We will see an example of this in Section 11.3.

### 11.1.2 Inspecting the LALR States

Lark provides no method that would print the LALR states it has computed. However, if the parser is created with `debug=True`, then the parse table is kept in a form in which the states are still the sets of marked rules that we have discussed in the previous chapter, so we can print them ourselves. Figure 11.5 on page 182 shows a function that does this. The same function is used in all notebooks of this chapter.

```
from lark.parsers.lalr_analysis import Shift

def dump_states(lark_parser):
  table  = lark_parser.parser.parser.parser.parse_table
  key    = lambda state: sorted(str(rule) for rule in state)
  start  = next(iter(table.start_states.values()))
  rest   = sorted((s for s in table.states if s != start), key=key)
  order  = [start] + rest
  number = { state: i for i, state in enumerate(order) }
  for state in order:
    print(f'state {number[state]}')
    for marked_rule in sorted(state, key=str):
      print(f'    {marked_rule}')
    print()
    for token, (action, arg) in sorted(table.states[state].items()):
      if action is Shift:
        print(f'    {token:<8} shift and go to state {number[arg]}')
      else:
        print(f'    {token:<8} reduce using rule {arg}')
    print()
```

Figure 11.5: A function that prints the LALR states computed by Lark.

The chain of attributes `lark_parser.parser.parser.parser` looks confusing, but it is easily explained: the object returned by `Lark` delegates to a parsing frontend, which delegates to an object of class `LALR_Parser`, which finally contains the object of class `_Parser` that stores the attribute

`parse_table`. The states of this table are *frozen sets of marked rules*, which is exactly the notion that we have developed in the previous chapter. They are not numbered, so the dictionary `number` assigns a number to every state in order to make the output readable. Since the states are sets, iterating over them is not reproducible from one run to the next. Therefore, the states are sorted before they are numbered and the start state is put first, so that it receives the number $0$.

Given the grammar shown in Figure 11.2, Lark creates seven different states. Figure 11.6 on page 183 shows the most interesting of these, together with its actions.

```
1  state 5
2      <expr : expr * PLUS expr>
3      <expr : expr * STAR expr>
4      <expr : expr PLUS expr * >
5
6      $END     reduce using rule <expr : expr PLUS expr>
7      PLUS     shift and go to state 2
8      STAR     shift and go to state 3
```

Figure 11.6: One of the states with a conflict.

Note that Lark marks the position of the parser inside a rule with the character "`*`" instead of the bullet "•" that we have been using so far. The state shown in Figure 11.6 is reached after an expression of the form *expr* '+' *expr* has been read. There are two shift-reduce conflicts in this state: both for the token PLUS and for the token STAR, the parser could either reduce with the rule

$$\textit{expr} \rightarrow \textit{expr}\ \text{'+'}\ \textit{expr}$$

or shift the operator. We see that both conflicts have been decided in favour of shifting: the only reduce action that is left in this state is the one for the end of input, which is called $END by Lark. Note that the printed state no longer shows the discarded reduce actions, nor does it show the follow sets of the marked rules: the parse table only records the decision that has actually been taken. If we want to know which actions have been discarded, we have to read the warnings shown in Figure 11.4.

Of course, resolving these conflicts in favour of shifting is wrong: the string

```
1 * 2 + 3
```

is then interpreted as "`1 * (2 + 3)`" and not as "`(1 * 2) + 3`". You can convince yourself of this by inspecting the parse tree that Lark produces for this string.

## 11.2 Operator Precedence

Consider the grammar for arithmetical expressions shown in Figure 11.7 on page 184. This grammar is ambiguous for the same reason as the grammar discussed in the previous section: it does not specify the precedences and associativities of the operators.

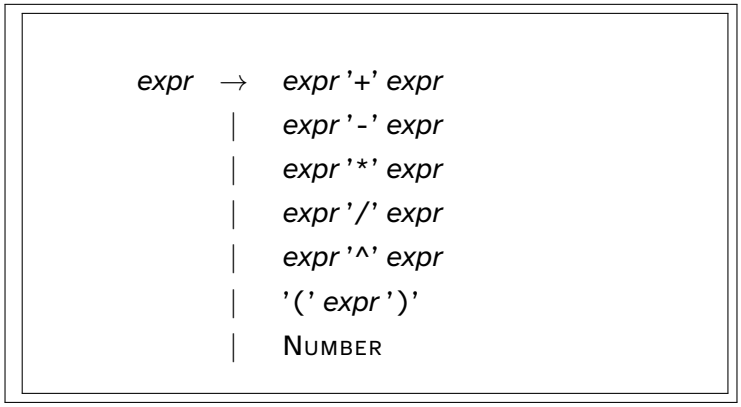



Figure 11.7: An ambiguous grammar for arithmetical expressions.

**There is no way to declare the precedence of an operator in LARK.** Neither a precedence nor an associativity can be attached to a terminal, and the resolution of an individual shift-reduce conflict cannot be influenced. Every shift-reduce conflict is resolved in favour of shifting.

As every shift-reduce conflict is resolved in favour of shifting, all operators effectively have the same precedence and, moreover, they all associate to the right, since shifting an operator of the same precedence is exactly what a right associative operator requires. This explains the parse trees that we have seen in the previous section: the string "`1*2+3`" is read as "`1*(2+3)`" because an operator that groups to the right collects everything that follows it.

Hence, if we want the operators to have different precedences, we have to encode these precedences into the grammar itself. This is done by introducing one syntactical variable for every precedence level. The technique is the one that we have already used in the chapter on context-free languages, and it should by now be familiar: the variable of a given precedence level is defined in terms of the variable of the next higher precedence level.

- If an operator of precedence level $n$ is left associative, the corresponding rule is left recursive:

  $$v_n \to v_n \; o \; v_{n+1}.$$

- If the operator is right associative, the rule is right recursive:

  $$v_n \to v_{n+1} \; o \; v_n.$$

- If the operator is non-associative, the rule uses neither recursion:

  $$v_n \to v_{n+1} \; o \; v_{n+1}.$$

Figure 11.8 on page 185 shows the grammar of Figure 11.7 rewritten in this way. The variable `expr` corresponds to precedence level 1, `prod` to precedence level 2, and `fact` to precedence level 3. The variable `atom` collects the expressions that bind most tightly, that is, numbers and parenthesised expressions.

```
?expr : expr "+" prod  -> add
      | expr "-" prod  -> sub
      | prod

?prod : prod "*" fact  -> mul
      | prod "/" fact  -> div
      | fact

?fact : atom "^" fact  -> power
      | atom

?atom : "(" expr ")"
      | NUMBER         -> number

NUMBER : /0|[1-9][0-9]*/

%import common.WS
%ignore WS
```

Figure 11.8: An unambiguous Lark grammar for arithmetical expressions.

Observe how the associativities are expressed: the rules for `expr` and `prod` are left recursive, because the additive and multiplicative operators associate to the left, while the rule

```
fact : atom "^" fact
```

is right recursive, because exponentiation associates to the right. This grammar is an LALR(1) grammar: if we build a parser for it with `strict=True`, no exception is raised and no warning is printed. The Jupyter notebook

Python/Chapter-11/02-Conflicts-Resolved.ipynb

contains this grammar. Checking a few examples confirms that the generated parser behaves as intended:

(a) "`1-2-3`" is parsed as "`(1-2)-3`",

(b) "`1-2*3`" is parsed as "`1-(2*3)`", and

(c) "`2^3^4`" is parsed as "`2^(3^4)`".

**Remark**: Encoding the precedences into the grammar has a price: the parse trees get deeper, since every expression is now wrapped into one node per precedence level. This is a nuisance if the parse tree is processed further. Lark offers two remedies for this.

1. If the name of a syntactical variable is prefixed with a question mark, as in `?expr`, then a node of this variable is removed from the tree whenever it has exactly one child. This inlines the chains `expr` → `prod` → `fact` → `atom` that carry no information.

2. Individual alternatives can be given a name using the operator "`->`", as in

   ```
   ?expr : expr "+" prod -> add
   ```

so that the resulting tree node is labelled `add` instead of `expr`.

Note that an alternative carrying an alias is never inlined, even if it has a single child. This is why the alternative `NUMBER -> number` survives although `?atom` begins with a question mark.

The notebook uses both devices: it writes `?expr`, `?prod`, `?fact` and `?atom`, and it labels the alternatives with the aliases `add`, `sub`, `mul`, `div`, `power` and `number`. An object of class `Transformer` that provides one method per alias then converts the parse tree into the nested tuples that we have been using as abstract syntax trees.

Both features concern the shape of the parse tree only, they have no influence on the parse table and therefore cannot introduce or remove conflicts. ◇

## 11.3 Resolving Shift-Reduce and Reduce-Reduce Conflicts

We start our discussion by categorizing conflicts with respect to their origin.

1. *Ambiguity Conflicts* are conflicts that originate from an ambiguity in the underlying grammar. Such conflicts therefore indicate an actual problem with the grammar. We saw an example of such conflicts when we tried in Figure 11.2 to describe the syntax of arithmetic expressions without the syntactic categories *product* and *factor*.

   In Lark, such conflicts are resolved by rewriting the grammar as shown in the previous section.

2. *Look-Ahead Conflicts* are Reduce-Reduce conflicts where the grammar is technically unambiguous, but for which a look-ahead of one token is insufficient to resolve the conflict.

3. *Mysterious Conflicts* arise only during the transition from LR states to Lalr states by merging states with the same core. These conflicts therefore occur precisely when the concept of an Lalr grammar is insufficient to describe the syntax of the language to be parsed.

We will now look at the last two cases in detail and show ways in which the conflicts can be resolved.

### 11.3.1 Look-Ahead Conflicts

A Look-Ahead Conflict exists when the grammar is unambiguous, but a look-ahead of one token is insufficient to decide which rule should be used for reduction. Figure 11.9 shows the grammar `03-Look-Ahead.ipynb`[1], which, although unambiguous, does not possess the LR(1) property and is therefore certainly not an Lalr(1) grammar.

```
1   a : b "U" "V"
2     | c "U" "W"
3   b : "X"
4   c : "X"
```

Figure 11.9: An unambiguous grammar without the LR(1) property.

If we calculate the LR states of this grammar, we find, among others, the following state:

[1] I found this grammar on the web on Pete Jinks' page at the address
http://www.cs.man.ac.uk/~pjj/cs212/ho/node19.html
.

$\{b \rightarrow$ 'X'• : 'U', $c \rightarrow$ 'X'• : 'U'$\}$.

Since the set of follow tokens is the same for both rules, we have a Reduce-Reduce conflict here. This conflict is caused by the fact that the parser, with a look-ahead of only one token, cannot decide whether an 'X' should be interpreted as a $b$ or as a $c$, because this is only decided when the character following 'U' is read: If this is a 'V', then the rule

$a \rightarrow b$ 'U' 'V'

will be used overall, and consequently the 'X' is to be interpreted as a $b$. If, however, the second token after the 'X' is a 'W', then the rule to be used is

$a \rightarrow c$ 'U' 'W'

and consequently the 'X' is to be read as $c$.

Lark refuses to build a parser for this grammar. Instead, an exception of class `GrammarError` is raised whose message is shown in Figure 11.10 on page 187. Note that the part of the message that shows the state is only printed if the parser is created with `debug=True`.

```
Reduce/Reduce collision in Terminal('U') between the following rules:
    - <c : X>
    - <b : X>
    collision occurred in state: {
        <c : X * >
        <b : X * >
    }
```

Figure 11.10: The error message produced for the grammar of Figure 11.9.

```
    a : b "V"
      | c "W"
    b : "X" "U"
    c : "X" "U"
```

Figure 11.11: An LR(1) grammar equivalent to Figure 11.9.

The problem with this grammar is that it attempts to interpret an 'X' either as a $b$ or as a $c$ depending on the context. It is obvious how the problem can be solved: If the context "U", which follows both $b$ and $c$, is included in the rules for $b$ and $c$, then the conflict disappears, because then the state in which the conflict previously occurred has the form

$\{b \rightarrow$ 'X' 'U'• : 'V', $c \rightarrow$ 'X' 'U'• : 'W'$\}$.

Here, the next token decides which rule we must reduce with in this state: If the next token is a 'V', we reduce with the rule

$b \rightarrow$ 'X' 'U',

if, however, the next token is the letter 'W', we take the rule

$c \rightarrow$ 'X' 'U'• : 'W'

instead. Figure 11.11 shows the correspondingly modified grammar, which you can find at

Python/Chapter-11/04-Look-Ahead-Solved.ipynb

on the web. For this grammar, Lark builds a parser without complaining. Applying the function `dump_states` to this parser shows the decisive state, which is reproduced in Figure 11.12 on page 188. The follow sets of the two rules are now disjoint and therefore the action table of this state offers exactly one reduce action per token: a 'V' reduces with the rule for $b$, while a 'W' reduces with the rule for $c$.

```
state 7
    <b : X U * >
    <c : X U * >

    V          reduce using rule <b : X U>
    W          reduce using rule <c : X U>
```

Figure 11.12: The state that had caused the conflict, now free of conflicts.

**Remark**: There is a second way out: the grammar can be parsed with the Earley parser, which is the default parser of Lark. If the parser is created with

```
Lark(grammar, start='a', parser='earley', keep_all_tokens=True)
```

then the grammar of Figure 11.9 is accepted as it stands, because the Earley algorithm does not need a bounded look-ahead. The keyword argument `keep_all_tokens` keeps the anonymous terminals in the parse tree, which would otherwise be filtered out. Notebook 03-Look-Ahead.ipynb demonstrates this: the Earley parser recognizes the 'X' of the string 'XUV' as a $b$ and the 'X' of the string 'XUW' as a $c$. The price is a parser that is considerably slower than an Lalr parser. ◇

### 11.3.2 Mysterious Reduce-Reduce Conflicts

A conflict is called a mysterious reduce-reduce conflict if the conflict results from the merger of states that happens when we go from an LR parsing table to an Lalr parsing table. The grammar in Figure 11.13 on page 189 is the same as the grammar shown in Figure 10.16 on page 176 in the previous chapter. Then we had seen that this grammar is an LR grammar, but not an Lalr grammar. Let us see what happens if we use Lark to generate the states for this grammar. This example is discussed in the notebook

Python/Chapter-11/05-Mysterious-Conflicts.ipynb.

When we compute the LR(1) states of this grammar, we find the two states

$\{a \rightarrow X\bullet : \text{'y'},\ b \rightarrow X\bullet : \text{'z'}\}$ and $\{b \rightarrow X\bullet : \text{'y'},\ a \rightarrow X\bullet : \text{'z'}\}$.

Taken by itself, neither of these two states has a conflict, since the follow sets of the respective rules are disjoint. However, these two states have the same core and are therefore merged when the Lalr table is computed. The resulting state has the form

$\{a \rightarrow X\bullet : \{\text{'y'}, \text{'z'}\}, b \rightarrow X\bullet : \{\text{'y'}, \text{'z'}\}\}$

```
s : "v" a "y"
  | "w" b "y"
  | "v" b "z"
  | "w" a "z"

a : X

b : X

X : "x"
```

Figure 11.13: A grammar that generates a mysterious reduce-reduce conflict.

and obviously has a reduce-reduce conflict if the next token is either 'y' or 'z'.

Lark performs this merger and therefore reports two reduce-reduce collisions, one for the token 'y' and one for the token 'z'. Figure 11.14 on page 189 shows the resulting error message. In other words, Lark really does construct LALR tables and therefore rejects this grammar, as the theory of the previous chapter predicts.

```
Reduce/Reduce collision in Terminal('Z') between the following rules:
    - <a : X>
    - <b : X>
    collision occurred in state: {
        <a : X * >
        <b : X * >
    }

Reduce/Reduce collision in Terminal('Y') between the following rules:
    - <a : X>
    - <b : X>
    collision occurred in state: {
        <a : X * >
        <b : X * >
    }
```

Figure 11.14: The error message for the grammar of Figure 11.13.

Note that the error message shows the merged state, but it does not show the follow sets of the two rules. These have to be reconstructed by hand, as we have just done.

**Remark**: A mysterious conflict cannot be repaired by rewriting the grammar in the way that we have used for the look-ahead conflict of the previous subsection, for the grammar is already an LR(1) grammar: the defect is not in the grammar but in the compression of the LR states into the LALR states. Hence, the only way to parse this language with Lark is to use the Earley parser:

```
Lark(grammar, start='s', parser='earley', keep_all_tokens=True)
```

The EARLEY algorithm builds no parse table at all and therefore never merges any states. The notebook shows that all four strings of the language are then parsed correctly: the 'x' in the middle is recognized as an $a$ in `'vxy'` and `'wxz'`, but as a $b$ in `'vxz'` and `'wxy'`. ◇

**Exercise 28**: Use LARK to implement a *Python* parser that is able to evaluate formulas from propositional logic. The language should support the operators

- "<->" (equivalence),
- "->" (implication),
- "|" (disjunction),
- "&" (conjunction), and
- "!" (negation).

The precedences and associativities are defined as follows:

1. The operator "!" binds stronger than the operator "&".
2. The operator "&" binds stronger than the operator "|".
3. The operators "|" and "&" are both left associative.
4. The operator "->" associates to the right and binds weaker than the operator "|".
5. The operator "<->" is not associative and binds weaker than the operator "->".

Furthermore, the language should support parenthesis, variables, and the constants "$\top$" and "$\bot$". Since LARK does not support precedence declarations, you have to introduce one syntactical variable for every precedence level. Create the parser with `parser='lalr'` and `strict=True` so that you notice immediately if your grammar has a conflict.

**Exercise 29**: Implement a parser (using LARK) that evaluates operations on sets of integers. Your parser must accept strings containing integer sets and operators. A set literal is defined as a comma-separated list of integers enclosed in curly braces. An empty set is allowed.

- Example: `{1, 2, 3}`
- Example: `{}`

You must support the following operators. The table is ordered from **highest binding power (top)** to **lowest binding power (bottom)**.

| Precedence | Name | Math Symbol | Associativity |
|---|---|---|---|
| 4 | Power Set | $2^A$ | Right |
| 3 | Intersection | $A \cap B$ | Left |
| 2 | Symmetric Difference | $A \Delta B$ | Left |
| 1 | Union, Difference | $A \cup B$, $A \setminus B$ | Left |

Table 11.1: Operator Precedence Table

Again, the precedences have to be encoded into the grammar. Note that the parser has to be an LALR parser: if you use the default EARLEY parser, an ambiguous grammar will not be rejected but will produce ambiguous parse trees instead.

## 11.4 Check your Understanding

(a) What do you have to do in order to see shift-reduce conflicts when using Lark?

(b) How does Lark resolve a shift-reduce conflict? How does it deal with a reduce-reduce conflict?

(c) What is the effect of the keyword argument `strict=True`?

(d) Are you able to inspect the Lalr states that Lark has computed? Which keyword argument is needed for this?

(e) Why is the string "`1*2+3`" parsed as "`1*(2+3)`" if the grammar does not specify the precedences of the arithmetical operators?

(f) Given the precedences and associativities of a set of operators, can you write down a grammar that is an Lalr(1) grammar and that respects these precedences and associativities?

(g) How does the grammar rule for an operator of a given precedence level depend on whether this operator is left associative, right associative, or non-associative?

(h) What is the purpose of the question mark in a rule of the form "`?expr : ...`" and what is the purpose of an alias that is introduced with the operator "`->`"?

Can these features introduce or remove a conflict?

(i) Which three kinds of conflicts have we distinguished according to their origin?

(j) How is the concept of a look-ahead conflict defined?

(k) How is the concept of a mysterious reduce-reduce conflict defined?

# Chapter 12

# Assembler

Figure 12.1: Jasmin.

A compiler translates programs written in a high level language like C or *Java* into some low level representation. This low level representation can be either machine code or some form of assembler code. For the programming language *Java*, the command `javac` compiles a program written in *Java* into *Java* byte code. This byte code is then executed using the *Java* virtual machine (JVM).

The compiler that we are going to develop in the next chapter generates a particular form of assembler code known as JVM assembler code. This assembler code can be translated directly into *Java* byte code, which is also the byte code generated by the program `javac`. The program for translating JVM assembler into bytecode is called *Jasmin*. You can download *Jasmin* at

http://sourceforge.net/projects/jasmin.

*Jasmin* takes an assembler file as input and outputs *Java* byte code. The byte code produced by *Jasmin*

can be executed using the command `java` just like any other "`.class`"-file. This chapter will discuss the syntax and semantics of *Jasmin* assembler code. *Jasmin* is discussed in more detail in the book *Java Virtual Machine* [MD97].

By the way, *Java* isn't the only programming language that is translated into *Java* Byte code. Here is a list of some of the better known programming languages that use the Jvm architecture.

1. Kotlin is developed by JetBrains. It is fully interoperable with *Java* and compiles to Jvm bytecode. It is widely used in Android development as a replacement for *Java*.

   As of November 2025, *Kotlin* occupies the $20^{\text{th}}$ place in the Tiobe index.

2. Scala blends functional and object-oriented programming paradigms. Scala compiles to Jvm bytecode and is often used for concurrent and distributed applications.

   Currently, *Scala* occupies the $35^{th}$ place in the Tiobe index.

3. Groovy is a dynamic scripting language that is easy to learn for *Java* programmers because its syntax is close to *Java*. It is often used in scripting, configuration, and in build systems like Gradle.

4. Clojure is a modern Lisp dialect designed for functional programming. It compiles to Jvm bytecode and supports concurrency.

5. JRuby is a Jvm-based version of the Ruby programming language. It is fully compatible with *Ruby* but additionally supports the use of *Java* libraries.

6. Jython is a Jvm-based version of Python version 2.7.

7. Frege is a version of the pure functional language Haskell that compiles to Jvm bytecode.

The main reason these languages compile to the Jvm is to take advantage of *Java*'s rich library ecosystem. Another reason is the fact that compiling a language to the Jvm makes this language available for all those hardware platforms that support *Java*.

## 12.1 Introduction into Jasmin Assembler

To get used to the syntax of *Jasmin* assembler, we start with a small program that prints the string

"`Hello World!`"

on the standard output stream. Figure 12.2 on page 194 shows the program `Hello.jas`. We discuss this program line by line.

1. Line 1 uses the directive "`.class`" to define the name of the class file that is to be produced by the assembler. In this case, the class name is `Hello`. Therefore, *Jasmin* will translate this file into the class file "`Hello.class`".

2. Line 2 uses the directive "`.super`" to specify the super class of the class `Hello`. In our examples, the super class will always be the class `Object`. Since this class resides in the package "`java.lang`", the super class has to be specified as

   `java/lang/Object`.

   Observe that the character "`.`" in the class name "`java.lang.Object`" has to be replaced by the character "`/`". This is true even if a *Windows* operating system is used.

```
1  .class public Hello
2  .super java/lang/Object
3
4  .method public <init>()V
5      aload 0
6      invokenonvirtual java/lang/Object/<init>()V
7      return
8  .end method
9
10 .method public static main([Ljava/lang/String;)V
11     .limit locals 1
12     .limit stack  2
13     getstatic     java/lang/System/out Ljava/io/PrintStream;
14     ldc           "Hello World!"
15     invokevirtual java/io/PrintStream/println(Ljava/lang/String;)V
16     return
17 .end method
```

Figure 12.2: An assembler program to print "`Hello World!`".

3. The lines 4 – 8 initialize the program. This code is always the same and corresponds to a constructor for the class `Hello`. It is equivalent to the following *Java* code:

```
1   public Hello() {
2       super(); // Calls the constructor of java.lang.Object
3   }
```

   As this code is copied verbatim to the beginning of every class file, we will not discuss it further.

4. The lines 10 – 17 defines the method `main` that does the actual work.

   (a) Line 10 uses the directive "`.method`" to declare the name of the method and its signature. The string

   ```
   main([Ljava/lang/String;)V
   ```

   specifies the signature:

   i. The string "`main`" is the name of the method that is defined.
   ii. The character "`[`" specifies that the first argument is an array.
   iii. The character "L" specifies that this array consists of objects.
   iv. The string "`java/lang/String;`" specifies that these objects are objects of the class "`java.lang.String`".
   v. Finally, the character "V" specifies that the return type of the method `main` is "`void`".

   (b) Line 11 uses the directive "`.limit locals`" to specify the number of local variables used by the method `main`. In this case, there is just one local variable. This variable corresponds to the parameter of the method main. The assembler file shown in Figure 12.2 on page 194 corresponds to the *Java* code shown in Figure 12.3 below. The method `main` has one local

variable, which is the parameter `args`. The information on the number of local variables is needed by the *Java* virtual machine in order to allocate memory for these variables on the stack.

```
1  public static void main(String[] args) {
2      System.out.println("Hello World!");
3  }
```

Figure 12.3: Printing `Hello world` in *Java*.

(c) For the purpose of the following discussion, we assume that there exist two types of processors: Those that store the objects they work upon in registers and those that store these objects on a stack residing in main memory. The Java virtual machine is of the second type. Hence, *Jasmin* assembler programs do not refer to registers but rather refer to this stack[1]. Line 12 uses the directive

> "`.limit stack`"

to specify the maximal height of the stack. In this case, the stack is allowed to contain a maximum of two objects. It is easy to see that we indeed do never place more than two objects onto the stack, since the command `getstatic` in line 13 pushes the object

> `java.lang.System.out`

onto the stack. This is an object of class "`java.io.PrintStream`". Then, the command `ldc` in line 14 pushes a reference to the string "`Hello World`" onto the stack. The other instructions do not push anything onto the stack.

(d) Line 15 calls the method `println` of the class "`java.io.PrintStream`". Therefore, we first have to put an object of this class onto the stack. This object is the object `java.lang.System.out` that was pushed on the stack using the method `getstatic` in line 13.

Line 15 also specifies that `println` takes one argument of type `java.lang.String` and that it returns `void`, i.e. nothing.

(e) Line 16 returns from the method `main`.

(f) In line 17 the directive "`.end`" marks the end of the code corresponding to the method `main`.

Before we proceed, let us assume that we are working on a Unix operating system and that there is an executable file called `jasmin` somewhere in our path that contains the following code:

```
#!/bin/bash
java -jar /usr/local/lib/jasmin.jar $@
```

Of course, for this to work the directory `/usr/local/lib/` has to contain the file "`jasmin.jar`". If we were working on a Windows operating system, we would have a file called `jasmin.bat` somewhere in our PATH. This file would contain the following line:

```
java -jar %USERPROFILE%\Software\jasmin-2.4\jasmin.jar %*
```

[1] In reality, all real processors make use of registers. However, it is possible to simulate a stack machine using a real processor and that is what is done in the Java virtual machine.

Of course, for this to work the directory %USERPROFILE%\Software\jasmin-2.4 has to contain the file "jasmin.jar".

In order to execute the assembler program discussed above, we first have to translate the assembler program into a class-file. This is done using the command

```
jasmin Hello.jas
```

Executing this command creates the file "Hello.class". This class file can then be executed just like any class file generated from javac by typing

```
java Hello
```

in the command line, provided the environment variable CLASSPATH contains the current directory, i.e. the CLASSPATH has to contain the directory ".".

We will proceed to discuss the different assembler commands in more detail later. To this end, we first have to discuss some background: One of the design goal of the programming language *Java* was compatibility. The idea was that it should be possible to execute *Java* class files on any computer. Therefore, the *Java* designers decided to create a so called *virtual machine*. A virtual machine is a computer architecture that, instead of being implemented in silicon, is simulated. Programs written in *Java* are first compiled into so called *class files*. These class files correspond to the machine code of the *Java* virtual machine (Jvm). The architecture of the virtual machine is a *stack machine*. A stack machine does not have any registers to store variables. Instead, there is a stack and all variables reside on the stack. Every command takes its arguments from the top of the stack and replaces these arguments with the result of the operation performed by the command. For example, if we want to add two values, then we first have to push both values onto the stack. Next, performing the add operation will pop these values from the stack and then push their sum onto the stack.

## 12.2 Assembler Instructions

We proceed to discuss some of the Jvm instructions. Since there are more than 160 Jvm instructions, we can only discuss a subset of all instructions. We restrict ourselves to those instructions that deal with integers: For example, there is an instruction called iadd that adds two 32 bit integers. There are also instructions like fadd that adds two floating point numbers and dadd that adds two double precision floating point numbers but, since our time is limited, we won't discuss these instructions. Before we are able to discuss the different instructions we have to discuss how the main memory is organized in the Jvm: In the Jvm, the memory is split into four parts:

1. The program memory contains the program code as a sequence of bytes.
2. The operands of the different machine instructions are put onto the stack. Furthermore, the stack contains the arguments and the local variables of a procedure. However, in the context of the Jvm the procedures are called *methods* instead of procedures.

   The register SP (stack pointer) points to the top of the stack. If a method is called, the arguments of the method are placed on the stack. The register LV (local variables) points to the first argument of the current method. On top of the arguments, the local variables of the method are put on the stack. Both the arguments and the local variables can be accessed via the register LV by specifying their offset from the first argument. We will discuss the register LV in more detail when we discuss the invocation of methods.
3. The heap is used for dynamically allocated memory. Newly created objects are located in the heap.

4. The constant pool contains the definitions of constants and also the addresses of methods in program memory.

Figure 12.4 shows the memory structure of the JVM. In the following, we will be mostly concerned with the stack. We proceed to discuss some of the assembler instructions.

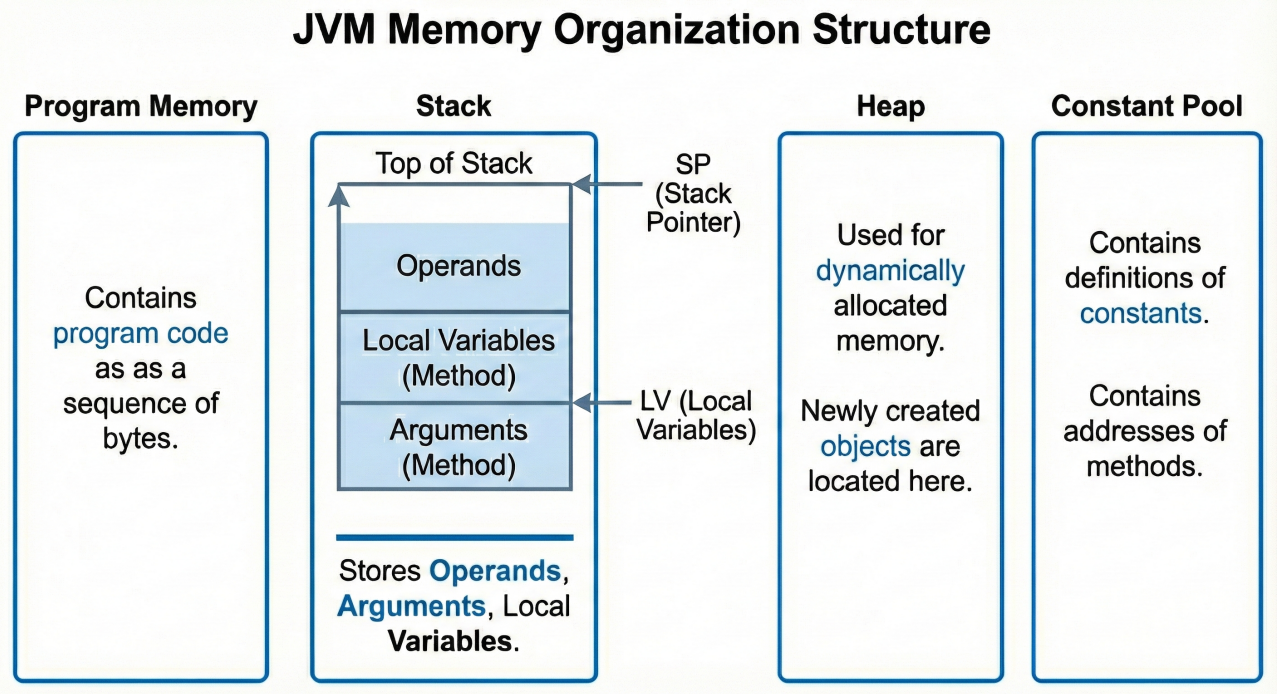



Figure 12.4: The memory structure of the JVM.

**Arithmetical and Logical Instructions**

1. The instruction "`iadd`" adds those values that are on top of the stack and replaces these values by their sum. Figure 12.5 on page 198 show how this command works. The left part of the figure shows the stack as it is before the command `iadd` is executed, while the right part of the figure depicts the situation after the execution of this command. Note that after the command is executed, the stack pointer points to the position where formerly the first argument had been stored.

2. The instruction "`isub`" subtracts the integer value on top of the stack from the value that is found on the position next to the top of the stack. Figure 12.6 on page 198 pictures this command. The left part of the figure shows the stack as it is before the command `isub` is executed, while the right part of the figure depicts the situation after the execution of this command. Note that after the command is executed, the stack pointer points to the position where formerly the first argument had been stored.

3. The instruction "`imul`" multiplies the two integer values which are on top of the stack. This instruction works similar to the instruction `iadd`. If the product does not fit in 32 bits, only the lowest 32 bits of the result are written onto the stack.

4. The instruction "`idiv`" divides the integer value that is found on the position next to the top of the stack by the value on top of the stack.

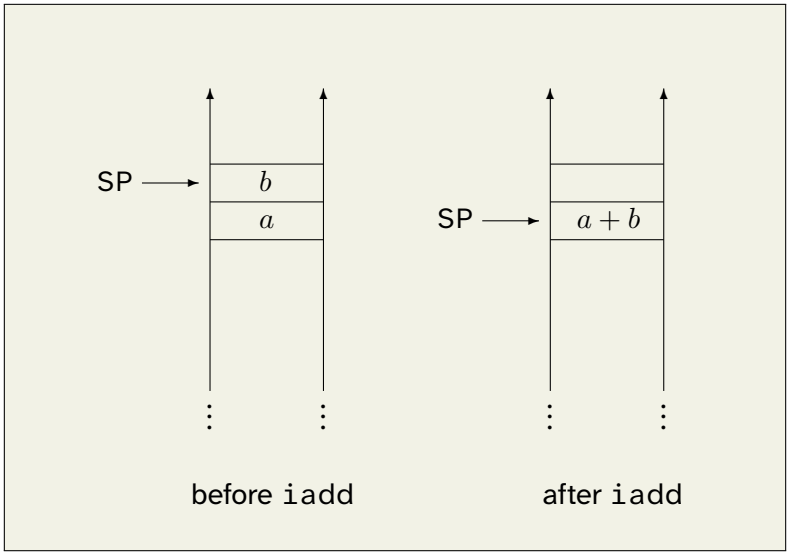



Figure 12.5: The effect of `iadd`.

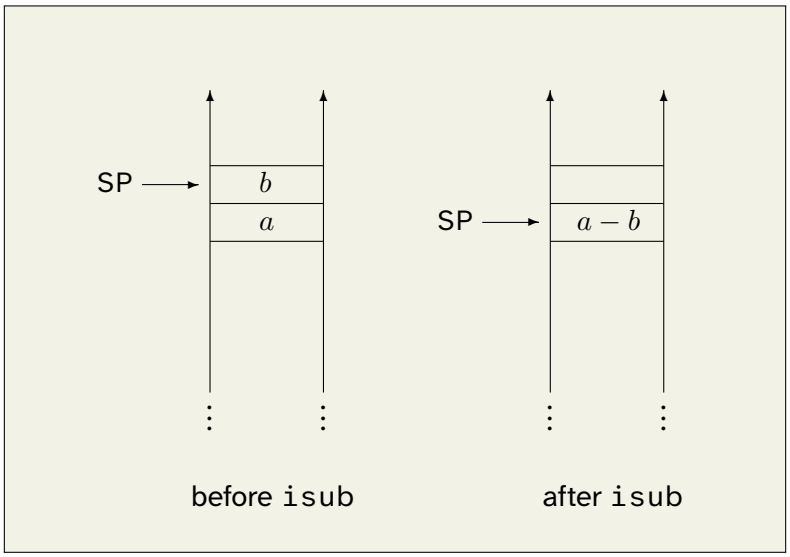



Figure 12.6: The effect of `isub`.

**Note**: Integer division in *Java* is different from integer division in *Python*. In *Python*, integer division is always rounded down (i.e. rounded to negative infinity), while integer division in *Java* rounds towards $0$. While there is no difference for natural numbers, we have $-5 \,/\, 2 = -2$ in *Java*, while we have $-5 \,//\, 2 = -3$ in *Python*.[2].

5. The instruction "`irem`" computes the remainder $a \,\%\, b$ of the division of $a$ by $b$ where $a$ and $b$ are integer values found on top of the stack.

   Again, it is important to note that the *Java* implementation of $a \,\%\, b$ differs from the implementation in *Python*. For example, in *Java* we have $-5 \,\%\, 3 = -2$, while we have $-5 \,\%\, 3 = 1$ in *Python*.

6. The instruction "`iand`" computes the bitwise "and" of the values on top of the stack.

[2] In mathematics, integer division, which is also known as Euclidean division is defined in the same was as it is implemented in *Python*. The different implementation of this operator in *Java* is mostly a reflection of the poor educational standards of the United States of America.

7. The instruction "`ior`" computes the bitwise "or" of the integer values that are on top of the stack.

8. The instruction "`ixor`" computes the bitwise "*exclusive or*" of the integer values that are on top of the stack.

**Shift Instructions**

1. The instruction "`ishl`" shifts the value $a$ to the left by $b$`[4:0]` bits. Here, $a$ and $b$ are assumed to be the two values on top of the stack: $b$ is the value on top of the stack and $a$ is the value below $b$. $b$`[4:0]` denotes the natural number that results from the 5 lowest bits of $b$. Figure 12.7 on page 199 pictures this command.

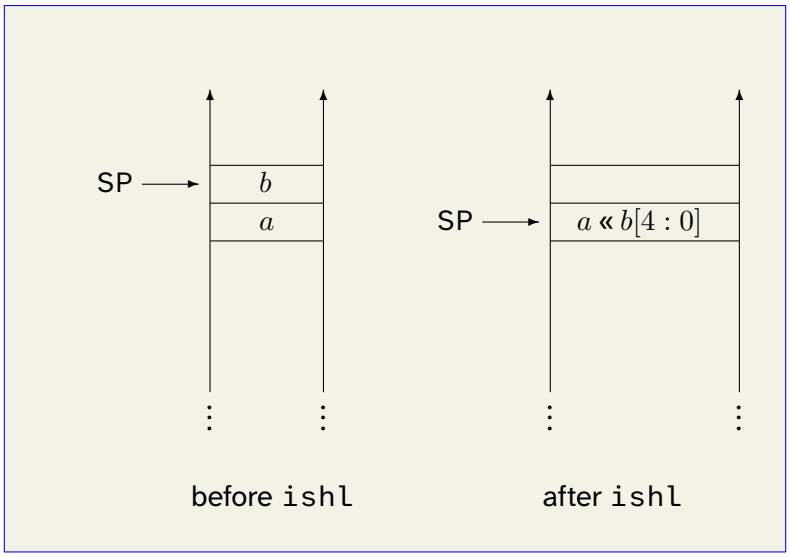



Figure 12.7: The effect of `ishl`.

2. The instruction "`ishr`" shifts the value $a$ to the right by $b$`[4:0]` bits. Here, $a$ and $b$ are assumed to be the two values on top of the stack: $b$ is the value on top of the stack and $a$ is the value below $b$. $b$`[4:0]` denotes the natural number that results from the 5 lowest bits of $b$. Note that this instruction performs an *arithmetic shift*, i.e. the sign bit is preserved.

### 12.2.1 Instructions to Manipulate the Stack

1. The instruction "`dup`" duplicates the value that is on top of the stack. Figure 12.8 on page 200 pictures this command.

2. The instruction "`pop`" removes the value that is on top of the stack. Figure 12.9 on page 200 pictures this command. The value is not actually erased from memory, only the stack pointer is decremented. The next instruction that puts a new value onto the stack will therefore overwrite the old value.

3. The instruction "`nop`" does nothing. The name is short for "`no operation`".

4. The instruction "`bipush` $b$" pushes the byte $b$ that is given as argument onto the stack. The argument $b$ is constricted to a byte in order to limit the number of bytes that are needed to represent this instruction in memory. Figure 12.10 on page 201 pictures this command.

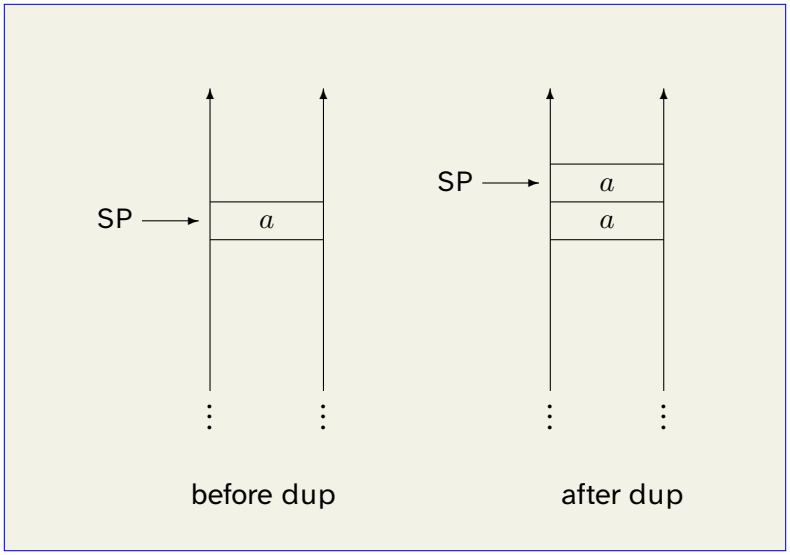



Figure 12.8: The effect of dup.

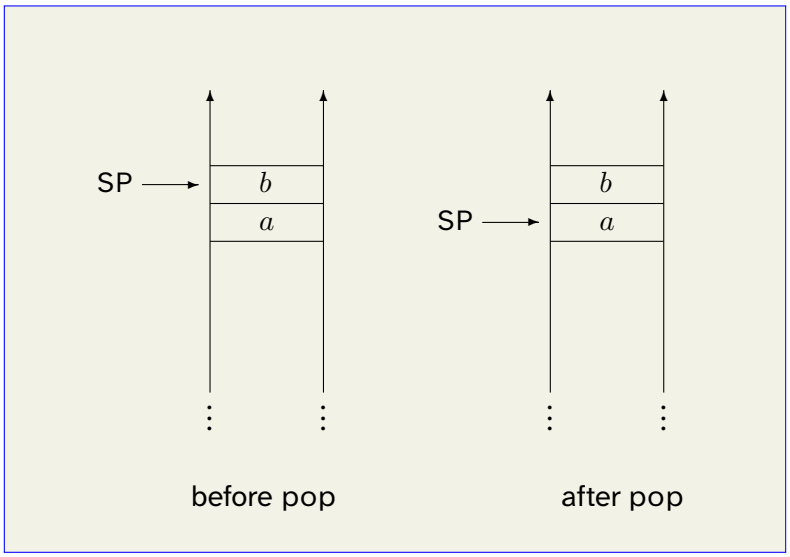



Figure 12.9: The effect of pop.

5. The instruction "`getstatic` $v$ $c$" takes two parameters: $v$ is the name of a static variable and $c$ is the type of this variable. For example,

   ```
   getstatic java/lang/System/out Ljava/io/PrintStream;
   ```

   pushes a reference to the `PrintStream` that is known as

   ```
   java.lang.System.out
   ```

   onto the stack.

6. The instruction "`iload` $v$" reads the local variable with index $v$ and pushes it on top of the stack. Figure 12.11 on page 201 pictures this command. Note that LV denotes the register pointing to the beginning of the local variables of a method.

7. The instruction "`istore` $v$" removes the value which is on top of the stack and stores this value at the location for the local variable with number $v$. Hence, $v$ is interpreted as an index into the

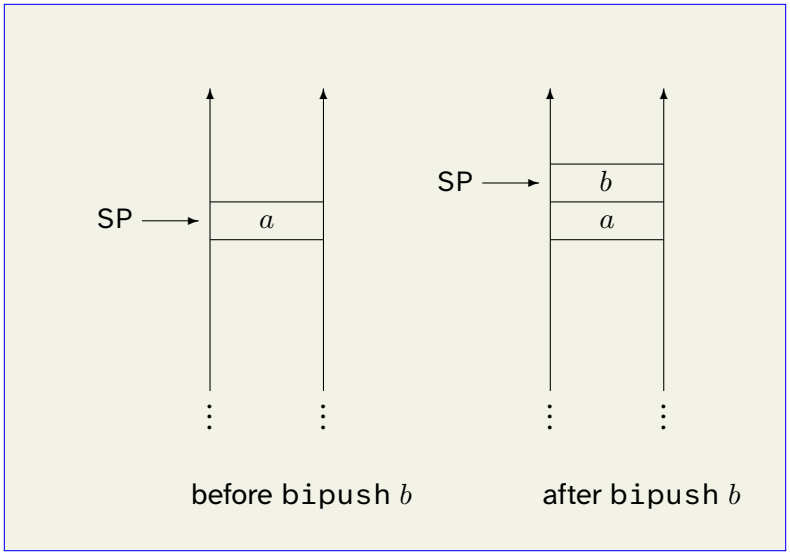



Figure 12.10: The effect of `bipush` $b$.

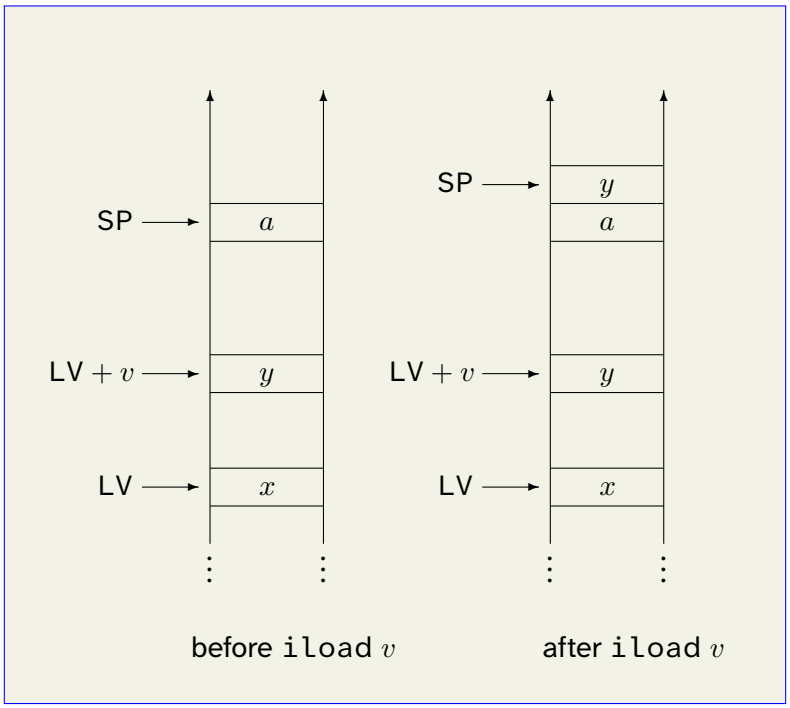



Figure 12.11: The effect of `iload` $v$.

local variable table of the method. Figure 12.12 on page 202 pictures this command.

8. The instruction "`ldc` $c$" pushes the constant $c$ onto the stack. This constant can be an integer, a single precision floating point number, or a (pointer to) a string. If $c$ is a string, this string is actually stored in the so called *constant pool* and in this case the command "`ldc` $c$" will only push a pointer to the string onto the stack.

### Branching Commands

In this subsection we discuss those commands that change the control flow.

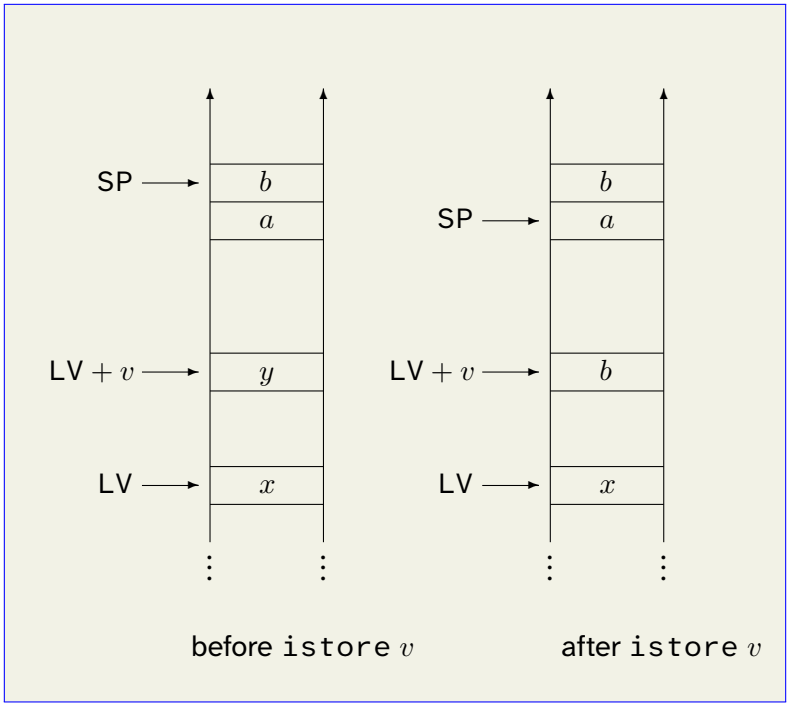



Figure 12.12: The effect of `istore` $v$.

1. The instruction "`goto` $l$" jumps to the label $l$. Here the label $l$ is a label name that has to be declared inside the method containing this `goto` command. A label with name *target* is declared using the syntax

   *target* :

   The next section presents an example assembler program that demonstrates this command.

2. The instruction "`if_icmpeq` $l$" checks whether the value on top of the stack is the same as the value preceding it. If this is the case, the program will branch to the label $l$. Otherwise, the control flow is not changed. Observe that both values that are compared are removed from the stack.

3. The instruction "`if_icmpne` $l$" checks whether the value on top of the stack is different from the value preceding it. If this is the case, the program will branch to the label $l$. Otherwise, the control flow is not changed. Observe that both values that are compared are removed from the stack.

4. The instruction "`if_icmplt` $l$" checks whether the value that is below the top of the stack is less than the value on top of the stack. If this is the case, the program will branch to the label $l$. Otherwise, the control flow is not changed. Observe that both values that are compared are removed from the stack.

5. The instruction "`if_icmple` $l$" checks whether the value that is below the top of the stack is less than or equal to the value on top of the stack. If this is the case, the program will branch to the label $l$. Otherwise, the control flow is not changed. Observe that both values that are compared are removed from the stack.

   There are similar commands called `if_icmpgt` and `if_icmpge`.

6. The instruction "`ifeq` $l$" checks whether the value on top of the stack is zero. If this is the case,

the program will branch to the label $l$. Otherwise, the control flow is not changed. Observe that the value that is tested is removed from the stack.

7. The instruction "`iflt` $l$" checks whether the value on top of the stack is less than zero. If this is the case, the program will branch to the label $l$. Otherwise, the control flow is not changed. Observe that the value that is tested is removed from the stack.

8. The instruction "`invokevirtual` $m$" is used to call the method $m$. Here $m$ has to specify the full name of the method. For example, in order to invoke the method `println` of the class `java.io.PrintStream` we have to write:

   ```
   invokevirtual java/io/PrintStream/println(I)V
   ```

   Before the command `invokevirtual` is executed, we have to put the arguments of the method onto the stack. For example, if we want to invoke the method `println` that takes an integer argument, we first have to push an object of type `PrintStream` onto the stack. Furthermore, we need to push an integer onto the stack.

9. The instruction "`invokestatic` $m$" is used to call the method $m$. Here $m$ has to specify the full name of the method. Furthermore, $m$ needs to be a static method. For example, in order to invoke a method called `sum` that resides in the class `Sum` and that takes one integer argument we have to write:

   ```
   invokestatic Sum/sum(I)I
   ```

   In the type specification "`sum(I)I`" the first "`I`" specifies that `sum` takes one integer argument, while the second "`I`" specifies that the method `sum` returns an integer.

   Before the command `invokestatic` is executed, we have to put the arguments of the method onto the stack. For example, if we want to invoke the method `sum` described above, then we have to push an integer $n$ onto the stack. After the method returns, this integer is replaced with the return value $\texttt{sum}(n)$.

10. The instruction "`ireturn`" returns from the method that is currently invoked. This method also returns a value to the calling procedure. In order for `ireturn` to be able to return a value $v$, this value $v$ has to be pushed onto the stack before the command `ireturn` is executed.

    In general, if a method taking $n$ arguments $a_1, \cdots, a_n$ is to be called, then first the arguments $a_1, \cdots, a_n$ have to be pushed onto the stack. When the method $m$ is done and has computed its result $r$, the arguments $a_1, \cdots, a_n$ will have been replaced with the single value $r$.

11. The instruction "`return`" returns from the method that is currently invoked. However, in contrast to the command `ireturn`, this command is used if the method that has been invoked does not return a result.

## 12.3 An Example Program

Figure 12.13 shows the C program `sum.c` that computes the sum

$$\sum_{i=1}^{6^2} i.$$

The function $\texttt{sum}(n)$ computes the sum $\sum_{i=1}^{n} i$ and the function `main` calls this function with the argument $6 \cdot 6$. Figure 12.14 on page 205 shows how this program can be translated into the assembler program `Sum.jas`. We discuss the implementation of this program next.

```
1  #import "stdio.h"
2
3  int sum(int n) {
4      int s;
5      s = 0;
6      while (n != 0) {
7          s = s + n;
8          n = n - 1;
9      };
10     return s;
11 }
12 int main() {
13     printf("%d\n", sum(6*6));
14     return 0;
15 }
```

Figure 12.13: A C function to compute the sum $\sum_{i=1}^{36} i$.

1. Line 1 specifies the name of the generated class which is to be `Sum`.
2. Line 2 specifies that the class `Sum` is a subclass of the class `java.lang.Object`.
3. Lines 4 – 6 initialize the class. The code used here is the same as in the example printing "`Hello World!`".
4. Line 8 declares the method `main`. This method takes an argument which is an array of strings. It does not return a value.
5. Line 9 specifies that there is just one local variable.
6. Line 10 specifies that the stack will contain at most 3 temporary values.
7. Line 11 pushes the object `java.lang.out` onto the stack. We need this object later in order to invoke `println`.
8. Line 12 pushes the number 6 onto the stack.
9. Line 13 duplicates the value 6. Therefore, after line 13 is executed, the stack contains three elements: The object `java.lang.out`, the number 6, and again the number 6.
10. Line 14 multiplies the two values on top of the stack and replaces them with their product, which happens to be 36.
11. Line 15 calls the method `sum` defined below. After this call has finished, the number 36 on top of the stack is replaced with the value $\texttt{sum}(36)$.
12. Line 16 prints the value that is on top of the stack. This is possible, because the object `java.lang.out` is directly below the value $\texttt{sum}(36)$.
13. Line 20 declares the method sum. This method takes one integer argument and returns an integer as result.

```
1   .class public Sum
2   .super java/lang/Object
3
4   .method public <init>()V
5       ...
6   .end method
7
8   .method public static main([Ljava/lang/String;)V
9       .limit locals 1
10      .limit stack  3
11      getstatic     java/lang/System/out Ljava/io/PrintStream;
12      ldc           6
13      dup
14      imul
15      invokestatic  Sum/sum(I)I
16      invokevirtual java/io/PrintStream/println(I)V
17      return
18  .end method
19
20  .method public static sum(I)I
21      .limit locals 2
22      .limit stack  2
23      ldc    0
24      istore 1                    ; s = 0;
25  loop:
26      iload  0                    ; n
27      ifeq   finish               ; if (n == 0) goto finish;
28      iload  1                    ; s
29      iload  0                    ; n
30      iadd
31      istore  1                   ; s = s + n;
32      iload   0                   ; n
33      ldc     1
34      isub
35      istore  0                   ; n = n - 1;
36      goto    loop
37  finish:
38      iload   1
39      ireturn                     ; return s;
40  .end method
```

Figure 12.14: An assembler program to compute the sum $\sum_{i=1}^{36} i$.

14. Line 21 specifies that the method `sum` has two local variables: The first local variable is the parameter $n$ and the second local variable corresponds to the variable $s$ in the C program.

15. The effect of lines 23 and 24 is to initialize this variable $s$ with the value $0$.

16. Line 26 pushes the local variable $n$ on the stack so that line 27 is able to test whether $n$ is already $0$. If $n = 0$, the program branches to the label `finish` in line 37, pushes the result $s$ onto the stack (line 38) and returns. If $n$ is not yet $0$, the execution proceeds to line 28.

17. Line 28 and line 29 push the sum $s$ and the variable $n$ onto the stack. These values are then added in line 30 and the result is written to the local variable $s$ in line 31. The combined effect of these instructions is therefore to perform the assignment

```
s = s + n;
```

18. The instructions in line 32 up to line 35 implement the assignment

```
n = n - 1;
```

19. In line 36 we jump back to the beginning of the loop and test whether $n$ is zero.

20. The declaration in line 40 terminates the definition of the method `sum`.

**Exercise 30**:

(a) Implement an assembler program that computes the function `factorial`, which takes a natural number $n$ as input and returns $n!$. Test your program by printing $n!$ for $n = 1, \cdots, 10$.

(b) Implement the function `factorial` in a recursive manner.

**Exercise 31**: Create a Jasmin assembler program that calculates the $n$-th number in the Fibonacci sequence. This exercise tests your ability to manage the stack during recursive calls or iterative state tracking.

The Fibonacci sequence is defined recursively as follows:

$$F(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

### Hint

If you choose a recursive approach, remember that calculating $F(n-1) + F(n-2)$ requires storing the result of the first call on the stack while the second call is being processed. Pay close attention to your `.limit stack` directive!

## 12.4 Disassembler*

Sometimes it is useful to transform a file consisting of *Java* byte code back into something that looks like assembler code. After all, a *Java* class file is a binary file and can therefore only be viewed via commands like od that produce an *octal* or *hexadecimal dump* of the given file. The command `javap` is a *disassembler*, i.e. it takes a *Java* byte code file and transforms it in something that looks similar to *Jasmin* assembler. For example, Figure 12.15 shows the class *Java* class `Sum` to compute the sum

$$\sum_{i=1}^{36} i$$

written in *Java*. If this program is stored in a file called "`Sum.java`", we can compile it via the following command:

```
javac Sum.java
```

This will produce a class file with the name "`Sum.class`" containing the byte code.

```
1  public class Sum {
2      public static void main(String[] args) {
3          System.out.println(sum(6 * 6));
4      }
5
6      static int sum(int n) {
7          int s = 0;
8          for (int i = 0; i <= n; ++i) {
9              s += i;
10         }
11         return s;
12     }
13 }
```

Figure 12.15: A *Java* program computing $\sum_{i=0}^{6^2} i$.

Next, in order to *decompile* the "`.class`" file, we run the command

```
javap -c Sum.class
```

The output of this command is shown in Figure 12.16. We see that the syntax used by `javap` differs a bit from the syntax used by *Jasmin*. It is rather unfortunate that the company developing *Java* has decided not to make their assembler public. Still, we can see that the output produced by `javap` is quite similar to the input accepted by `jasmin`.

```
Compiled from "Sum.java"
public class Sum {
  public Sum();
    Code:
      0: aload_0
      1: invokespecial #1 // Method java/lang/Object."<init>":()V
      4: return

  public static void main(java.lang.String[]);
    Code:
      0: getstatic     #2 // Field java/lang/System.out:
                          //       Ljava/io/PrintStream;
      3: bipush        36
      5: invokestatic  #3 // Method sum:(I)I
      8: invokevirtual #4 // Method java/io/PrintStream.println:(I)V
     11: return

  static int sum(int);
    Code:
      0: iconst_0
      1: istore_1
      2: iconst_0
      3: istore_2
      4: iload_2
      5: iload_0
      6: if_icmpgt     19
      9: iload_1
     10: iload_2
     11: iadd
     12: istore_1
     13: iinc          2, 1
     16: goto          4
     19: iload_1
     20: ireturn
}
```

Figure 12.16: The output of “`javap -c Sum.class`”.

# Chapter 13

# Development of a Simple Compiler

Figure 13.1: A medieval compiler.

In this chapter, we construct a compiler that translates a fragment of the C language into *Java* assembler. The fragment of the C language translated by the compiler is referred to as *Integer*-C, because only the data type `int` is available. Although it would be feasible to support additional data types, the extra

effort would no longer be proportionate to the educational benefit of the example.

A compiler essentially consists of the following components:

1. The scanner reads the file to be translated and breaks it down into a sequence of tokens.
2. The parser reads the sequence of tokens and produces an abstract syntax tree as a result.

   We will generate both the scanner and the parser with Lark from a single grammar.
3. The type-checker checks the abstract syntax tree for type errors. Since the language *Integer*-C contains only a single data type, this phase is not needed for our compiler.
4. The code generator then translates the parse tree into a sequence of assembler commands that conform to the syntax of *Jasmin*.
5. This can be followed by an *optimization phase*. Due to time constraints we cannot cover this topic.
6. The resulting assembler program can be translated into Java bytecode using *Jasmin*.
7. The Java bytecode is then executed by the Jvm, i.e. the *Java* virtual machine.

In compilers whose target code is a Risc assembler program, a code similar to Jvm code is usually generated first. Such a code is referred to as *intermediate code*. It then remains the task of a so-called *backend* to generate an assembler program for a given processor architecture. The most challenging task here is to find a register allocation for the variables, such that as many variables as possible can be held in registers.

## 13.1 The Programming Language *Integer*-C

We now introduce the language *Integer*-C, which our compiler is intended to translate. In this context, we also refer to it as the *source language* of our compiler. Figure 13.2 on page 211 shows the grammar of the source language in Extended Backus-Naur Form (Ebnf). The grammar for *Integer*-C uses the following terminals:

1. Id is an abbreviation for Identifier and represents a sequence of digits, letters, and underscores that begins with a letter. An Id designates either a variable or the name of a function.
2. Number stands for a sequence of digits interpreted as a decimal number.
3. In addition, there are a number of operators such as `"+"`, `"-"`, etc., as well as keywords like `"if"`, `"else"`, and `"while"`.

According to the grammar provided above, a program is a list of functions. A function consists of the declaration of its signature, followed by a list of declarations (*decl*) and commands (*stmt*) enclosed in curly braces. The grammar shown in Figure 13.2 is ambiguous:

$$
\begin{array}{rcl}
\textit{program} & \rightarrow & \textit{function}+ \\
\textit{function} & \rightarrow & \texttt{“int”}\ \textsc{Id}\ \texttt{“(”}\ \textit{paramList}\ \texttt{“)”}\ \texttt{“\{”}\ \textit{decl}+\ \textit{stmnt}+\ \texttt{“\}”} \\
\textit{paramList} & \rightarrow & (\texttt{“int”}\ \textsc{Id}\ (\texttt{“,”}\ \texttt{“int”}\ \textsc{Id})^*)^? \\
\textit{decl} & \rightarrow & \texttt{“int”}\ \textsc{Id}\ \texttt{“;”} \\
\textit{stmnt} & \rightarrow & \texttt{“\{”}\ \textit{stmnt}^*\ \texttt{“\}”} \\
 & | & \textsc{Id}\ \texttt{“=”}\ \textit{expr}\ \texttt{“;”} \\
 & | & \texttt{“if”}\ \texttt{“(”}\ \textit{boolExpr}\ \texttt{“)”}\ \textit{stmnt} \\
 & | & \texttt{“if”}\ \texttt{“(”}\ \textit{boolExpr}\ \texttt{“)”}\ \textit{stmnt}\ \texttt{“else”}\ \textit{stmnt} \\
 & | & \texttt{“while”}\ \texttt{“(”}\ \textit{boolExpr}\ \texttt{“)”}\ \textit{stmnt} \\
 & | & \texttt{“return”}\ \textit{expr}\ \texttt{“;”} \\
 & | & \textit{expr}\ \texttt{“;”} \\
\textit{boolExpr} & \rightarrow & \textit{boolExpr}\ (\texttt{“\&\&”} \mid \texttt{“||”})\ \textit{boolExpr} \\
 & | & \texttt{“!”}\ \textit{boolExpr} \\
 & | & \texttt{“(”}\ \textit{boolExpr}\ \texttt{“)”} \\
 & | & \textit{expr}\ (\texttt{“==”} \mid \texttt{“!=”} \mid \texttt{“<=”} \mid \texttt{“>=”} \mid \texttt{“<”} \mid \texttt{“>”})\ \textit{expr} \\
\textit{expr} & \rightarrow & \textit{expr}\ (\texttt{“+”} \mid \texttt{“-”} \mid \texttt{“*”} \mid \texttt{“/”} \mid \texttt{“\%”})\ \textit{expr} \\
 & | & \texttt{“(”}\ \textit{expr}\ \texttt{“)”} \\
 & | & \textsc{Number} \\
 & | & \textsc{Id}\ (\texttt{“(”}(\textit{expr}\ (\texttt{“,”}\ \textit{expr})^*)^?\ \texttt{“)”})^?
\end{array}
$$

Figure 13.2: An EBNF grammar for *Integer*-C

1. The grammar has the Dangling-Else Problem. We will see that the LALR parser generated by LARK resolves this problem in the way we want.
2. The rules for arithmetic and Boolean expressions do not specify the precedences and associativities of the operators. We will encode this information into the grammar by stratifying it, i.e. by introducing one syntactical variable for every precedence level.

Figure 13.3 shows a simple INTEGER-C program. The function *sum*$(n)$ calculates the sum

$$\sum_{i=1}^{n} i$$

and the function *main*$()$ calls the function `sum` with the argument $6 \cdot 6$. The function `println` used in the program outputs its argument followed by a newline. We assume that this function is predefined.

## 13.2 Developing the Scanner and the Parser

We construct both the scanner and the parser using LARK. In contrast to parser generators like PLY, which separate the specification of the scanner from the specification of the parser, LARK reads a *single*

```
1   int sum(int n) {
2       int s;
3       s = 0;
4       while (n != 0) {
5           s = s + n;
6           n = n - 1;
7       }
8       return s;
9   }
10  int main() {
11      int n;
12      n = 6 * 6;
13      println(sum(n));
14  }
```

Figure 13.3: A simple Integer-C program to compute the sum $\sum_{i=1}^{6\cdot 6} i$.

grammar that specifies both the tokens and the grammar rules. Recall that the names of terminals are written in upper case, while the names of syntactical variables are written in lower case. We build this grammar as a string: The part of the grammar that describes the scanner is collected in the variable `terminal_definitions`, while the grammar rules are collected in the variable `syntax`. At the end, both strings are concatenated. The notebook

`Python/Chapter-13/Compiler.ipynb`

contains the complete compiler. In this notebook, the strings `terminal_definitions` and `syntax` are built step by step. For the presentation in these lecture notes, we have combined these steps. Figure 13.4 on page 213 shows the specification of the scanner.

1. Line 5 defines the token NUMBER, which specifies a natural number. In Lark, a regular expression is enclosed in forward slashes.

2. Line 7–12 define the tokens for those operators that consist of more than one character. Strictly speaking, these definitions are not necessary: A string that occurs literally inside a grammar rule, for example "==", is automatically turned into an anonymous terminal. However, the name that Lark generates for an anonymous terminal built from a symbol like "==" has the form __ANON_1, and such a name is of little help when we inspect the token stream or read an error message. If a terminal with the same definition has been declared explicitly, Lark uses this terminal instead of creating an anonymous one. Hence, these declarations only serve to give the tokens readable names. Note that a terminal that is defined by a string needs no escaping: We can simply write "||". Furthermore, Lark prefers longer strings, so "!=" is preferred over "!" and "<=" is preferred over "<".

   The operators that consist of a single character need no declaration at all: Lark gives the corresponding anonymous terminals names like PLUS, LPAR, or SEMICOLON.

3. Our version of C supports both single-line comments and multi-line comments. These are defined by the token COMMENT in line 14 and 15, which shows that a terminal can be defined as an alternative of several regular expressions.

```
1  from lark import Lark, Transformer, v_args
2  from lark.exceptions import UnexpectedInput
3
4  terminal_definitions = r"""
5  NUMBER: /0|[1-9][0-9]*/
6
7  EQ : "=="
8  NE : "!="
9  LE : "<="
10 GE : ">="
11 AND: "&&"
12 OR : "||"
13
14 COMMENT: /\/\*(.|\n)*?\*\//
15         | /\/\/[^\n]*/
16 %ignore COMMENT
17
18 NAME: /[a-zA-Z][a-zA-Z0-9_]*/
19
20 %ignore /[ \t\r\n]+/
21 """
```

Figure 13.4: The scanner for *Integer*-C.

(a) Multi-line comments start with the string “`/*`” and end with the string “`*/`”. Note that we have to use the *non-greedy* quantifier “`*?`”. Otherwise, a line like

```
/* blah */ a = 1; /* blub */
```

would be recognized as a single comment.

(b) Single-line comments start with the string “`//`” and extend to the end of the line.

As the forward slash is the delimiter of a regular expression in Lark, every occurrence of the character “`/`” inside a regular expression has to be escaped as “`\/`”. The directive `%ignore` in line 16 tells the scanner to discard all comments, so that they never show up in the token stream.

4. The most interesting aspect is the implementation of keywords like “`while`” or “`return`”. Syntactically, all keywords are identifiers, i.e. they are also matched by the regular expression defining the token NAME in line 18. In the grammar of Figure 13.2, this token has been called Id. Lark deals with keywords automatically: If a string terminal like “`if`” is also matched by the regular expression of another terminal, then Lark first scans a NAME and afterwards checks whether the resulting string is one of the keywords. If it is, the token type is changed accordingly. Because of this, the string `iffy` is scanned as a single NAME and not as the keyword `if` followed by the identifier `fy`. Therefore, the keywords do not need to be declared as tokens. They are simply written as strings inside the grammar rules.

5. Finally, line 20 specifies that white space is ignored. Lark keeps track of line numbers and column numbers on its own: every token $t$ provides the attributes `t.line` and `t.column`, which are used in error messages.

Now we are ready to present the grammar rules. As Lark supports Ebnf, the grammar shown in

```
1  syntax = r"""
2  program: function+
3
4  function: "int" NAME "(" parameters ")" "{" declarations statements "}"
5
6  parameters  : ("int" NAME ("," "int" NAME)*)?
7
8  declarations: ("int" NAME ";")*
9
10 statements  : statement*
11
12 statement: "{" statements "}"                                -> block
13          | NAME "=" expr ";"                                 -> assign
14          | "if" "(" bool_expr ")" statement                  -> if_stmnt
15          | "if" "(" bool_expr ")" statement "else" statement -> if_else_stmnt
16          | "while" "(" bool_expr ")" statement               -> while_stmnt
17          | "return" expr ";"                                 -> return_stmnt
18          | expr ";"                                          -> expr_stmnt
19 """
```

Figure 13.5: Grammar for Integer-C, part 1: programs, functions, and statements.

Figure 13.2 can be used nearly unchanged. There are only two things we have to take care of: First, the rules for arithmetic and Boolean expressions have to be stratified. Second, we have to attach names to those alternatives of a rule that we want to distinguish when we build the abstract syntax tree. Figure 13.5 on page 214 shows the first part of the grammar.

1. Line 2 specifies that a `program` is a non-empty list of function definitions.

2. A `function` definition starts with the keyword "`int`" that is followed by the name of the function, an opening parenthesis, a list of the parameters of the function, a closing parenthesis, an opening brace, a list of declarations, a list of statements, and finally a closing brace. The strings "`int`", "`(`", "`)`", "`{`", and "`}`" are anonymous terminals. Lark removes anonymous string terminals from the parse tree. Therefore, the node of the parse tree that represents a function has exactly four children: the name of the function, its parameters, its declarations, and its statements.

   The initial version of the programming language C, as detailed in the book "*The C Programming Language*" [KR78], required that all variable declarations precede any statements, prohibiting the mixing of declarations and statements. While this limitation was lifted in subsequent versions of the language, our implementation still enforces this restriction.

3. The syntactical variables `parameters`, `declarations`, and `statements` defined in line 6, 8, and 10 describe lists. Using the Ebnf operators "`*`" and "`?`", one rule per list suffices. Note that a parameter list may be empty and that different parameters are separated by commas, while every declaration is terminated by a semicolon. The parse trees of `parameters` and `declarations` contain only the tokens of type NAME, since all other tokens occurring in these rules are anonymous terminals.

4. Line 12–18 define the syntactical variable `statement`. The names given after the operator "`->`" are the names of the methods that later build the abstract syntax tree for the corresponding alternative.

The two alternatives `if_stmnt` and `if_else_stmnt` cause the dangling else problem: When the parser has read a statement of the form

`if (` *bool_expr* `)` *statement*

and the next token is the keyword "`else`", then the parser can either *reduce* this statement or it can *shift* the token "`else`". Lark resolves a shift-reduce conflict in favour of shifting. This is exactly what we want: the keyword "`else`" is attached to the innermost "`if`" that does not yet have an "`else`" part. Hence, a statement of the form

```
if (a == 1) if (b == 1) x = 2; else x = 3;
```

is interpreted as

```
if (a == 1) { if (b == 1) x = 2; else x = 3; }.
```

```
syntax += r"""
?bool_expr: bool_expr "||" conjunction -> or_expr
          | conjunction

?conjunction: conjunction "&&" negation -> and_expr
            | negation

?negation: "!" negation -> not_expr
         | bool_atom

?bool_atom: expr "==" expr -> equal
          | expr "!=" expr -> unequal
          | expr "<=" expr -> less_or_equal
          | expr ">=" expr -> greater_or_equal
          | expr "<"  expr -> less
          | expr ">"  expr -> greater
          | "(" bool_expr ")"

?expr: expr "+" product -> add
     | expr "-" product -> subtract
     | product

?product: product "*" factor -> multiply
        | product "/" factor -> divide
        | product "%" factor -> modulo
        | factor

?factor: NUMBER                  -> number
       | NAME                    -> variable
       | NAME "(" arguments ")" -> function_call
       | "(" expr ")"

arguments: (expr ("," expr)*)?
"""
```

Figure 13.6: Grammar for Integer-C, part 2: Boolean and arithmetic expressions.

Figure 13.6 on page 215 shows the grammar rules for Boolean and arithmetic expressions. The Lalr

parser of Lark does not support precedence declarations. Therefore, we encode the precedences and associativities of the operators into the grammar by introducing one syntactical variable per precedence level, as discussed in Section 11.2.

1. Line 2–17 define Boolean expressions:
   (a) A `bool_expr` is a list of `conjunction`s that are separated by the operator "||", which represents logical or.
   (b) A `conjunction` is a list of `negation`s that are separated by the operator "&&", which represents logical and. Hence, "&&" binds stronger than "||".
   (c) A `negation` is a `bool_atom` that is preceded by an arbitrary number of negation operators "!".
   (d) A `bool_atom` is either a comparison of two arithmetic expressions or a Boolean expression in parentheses.

   As the operators "||" and "&&" are left associative, the corresponding rules are left recursive: the syntactical variable on the left hand side of the rule occurs as the first symbol on the right hand side. The operator "!" is right associative, since we want to interpret an expression of the form `!!`$a$ as `!(!`$a$`)`. Hence, the rule for `not_expr` is right recursive. The comparison operators "==", "!=", "<=", ">=", "<", and ">" are non-associative, since our programming language disallows the chaining of comparison operators. Therefore, the rules for `bool_atom` are not recursive at all and an expression of the form $x < y < z$ is syntactically invalid.
2. The question mark that precedes the names `bool_expr`, `conjunction`, `negation`, and `bool_atom` tells Lark to *inline* these syntactical variables: If a node of the parse tree has only a single child and carries no name of its own, then this node is replaced by its child. Without these question marks, the parse tree of the expression `x == 1` would contain a chain of four nodes.
3. Line 19–31 define arithmetic expressions in the same way: The operators "+" and "-" have a lower precedence than the operators "*", "/", and "%". As all of these operators are left associative, all rules are left recursive. The last alternative of the rule for `factor` has no name, as a parenthesized expression should not create a node of its own.
4. Line 33 defines the arguments of a function call as a possibly empty list of expressions that are separated by commas.

```
1 grammar = syntax + terminal_definitions
2
3 import logging
4 import lark
5
6 lark.logger.setLevel(logging.WARNING)
7 parser = Lark(grammar, parser='lalr', start='program', debug=True)
```

Figure 13.7: Creating the parser.

Figure 13.7 on page 216 shows how the parser is created. Line 1 concatenates the grammar rules and the terminal definitions. In line 7, the keyword argument `parser='lalr'` selects the Lalr parser, while `start='program'` declares `program` to be the start variable of our grammar. By default,

Lark reports neither shift-reduce conflicts nor reduce-reduce conflicts. Setting `debug` to `True` and lowering the log level of the logger of Lark to `WARNING` in line 6 makes these conflicts visible. For our grammar, Lark reports only one conflict:

```
Shift/Reduce conflict for terminal ELSE: (resolving as shift)
```

This is the conflict caused by the dangling else problem that has been discussed above. All other conflicts have been eliminated by stratifying the grammar.

```
@v_args(inline=True)
class ASTBuilder(Transformer):
    # program and function definitions
    def program(self, *functions):
        return ('.',) + functions

    def function(self, name, parameters, declarations, statements):
        return ('fct', str(name), parameters, declarations, statements)

    def parameters(self, *names):
        return ('.',) + tuple(str(name) for name in names)

    def declarations(self, *names):
        return ('.',) + tuple(str(name) for name in names)

    def statements(self, *stmnts):
        return ('.',) + stmnts

    # statements
    def block(self, statements):
        return statements

    def assign(self, name, expr):
        return ('=', str(name), expr)

    def if_stmnt(self, condition, stmnt):
        return ('if', condition, stmnt)

    def if_else_stmnt(self, condition, then_stmnt, else_stmnt):
        return ('if-else', condition, then_stmnt, else_stmnt)

    def while_stmnt(self, condition, body):
        return ('while', condition, body)

    def return_stmnt(self, expr):
        return ('return', expr)

    def expr_stmnt(self, expr):
        return expr
```

Figure 13.8: Construction of the abstract syntax tree, part 1.

The parse tree that is computed by Lark is an object of class `lark.Tree`. As the code generator presented in the next section works on nested tuples, we have to convert the parse tree into a nested

```
40    # Boolean expressions
41    def or_expr (self, lhs, rhs): return ('||', lhs, rhs)
42    def and_expr(self, lhs, rhs): return ('&&', lhs, rhs)
43    def not_expr(self, arg):      return ('!' , arg)
44
45    def equal           (self, lhs, rhs): return ('==', lhs, rhs)
46    def unequal         (self, lhs, rhs): return ('!=', lhs, rhs)
47    def less_or_equal   (self, lhs, rhs): return ('<=', lhs, rhs)
48    def greater_or_equal(self, lhs, rhs): return ('>=', lhs, rhs)
49    def less            (self, lhs, rhs): return ('<' , lhs, rhs)
50    def greater         (self, lhs, rhs): return ('>' , lhs, rhs)
51
52    # arithmetical expressions
53    def add     (self, lhs, rhs): return ('+', lhs, rhs)
54    def subtract(self, lhs, rhs): return ('-', lhs, rhs)
55    def multiply(self, lhs, rhs): return ('*', lhs, rhs)
56    def divide  (self, lhs, rhs): return ('/', lhs, rhs)
57    def modulo  (self, lhs, rhs): return ('%', lhs, rhs)
58
59    def number(self, n):
60        return ('Number', str(n))
61
62    def variable(self, name):
63        return str(name)
64
65    def function_call(self, name, arguments):
66        return ('call', str(name)) + arguments[1:]
67
68    def arguments(self, *exprs):
69        return ('.',) + exprs
```

Figure 13.9: Construction of the abstract syntax tree, part 2.

tuple. This is done by the class `ASTBuilder` that is shown in Figure 13.8 on page 217 and Figure 13.9 on page 218. This class is a subclass of the class `Transformer` provided by Lark.

1. A transformer traverses the parse tree bottom-up. For every node, it calls the method whose name is the name of the corresponding grammar rule, respectively the name that has been specified after the operator "->". The arguments of this method are the results that have already been computed for the children of the node. The decorator `@v_args(inline=True)` in line 1 specifies that these results are passed as separate arguments instead of being packed into a single list.

2. The abstract syntax trees are represented as follows:

   (a) A list of $n$ items is represented as the nested tuple $(\text{“.”}, \mathit{item}_1, \cdots, \mathit{item}_n)$. For example, a program consisting of the functions $f_1, \cdots, f_n$ is represented as $(\text{“.”}, f_1, \cdots, f_n)$.

   (b) A function definition is represented as the nested tuple

   $$(\text{“fct”}, \mathit{name}, \mathit{parameters}, \mathit{variables}, \mathit{statements}).$$

   (c) A variable is represented as a string, while a number is represented as a pair of the form $(\text{“Number”}, \mathit{digits})$.

(d) All other nodes are represented as tuples whose first component is the operator. For example, the abstract syntax tree of an assignment is a tuple of the form $(\text{“=”}, \mathit{name}, \mathit{expr})$.

3. The tokens computed by Lark are objects of the class `lark.Token`, which is a subclass of `str`. We convert these tokens into ordinary strings via the function `str`, so that the abstract syntax tree contains no objects that are specific to Lark.

4. A block is represented by the list of its statements, while an expression statement is represented by the expression itself.

5. In the method `function_call`, the expression `arguments[1:]` discards the key “`.`” of the list of arguments. Hence, a function call is represented as a tuple of the form

$$(\text{“call”}, \mathit{name}, \mathit{arg}_1, \cdots, \mathit{arg}_n).$$

```
ast_builder = ASTBuilder()

def parse_program(program):
    try:
        parse_tree = parser.parse(program)
    except UnexpectedInput as error:
        print(f'Syntax error in line {error.line}, column {error.column}:')
        print(error.get_context(program))
        return None
    return ast_builder.transform(parse_tree)
```

Figure 13.10: Parsing a program.

Finally, Figure 13.10 on page 219 shows the function `parse_program`. This function takes a string containing a program, parses it, and returns the abstract syntax tree of this program. If the program contains a syntax error, Lark raises an exception of class `UnexpectedInput`. This exception provides the attributes `line` and `column`, while the method `get_context` returns the line of the program that contains the error together with a marker that points to the offending token. In this case, `parse_program` returns `None`. For example, for the erroneous program

```
int main() { x = 1 + ; }
```

the following error message is printed:

```
Syntax error in line 1, column 22:
int main() { x = 1 + ; }
                     ^
```

**Exercise 32**: Extend the parser of the notebook

`Python/Chapter-13/Compiler.ipynb`

so that it supports the ternary C-operator for conditional expressions. For example, in C, the following assignment using the ternary operator can be used to assign to the variable `max` the maximum of the values of the variables `x` and `y`:

```
max = x > y ? x : y;
```

Extend the grammar and the class `ASTBuilder` so that they can process the assignment shown above. ◇

## 13.3 Code Generation

Next, we discuss the generation of code. We structure our representation by discussing the code generation for arithmetic expressions, Boolean expression, statements, and function definitions separately.

### 13.3.1 Translation of Arithmetic Expressions

Given an arithmetic expression $e$, the translation of $e$ is supposed to generate some code that, when executed, places the result of evaluating the expression $e$ onto the stack. To this end we define a function `compile_expr` that has the following signature:

$$\texttt{compile_expr} : \mathit{Expr} \times \mathit{SymbolTable} \times \mathit{ClassName} \to \big\langle \mathit{List}[\mathit{AsmCmd}], \mathbb{N} \big\rangle.$$

A call of this function has the form

$$\texttt{compile_expr}(\texttt{expr}, \texttt{st}, \texttt{name}).$$

The interpretation of the arguments is as follows:

(a) `expr` is the arithmetic expression that is to be translated into assembler code.

(b) `st` is the symbol table: Concretely, this is a dictionary that maps the variable names to their position in the local variable frame. For example, if "x" is a variable that occupies the third slot in the local variable frame, then we have

```
st['x'] = 2,
```

because the first variable in the local variable frame has index `0`. We will discuss later how the positions of the variables in the local variable frame are assigned.

(c) `name` is the name of the class that is to be used by our compiler.

As we generate *Java* assembler and in *Java* every function has to be a part of a class, all functions that we create have to be static functions that are defined inside the *Java* `.class` file that is generated. The parameter `name` specifies the name of this class.

The argument `name` is only needed when function calls are translated.

The function `compile_expr` returns a pair.

(a) The first component of this pair is a list of *Java* assembler commands that adhere to the syntax recognized by *Jasmin*. In general, when the expression that is translated is complex, the execution of these assembler commands might need considerable room on the stack. However, it has to be guaranteed that when the execution of these commands finishes, the stack is back to its original height plus one because the net effect of executing these commands must be to put the value of the expression on the stack.

(b) The second component of the return value of `compile_expr` is a natural number. This natural number tells us how much the stack might grow when `expr` is evaluated. This information is needed because the *Java* virtual machine needs this information in advance: In *Java*, every function has to declare how much space it might use on the stack. This declaration is done using the pseudo assembler command

```
.limit stack sz
```

where `sz` is the maximum size of the stack. Controlling the maximum height of the stack is a security feature of *Java* that prevents exploits that utilize stack overflows.

All assembler commands that are generated are indented by eight space characters, while labels are indented by four space characters. This makes the generated assembler code easier to read. The indentation of the commands is done by the function `indent` that is shown in Figure 13.11.

```
def indent(s):
    return ' ' * 8 + s
```

Figure 13.11: Indentation of assembler commands.

The function `compile_expr` is implemented via a case distinction. Figure 13.12 on page 221 shows the skeleton of this function. In the following, we discuss the implementation of the different expressions one by one.

```
def compile_expr(expr, st, class_name):
    match expr:
        case str(var): # expr is a variable name
            ...
        case 'Number', n:
            ...
        case ('+' | '-' | '*' | '/' | '%') as op, lhs, rhs:
            ...
        case 'call', 'println', *args:
            ...
        case 'call', f, *args:
            ...
        case _:
            assert False, f'Error in compile_expr({expr}, {st}, {class_name})'
```

Figure 13.12: Translation of an expression.

### Translation of a Variable

If the expression that is to be compiled is a variable $v$, we have to load this variable onto the stack using the command `iload`. This command has one parameter which is the index of the variable on the local variable frame. This index is stored in the symbol table `st`. Since we just need one entry on the stack to store the variable, we have

$$\texttt{compile_expr}(v, \texttt{st}, \texttt{name}) = \langle \texttt{[ iload st[}v\texttt{] ]}, 1 \rangle \quad \text{if } v \text{ is a variable.}$$

The code for translating a variable is shown in Figure 13.13 on page 222.

### Translation of a Constant

We can load a constant $c$ onto the stack using the assembler command

$$\texttt{ldc } c.$$

As we only need the room to store $c$ on the stack, we have

$$\texttt{compile_expr}(c, \texttt{st}, \texttt{name}) = \langle \texttt{[ ldc } c \texttt{ ]}, 1 \rangle \quad \text{if } c \text{ is a constant.}$$

```
def compile_expr(expr, st, class_name):
    match expr:
        case str(var): # expr is a variable name
            Cmd = indent(f'iload {st[var]}')
            return [Cmd], 1
        ...
```

Figure 13.13: Translation of a variable.

The code for translating a constant is shown in Figure 13.14 on page 222.

```
def compile_expr(expr, st, class_name):
    match expr:
        ...
        case 'Number', n:
            Cmd = indent(f'ldc {n}')
            return [Cmd], 1
        ...
```

Figure 13.14: Translation of a constant.

### Translation of an Arithmetic Operator

In order to translate an expression of the form

*lhs* "+" *rhs*

into assembler code, we first have to recursively translate the expressions *lhs* and *rhs* into assembler code. Later, when this code is executed the values of the expressions *lhs* and *rhs* are placed on the stack. We add these values using the command `iadd`. If the evaluation of *lhs* needs $s_1$ words on the stack and the evaluation of *rhs* needs $s_2$ words on the stack, then the evaluation of *lhs* + *rhs* needs

$$\max(s_1, 1 + s_2)$$

words on the stack, because when *rhs* is evaluated, the value of *lhs* occupies already one position on the stack and hence the evaluation of *rhs* can only use the memory cells that are above the position where *lhs* is stored. Therefore, the compilation of *lhs* + *rhs* can be specified as follows:

$$\begin{aligned} & \texttt{compile_expr}(\mathit{lhs}, \texttt{st}, \texttt{name}) = \langle L_1, s_1 \rangle \\ \wedge\ & \texttt{compile_expr}(\mathit{rhs}, \texttt{st}, \texttt{name}) = \langle L_2, s_2 \rangle \\ \rightarrow\ & \texttt{compile_expr}(\mathit{lhs} + \mathit{rhs}, \texttt{st}, \texttt{name}) = \big\langle L_1 + L_2 + \texttt{[ iadd ]}, \max(s_1, 1 + s_2) \big\rangle. \end{aligned}$$

Figure 13.15 on page 223 shows how the translation of arithmetic operators is implemented. The translation of the subtraction, the multiplication, the division, and the modulo operator is similar. Note how we use the matching statement to extract the arithmetic operator and the dictionary `OpToCmd` to pick the correct assembler command.

```
def compile_expr(expr, st, class_name):
    match expr:
        ...
        case ('+' | '-' | '*' | '/' | '%') as op, lhs, rhs:
            L1, sz1 = compile_expr(lhs, st, class_name)
            L2, sz2 = compile_expr(rhs, st, class_name)
            OpToCmd = { '+': 'iadd', '-': 'isub', '*': 'imul',
                        '/': 'idiv', '%': 'irem' }
            Cmd     = indent(OpToCmd[op])
            return L1 + L2 + [Cmd], max(sz1, 1 + sz2)
        ...
```

Figure 13.15: Translation of the addition of expressions.

### Translation of a Function Call

In order to translate a function call of the form $f(a_1, \cdots, a_n)$ we first have to translate the arguments $a_1, \cdots, a_n$. Then there are two cases:

1. If $f$ is the function `println`, then *Jasmin* treats the function $f$ as a method of the predefined *Java* object `java.lang.System.out`. This method can be invoked using the assembler command `invokevirtual`. As before, let us assume that we have

   $$\texttt{compile_expr}(a_i, \texttt{st}, \texttt{class_name}) = \langle L_i, s_i \rangle.$$

   This time, we have to start by putting the object `java.lang.System.out` onto the stack before we can evaluate any of the arguments. Therefore, in order to calculate the stack size we now define

   $$s := \max(1 + s_1, 2 + s_2, \cdots, i + s_i, \cdots, n + s_n).$$

   In this case we have to define the signature `fs` of $f$ as the string

   $$\texttt{fs} := \texttt{java/io/PrintStream/println(}\underbrace{\texttt{I}\cdots\texttt{I}}_{n}\texttt{)V}.$$

   Furthermore, we define the command `gs` for putting the object `java.lang.System.out` onto the stack as

   $$\texttt{gs} := \texttt{getstatic java/lang/System/out Ljava/io/PrintStream;}.$$

   Then we can define the value of $\texttt{compile_expr}\big(f(a_1, \cdots, a_n), \texttt{st}, \texttt{class_name}\big)$ as follows:

   $$\texttt{compile_expr}\big(f(a_1, \cdots, a_n), \texttt{st}, \texttt{class_name}\big) := \\ \big\langle \texttt{[ gs ]} + L_1 + \cdots + L_n + \texttt{[ invokevirtual fs ]}, s \big\rangle.$$

   Figure 13.16 on page 224 shows how the translation of a call to `println` is implemented.

```
def compile_expr(expr, st, class_name):
    match expr:
        ...
        case 'call', 'println', *args:
            CmdLst    = [
                indent('getstatic java/lang/System/out Ljava/io/PrintStream;')
            ]
            stck_size = 0
            cnt       = 0
            for arg in args:
                L, sz_arg = compile_expr(arg, st, class_name)
                stck_size = max(stck_size, cnt + 1 + sz_arg)
                CmdLst   += L
                cnt      += 1
            CmdLst += [
                indent(f'invokevirtual java/io/PrintStream/println({"I"*cnt})V')
            ]
            return CmdLst, stck_size
        ...
```

Figure 13.16: Translation of a call to `println`.

2. $f$ is a user defined function. In this case, the byte code treats $f$ as a static function of the class `class_name`. This static function can be called using the assembler command `invokestatic`. In order to calculate the stack size that needs to be reserved for the evaluation of $f(a_1, \cdots, a_n)$, let us assume that we have

   $$\texttt{compile_expr}(a_i, \texttt{st}, \texttt{class_name}) = \langle L_i, s_i \rangle,$$

   i.e. evaluation of the $i^{\text{th}}$ argument is done by the list of assembler commands $L_i$ and needs a stack size of $s_i$. As we are evaluating the arguments $a_i$ in the order from $a_1$ to $a_n$, the evaluation of $a_i$ needs a stack size of $i - 1 + s_i$, since the results from the evaluation of the arguments $a_1$, $\cdots$, $a_{i-1}$ are already placed on the stack. Therefore, if we define

   $$s := \max(s_1, 1 + s_2, \cdots, i - 1 + s_i, \cdots, n - 1 + s_n),$$

   then $s$ is the total amount of stack size needed to evaluate the arguments. Furthermore, let us define the signature `fs` of the function $f$ as the string

   $$\texttt{fs} := \texttt{class_name/f(}\underbrace{\texttt{I}\cdots\texttt{I}}_{n}\texttt{)I}.$$

   Then we can define the value of $\texttt{compile_expr}\big(f(a_1, \cdots, a_n), \texttt{st}, \texttt{class_name}\big)$ as follows:

   $$\texttt{compile_expr}\big(f(a_1, \cdots, a_n), \texttt{st}, \texttt{class_name}\big) := \\ \big\langle L_1 + \cdots + L_n + \texttt{[ invokestatic fs ]}, \max(s, 1) \big\rangle.$$

   We have to take the maximum of $s$ and $1$ to cover the case that the function does not take any parameters.

   Figure 13.17 on page 225 shows how the translation of function calls is implemented.

### 13.3.2 Translation of Boolean Expressions

Boolean expressions are built from equations and inequations using the logical operators "!" (logical not), "&&" (logical and), and "||" (logical or). To compile Boolean expressions we define a function

```
def compile_expr(expr, st, class_name):
    match expr:
        ...
        case 'call', f, *args:
            CmdLst    = []
            stck_size = 0
            cnt       = 0
            for arg in args:
                L, sz_arg = compile_expr(arg, st, class_name)
                stck_size = max(stck_size, cnt + sz_arg)
                CmdLst   += L
                cnt      += 1
            CmdLst += [indent(f'invokestatic {class_name}/{f}({"I"*cnt})I')]
            return CmdLst, max(stck_size, 1)
        ...
```

Figure 13.17: Translation of function calls.

`compile_bool` that has the following signature:

$$\texttt{compile_bool} : \mathit{BoolExpr} \times \mathit{SymbolTable} \times \mathit{ClassName} \to \big\langle \mathit{List}[\mathit{AsmCmd}], \mathbb{N} \big\rangle.$$

The interpretation of the arguments is similar to the interpretation of the arguments of the function `compile_expr`. When we execute the list of assembler commands that result from the compilation of a Boolean expression, we expect that the execution of these commands either puts the number $1$ (representing `True`) or the number $0$ (representing `False`) onto the stack. The translation of Boolean expressions needs labels. These labels are created by the function `new_label` that is shown in Figure 13.18. The global variable `label_counter` counts the labels that have already been created, so that every call of `new_label` returns a new label of the form `l1`, `l2`, `l3`, etc.

```
label_counter = 0

def new_label():
    global label_counter
    label_counter += 1
    return 'l' + str(label_counter)
```

Figure 13.18: Creating new labels.

We start our discussion of the function `compile_bool` with the translation of equations.

### Translation of Equations

In the *Java* virtual machine, the logical values `True` and `False` are represented by the integers $1$ and $0$, respectively. Therefore, the code produced from translating an equation of the form

$$\mathit{lhs} == \mathit{rhs}$$

should either place the number $1$ or $0$ onto the stack: If the value of *lhs* is equal to the value of *rhs*, the number $1$ has to be put on the stack, otherwise the value $0$ has to be put onto the stack. Let us assume

that we have

$$\texttt{compile_expr}(\textit{lhs}, \texttt{st}, \texttt{class_name}) = \langle L_1, s_1 \rangle \quad \text{and}$$
$$\texttt{compile_expr}(\textit{rhs}, \texttt{st}, \texttt{class_name}) = \langle L_2, s_2 \rangle.$$

Then the stack size needed for evaluating the equation *lhs* == *rhs* is given by the expression

$$\max(s_1, 1 + s_2)$$

and the function `compile_bool` can be specified as follows:

$$\begin{aligned} \textit{compile_bool}(\textit{lhs} == \textit{rhs}, \texttt{st}, \texttt{class_name}) \;=\; \langle \;& L_1 \\ & + L_2 \\ & + [\,\texttt{if_icmpeq}\ \textit{true}\,] \\ & + [\,\texttt{bipush}\ 0\,] \\ & + [\,\texttt{goto}\ \textit{next}\,] \\ & + [\,\textit{true}\texttt{:}\,] \\ & + [\,\texttt{bipush}\ 1\,] \\ & + [\,\textit{next}\texttt{:}\,], \qquad s \rangle \end{aligned}$$

Here, *true* and *next* have to be new labels that do not occur elsewhere in the code for the function that is compiled. Let me explain this equation in detail:

1. $L_1$ and $L_2$ are the lists of assembler commands that evaluate *lhs* and *rhs*, respectively.
2. Executing $L_1$ and $L_2$ leaves two values on the stack. These values are then compared using the assembler command `if_icmpeq`. If these value are the same, execution proceeds at the label *true*.
3. Otherwise execution commences with the next instruction and hence the value $0$ is pushed onto the stack.
4. Next, the control flow jumps to the label *next*, which is at the end of the generated list of assembler commands. Hence, in this case the evaluation of the equation *lhs* == *rhs* is finished.
5. If the values of *lhs* and *rhs* are the same, the number $1$ is pushed onto the stack.

The translation of a negated equation of the form

*lhs* != *rhs*

is similar to the translation of an equation: We only have to replace the command `if_icmpeq` with the command `if_icmpne`. Similarly, if the inequation has the form

*lhs* <= *rhs*

we have to replace the command `if_icmpeq` with the command `if_icmple`. If the inequation has the form

*lhs* >= *rhs*

we have to replace the command `if_icmpeq` with the command `if_icmpge`. If the inequation has the form

*lhs* < *rhs*

we have to replace the command `if_icmpeq` with the command `if_icmplt`. If the inequation has the form

*lhs* > *rhs*

we have to replace the command `if_icmpeq` with the command `if_icmpgt`. Figure 13.19 on page 227 shows how the translation of equations and inequations is implemented.

```
def compile_bool(expr, st, class_name):
    match expr:
        case ('==' | '!=' | '<=' | '>=' | '<' | '>') as op, lhs, rhs:
            OpToCmd = { '==': 'if_icmpeq',
                        '!=': 'if_icmpne',
                        '<=': 'if_icmple',
                        '>=': 'if_icmpge',
                        '<' : 'if_icmplt',
                        '>' : 'if_icmpgt'
                      }
            L1, sz1    = compile_expr(lhs, st, class_name)
            L2, sz2    = compile_expr(rhs, st, class_name)
            true_label = new_label()
            next_label = new_label()
            CmdLst     = L1 + L2
            cmd        = OpToCmd[op]
            CmdLst    += [indent(cmd + ' ' + true_label)]
            CmdLst    += [indent('bipush 0')]
            CmdLst    += [indent('goto ' + next_label)]
            CmdLst    += [' ' * 4 + true_label + ':']
            CmdLst    += [indent('bipush 1')]
            CmdLst    += [' ' * 4 + next_label + ':']
            return CmdLst, max(sz1, 1 + sz2)
        ...
```

Figure 13.19: Translation of equations.

### Translation of Binary Boolean Operators

In order to translate an expression of the form

*lhs* && *rhs*

into assembler code, we first have to recursively translate the expressions *lhs* and *rhs* into assembler code. Later, when this code is executed the values of the expressions *lhs* and *rhs* are placed on the stack. We combine these values using the command `iand`. If the evaluation of *lhs* needs $s_1$ words on the stack and the evaluation of *rhs* needs $s_2$ words on the stack, then the evaluation of *lhs* && *rhs* needs

$$\max(s_1, 1 + s_2)$$

words on the stack, because when *rhs* is evaluated, the value of *lhs* occupies already one position on the stack and hence the evaluation of *rhs* can only use the memory cells that are above the position where *lhs* is stored. Therefore, the compilation of *lhs* && *rhs* can be specified as follows:

$$\begin{array}{rl} & \texttt{compile_bool}(\textit{lhs}, \texttt{st}, \texttt{name}) = \langle L_1, s_1 \rangle \\ \wedge & \texttt{compile_bool}(\textit{rhs}, \texttt{st}, \texttt{name}) = \langle L_2, s_2 \rangle \\ \rightarrow & \texttt{compile_bool}(\textit{lhs}\ \texttt{\&\&}\ \textit{rhs}, \texttt{st}, \texttt{name}) = \big\langle L_1 + L_2 + \texttt{[ iand ]}, \max(s_1, 1 + s_2) \big\rangle. \end{array}$$

The translation of an expression of the form

*lhs* || *rhs*

is similar: Instead of using the command `iand` we have to use the command `ior` instead. Figure 13.20 on page 228 shows how the translation of binary logical operators is implemented.

```
def compile_bool(expr, st, class_name):
    match expr:
        ...
        case ('&&' | '||') as op, lhs, rhs:
            OpToCmd      = { '&&': 'iand', '||': 'ior' }
            L1, sz1      = compile_bool(lhs, st, class_name)
            L2, sz2      = compile_bool(rhs, st, class_name)
            cmd          = OpToCmd[op]
            CmdLst       = L1 + L2 + [indent(cmd)]
            return CmdLst, max(sz1, 1 + sz2)
        ...
```

Figure 13.20: Translation of binary logical operators.

It should be noted that our translation of the binary logical operators is different from what happens in the language C. In C, the evaluation of logical operators is lazy: if an expression of the form

*lhs* && *rhs*

is evaluated, the evaluation is stopped as soon the evaluation of *lhs* returns $0$ because then there is no point to evaluate *rhs*, since if the value of *lhs* is $0$, the result of the expression *lhs* && *rhs* is $0$, independent from the value of the expression *rhs*. Similarly, if an expression of the form

*lhs* || *rhs*

is evaluated and the evaluation of *lhs* is $1$, then the result of the evaluation of the expression *lhs* || *rhs* is $1$, regardless of the value of *rhs*.

#### Translation of Negations

The translation of an expression of the form `!`*expr* is not as straightforward as the translation of conjunctions and disjunctions. The reason is that *Jasmin* has no assembler command of the form `inot` that negates a logical value. But there is another way: Since we represent the truth values as $0$ and $1$, we can specify negation arithmetically as follows:

$$!x = 1 - x.$$

Therefore, if we have

$$\texttt{compile_bool}(\textit{arg}, \texttt{st}, \texttt{class_name}) = \langle L, s \rangle$$

we can define:

$$\texttt{compile_bool}(\texttt{!}\textit{arg}, \texttt{st}, \texttt{class_name}) = \langle \texttt{[bipush 1]} + L + \texttt{[isub]}, 1 + s \rangle.$$

Figure 13.21 on page 229 shows how the translation of the negation operator is implemented.

### 13.3.3 Translation of Statements

Next, we show how statements are compiled. First of all, we agree that the execution of a statement must not change the size of the stack: The size of stack before the execution of a statement must

```
def compile_bool(expr, st, class_name):
    match expr:
        ...
        case '!', arg:
            L, sz  = compile_bool(arg, st, class_name)
            CmdLst = [indent('bipush 1')] + L + [indent('isub')]
            return CmdLst, sz + 1
        ...
```

Figure 13.21: Translation of the negation operator.

be the same as the size of the stack after the statement has been executed. Of course, during the execution of the statement the stack may well grow and shrink. But once the execution of the statement has finished, the stack has to be cleaned from all intermediate values that have been put on the stack during the execution of the statement.

To compile statements we define a function `compile_stmnt` that has the following signature:

$$\texttt{compile_stmnt} : \mathit{Stmnt} \times \mathit{SymbolTable} \times \mathit{ClassName} \to \big\langle \mathit{List}[\mathit{AsmCmd}], \mathbb{N} \big\rangle.$$

The interpretation of the arguments is similar to the interpretation of the arguments of the function `compile_expr`. The only difference is that the first argument now is an abstract syntax tree that represents a statement.

### Translation of Assignments

To begin with, we show how an assignment of the form

$$x = \mathit{expr}$$

is translated. The idea is to evaluate *expr*. As a consequence of this evaluation the value of *expr* remains on the stack, from where it can be stored in the variable $x$ using the assembler command `istore`. Therefore, if we have

$$\texttt{compile_expr}(\mathit{expr}, \texttt{st}, \texttt{class_name}) = \langle L, s \rangle$$

we can define:

$$\texttt{compile_stmnt}(x = \mathit{expr}, \texttt{st}, \texttt{class_name}) = \big\langle L + \texttt{[istore st[x]]}, s \big\rangle.$$

Figure 13.22 on page 229 shows how the translation of an assignment is implemented.

```
def compile_stmnt(stmnt, st, class_name):
    match stmnt:
        case '=', var, expr:
            CmdLst, sz = compile_expr(expr, st, class_name)
            CmdLst    += [indent(f'istore {st[var]}')]
            return CmdLst, sz
        ...
```

Figure 13.22: Translation of an assignment statement.

### The Translation of Branching Commands

Next, we show how to translate a branching command of the form

`if` (*bool_expr*) *stmnt*.

Obviously, we first have to translate the Boolean expression *bool_expr*. If we have

$$\texttt{compile_bool}(\textit{bool_expr}, \texttt{st}, \texttt{class_name}) = \langle L_1, s_1 \rangle$$

and, furthermore,

$$\texttt{compile_stmnt}(\textit{stmnt}, \texttt{st}, \texttt{class_name}) = \langle L_2, s_2 \rangle,$$

then we can define:

$$\begin{aligned} \textit{compile_stmnt}\big(\texttt{if}\ (\textit{expr})\ \textit{stmnt}, \texttt{st}, \texttt{class_name}\big) \;=\; \big\langle\ & L_1 \\ & + [\ \texttt{ifeq}\ \textit{else}\ ] \\ & + L_2 \\ & + [\ \textit{else}\texttt{:}\ ], \qquad \max(s_1, s_2) \big\rangle \end{aligned}$$

This works because executing the list of assembler commands $L_1$ leaves either a $0$ or a $1$ on the stack. If *expr* is false, the top of the stack stores the number $0$. In this case, the branching command `ifeq` branches to the label *else* and the assembler commands in the list $L_2$ are not executed. If *expr* is true, the top of the stack stores the number $1$. Therefore, the branching command `ifeq` does not branch and the assembler commands in the list $L_2$ are executed, i.e. *stmnt* is executed.

The translation of a branching command of the form

`if` (*bool_expr*) $\textit{stmnt}_1$ `else` $\textit{stmnt}_2$

is similar. Let us assume that we have the following:

1. $\texttt{compile_bool}(\textit{bool_expr}, \texttt{st}, \texttt{class_name}) = \langle L_1, s_1 \rangle$
2. $\texttt{compile_stmnt}(\textit{stmnt}_1, \texttt{st}, \texttt{class_name}) = \langle L_2, s_2 \rangle$, and
3. $\texttt{compile_stmnt}(\textit{stmnt}_2, \texttt{st}, \texttt{class_name}) = \langle L_3, s_3 \rangle$.

Then we define:

$$\textit{compile_stmnt}(\texttt{if}\ (\textit{bool_expr})\ \textit{stmnt}_1\ \texttt{else}\ \textit{stmnt}_2, \texttt{st}, \texttt{class_name}) = \langle L_1 + [\,\texttt{ifeq}\ \textit{else}\,] + L_2 + [\texttt{goto}\ \textit{next}] + [\,\textit{else:}\,] + L_3 + [\,\textit{next:}\,],\ \max(s_1, s_2, s_3)\rangle$$

Figure 13.23 on page 231 shows how the translation of branching statements is implemented.

```
def compile_stmnt(stmnt, st, class_name):
    match stmnt:
        ...
        case 'if', expr, sub_stmnt:
            L1, sz1    = compile_bool(expr, st, class_name)
            L2, sz2    = compile_stmnt(sub_stmnt, st, class_name)
            else_label = new_label()
            lbl_stmnt  = ' ' * 4 + else_label + ':'
            CmdLst = L1 + [indent(f'ifeq {else_label}')] + L2 + [lbl_stmnt]
            return CmdLst, max(sz1, sz2)
        case 'if-else', expr, then_stmnt, else_stmnt:
            L1, sz1    = compile_bool(expr, st, class_name)
            L2, sz2    = compile_stmnt(then_stmnt, st, class_name)
            L3, sz3    = compile_stmnt(else_stmnt, st, class_name)
            else_label = new_label()
            next_label = new_label()
            if_stmnt   = indent(f'ifeq {else_label}')
            else_stmnt = ' ' * 4 + else_label + ':'
            next_stmnt = ' ' * 4 + next_label + ':'
            goto_stmnt = indent(f'goto {next_label}')
            CmdLst = L1 + [if_stmnt] + L2 + [goto_stmnt, else_stmnt] + \
                     L3 + [next_stmnt]
            return CmdLst, max(sz1, sz2, sz3)
        ...
```

Figure 13.23: Translation of branching statements.

### Translation of a Loop

If we want to translate a `while` loop of the form

$$\texttt{while}\ (\textit{cond})\ \textit{stmnt}$$

we recursively compute

$$\texttt{compile_bool}(\textit{cond}, \texttt{st}, \texttt{class_name}) = \langle L_1, s_1\rangle$$

and

$$\texttt{compile_stmnt}(\textit{stmnt}, \texttt{st}, \texttt{class_name}) = \langle L_2, s_2\rangle,$$

Then we can define

$$\begin{aligned}\textit{compile_stmnt}(\texttt{while}\ (\textit{cond})\ \textit{stmnt}, \texttt{st}, \texttt{class_name}) = \langle\ & [\textit{loop}:] \\ +\ & L_1 \\ +\ & [\,\texttt{ifeq}\ \textit{next}\,] \\ +\ & L_2 \\ +\ & [\texttt{goto}\ \textit{loop}] \\ +\ & [\,\textit{next:}\,],\quad \max(s_1, s_2)\rangle\end{aligned}$$

Figure 13.24 on page 232 shows how the translation of a loop is implemented.

```
def compile_stmnt(stmnt, st, class_name):
    match stmnt:
        ...
        case 'while', expr, body_stmnt:
            L1, sz1    = compile_bool(expr, st, class_name)
            L2, sz2    = compile_stmnt(body_stmnt, st, class_name)
            loop_label = new_label()
            next_label = new_label()
            if_stmnt   = indent(f'ifeq {next_label}')
            loop_stmnt = ' ' * 4 + loop_label + ':'
            next_stmnt = ' ' * 4 + next_label + ':'
            goto_stmnt = indent(f'goto {loop_label}')
            CmdLst = [loop_stmnt] + L1 + [if_stmnt] + \
                     L2 + [goto_stmnt, next_stmnt]
            return CmdLst, max(sz1, sz2)
        ...
```

Figure 13.24: Translation of a loop.

### Translation of a Return Statement

In order to translate a return statement of the form

`return` *expr*`;`

we first have to translate the expression *expr*. Assume that

`compile_expr`(*expr*, `st`, `class_name`) $= \langle L, s \rangle$.

Then we can define

`compile_stmnt`(`return` *expr*`;`, `st`, `class_name`) $= \big\langle L + [\texttt{ireturn}], s \big\rangle$.

Figure 13.25 on page 232 shows how a return statement is translated.

```
def compile_stmnt(stmnt, st, class_name):
    match stmnt:
        ...
        case 'return', expr:
            CmdLst, sz = compile_expr(expr, st, class_name)
            CmdLst    += [indent('ireturn')]
            return CmdLst, sz
        ...
```

Figure 13.25: Translation of a return statement.

### Translating a Block Statement

The translation of block statement of the form

$\{stmnt_1\,; \cdots stmnt_n\,;\}$

proceeds by translating the statements $\mathit{stmnt}_i$ separately. Assume that

$$\texttt{compile_stmnt}(\mathit{stmnt}_i, \texttt{st}, \texttt{class_name}) = \langle L_i, s_i \rangle \quad \text{for } i = 1, \cdots, n.$$

Then we define

$$\mathit{compile_stmnt}\big(\{\mathit{stmnt}_1\,; \cdots \mathit{stmnt}_n\,;\}, \texttt{st}, \texttt{class_name}\big) := \big\langle L_1 + \cdots + L_n, \max(s_1, \cdots, s_n) \big\rangle.$$

Figure 13.26 on page 233 shows how the translation of a block statement is implemented.

```
def compile_stmnt(stmnt, st, class_name):
    match stmnt:
        ...
        case '.', *stmnt_lst:
            CmdLst = []
            size   = 0
            for s in stmnt_lst:
                L, sz = compile_stmnt(s, st, class_name)
                CmdLst += L
                size   = max(size, sz)
            return CmdLst, size
        ...
```

Figure 13.26: Translation of a block statement.

### Translation of Expression Statements

When we translate an expression statement of the form

*expr* ;

we assume that the evaluation of the expression does not leave a value on the stack. At present, the only expression whose evaluation does not create a value is an invocation of the function `println`. Then in order to evaluate the expression statement, we just have to evaluate the expression. Therefore, we have

$$\texttt{compile_stmnt}(\mathit{expr}\,;, \texttt{st}, \texttt{class_name}) = \texttt{compile_expr}(\mathit{expr}, \texttt{st}, \texttt{class_name}).$$

Figure 13.27 on page 233 shows how the translation of an expression statement is implemented.

```
def compile_stmnt(stmnt, st, class_name):
    match stmnt:
        ...
        case _: # it must be an expression statement
            return compile_expr(stmnt, st, class_name)
```

Figure 13.27: Translation of an expression statement.

### 13.3.4 Translation of a Function Definition

```
def compile_fct(fct_def, class_name):
    global label_counter
    label_counter = 0
    _, name, parameters, variables, stmnts = fct_def
    _, *parameters = parameters
    _, *variables  = variables
    _, *stmnts     = stmnts
    m   = len(parameters)
    n   = len(variables)
    st  = {}
    cnt = 0
    for var in parameters + variables:
        st[var] = cnt
        cnt    += 1
    CmdLst = []
    size   = 0
    for stmnt in stmnts:
        L, sz = compile_stmnt(stmnt, st, class_name)
        CmdLst += L
        size = max(size, sz)
    limit_locals = f'.limit locals {m+n}'
    limit_stack  = f'.limit stack  {size}'
    return_stmnt = indent('return')
    if name != 'main':
        method = f'.method public static {name}({"I"*m})I'
        CmdLst = [method, limit_locals, limit_stack] + CmdLst + \
                 ['.end method']
        return CmdLst, size
    else:
        method = '.method public static main([Ljava/lang/String;)V'
        CmdLst = [method, limit_locals, limit_stack] + CmdLst + \
                 [return_stmnt, '.end method']
        return CmdLst, size
```

Figure 13.28: Translation of a function definition.

Figure 13.28 on page 234 shows how a function definition is translated. In order to understand how this works we have to understand what type of code the function `compile_fct` has to generate. Depending on whether the function that is translated is the function `main` or not there are two cases.

1. If the function that is to be compiled has the name “`main`”, the code that has to be generated has the following form:

```
1  .method public static main([Ljava/lang/String;)V
2  .limit locals l
3  .limit stack s
4     s_1
5     ⋮
6     s_n
7     return
8  .end method
```

Here $l$ is the number of all variables used by the function `main`, while $s$ is the maximum height of the stack. Finally, $s_1, \cdots, s_n$ are the assembler commands that result from translating the block inside the function.

2. Otherwise, the following code is to be generated:

```
1     .method public static f(I···I)I
2     .limit locals l
3     .limit stack s
4         s_1
5         ⋮
6         s_n
7     .end method
```

Here $f$ is the name of the function. The signature of $f$ is a string of the form

$$f(\underbrace{\texttt{I}\cdots\texttt{I}}_{m})\texttt{I}$$

where $m$ is the number of the parameters. Furthermore, $l$ is the number of all local variables and $s$ is the maximum height of the stack. Finally, $s_1, \cdots, s_n$ are the assembler commands that result from translating the block inside the function.

We proceed to discuss the implementation shown in Figure 13.28 line by line.

1. We initialize the global variable `label_counter` to $0$. Every time a new label is generated, this counter will be incremented by the function `new_label` that creates labels of the form `l1`, `l2`, etc.

2. The nested tuple that is produced by the parser from a function definition has the form

   $$(\texttt{'fct'}, \mathit{name}, \mathit{parameters}, \mathit{variables}, \mathit{stmnts}).$$

   (a) *name* is the name of the function.

   (b) *parameters* is the abstract syntax tree representing the list of parameters.

   (c) *variables* is the abstract syntax tree representing the list of all local variables declared in the function.

   (d) *stmnts* is the the abstract syntax tree representing the list of all statements in the body of the function.

3. Initially, the parameters, variables, and statements are stored as nested tuples. In line 5-7 these nested tuples are transformed into proper lists.

4. $m$ is the number of parameters.
5. $n$ is the number of local variables.
6. The `for` loop in line 12–14 initializes the symbol table `st`. The parameters occupy the first positions of the local variable frame and the local variables follow the parameters. Given a parameter or local variable $x$, the expression `st[`$x$`]` returns the position in the local variable frame where the parameter or variable is stored.
7. The `for` loop in line 17–20 translates the statements in the body of the function one by one. Furthermore, it computes the size of the stack that is needed.
8. Line 21 declares the size of the local variable frame. The pseudo-command `.limit locals` reserves memory for the parameters and the local variables.
9. Line 22 declares the size of the stack needed by the function.
10. Line 23 creates the command `return` that has to terminate the function `main`, since this function does not return a value.
11. Depending on whether the function that is compiled is the function `main` or not, the correct code is returned.

### 13.3.5 Compiling a Program

We conclude this chapter by presenting the function `compile_program` that is shown in Figure 13.29 on page 237. This function takes the name of a file as its argument. This file is expected to contain a program written in the subset of the programming language C that has been presented in this chapter. If the name of this file is, for example, `Test.c`, then the function `compile_program` generates a file with the name `Test.jas`, that contains an assembler program that adheres to the rules of *Jasmin*. We proceed to discuss the function `compile_program` line by line.

1. In line 1 we import the module `os`. This module contains the submodule `path` which contains the functions `dirname` and `basename`.

   (a) The function `dirname` takes a string specifying a *file path*, e.g. something like

   ```
   '~/Formal-Languages/Python/Chapter-13/Examples/Test.c'
   ```

   and returns the name of the directory. In the example given above it would return the string

   ```
   '~/Formal-Languages/Python/Chapter-13/Examples'.
   ```

   (b) The function `basename` takes a *file path* and returns the name of the filename of the directory. In the example given previously it would return the string `'Test.c'`.

   The module `os` is aware of the differences in naming directories that exist in different operating systems.

2. After computing the `directory` of the file and its `base` in line 4 and 5, we remove the file extension `'.c'` in line 6. Line 7 computes the name of the output file. The function `os.path.join` combines the name of the directory and the name of the file using the separator of the operating system.
3. Line 8 and 9 read the file and store its content into the string `program`.

```
1  import os
2
3  def compile_program(file_name):
4      directory = os.path.dirname(file_name)
5      base      = os.path.basename(file_name)
6      base      = base[:-2]
7      outfile   = os.path.join(directory, base + '.jas')
8      with open(file_name, 'r') as handle:
9          program = handle.read()
10     ast = parse_program(program)
11     _, *fct_lst = ast
12     CmdLst = []
13     for fct in fct_lst:
14         L, _ = compile_fct(fct, base)
15         CmdLst += L + ['\n']
16     with open(outfile, 'w') as handle:
17         handle.write('.class public ' + base + '\n');
18         handle.write('.super java/lang/Object\n\n');
19         handle.write('.method public <init>()V\n');
20         handle.write('    aload 0\n');
21         handle.write('    invokenonvirtual java/lang/Object/<init>()V\n');
22         handle.write('    return\n');
23         handle.write('.end method\n\n');
24         for cmd in CmdLst:
25             handle.write(cmd + '\n')
```

Figure 13.29: Translation of a program.

4. In line 10 the function `parse_program` shown in Figure 13.10 converts the program into an abstract syntax tree that is represented as a nested tuple.
5. `fct_lst` is a list of all function definitions occurring in the given program.
6. `CmdLst` is a list of all assembler commands that will be generated.
7. The `for` loop in line 13–15 translates all function definitions into assembler code and concatenates the assembler statements into the list `CmdLst`.
8. In line 16 the output file is opened for writing.
9. Line 17–23 write the initialization code to the output file.
10. The `for` loop in line 24–25 writes the generated assembler commands to the output file.

Figure 13.30 on page 238 shows an integer C program for computing the sum $\sum_{i=1}^{6^2} i$. When we translate this program using the compiler developed in this chapter, we get the program that is shown in Figure 13.31 on page 239.

```
int sum(int n) {
    int s;
    s = 0;
    while (n != 0) {
        s = s + n;
        n = n - 1;
    }
    return s;
}

int main() {
    int n;
    n = 6 * 6;
    println(sum(n));
}
```

Figure 13.30: A C program for computing the sum $\sum_{i=1}^{6^2} i$.

```
1   .class public MySum
2   .super java/lang/Object
3
4   .method public <init>()V
5       ...
6   .end method
7
8   .method public static sum(I)I
9   .limit locals 2
10  .limit stack  2
11          ldc 0
12          istore 1
13      l3:
14          iload 0
15          ldc 0
16          if_icmpne l1
17          bipush 0
18          goto l2
19      l1:
20          bipush 1
21      l2:
22          ifeq l4
23          iload 1
24          iload 0
25          iadd
26          istore 1
27          iload 0
28          ldc 1
29          isub
30          istore 0
31          goto l3
32      l4:
33          iload 1
34          ireturn
35  .end method
36
37  .method public static main([Ljava/lang/String;)V
38  .limit locals 1
39  .limit stack  2
40          ...
41  .end method
```

Figure 13.31: Assembler program generated by our compiler.

**Exercise 33**: Extend the compiler that is presented in this chapter and that is available at

github.com/karlstroetmann/Formal-Languages/blob/master/Python/Chapter-13/Compiler.ipynb

so that a new kind of `for`-loops is supported. Figure 13.32 on page 240 shows an example of the use of this kind of `for` loop. ◇

```
int sum(int n) {
    int s;
    int i;
    s = 0;
    for (i = 1 .. n) {
        s = s + i;
    }
    return s;
}
```

Figure 13.32: A function for computing the sum $\sum_{i=1}^{n} i$.

# Bibliography


[ASUL06] Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, and Monica S. Lam. *Compilers — Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2006.

[Bac59] John W. Backus. The syntax and semantics of the proposed international algebraic language of the zurich ACM-GAMM conference. In *IFIP Congress*, pages 125–131, 1959.

[CS70] John Cocke and Jacob T. Schwartz. Programming languages and their compilers: Preliminary notes. Technical report, Courant Institute of Mathematical Sciences, New York University, 1970.

[DeR71] Franklin L. DeRemer. Simple LR(k) grammars. *Communications of the ACM*, 14(7):453–460, 1971.

[Ear70] Jay C. Earley. An efficient context-free parsing algorithm. *Communications of the ACM*, 13(2):94–102, 1970.

[HMU06] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3rd edition, 2006.

[Kas65] Tadao Kasami. An efficient recognition and syntax-analysis algorithm for context-free languages. Technical report, Air Force Cambridge Research Lab, Bedford, MA, 1965.

[Kle56] Stephen C. Kleene. Representation of events in nerve nets and finite automata. In C. E. Shannon and J. McCarthy, editors, *Automata studies*, pages 3–40. Princeton University Press, 1956.

[Knu65] Donald E. Knuth. On the translation of languages from left to right. *Information and Control*, 8(6):607–639, December 1965.

[KR78] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, Englewood Cliffs, NJ, 1978.

[MD97] Jon Meyer and Troy Downing. *Java Virtual Machine*. O'Reilly, 1997.

[NBB+60] P. Naur, J. W. Backus, F. L. Bauer, J. Green, C. Katz, J. MacCarthy, A. J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J. H. Wegstein, A. van Wijngaarden, and M. Woodger. Report on the algorithmic language Algol 60. *Numerische Mathematik*, 2:106–136, 1960.

[RS59] Michael Rabin and Dana Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2):114–125, 1959.

[Sip12] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, third edition, 2012.

[You67] Daniel H. Younger. Recognition and parsing of context-free languages in time $n^3$. *Information and Control*, 10(2):189–208, 1967.

# Index


$L(F)$, 60, 65
$\Sigma$, 59
$\doteq$, 18
$\emptyset$, 15
$\lambda$-generating, 158
$\rightsquigarrow$, 65
*det*$(F)$, 67
$\varepsilon$, 15
$\varepsilon$-closure, 66
$\varepsilon$-transition, 64
$\Sigma^*$, 7
$\lambda$, 7
*closure*$(\mathcal{M})$, 156

abstract syntax tree, 102
accepted language, 60
accepting state, 58, 59
*action* (shift-reduce parser), 148
alias (Lark), 110
alphabet, 7
ambiguous grammar, 104
anonymous terminal (Lark), 126, 180
Ascii alphabet, 7
assertion, 33
augmented grammar, 157

back-reference, 34
Backus-Naur form, 106
block formula, 51
bottom-up parser, 145

calculator, symbolic, 109
canonical LR language, 175
canonical LR parser, 146
character class, 32
    negated, 33
    predefined, 33
choice
    of regular expressions, 29
`chr`, 46
closure of a set of marked rules, 156
closure properties of regular languages, 83
Cocke-Younger-Kasami algorithm, 137
code generation, 220
`compile`, 41
`re.compile`, 53
complement of a language, 83, 86
complete, finite state machine, 60
computer algebra, 122
concatenation, 8
    of regular expressions, 29
configuration (of an Nfa), 65
context-free grammar, 99
context-free language, 9
core of a set of extended marked rules, 173
Cyk algorithm, 137

dangling else problem, 211
dead state, 60
death of a Dfa, 58
derivation step, 100
deterministic finite automaton, 58
Dfa, 59
differentiation
    chain rule, 121
    product rule, 119
    quotient rule, 120
    sum rule, 119
`re.DOTALL`, 45

e.m.R., *see* extended marked rule
Earley object, 138
Earley's algorithm, 138
    completion, 139
    prediction, 139
    reading, 139
Ebnf grammar, 105
empty string, $\lambda$, 7
entity
    named, 44
    numerical, 44
equivalence class, 78
equivalence of regular expressions, 18
equivalent states, 78
escape sequence, 28

escaping, 28
exam evaluation, 38
extended Backus-Naur form, 105
extended marked rule, 168
extended union, 174

`findall`, 27
finite automaton, 57
finite state machine, 57
*First*, 158
*Follow*, 160
formal language, 7, 9

generator function, 25
grammar rule, 99
greedy, 32
group, 34
`group`, 41
grouping, 34

HTML tag, 44
HTML to text, 43
`html5`, 46

`re.IGNORECASE`, 28
inlining directive (LARK), 110
input alphabet, 59
*Integer*-C, 209
interpreter, 124
intersection of languages, 84

*Jasmin*, 192
*Java* byte code, 192
*Java* virtual machine, 192
JVM, 192
    constant pool, 197
    heap, 196
    local variables, 196
    program memory, 196
    stack, 196
    stack pointer, 196

Kleene closure, 15

LALR language, 175
LALR parser, 146
LARK, 109
lazy evaluation, 228
left associative, 184
left-recursive, 105
leftmost derivation, 147
length of a string, 8
`Lex`, 21
lookahead, 55
lookahead assertion, 36
    negative, 37
LR-goto-graph, 172

marked rule, 155
master pattern, 22, 24
`match`, 42, 53
match object, 25
meta-character, 28

named group, 22, 25, 35
nested tuple, 112, 217
NFA, 64
non-associative, 184
non-deterministic finite automaton, 57, 64
non-greedy, 32
non-terminal, 98, 99
normal form, 122

palindrome, 101
parse tree, 102
    inner node, 102
    leaf node, 102
parser, 96
parser configuration, 148
parsing, 97
pattern matching, 113
    capture pattern, 119
    or pattern, 114
    star pattern, 133
pattern object, 41
`Ply`, 21
Post correspondence problem, 104
power of a language, 14
power of a string, 15
power set, 15
preamble, 49
precedence
    of arithmetic operators, 110
    of regular expression operators, 16
prime number, 10
product automaton, 85
product of languages, 14
pumping lemma for regular languages, 90

quantifier, 30
    non-greedy, 32

range, 32

raw string, 28
`re` (*Python* module), 22, 27
reachable, 78
read-eval-print loop, 114
recursive descent parser, 105
reduce-reduce conflict, 163, 170
    mysterious, 188
reflexive-transitive closure, 65
regular expression, 13, 15
    syntax in *Python*, 27
regular language, 9, 83
repetition qualifier, *see* quantifier
reversal of a string, 88
rewrite rule, 122
right associative, 115, 184
right recursive, 116
rightmost derivation, 147

scanner, 21, 22
    building with regular expressions, 24
scanner state, 22, 47, 53
`search`, 40
semantic action, 109
semi-decidable, 10
separable, 79
separates, 79
set difference, 87
shift-reduce conflict, 163, 170
    resolution by LARK, 181
shift-reduce parser, 147
signature of a method, 223
SLR grammar, 163
SLR language, 175
SLR parser, 145
stack, 22, 54
start state, 58, 59
start symbol, 99
string, 7
`sub`, 45
substitution pipeline, 22, 44
symbol, 7
symbol table, 220
symbolic differentiation, 115
syntactic category, 98
syntactic variable, 98, 99
syntax tree, 97

terminal, 98, 99
`Token` (class), 25
token, 21, 22, 99
    attributes, 27
token specification, 24
token type, 213
top-down parser, 104
transition function, 59
type annotation, 24

unary minus, 115
UNICODE alphabet, 7
union of languages, 84
universal language, 10

word boundary, 33